

# 基于位置型 DOP 的无线电干扰源定位与清除策略

## 摘要

无线电干扰源的持续存在会对正常通信和频谱秩序造成显著影响，需要尽快定位和清除。为提高干扰源的定位和清除效率，本文针对测向误差、接收范围及发射方向等多重不确定性，建立机器狗定位和清除模型，在保证干扰源全部清除的前提下尽可能缩短定位清除时间。

针对问题一，将示向度的  $\pm 1^\circ$  有界误差转化为楔形约束，并进一步用二维叉积表示为线性半平面交，利用集合成员估计构造凸多边形定位区域；在此基础上采用旋转卡壳法求取区域直径。用反证法证明该同直径圆不一定能完全覆盖该区域，并建立最小覆盖圆模型，结合 Carathéodory 定理与 Jung 定理分析其支撑结构和覆盖半径上下界，最后采用 Welzl 算法完成最小覆盖圆的最优圆心和半径求解。

针对问题二，根据首次示向结果和最大有效接收距离构造干扰源位置一致集，并利用首次成功接收信息的最小距离约束，建立第二检测点的稳定接收可行域。在保证再次成功接收信号的基础上，构造同时考虑检测距离与交会角的位置误差型 DOP，并以区域深度进行加权，降低极端边界位置对优化结果的过度影响；随后采用连续约束优化获得最优检测位置，并通过相对性能容差  $\tau$  构造近优水平集，得到第二检测点的候选区域。

针对问题三，在全向干扰源场景下，依据 1000 m 最小有效接收半径建立全域覆盖模型，综合移动与检测成本得到由中心点和八个外围点组成的九点扫描方案；随后按频道建立动态维护策略和滚动开放路径规划，完成机器狗的定位与清除任务。在此基础上，结合 Fisher 信息增益、单位时间信息效用和区域加权 DOP 选择补测位置，并引入试探清除机制和最小覆盖圆半径条件完成清除任务。再根据蒙特卡洛仿真实验结果进一步优化定位与清除策略，最后三次正式测试分别成功清除 12、14、10 个干扰源，平均定位清除时间分别为 279.93 s/个、262.52 s/个和 264.72 s/个。

针对问题四，进一步考虑定向干扰源未知发射方向带来的部分可观测性，将问题三的全域覆盖模型通过中心—双环优化得到 21 点全方向覆盖模型。在此基础上建立联合后验策略，根据观测信息动态更新目标状态，并综合预测接收概率优化问题三的补测决策模型。三次正式测试分别成功清除 12、12、10 个干扰源，平均定位清除时间分别为 533.54 s/个、486.33 s/个和 613.65 s/个，表明所建策略在复杂定向源场景下仍具备较好的任务完成能力和效率。

关键词：无线电干扰源 最小覆盖圆 位置型 DOP 全域覆盖模型 联合后验策略

# 一、问题重述

## 1.1 问题背景

无线电干扰会影响正常通信和无线电频谱管理，因而需要尽快确定干扰源位置并将其清除。实际排查通常先由固定监测站、车辆或无人机缩小目标范围，再由技术人员携带测向设备在目标分布区域徒步排查。现考虑利用机器狗在危险环境中代替人工执行搜索与清除任务，以提高定位及清除效率。

## 1.2 题目信息

- 待搜索区域是以坐标原点为圆心、半径为  $1800m$  的圆形区域。区域内分布着数量、位置未知的信号源，各干扰源占用互不相同的频道，且不随时间变化。
- 干扰源可分为全向源和定向源，其中全向源向各个方向辐射信号，定向源仅在发射方向左右各  $90^\circ$  范围内产生可接收信号；
- 每个干扰源的有效接受半径在  $1000m$  到  $1500m$  之间，实际取值未知。测向机给出的示向度存在  $\pm 1^\circ$  的误差，单次检测实际上只能把干扰源限制楔形区域。
- 机器狗移动速度为  $5m/s$ ，切换频道、完成测向和清除均会消耗时间，需要合理组织搜索、定位和清除过程；
- 当机器狗进入距干扰源  $20m$  以内时，可通过光学设备精确定位；进入  $5m$  范围且位于干扰源有效覆盖角度，会因信号过强而无法测向，但可直接转入光学定位并清除。

## 1.3 问题要求

**问题 1：**根据若干监测点及其测向结果，综合考虑方位角误差，确定干扰源可能所在的定位区域，计算该区域内任意两点的最大距离，即定位区域直径。进一步判断，直径等于该数值的圆是否必然能够覆盖整个定位区域；

**问题 2：**对于一个全向信号源，需要根据首次观测得到的方位信息，设计第二个检测点的选择方法，以获得较好的定位性能，并给出满足条件的候选区域；

**问题 3：**假设区域内共有 10—16 个全向干扰源，具体数量未知。设计能够清除全部干扰源的搜索、定位与清除策略，并在保证任务完成率的情况下尽量缩短总时间。通过模拟器演练和正式测试，验证策略的可靠性和效率；

**问题 4：**在区域内同时存在全向干扰源和定向干扰源，总数仍为 10—16 个但具体数量未知。进一步改进问题 3 的策略，扩展搜索覆盖、测向选点和清除算法，同时完成正式测试及日志整理。

## 二、问题分析

本题围绕机器狗对未知干扰源的自主定位与清除展开，是不完全信息下的感知与决策优化问题。这其中包括将带误差的测向信息转化为目标可能区域，选择有利的检测位置缩小该区域，最后通过全域搜索平衡各选择的时间消耗完成多目标清除。

### 2.1 问题一分析

问题一要求根据多个检测点的位置和带误差的示向度确定干扰源的定位区域，计算其直径，并判断该直径圆能否覆盖整个区域。题目中仅给出示向误差为  $\pm 1^\circ$ ，故本文不额外引入某一具体的概率分布。为完整保留误差界所包含的信息，采用集合成员估计 (Set Membership Estimation, SME) 描述角度不确定性，并据此构造包含真实干扰源的定位区域。再利用凸多边形的几何性质和旋转卡壳算法求最远点对，在此基础上分析该点对距离为直径的圆能否覆盖整个区域；若不能，则考虑引入最小包围圆去求得直径。

### 2.2 问题二分析

问题二需要根据一次带误差的示向观测选择第二检测位置。由于干扰源的实际位置和有效接收半径均未知，本文首先根据首次示向结果、测角误差和有效接收距离构造干扰源的位置一致集；随后消去未知接收半径，建立第二检测点的鲁棒接收约束；在此基础上，以区域深度加权的平均 DOP 衡量双站交会几何，求得最优检测位置，并利用相对性能容差提取近优水平集给出连续候选区域。

### 2.3 问题三分析

问题三可看作多目标在线搜索问题，已知目标区域中均为全向干扰源，要求在保证全部干扰源都被清除的前提下，尽量缩短任务总时间。为此，本文首先依据最小有效接收半径设计能够覆盖整个区域的扫描点集，并初步规划搜索路线；发现信号后，按频道维护目标定位区域，利用最小覆盖圆判断是否具备可靠清除条件。若定位精度不足，则在保证再次接收信号的范围内，根据高斯后验和 Fisher 信息计算候选位置的概率信息增益，并以区域加权 DOP 辅助选取补测点。每完成一次扫描、补测或清除，立即更新频道状态、定位区域和任务队列，再重新规划下一阶段路线，直至全部频道完成搜索且所有已发现目标均被清除。

### 2.4 问题四分析

问题四则将上一问的搜索与清除策略扩展到全向源和定向源混合的场景。该题需要注意未收到信号既可能由距离过远造成，也可能是机器狗位于定向源的辐射背面，因此不能再直接利用负观测排除目标。求解时需要联合描述干扰源的位置与发射方向，采用多视角覆盖保证定向源能够被发现，并根据正、负观测持续收缩联合可行域。

### 三、模型假设

为简化问题，本文做出以下假设：

- 假设机器狗可在二维平面内自由通行，忽略地形、障碍物、转向及加减速过程的影响，任意两任务点之间均沿直线移动；
- 假设定位区域非空且观测总体相容，即在示向误差不超过  $\pm 1^\circ$  的条件下，同一频道的多次观测存在共同可行解，不存在空集情况。
- 假设定向源类型与位置先验无特殊偏好，默认其位置是圆域均匀分布。

### 四、符号说明

| 符号                                                 | 符号说明                                            |
|----------------------------------------------------|-------------------------------------------------|
| $\mathbf{s}_i$                                     | 第 $i$ 个检测点的位置                                   |
| $\theta_i$                                         | 第 $i$ 个检测点测得的示向角                                |
| $W_i$                                              | 第 $i$ 次示向观测对应的集合成员一致集                           |
| $P$                                                | 多次示向观测共同确定的凸多边形定位区域                             |
| $r_*$                                              | 定位区域的最小覆盖半径                                     |
| $\Omega_1$                                         | 第一次检测后干扰源的目标可行区域                                |
| $C_{\text{recv}}$                                  | 第二检测点的鲁棒接收可行域                                   |
| $D(\mathbf{p}, \mathbf{g})$                        | 检测点 $\mathbf{p}$ 对候选源位置 $\mathbf{g}$ 的位置误差型 DOP |
| $\mathcal{R}_\tau$                                 | 第二检测点的近优候选区域                                    |
| $\mathcal{S}$                                      | 全局覆盖扫描点集合                                       |
| $P_{j,t}$                                          | 时刻 $t$ 频道 $j$ 的凸多边形定位区域                         |
| $\pi_{j,t}(\mathbf{z})$                            | 时刻 $t$ 频道 $j$ 的高斯规划后验密度                         |
| $I(\mathbf{p})$                                    | 候选点 $\mathbf{p}$ 的 Fisher 信息增益                  |
| $U(\mathbf{p})$                                    | 候选点 $\mathbf{p}$ 的单位时间信息效用                      |
| $\tau_j$                                           | 频道 $j$ 对应干扰源的类型                                 |
| $\phi_j$                                           | 频道 $j$ 对应定向源的发射朝向                               |
| $p_{j,t}(\mathbf{z}, \eta \mid \mathcal{H}_{j,t})$ | 频道 $j$ 的位置–类型–朝向联合后验密度                          |
| $P_{\text{rec}}(\mathbf{p})$                       | 候选点 $\mathbf{p}$ 的预测接收概率                        |

## 五、问题一的模型建立和求解

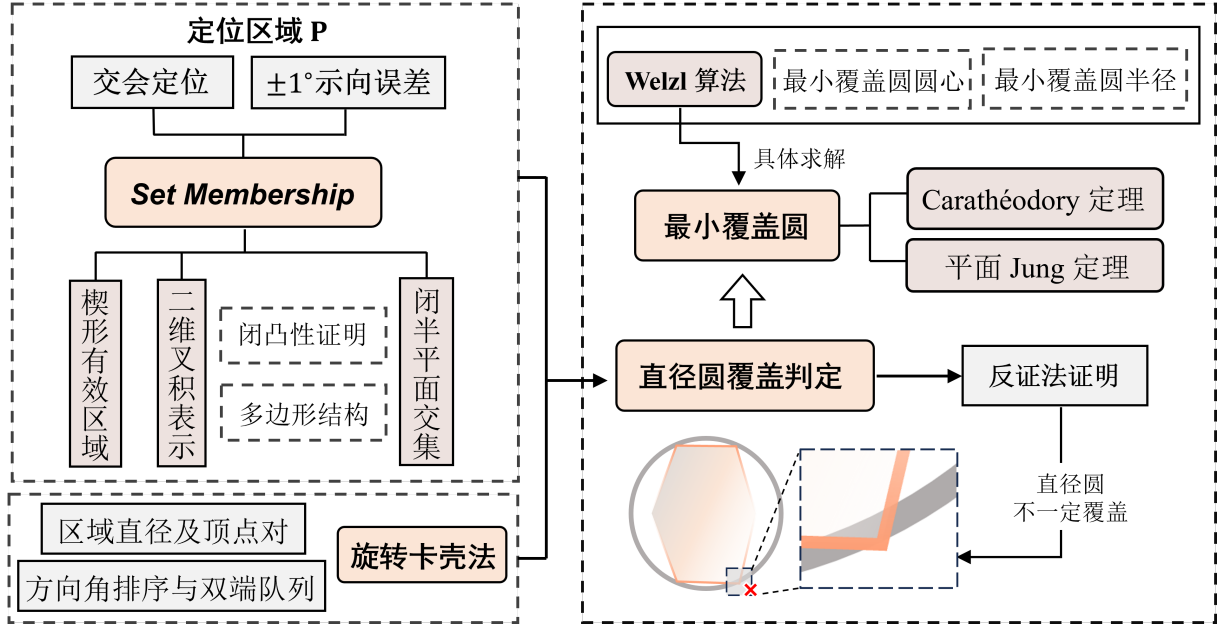


图 1 问题一思路框架

### 5.1 测向约束的半平面表示

#### 5.1.1 Set Membership 定位模型

SME 是一类基于确定性误差界的不确定性估计方法<sup>[1-2]</sup>。该方法将每次观测转化为由误差界限定的可行集合，再对多次观测全部可行集合求交，得到与所有观测同时一致的状态集合。只要实际误差未超出给定边界，真实状态就必然包含在最终一致集中。

设第  $i$  个检测点为  $\mathbf{s}_i = (x_i, y_i)^\top$ ，测得示向角为  $\theta_i$ ，干扰源候选位置为  $\mathbf{z} = (x, y)^\top$ ，误差上界记为  $\delta = \pi/180$ 。定义二维叉积  $[\mathbf{a}, \mathbf{b}] = a_x b_y - a_y b_x$ ，并令

$$\mathbf{u}_i^- = \begin{pmatrix} \cos(\theta_i - \delta) \\ \sin(\theta_i - \delta) \end{pmatrix}, \quad \mathbf{u}_i^+ = \begin{pmatrix} \cos(\theta_i + \delta) \\ \sin(\theta_i + \delta) \end{pmatrix}. \quad (1)$$

由于真实方位角必定位于  $[\theta_i - \delta, \theta_i + \delta]$  内，第  $i$  次观测对应的集合成员一致集为

$$W_i = \{\mathbf{z} \in \mathbb{R}^2 : [\mathbf{u}_i^-, \mathbf{z} - \mathbf{s}_i] \geq 0, [\mathbf{u}_i^+, \mathbf{z} - \mathbf{s}_i] \leq 0\}. \quad (2)$$

集合  $W_i$  是以  $\mathbf{s}_i$  为顶点、张角为  $2\delta$  的闭角锥。又因为  $0 < 2\delta < \pi$ ，式 (2) 描述的是楔形，不包含与示向方向相反的区域。

叉积约束可进一步写成线性半平面约束。令

$$\begin{aligned} \mathbf{a}_{i,-} &= \begin{pmatrix} \sin(\theta_i - \delta) \\ -\cos(\theta_i - \delta) \end{pmatrix}, \quad b_{i,-} = \mathbf{a}_{i,-}^\top \mathbf{s}_i, \\ \mathbf{a}_{i,+} &= \begin{pmatrix} -\sin(\theta_i + \delta) \\ \cos(\theta_i + \delta) \end{pmatrix}, \quad b_{i,+} = \mathbf{a}_{i,+}^\top \mathbf{s}_i. \end{aligned} \quad (3)$$

则每个楔形等价于两个半平面交：

$$W_i = \{ \mathbf{z} : \mathbf{a}_{i,-}^\top \mathbf{z} \leq b_{i,-}, \quad \mathbf{a}_{i,+}^\top \mathbf{z} \leq b_{i,+} \}. \quad (4)$$

将全部  $2n$  个半平面约束按行堆叠为矩阵  $A$  和向量  $\mathbf{b}$ ，其中矩阵  $A \in \mathbb{R}^{2n \times 2}$ ，向量  $\mathbf{b} \in \mathbb{R}^{2n}$ ，可得多次观测共同允许的定位区域

$$P = \bigcap_{i=1}^n W_i = \{ \mathbf{z} \in \mathbb{R}^2 : A\mathbf{z} \leq \mathbf{b} \}. \quad (5)$$

式 (5) 即为求得的 Set Membership。至此，测向交会定位已经转化为  $2n$  个闭半平面的交集计算。

### 5.1.2 定位区域的凸性和多边形结构

在计算定位区域直径之前，需首先明确可行集合  $P$  的几何性质。由式 (5) 可知，定位区域  $P$  是有限个闭半平面的交，因此具有闭性和凸性。具体而言，任取  $\mathbf{z}, \mathbf{w} \in P$  及  $\lambda \in [0, 1]$ 。对每个  $k$ ，均有

$$\mathbf{a}_k^\top (\lambda \mathbf{z} + (1 - \lambda) \mathbf{w}) = \lambda \mathbf{a}_k^\top \mathbf{z} + (1 - \lambda) \mathbf{a}_k^\top \mathbf{w} \leq b_k.$$

故  $\lambda \mathbf{z} + (1 - \lambda) \mathbf{w} \in P$ ，所以  $P$  为凸集；又因每个  $W_i$  均为闭集，有限个闭集的交仍为闭集，所以  $P$  也是闭集。

进一步，若  $P$  非空、有界且具有非空内部，则其边界由有限条约束直线上的线段构成。若某点严格满足全部约束  $\mathbf{a}_k^\top \mathbf{z} < b_k$ ，由约束的有限性与连续性，该点周围存在一个仍满足全部约束的小邻域，因此该点为  $P$  的内点。于是，任意边界点至少使一个约束取等，从而有

$$\partial P \subseteq \bigcup_{k=1}^{2n} \{ \mathbf{z} : \mathbf{a}_k^\top \mathbf{z} = b_k \}.$$

由前述闭凸性及  $P$  的有界性可知，每条约束直线与  $P$  的交至多是一条闭线段，也可能是单点或空集。因此， $P$  的边界由有限条线段组成。结合非空内部条件， $P$  为二维凸多边形。每个半平面至多贡献一条边，故其边数和顶点数均不超过  $2n$ 。

### 5.1.3 半平面交构造定位多边形

直径计算需要用到多边形顶点，因此还需通过半平面交构造定位多边形。将  $A\mathbf{z} \leq \mathbf{b}$  的第  $k$  个约束记为  $\mathbf{a}_k^\top \mathbf{z} \leq b_k$ 。由于  $\|\mathbf{a}_k\|_2 = 1$ ，可令

$$\mathbf{d}_k = (-a_{k,y}, a_{k,x})^\top, \quad \mathbf{t}_k = b_k \mathbf{a}_k. \quad (6)$$

此时  $\mathbf{t}_k$  位于第  $k$  条约束的边界上， $\mathbf{d}_k$  为边界方向向量。将边界写成有向直线  $\ell_k = \mathbf{t}_k + \tau \mathbf{d}_k$ ，则

$$[\mathbf{d}_k, \mathbf{z} - \mathbf{t}_k] = b_k - \mathbf{a}_k^\top \mathbf{z}, \quad H_k = \{\mathbf{z} : [\mathbf{d}_k, \mathbf{z} - \mathbf{t}_k] \geq 0\}. \quad (7)$$

因此，每个半平面都被统一表示为有向边界直线左侧的区域。对于两条不平行的边界直线，其交点为

$$I(\ell_k, \ell_l) = \mathbf{t}_k + \frac{[\mathbf{t}_l - \mathbf{t}_k, \mathbf{d}_l]}{[\mathbf{d}_k, \mathbf{d}_l]} \mathbf{d}_k. \quad (8)$$

具体算法如下：

1. 半平面交首先按照  $\mathbf{d}_k$  的方向角对边界排序。在单位法向量相同的表示下，保留较小的  $b_k$ 。
2. 随后用双端队列维护当前有效边界。每加入一个新半平面，若队尾两条边界的交点落在新半平面之外，则删除队尾边界并继续检查；队首采用相同处理。
3. 所有约束加入后，再检查队列首尾，使交集闭合。
4. 最后利用式 (8) 求相邻边界的交点，并删除重复点及连续共线的中间点，得到按逆时针排列的顶点集

$$\mathcal{V} = \{\mathbf{v}_0, \mathbf{v}_1, \dots, \mathbf{v}_{m-1}\}, \quad P = \text{conv}(\mathcal{V}). \quad (9)$$

### 5.2 定位区域直径的计算

定位区域直径定义为  $D = \text{diam}(P) = \max_{\mathbf{z}, \mathbf{w} \in P} \|\mathbf{z} - \mathbf{w}\|_2$ 。由于  $P$  非空、有界且闭，其最大值存在。进一步，凸多边形的直径必由一对顶点取得，即

$$D = \max_{0 \leq i, j < m} \|\mathbf{v}_i - \mathbf{v}_j\|_2. \quad (10)$$

为说明这一点，任取  $\mathbf{z}, \mathbf{w} \in P$ ，可分别写成顶点的凸组合

$$\mathbf{z} = \sum_{i=0}^{m-1} \alpha_i \mathbf{v}_i, \quad \mathbf{w} = \sum_{j=0}^{m-1} \beta_j \mathbf{v}_j,$$

其中  $\alpha_i, \beta_j \geq 0$ , 且  $\sum_i \alpha_i = \sum_j \beta_j = 1$ 。于是

$$\begin{aligned}\|\mathbf{z} - \mathbf{w}\|_2 &= \left\| \sum_{i,j} \alpha_i \beta_j (\mathbf{v}_i - \mathbf{v}_j) \right\|_2 \\ &\leq \sum_{i,j} \alpha_i \beta_j \|\mathbf{v}_i - \mathbf{v}_j\|_2 \\ &\leq \max_{0 \leq i, j < m} \|\mathbf{v}_i - \mathbf{v}_j\|_2.\end{aligned}$$

另一方面, 所有顶点均属于  $P$ , 因此上述上界可由某一对顶点取得, 连续区域上的直径计算由此转化为有限顶点对上的极值计算。式 (10) 可以选择直接枚举顶点对求直径, 但直接枚举需要  $O(m^2)$  时间。本文进一步利用凸性, 采用旋转卡壳法进行计算。

旋转卡壳法是求解凸多边形极值距离的几何算法。该方法利用一对相互平行的支撑线夹住凸多边形, 并沿多边形边界连续转动支撑线, 从而依次枚举可能形成最远点对的对踵点。前述半平面交已经得到按逆时针排列的凸多边形顶点, 因此可直接采用旋转卡壳法计算定位区域直径及其端点, 将候选范围缩小到对踵点对。

对每条边  $\mathbf{v}_i \mathbf{v}_{i+1}$ , 定义

$$T(i, j) = [\mathbf{v}_{i+1} - \mathbf{v}_i, \quad \mathbf{v}_j - \mathbf{v}_i] \quad (11)$$

表示相应平行四边形的有向面积。对固定的  $i$ , 最大化  $T(i, j)$  等价于寻找距离该边所在直线最远的顶点。随着边下标  $i$  沿逆时针方向增加, 只要  $T(i, j+1) > T(i, j)$ , 就将指针  $j$  向前移动, 并检查  $\mathbf{v}_j$  与边的两个端点之间的距离; 若两面积相等, 还需同时检查  $\mathbf{v}_{j+1}$ , 以免遗漏平行支撑边对应的最远点对。指针在整个过程中单向移动, 因此总推进次数为  $O(m)$ 。最终可得到直径  $D$  及一对直径端点  $\mathbf{p}_*, \mathbf{q}_*$ 。具体如 2 所示。

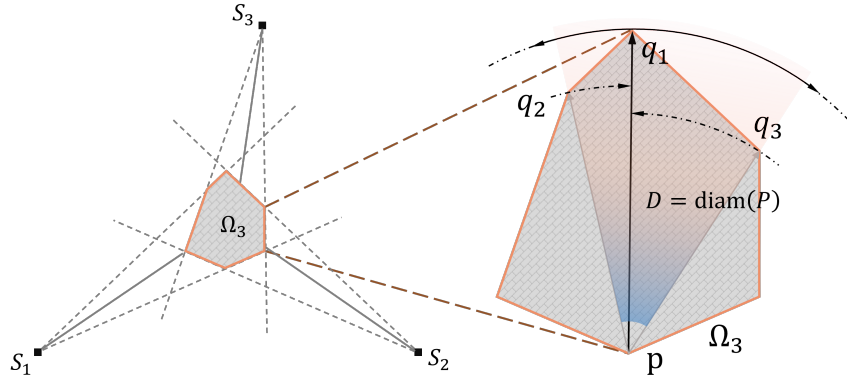


图 2 旋转卡壳示意图

令

$$\mathbf{c}_D = \frac{\mathbf{p}_* + \mathbf{q}_*}{2}, \quad r_D = \frac{D}{2}. \quad (12)$$

以最远点对为直径构造圆盘  $\mathbb{B}(\mathbf{c}_D, D/2)$ 。半径为  $D/2$  的圆盘能够覆盖  $P$ , 当且仅当

$$\max_{0 \leq j < m} \|\mathbf{v}_j - \mathbf{c}_D\|_2 \leq \frac{D}{2}. \quad (13)$$

该条件的必要性可由直径端点确定。若  $P \subseteq \mathbb{B}(\mathbf{c}, D/2)$ ，则

$$D = \|\mathbf{p}_* - \mathbf{q}_*\|_2 \leq \|\mathbf{p}_* - \mathbf{c}\|_2 + \|\mathbf{q}_* - \mathbf{c}\|_2 \leq D.$$

上式中的不等号必须同时取等，故  $\mathbf{p}_*, \mathbf{q}_*$  是该圆的一对对径点，圆心只能为  $\mathbf{c}_D$ 。反之，若式 (13) 成立，则该圆盘包含全部顶点；圆盘具有凸性，因而也包含这些顶点的凸包  $P$ 。因此，式 (13) 给出了直径圆覆盖定位区域的充要判据。

➤ 区域直径只刻画最远点对之间的距离，不能必然保证覆盖定位区域。

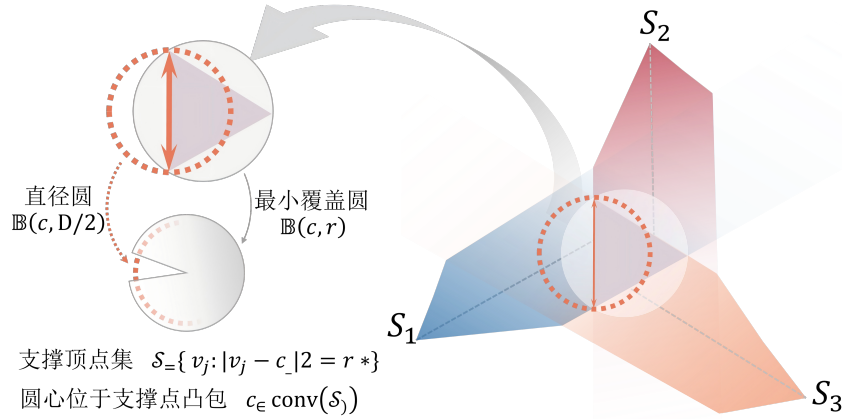


图 3 最小覆盖圆失效反例示意图

下面考虑图 3 所示的三次示向角锥交会，其公共一致集为边长为  $a$  的等边三角形。经平移与旋转后，可将三个顶点写为

$$\mathbf{v}_1 = \left(-\frac{a}{2}, 0\right)^\top, \quad \mathbf{v}_2 = \left(\frac{a}{2}, 0\right)^\top, \quad \mathbf{v}_3 = \left(0, \frac{\sqrt{3}a}{2}\right)^\top. \quad (14)$$

等边三角形任意两顶点间的距离均为  $a$ ，故定位区域的直径为

$$D = \max_{1 \leq i < j \leq 3} \|\mathbf{v}_i - \mathbf{v}_j\|_2 = a. \quad (15)$$

不妨选取  $\mathbf{v}_1, \mathbf{v}_2$  作为直径端点，则直径圆的圆心与半径分别为  $\mathbf{c}_D = (0, 0)^\top$  和  $D/2 = a/2$ 。然而第三个顶点到该圆心的距离满足

$$\|\mathbf{v}_3 - \mathbf{c}_D\|_2 = \frac{\sqrt{3}a}{2} > \frac{a}{2} = \frac{D}{2}, \quad (16)$$

因此第三个顶点位于直径圆之外。由于等边三角形的三组最远点对地位相同，选择其余任一边作为直径均会得到相同结论。

该三角形真正的最小覆盖圆为其外接圆，圆心位于三角形重心，最小覆盖半径为

$$\mathbf{c}_* = \left(0, \frac{\sqrt{3}a}{6}\right)^\top, \quad r_* = \frac{a}{\sqrt{3}} = \frac{D}{\sqrt{3}} > \frac{D}{2}. \quad (17)$$

故不存在直径为  $D$  的圆盘覆盖该定位区域。这说明直径圆必须经过式 (13) 的实际检验，不能由区域直径直接推出覆盖结论。

### 5.3 求解最小覆盖圆

最小覆盖圆法用于寻找能够覆盖给定点集或凸集的最小圆盘，其本质是一个单中心极小极大优化问题。定位区域直径只描述最远点对之间的距离，不一定能完全覆盖定位区域，那就需要重新优化圆心。为此，本文采用最小覆盖圆法求取最优圆心及最小覆盖半径。模型表示为：

$$r_* = \min_{\mathbf{c} \in \mathbb{R}^2} \max_{\mathbf{z} \in P} \|\mathbf{z} - \mathbf{c}\|_2. \quad (18)$$

对任意  $\mathbf{z} \in P$ ，将其写成顶点的凸组合可得

$$\|\mathbf{z} - \mathbf{c}\|_2 \leq \max_{0 \leq j < m} \|\mathbf{v}_j - \mathbf{c}\|_2. \quad (19)$$

因此，覆盖整个凸多边形等价于覆盖全部顶点，式 (18) 可写成二阶锥规划

$$\begin{aligned} & \min_{\mathbf{c}, r} r \\ \text{s.t.} \quad & \begin{cases} \|\mathbf{v}_j - \mathbf{c}\|_2 \leq r, & j = 0, 1, \dots, m-1, \\ r \geq 0. \end{cases} \end{aligned} \quad (20)$$

#### 5.3.1 Carathéodory 定理

记式 (20) 的最优解为  $(\mathbf{c}_*, r_*)$ 。目标函数连续且随  $\|\mathbf{c}\|_2$  增大而趋于无穷，故最优圆存在。若存在两个不同的最优圆心  $\mathbf{c}_1, \mathbf{c}_2$ ，则对任意顶点  $\mathbf{v}_j$  有

$$\left\| \mathbf{v}_j - \frac{\mathbf{c}_1 + \mathbf{c}_2}{2} \right\|_2^2 = \frac{\|\mathbf{v}_j - \mathbf{c}_1\|_2^2 + \|\mathbf{v}_j - \mathbf{c}_2\|_2^2}{2} - \frac{\|\mathbf{c}_1 - \mathbf{c}_2\|_2^2}{4} < r_*^2,$$

这与  $r_*$  的最小性矛盾，因此最优圆心唯一。再定义落在最优圆周上的支撑顶点集

$$\mathcal{S}_* = \{\mathbf{v}_j : \|\mathbf{v}_j - \mathbf{c}_*\|_2 = r_*\}.$$

若  $\mathbf{c}_* \notin \text{conv}(\mathcal{S}_*)$ ，由严格分离定理可找到一个方向，使圆心沿该方向作充分小的移动后，所有支撑顶点到新圆心的距离同时减小；非支撑顶点原有严格余量也不会被破坏，于是可得到半径小于  $r_*$  的覆盖圆，与最优性矛盾。因此

$$\mathbf{c}_* \in \text{conv}(\mathcal{S}_*).$$

平面 Carathéodory 定理指出，平面点集凸包中的任一点均可表示为其中至多三个点的凸组合。因而存在  $t \leq 3$  个支撑顶点  $\mathbf{w}_1, \dots, \mathbf{w}_t$  及权重  $\lambda_l \geq 0$ ，使

$$\sum_{l=1}^t \lambda_l = 1, \quad \mathbf{c}_* = \sum_{l=1}^t \lambda_l \mathbf{w}_l.$$

对任意候选圆心  $\mathbf{c}$ ，由  $\|\mathbf{w}_l - \mathbf{c}_*\|_2 = r_*$  可得

$$\sum_{l=1}^t \lambda_l \|\mathbf{w}_l - \mathbf{c}\|_2^2 = r_*^2 + \|\mathbf{c} - \mathbf{c}_*\|_2^2 \geq r_*^2.$$

故仅至多三个顶点就足以阻止覆盖半径继续减小。非退化情形只有两类：

1. 若由两个顶点确定，则二者为对径点，圆心是其中点且  $r_* = D/2$ ;
2. 若由三个顶点确定，则圆心为其中心点，三点构成锐角三角形，最优圆为其外接圆且  $r_* > D/2$ 。

其中，直角三角形可由斜边两端点支持，归入两点情形；钝角三角形的外接圆不是其最小覆盖圆。由此得到直径圆与最小覆盖圆之间的等价关系：

$$r_* = \frac{D}{2} \iff P \subseteq \mathbb{B}\left(c_D, \frac{D}{2}\right). \quad (21)$$

### 5.3.2 平面 Jung 定理与最坏覆盖上界

Jung 定理是凸几何中联系有界集合直径与最小覆盖球半径的基本结论。设  $K \subset \mathbb{R}^d$  为非空有界集合，其直径和最小覆盖球半径分别为  $D_K$  与  $R_K$ ，则 Jung 定理指出

$$R_K \leq \sqrt{\frac{d}{2(d+1)}} D_K.$$

该定理给出了在只知道集合最大两点距离的情况下，至少需要多大的球才能保证覆盖所有可能形状。其给出的是与集合具体形状无关的最坏情形上界。

本题的定位区域位于二维平面，令  $d = 2$ 、 $K = P$ ，便得到  $r_* \leq D/\sqrt{3}$ 。另一方面，任意覆盖圆都必须同时容纳一对距离为  $D$  的直径端点，故由三角不等式有  $r_* \geq D/2$ 。将二者合并可得

$$\frac{D}{2} \leq r_* \leq \frac{D}{\sqrt{3}}. \quad (22)$$

结合上一节得到的支撑点结构，还可在平面情形下直接验证 Jung 上界。若最优圆由两个顶点确定，则  $r_* = D/2$ ；若由三个顶点确定，设支撑三角形的最大内角为  $\gamma$ ，其对边长为  $a$ ，则  $\pi/3 \leq \gamma < \pi/2$  且  $a \leq D$ 。由扩展正弦定理，

$$r_* = \frac{a}{2 \sin \gamma} \leq \frac{a}{\sqrt{3}} \leq \frac{D}{\sqrt{3}}.$$

当且仅当三个支撑顶点构成边长为  $D$  的等边三角形时，上界取等。因此，式 (22) 给出的是所有同直径定位区域中不可再改进的最坏覆盖上界。

### 5.3.3 基于 Welzl 算法的最小覆盖圆求解

在对最小覆盖圆进行求解时，虽然已知最优圆由至多三个支撑顶点确定，但若直接枚举所有点对和三元组，再逐一检验其候选圆能否覆盖全部顶点，计算量会随顶点数迅速增加。经查阅资料，本文采用 Welzl 算法进行求解。

该算法先随机打乱顶点次序，并维护已处理点集的最小覆盖圆。若新顶点位于当前圆盘内，它不会改变当前解；若新顶点位于圆盘外，则更新后的最小覆盖圆必须经过该点，因此可将其加入待确定的边界集并递归求解。随着越界点不断加入，边界集逐步锁

定真正的支撑顶点；由于最优圆至多由三个支撑顶点确定，边界集达到三个点时即可直接构造唯一候选圆，递归由此具有固定规模的终止情形。

记  $\text{MEC}(Q, B)$  为覆盖  $Q \cup B$  且要求边界点集  $B$  位于圆周上的最小覆盖圆盘，其中  $Q$  为尚待检验的点集，且  $|B| \leq 3$ 。取  $p \in Q$ ，先计算  $C = \text{MEC}(Q \setminus \{p\}, B)$ ，则递推关系为

$$\text{MEC}(Q, B) = \begin{cases} C, & p \in C, \\ \text{MEC}(Q \setminus \{p\}, B \cup \{p\}), & p \notin C, \end{cases}$$

当  $Q = \emptyset$  或  $|B| = 3$  时递归终止，并由  $B$  直接构造圆盘。边界集中只有一个点时圆心即为该点、半径为零；有两个点时取二者中点及半距离；有三个非共线点时，通过

$$\begin{aligned} 2(\mathbf{w}_2 - \mathbf{w}_1)^T \mathbf{c} &= \|\mathbf{w}_2\|_2^2 - \|\mathbf{w}_1\|_2^2, \\ 2(\mathbf{w}_3 - \mathbf{w}_1)^T \mathbf{c} &= \|\mathbf{w}_3\|_2^2 - \|\mathbf{w}_1\|_2^2 \end{aligned}$$

求得外接圆心，再计算  $r = \|\mathbf{w}_1 - \mathbf{c}\|_2$ ；三点近似共线时，退化为覆盖三点的最远点对直径圆。这样，圆心优化与覆盖尺度计算在每次边界更新中同步完成，最终输出  $\mathbf{c}_*$ 、 $r_*$  及最小覆盖圆直径  $2r_*$ 。随机次序使算法的期望时间复杂度为  $O(m)$ 。

## 六、问题二的模型建立和求解

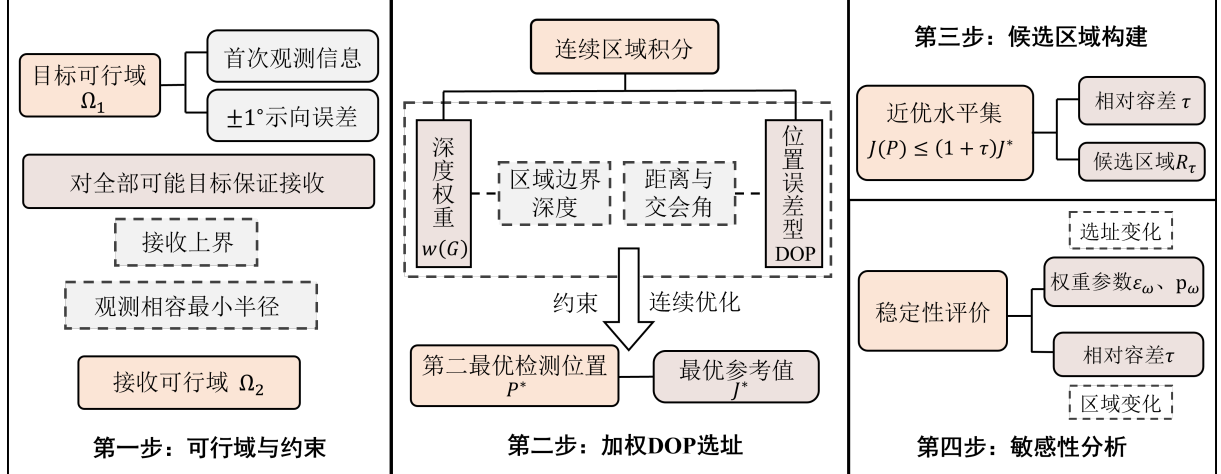


图 4 问题二思路框架

### 6.1 目标位置可行区域

在没有第二条示向约束的情况下，无法直接形成交会定位多边形，因而本文将首次示向得到的楔形与可接收信号范围相交，保留所有仍可能为真实位置的点，并以此作为后续鲁棒选点的输入。

设第一检测点为  $\mathbf{s}_1 = (x_1, y_1)^T$ ，测得示向度为  $\theta_1$ ，干扰源位置记为  $\mathbf{g} = (u, v)^T$ ，测角误差上界为  $\delta = \frac{\pi}{180}$ 。为了减少目标分布圆域对楔形区域的影响，本文将目标分布圆

域  $\Omega_0 = \{\mathbf{g} \in \mathbb{R}^2 : \|\mathbf{g}\|_2 \leq 1800\}$  忽略，将首次测向楔形表示为：

$$W_1 = \{\mathbf{g} : d_{\text{ang}}(\arg(\mathbf{g} - \mathbf{s}_1), \theta_1) \leq \delta\}, \quad (23)$$

其中  $d_{\text{ang}}$  表示圆周上的最小角度差。由于第一次检测已经获得稳定示向度，干扰源必在第一检测点的最大有效接收范围内，故其位置一致集为

$$\Omega_1 = W_1 \cap \mathbb{B}(\mathbf{s}_1, 1500). \quad (24)$$

虽然题目信息指出目标距观测点距离小于  $5m$  时接收不到信号，但在实际计算中，若严格删去该小圆盘会破坏凸性。考虑到该尺度远小于有效接收半径，模型暂保留这部分区域。为便于计算，又将两段圆弧用切线外接多边形逼近，将楔形继续表示成两个半平面，从而得到

$$\Omega_1^{(m)} = \{\mathbf{g} : \mathbf{a}_j^\top \mathbf{g} \leq b_j, j = 1, 2, \dots, m\}, \quad \Omega_1 \subseteq \Omega_1^{(m)}. \quad (25)$$

## 6.2 第二检测点的鲁棒接收可行域

设干扰源的实际有效接收半径为  $R$ ，则由题意可得： $1000 \leq R \leq 1500$ 。对任意  $\mathbf{g} \in \Omega_1$ ，与该目标位置相容的接收半径集合为

$$\mathcal{R}(\mathbf{g}) = [\max\{1000, \|\mathbf{g} - \mathbf{s}_1\|_2\}, 1500]. \quad (26)$$

记第二检测点为  $\mathbf{p} = (x, y)^\top$ 。为了在实际半径未知时仍保证第二次检测成功， $\mathbf{p}$  必须对每个可能的  $\mathbf{g}$  以及与其相容的每个  $R$  均满足  $\|\mathbf{p} - \mathbf{g}\|_2 \leq R$ 。该条件等价于仅对区间左端点进行约束，将其转化为约束函数形式：

$$H(\mathbf{p}) = \sup_{\mathbf{g} \in \Omega_1} \left\{ \|\mathbf{p} - \mathbf{g}\|_2^2 - \max\{1000, \|\mathbf{g} - \mathbf{s}_1\|_2\}^2 \right\}, \quad (27)$$

$$C_{\text{recv}} = \{\mathbf{p} : H(\mathbf{p}) \leq 0\}$$

如图 5 所示，第二检测点需同时满足首次观测导出的距离约束与稳定接收约束，其交叠部分构成鲁棒接收可行域  $C_{\text{recv}}$ 。

## 6.3 加权 DOP 连续优化模型

DOP 用于刻画观测几何对定位误差的放大程度<sup>[3-4]</sup>。对于双站测向，定位效果同时受检测距离和交会角影响：检测距离越大，角度误差对应的横向位置误差越大；两条测向线越接近平行，交会误差的放大越显著。因此，记

$$r_1(\mathbf{g}) = \|\mathbf{g} - \mathbf{s}_1\|_2, \quad r_2(\mathbf{p}, \mathbf{g}) = \|\mathbf{g} - \mathbf{p}\|_2, \quad (28)$$

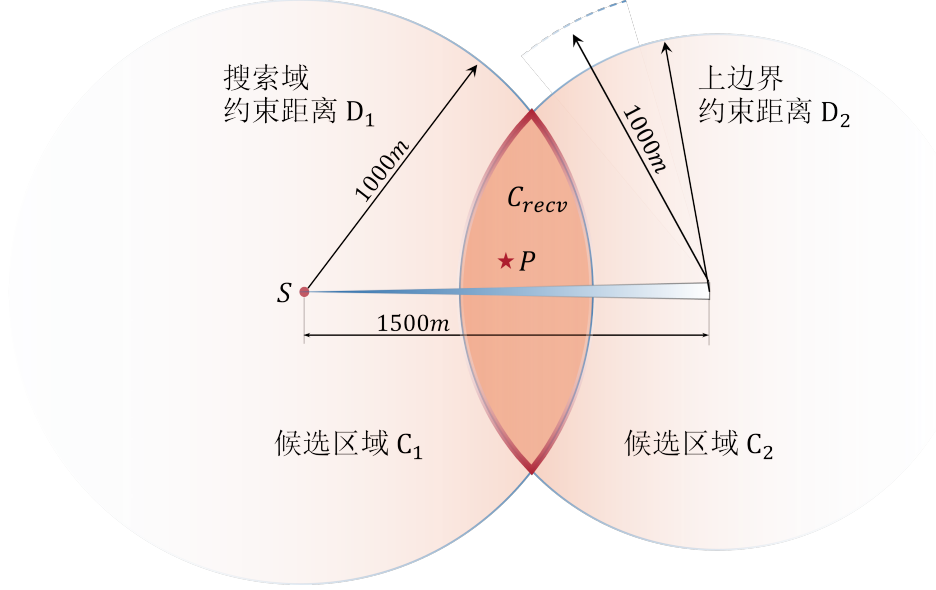


图 5 第二检测点候选区域

并以  $\alpha(\mathbf{p}, \mathbf{g}) = \angle(\mathbf{s}_1 - \mathbf{g}, \mathbf{p} - \mathbf{g})$  表示两条测向线的交会角，定义位置误差型 DOP 为

$$D(\mathbf{p}, \mathbf{g}) = \frac{r_1(\mathbf{g})r_2(\mathbf{p}, \mathbf{g})\sqrt{r_1^2(\mathbf{g}) + r_2^2(\mathbf{p}, \mathbf{g})}}{|\det(\mathbf{g} - \mathbf{s}_1, \mathbf{g} - \mathbf{p})|}, \quad (29)$$

其值越小，双站交会几何越有利。又因为单个目标位置不能代表整个集合  $\Omega_1$ 。为兼顾全域性能并适当提高区域内部的影响，本文引入目标位置到边界的深度

$$d(\mathbf{g}) = \text{dist}(\mathbf{g}, \partial\Omega_1), \quad d^* = \max_{\mathbf{g} \in \Omega_1} d(\mathbf{g}). \quad (30)$$

若采用式 (25) 的半平面表示，则

$$d(\mathbf{g}) = \min_{1 \leq j \leq m} \frac{b_j - \mathbf{a}_j^\top \mathbf{g}}{\|\mathbf{a}_j\|_2}, \quad (31)$$

而  $d^*$  可由最大内切圆对应的线性规划求得：

$$\begin{aligned} & \max_{\mathbf{c}, t} t \\ \text{s.t.} \quad & \begin{cases} \mathbf{a}_j^\top \mathbf{c} + t\|\mathbf{a}_j\|_2 \leq b_j, & j = 1, 2, \dots, m, \\ t \geq 0. \end{cases} \end{aligned} \quad (32)$$

据此构造归一化深度  $q(\mathbf{g}) = \frac{d(\mathbf{g})}{d^*}$  以及权重函数

$$w(\mathbf{g}; \varepsilon_w, p_w) = \varepsilon_w + (1 - \varepsilon_w)q^{p_w}(\mathbf{g}), \quad 0 < \varepsilon_w < 1, \quad p_w \geq 1. \quad (33)$$

其中  $\varepsilon_w$  为边界附近保留的基础权重， $p_w$  控制附加权重向区域内部集中的程度。令

$$Z(\varepsilon_w, p_w) = \int_{\Omega_1} w(\mathbf{g}; \varepsilon_w, p_w) dA > 0, \quad (34)$$

则第二检测点关于整个位置可行域的加权平均 DOP 为

$$J(\mathbf{p}; \varepsilon_w, p_w) = \frac{1}{Z(\varepsilon_w, p_w)} \int_{\Omega_1} w(\mathbf{g}; \varepsilon_w, p_w) D(\mathbf{p}, \mathbf{g}) dA. \quad (35)$$

当两条测向线趋于共线时，式 (29) 的分母趋于零。若共线直线穿过  $\Omega_1$  的内部，其邻域中的 DOP 通常不可积，不能因共线点本身面积为零而将其忽略。为此，将积分发散时的目标函数定义为  $J(\mathbf{p}) = +\infty$ ，并记

$$C_{\text{fin}} = \{\mathbf{p} \in C_{\text{recv}} : J(\mathbf{p}) < +\infty\}. \quad (36)$$

最终的检测点优化模型为

$$\begin{aligned} J^* &= \min_{\mathbf{p} \in \mathbb{R}^2} J(\mathbf{p}; \varepsilon_w, p_w) \\ \text{s.t.} \quad &\begin{cases} H(\mathbf{p}) \leq 0, \\ \mathbf{p} \in C_{\text{fin}}. \end{cases} \end{aligned} \quad (37)$$

相应的最优检测位置满足

$$\mathbf{p}^* \in \arg \min_{\mathbf{p} \in C_{\text{fin}}} J(\mathbf{p}). \quad (38)$$

求解时，先按照  $\|\mathbf{g} - \mathbf{s}_1\|_2 \leq 1000$  与否将约束函数分段。前一部分关于  $\mathbf{g}$  为凸二次函数，后一部分为仿射函数，其最大值均可在相应子区域的边界上搜索，从而将  $H(\mathbf{p})$  的连续核验化为分段边界的一维极值计算。随后对  $\Omega_1$  进行三角剖分，采用自适应高斯求积计算式 (35)，并在  $C_{\text{recv}}$  内进行多初值约束优化。对共线直线穿过区域内部的检测位置直接赋予无穷目标值，避免有限求积节点漏检奇异性。

## 6.4 近优候选区域与敏感性分析

### 6.4.1 近优候选区域

单个数值最优点容易受到参数和计算误差影响，也不能直接满足题目给出候选区域的要求。为使单个数值最优点扩展为近优候选区域，本文引入相对容差系数  $\tau > 0$ ，赋予区域内各点具有统一的评价标准。定义

$$\mathcal{R}_\tau = \{\mathbf{p} \in C_{\text{fin}} : J(\mathbf{p}; \varepsilon_w, p_w) \leq (1 + \tau)J^*\}. \quad (39)$$

图 6 中严格接收候选区域内的加权 DOP 存在明显空间差异，低值区域对应更有利的双站交会几何。近优区域沿低 DOP 带分布，并围绕最优点形成连续可选范围，从而兼顾定位性能与选点灵活性。

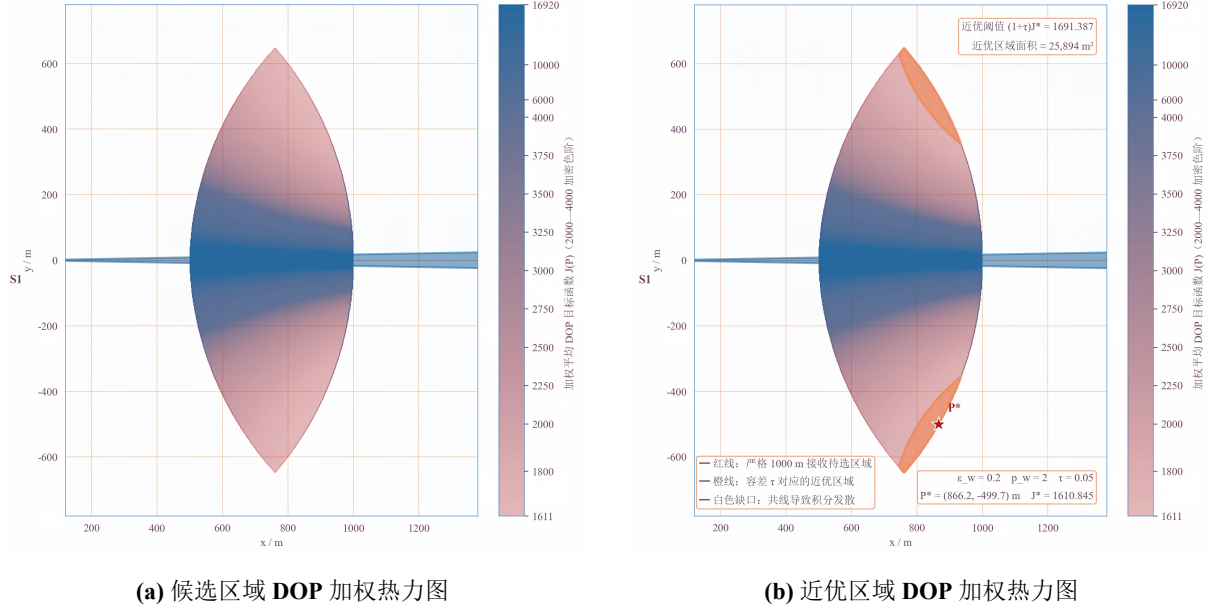


图 6 权重参数对评价指标和最优位置偏转角的影响

### 6.4.2 敏感性分析

模型中的  $\varepsilon_w$ 、 $p_w$  和  $\tau$  分别控制边界基础权重、深度权重形状和候选区域面积大小。本文在实际求解时取  $\varepsilon_w \in \{0.1, 0.2, 0.3, 0.5\}$ 、 $p_w \in \{1, 2, 3\}$  和  $\tau \in \{0.01, 0.03, 0.05, 0.10, 0.20\}$  进行敏感性分析。又因为改变  $\varepsilon_w$  或  $p_w$  会改变目标函数，不同权重下的  $J^*$  不能直接比较，故选定统一参考权重  $w_0$ ，用

$$J_0(\mathbf{p}) = \frac{\int_{\Omega_1} w_0(\mathbf{g}) D(\mathbf{p}, \mathbf{g}) dA}{\int_{\Omega_1} w_0(\mathbf{g}) dA} \quad (40)$$

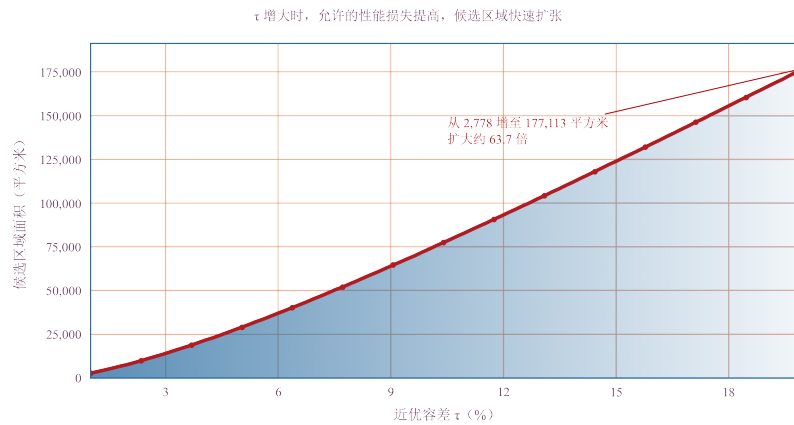
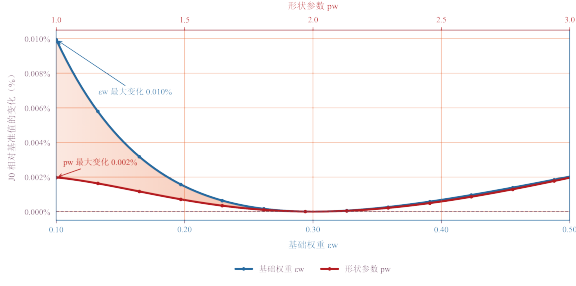


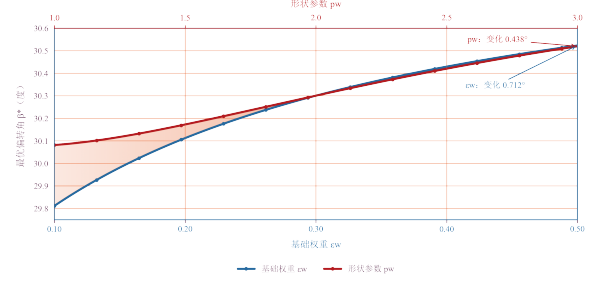
图 7 近优容差对候选区域面积的影响

图 7 表明，随着近优容差  $\tau$  增大，候选区域面积单调扩张，反映了选点范围与允许性能损失之间的权衡。

由图 8 可知，参数变化引起的  $J_0$  相对变化不超过 0.010%，最优位置偏转角的变化小于  $1^\circ$ ，其中基础权重  $\varepsilon_w$  的影响略大于形状参数  $p_w$ 。



(a) 权重参数对评价指标  $J_0$  的影响



(b) 权重参数对最优位置偏转角的影响

图 8 权重参数对评价指标和最优位置偏转角的影响

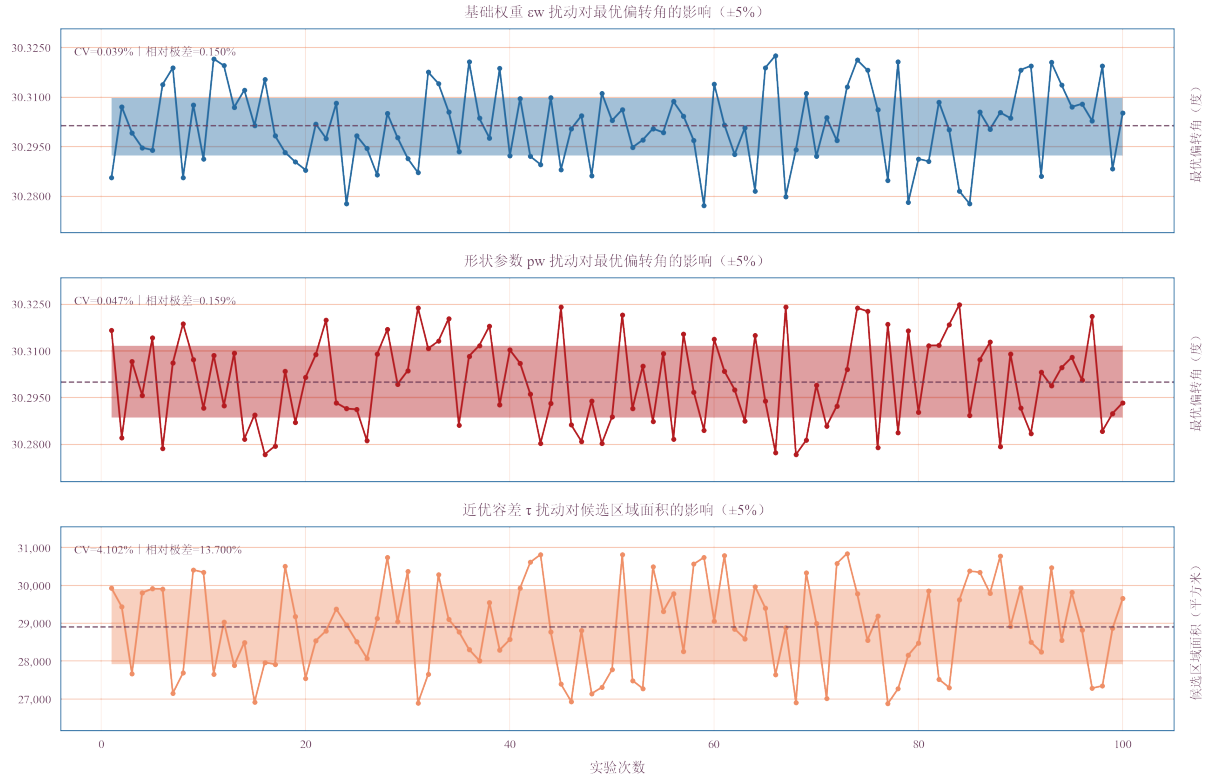


图 9 三个参数的 100 次局部扰动鲁棒性评估

而  $\varepsilon_w$  与  $p_w$  扰动下最优偏转角的变异系数均低于 0.05%，候选区域面积对  $\tau$  更为敏感，说明最优方向稳定但区域尺度主要受近优容差控制。具体如图 9 显示

## 七、问题三的模型建立与求解

### 7.1 全局覆盖扫描

设目标分布区域为

$$\Omega = \mathbb{B}(\mathbf{0}, R_0), \quad R_0 = 1800 \text{ m},$$

实际干扰源数量为  $N$ ，满足  $10 \leq N \leq 16$ ；频道编号为  $j \in \{1, \dots, 20\}$ ，且每个频道至多对应一个静止全向干扰源。第  $j$  个干扰源的位置和有效接收半径分别记为  $\mathbf{z}_j$  和

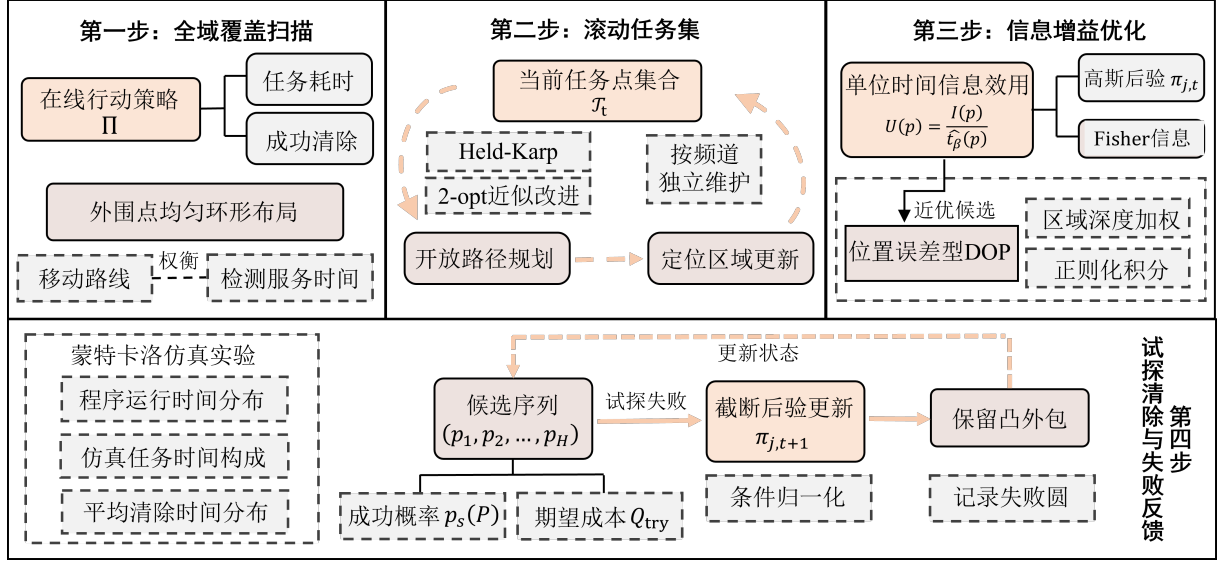


图 10 问题三思路框架

$\rho_j$ , 其中

$$1000 \leq \rho_j \leq 1500.$$

机器狗移动速度为  $v = 5 \text{ m/s}$ , 示向度误差满足确定性边界  $\delta = 1^\circ$ , 同一位置的测角误差不因重复检测而改变。记累计移动距离为  $L$ 。频道切换、检测、光学定位和成功清除的次数依次记为

$$N_{\text{sw}}, \quad N_{\text{m}}, \quad N_{\text{a}}, \quad N_{\text{c}}.$$

根据题设给出的动作耗时, 实际任务时间为

$$T = \frac{L}{5} + N_{\text{sw}} + 5N_{\text{m}} + 3N_{\text{a}} + 2N_{\text{c}}. \quad (41)$$

此外, 每个频道均维护未知、已发现、已清除和已认证不存在四种状态。记在线行动策略为  $\Pi$ , 则总体目标写为

$$\min_{\Pi} \mathbb{E}_{\Pi}[T], \quad (42)$$

s.t. 全部真实干扰源最终获得成功清除反馈.

其中, 期望由后文构造的规划概率模型计算; 真实任务耗时始终按式 (41) 统计。

所有全向干扰源的有效接收半径均不小于  $a = 1000 \text{ m}$ 。因此, 只要扫描点集合  $\mathcal{S}$  满足

$$\Omega \subseteq \bigcup_{s \in \mathcal{S}} \mathbb{B}(s, a), \quad (43)$$

并在各扫描点完成相应频道的检测, 便可保证每个真实干扰源至少被发现一次。为兼顾覆盖性和路线长度, 保留原点作为中心扫描点, 并在半径为  $R$  的圆周上均匀布置  $n$

个外围点：

$$\begin{aligned} \mathbf{s}_0 &= (0, 0)^\top, \\ \mathbf{s}_{k+1} &= R \begin{bmatrix} \cos(2\pi k/n) \\ \sin(2\pi k/n) \end{bmatrix}, \quad k = 0, \dots, n-1. \end{aligned} \quad (44)$$

中心点能够覆盖半径为  $a$  的内盘。对于外环中极径为  $r \in [a, R_0]$  的任意位置，其与最近外围点的极角差不超过  $\pi/n$ ，故二者距离平方的最大值为

$$g_n(r; R) = r^2 + R^2 - 2rR \cos \frac{\pi}{n}. \quad (45)$$

函数  $g_n(r; R)$  关于  $r$  为凸二次函数，闭区间上的最大值必在端点取得。因此，中心点与外围点共同覆盖  $\Omega$  的充要条件为

$$g_n(a; R) \leq a^2, \quad g_n(R_0; R) \leq a^2. \quad (46)$$

令  $c_n = \cos(\pi/n)$  及

$$\Delta_n = a^2 - R_0^2 \sin^2 \frac{\pi}{n}.$$

当  $\Delta_n < 0$  时不存在可行布点半径；当  $\Delta_n \geq 0$  时，式 (46) 给出当  $R \in [R_n^-, R_n^+]$  时：

$$\begin{aligned} R_n^- &= \max \left\{ 0, R_0 c_n - \sqrt{\Delta_n} \right\}, \\ R_n^+ &= \min \left\{ 2a c_n, R_0 c_n + \sqrt{\Delta_n} \right\}. \end{aligned} \quad (47)$$

若机器狗从中心出发，先到达一个外围点，再沿圆周依次访问其余外围点且不返回，则基础路线长度为

$$L_n(R) = R \left[ 1 + 2(n-1) \sin \frac{\pi}{n} \right]. \quad (48)$$

固定  $n$  时， $L_n(R)$  随  $R$  单调增加，故取可行下界  $R_n^-$ 。枚举  $n = 0, \dots, 10$  可知， $n \leq 5$  时均不可行，可行方案如下表所示。

表 2 均匀环形扫描点方案比较

| 外围点数 $n$ | 总点数 $n+1$ | 可行半径区间/m             | 基础路径/m   | $\tilde{T}_n(c_s = 30)/s$ |
|----------|-----------|----------------------|----------|---------------------------|
| 6        | 7         | [1122.956, 1732.051] | 6737.735 | 1557.547                  |
| 7        | 8         | [997.201, 1801.938]  | 6189.235 | 1477.847                  |
| 8        | 9         | [938.060, 1847.759]  | 5963.783 | 1462.757                  |
| 9        | 10        | [903.416, 1879.385]  | 5847.201 | 1469.440                  |
| 10       | 11        | [880.874, 1902.113]  | 5780.563 | 1486.113                  |

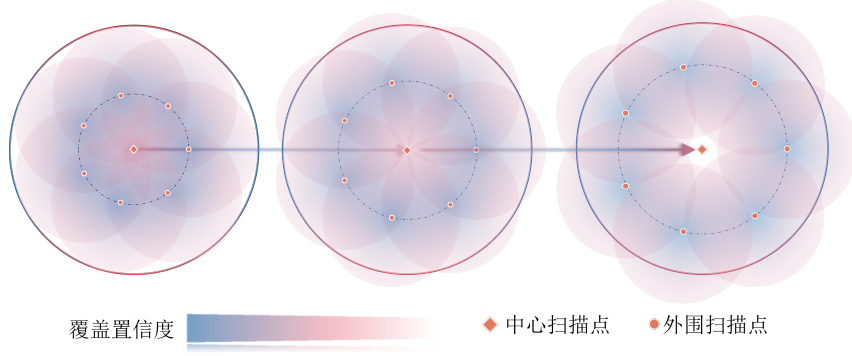


图 11 均匀取点全覆盖方案示意图

由图 8 可见，不同外围点数下中心—单环布局的覆盖效果，用于比较覆盖完整性与边界冗余。

而图 12 则刻画了相邻外围点与目标区域最不利边界点之间的几何关系，为式 (47) 的可行布点半径区间提供直观解释。

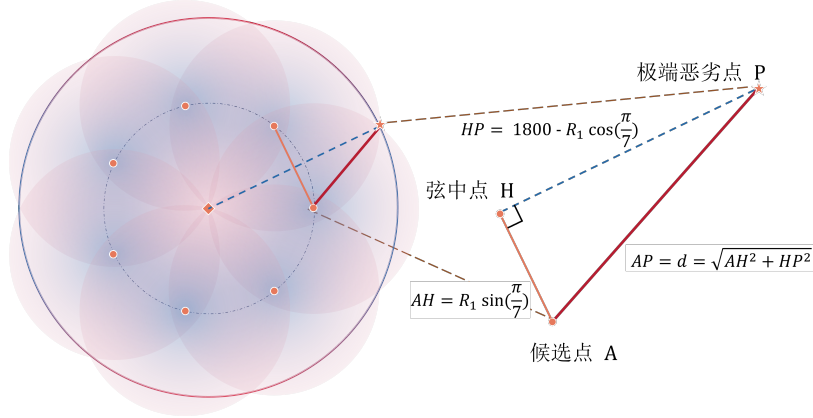


图 12 可行布点半径求解示意图

增加外围点能够缩短移动路线，但也会增加检测服务时间。设每个扫描点的平均有效服务成本为  $c_s$ ，构造布局选型代理

$$\tilde{T}_n = \frac{L_n(R_n^-)}{5} + (n+1)c_s. \quad (49)$$

当  $23.316 \leq c_s \leq 45.090$  秒时，八外围点方案在该对称布点族内不劣于其余可行方案。取  $c_s = 30$  秒时，表 2 中  $n = 8$  的代理时间最小。因此，本文采用中心加外围八点的九点扫描方案，并将理论下界 938.060414 米向上取整为 939 米：

$$\mathcal{S} = \{(0,0)^T\} \cup \left\{ 939 \begin{bmatrix} \cos(k\pi/4) \\ \sin(k\pi/4) \end{bmatrix} : k = 0, \dots, 7 \right\}. \quad (50)$$

取整后，覆盖条件两个端点处的最坏距离约为 382.982 米和 999.319 米，均小于 1000 米，故全域覆盖仍然成立，相应基础路径约为 5969.756 米。

## 7.2 滚动任务集

设  $\mathcal{U}_t, \mathcal{F}_t, \mathcal{D}_t, \mathcal{A}_t$  分别表示时刻  $t$  的未知频道、已发现未清除频道、已清除频道和已认证不存在频道。每个频道  $j$  还保存已检测扫描点编号集合  $E_{j,t}$  以及全部观测历史  $\mathcal{H}_{j,t}$ 。若未知频道  $j$  在位置  $\mathbf{q}$  返回无信号，则由最小接收半径可排除

$$\mathbf{z}_j \in \mathbb{B}(\mathbf{q}, 1000), \quad \text{即 } \mathbf{z}_j \notin \mathbb{B}(\mathbf{q}, 1000). \quad (51)$$

单次无信号只能排除局部位置；若某未知频道在九个扫描点均无信号，则其存在性与式 (43) 矛盾，因而可将该频道转入  $\mathcal{A}_t$ 。已发现频道不会因后续无信号而被改判。

将仍有未知频道待扫描的扫描点集合记为  $\mathcal{S}_t^{\text{pending}}$ 。对于已发现目标，采用近似后验均值  $\boldsymbol{\mu}_{j,t}$  作为全局调度中的代表位置；若概率求积暂时失效，则退回其集合成员区域的最小覆盖圆心  $\mathbf{c}_{j,t}^*$ 。当前任务节点集合为

$$\mathcal{T}_t = \mathcal{S}_t^{\text{pending}} \cup \{\hat{\mathbf{z}}_{j,t} : j \in \mathcal{F}_t\}, \quad \hat{\mathbf{z}}_{j,t} = \begin{cases} \boldsymbol{\mu}_{j,t}, & \text{后验可用,} \\ \mathbf{c}_{j,t}^*, & \text{后验不可用.} \end{cases} \quad (52)$$

若除当前位置外共有  $M$  个任务节点，坐标为  $\mathbf{x}_1, \dots, \mathbf{x}_M$ ，机器狗当前位置为  $\mathbf{q}_t$ ，则开放路径模型为

$$\min_{\sigma} \left\{ \|\mathbf{q}_t - \mathbf{x}_{\sigma_1}\|_2 + \sum_{k=1}^{M-1} \|\mathbf{x}_{\sigma_k} - \mathbf{x}_{\sigma_{k+1}}\|_2 \right\}. \quad (53)$$

开放路径不设置返回原点的边。式 (53) 仅优化当前已知任务节点间的移动距离，未来任务和服务成本通过滚动更新处理。当  $M \leq 10$  时，采用 Held—Karp 动态规划精确求解。令  $F(A, j)$  表示从  $\mathbf{q}_t$  出发，恰好访问子集  $A$  并终止于节点  $j$  的最短距离，则

$$\begin{aligned} F(\{j\}, j) &= \|\mathbf{q}_t - \mathbf{x}_j\|_2, \\ F(A, j) &= \min_{i \in A \setminus \{j\}} \{F(A \setminus \{j\}, i) + \|\mathbf{x}_i - \mathbf{x}_j\|_2\}. \end{aligned} \quad (54)$$

节点较多时采用多初值最近邻与 2-opt 近似改进，以避免指数复杂度影响运行。每轮仅执行路径中的下一项任务；扫描批次、补测或清除结束后，立即更新频道状态、目标区域及代表位置并重新规划，从而使搜索与定位过程交替推进。图 17 给出了滚动任务的示意图

## 7.3 定位区域更新

对每个已发现频道，沿用问题一的楔形一致集与半平面裁剪方法，并按频道独立维护定位区域；后续引入的概率模型只用于动作排序，不参与可靠性判定。设频道  $j$  在第  $i$  个成功测站  $\mathbf{s}_{j,i}$  获得示向度  $\theta_{j,i}$ ，定义角锥两条边界射线的单位方向向量

$$\mathbf{u}_{j,i}^{\pm} = \begin{bmatrix} \cos(\theta_{j,i} \pm \delta) \\ \sin(\theta_{j,i} \pm \delta) \end{bmatrix}.$$

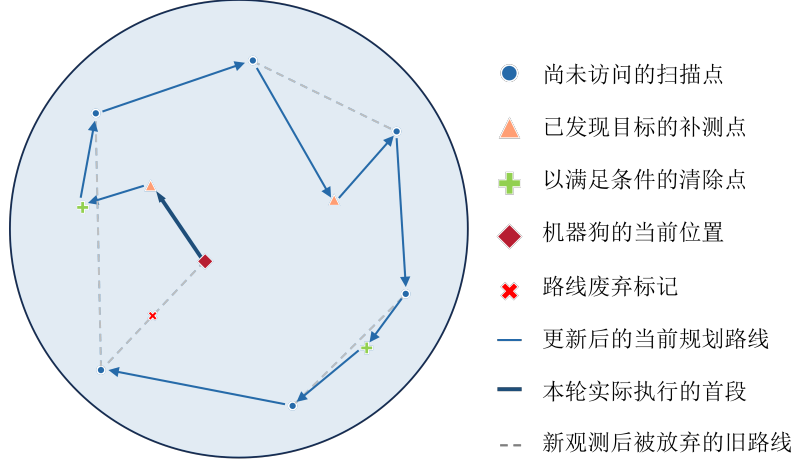


图 13 全局局部任务耦合-滚动路径重规划图

则该次观测对应的前向楔形为

$$W_{j,i} = \{z : [u_{j,i}^-, z - s_{j,i}] \geq 0, [u_{j,i}^+, z - s_{j,i}] \leq 0\}, \quad (55)$$

其中  $[\cdot, \cdot]$  表示二维叉积。每个角锥等价于两个闭半平面的交。若当前共有  $k_j$  次成功观测，则目标的精确硬约束区域为

$$C_{j,t} = \Omega \cap \bigcap_{i=1}^{k_j} W_{j,i} \cap \bigcap_{i=1}^{k_j} \mathbb{B}(s_{j,i}, 1500). \quad (56)$$

区域  $C_{j,t}$  可能含有圆弧。数值实现中，以  $m = 96$  个均匀单位法向量将每个圆盘外包为切线多边形，再与楔形半平面逐次裁剪，得到凸多边形  $P_{j,t}$ ，并始终保持

$$z_j \in C_{j,t} \subseteq P_{j,t}. \quad (57)$$

沿用问题一的最小覆盖圆模型，可将当前位置不确定性直接转化为可靠清除条件。设  $V(P_{j,t})$  为多边形顶点集合，定义其直径和最小覆盖半径为

$$\begin{aligned} D_{j,t} &= \max_{u, w \in V(P_{j,t})} \|u - w\|_2, \\ r_{j,t}^* &= \min_c \max_{u \in V(P_{j,t})} \|u - c\|_2. \end{aligned} \quad (58)$$

由问题一的 Carathéodory 支撑结构与 Jung 上界，覆盖全部顶点等价于覆盖整个凸多边形，且有

$$\frac{D_{j,t}}{2} \leq r_{j,t}^* \leq \frac{D_{j,t}}{\sqrt{3}}. \quad (59)$$

又因为真实目标必在  $P_{j,t}$  内，到达最小覆盖圆心  $c_{j,t}^*$  后有

$$\|z_j - c_{j,t}^*\|_2 \leq r_{j,t}^*.$$

因此， $r_{j,t}^* \leq 20$  米是保证清除干扰源的充要条件，在全部可靠清除位置中选择距离当前位置最近的点：

$$\begin{aligned} p_{j,t}^{\text{clr}} &= \arg \min_p \|p - q_t\|_2^2 \\ \text{s.t. } &\|p - u\|_2 \leq 20, \quad \forall u \in V(P_{j,t}). \end{aligned} \quad (60)$$

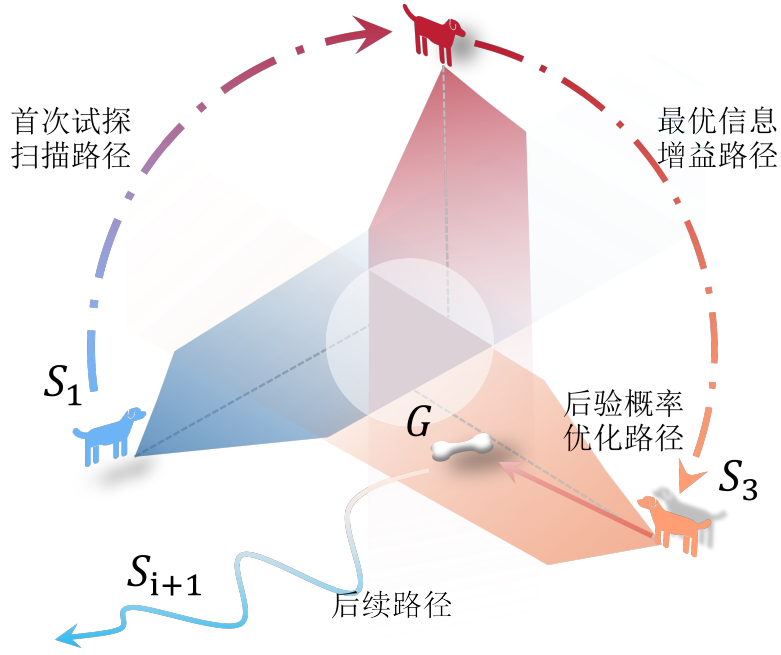


图 14 局部可行域逐步收缩图

#### 7.4 信息增益优化

当  $r_{j,t}^* > 20$  米时，需要增加示向观测。补测点首先必须保证能够接收到目标信号。对任意候选真实位置  $\mathbf{z} \in P_{j,t}$ ，既往成功接收记录说明与该位置相容的接收半径下界为

$$\underline{\rho}_j(\mathbf{z}) = \max \left\{ 1000, \max_{1 \leq i \leq k_j} \|\mathbf{z} - \mathbf{s}_{j,i}\|_2 \right\}. \quad (61)$$

因此，补测点  $\mathbf{p}$  的强接收可行域为

$$C_{j,t}^{\text{recv}} = \left\{ \mathbf{p} : \|\mathbf{p} - \mathbf{z}\|_2 \leq \underline{\rho}_j(\mathbf{z}), \quad \forall \mathbf{z} \in P_{j,t} \right\}. \quad (62)$$

由于真实位置  $\mathbf{z}_j \in P_{j,t}$ ，且实际半径满足  $\rho_j \geq \underline{\rho}_j(\mathbf{z}_j)$ ，故式 (62) 能够保证补测成功。为将式中的连续全称约束转化为有限次可计算检验，本文将  $P_{j,t}$  划分为若干三角形  $\tau$ 。对三角形顶点集  $V_\tau$  定义

$$\begin{aligned} A_0(\tau, \mathbf{p}) &= \max_{\mathbf{u} \in V_\tau} (\|\mathbf{p} - \mathbf{u}\|_2^2 - 1000^2), \\ A_i(\tau, \mathbf{p}) &= \max_{\mathbf{u} \in V_\tau} (\|\mathbf{p} - \mathbf{u}\|_2^2 - \|\mathbf{s}_{j,i} - \mathbf{u}\|_2^2). \end{aligned} \quad (63)$$

若每个三角形均满足

$$\min \{A_0(\tau, \mathbf{p}), A_1(\tau, \mathbf{p}), \dots, A_{k_j}(\tau, \mathbf{p})\} \leq 0, \quad (64)$$

则  $\mathbf{p} \in C_{j,t}^{\text{recv}}$ 。为避免同地点重复检测和近距离线性化不稳定，另加

$$\min_i \|\mathbf{p} - \mathbf{s}_{j,i}\|_2 \geq 5, \quad \|\mathbf{p} - \boldsymbol{\mu}_{j,t}\|_2 \geq 20. \quad (65)$$

为保持同一目标接收半径在任务过程中固定，取规划先验

$$\rho_j \sim U[1000, 1500],$$

并依据全部正、负接收记录计算与位置  $\mathbf{z}$  相容的半径区间。记成功和失败接收位置集合分别为  $\mathcal{S}_{j,t}^+$  与  $\mathcal{S}_{j,t}^-$ ，则

$$\begin{aligned}\underline{\rho}_{j,t}(\mathbf{z}) &= \max \left\{ 1000, \max_{\mathbf{s} \in \mathcal{S}_{j,t}^+} \|\mathbf{z} - \mathbf{s}\|_2 \right\}, \\ \bar{\rho}_{j,t}(\mathbf{z}) &= \min \left\{ 1500, \min_{\mathbf{q} \in \mathcal{S}_{j,t}^-} \|\mathbf{z} - \mathbf{q}\|_2 \right\}.\end{aligned}\tag{66}$$

相应接收似然取为

$$\ell_{\rho,j,t}(\mathbf{z}) = \frac{[\bar{\rho}_{j,t}(\mathbf{z}) - \underline{\rho}_{j,t}(\mathbf{z})]_+}{500}.$$

首次位置先验取目标圆域上的均匀分布。将测角误差近似为零均值高斯变量，并与区间  $[-\delta, \delta]$  等方差的尺度

$$\sigma_\theta = \frac{\pi/180}{\sqrt{3}}.$$

结合测角似然、几何支持集、接收似然及历史失败清除信息，构造规划后验

$$\pi_{j,t}(\mathbf{z}) = \frac{\mathbf{1}_{\{\mathbf{z} \in P_{j,t}\}} \ell_{\rho,j,t}(\mathbf{z}) \prod_{i=1}^{k_j} \exp \left[ -\frac{d_{\text{ang}}^2(\arg(\mathbf{z} - \mathbf{s}_{j,i}), \theta_{j,i})}{2\sigma_\theta^2} \right] \prod_{\mathbf{p}_h \in \mathcal{H}_{j,t}^{\text{fail}}} \mathbf{1}_{\{\|\mathbf{z} - \mathbf{p}_h\|_2 > 20\}}}{\int_{P_{j,t}} \ell_{\rho,j,t}(\mathbf{u}) \prod_{i=1}^{k_j} \exp \left[ -\frac{d_{\text{ang}}^2(\arg(\mathbf{u} - \mathbf{s}_{j,i}), \theta_{j,i})}{2\sigma_\theta^2} \right] \prod_{\mathbf{p}_h \in \mathcal{H}_{j,t}^{\text{fail}}} \mathbf{1}_{\{\|\mathbf{u} - \mathbf{p}_h\|_2 > 20\}} d\mathbf{u}}\tag{67}$$

采用三角形求积计算后验均值  $\boldsymbol{\mu}_{j,t}$  和协方差  $\boldsymbol{\Sigma}_{j,t}$ ；若求积失效，则以几何矩代替，且不据此判定频道无源。对候选点  $\mathbf{p}$ ，弧度制示向函数

$$g(\mathbf{z}, \mathbf{p}) = \text{atan2}(z_y - p_y, z_x - p_x)$$

在  $\mathbf{z} = \boldsymbol{\mu}_{j,t}$  处的梯度为

$$\mathbf{h}_\mathbf{p} = \frac{1}{\|\boldsymbol{\mu}_{j,t} - \mathbf{p}\|_2^2} \begin{bmatrix} -(\mu_{j,t,y} - p_y) \\ \mu_{j,t,x} - p_x \end{bmatrix}.\tag{68}$$

在线性高斯近似下，单次观测增加的 Fisher 信息为  $\mathbf{h}_\mathbf{p} \mathbf{h}_\mathbf{p}^\top / \sigma_\theta^2$ ，由矩阵行列式引理得到近似熵下降

$$I(\mathbf{p}) = \frac{1}{2} \log \left( 1 + \frac{\mathbf{h}_\mathbf{p}^\top \boldsymbol{\Sigma}_{j,t} \mathbf{h}_\mathbf{p}}{\sigma_\theta^2} \right).\tag{69}$$

为兼顾当前移动成本和补测后接近目标的尾程代价<sup>[5]</sup>，定义

$$\begin{aligned}\widehat{t}_\beta(\mathbf{p}) &= 10 + \frac{\|\mathbf{p} - \mathbf{q}_t\|_2}{5} + \beta \frac{\|\mathbf{p} - \boldsymbol{\mu}_{j,t}\|_2}{5}, \\ U(\mathbf{p}) &= \frac{I(\mathbf{p})}{\widehat{t}_\beta(\mathbf{p})}, \quad \beta = 0.5.\end{aligned}\tag{70}$$

模型在式 (62)、(64) 和 (65) 的约束下，采用多初值连续优化寻找  $U(\mathbf{p})$  较大的候选点。当候选点的单位时间信息效用接近时，进一步沿用问题二的区域深度加权 DOP 评价交会几何。取该目标首次成功测站  $\mathbf{s}_{j,1}$ ，令

$$r_1(\mathbf{z}) = \|\mathbf{z} - \mathbf{s}_{j,1}\|_2, \quad r_2(\mathbf{p}, \mathbf{z}) = \|\mathbf{z} - \mathbf{p}\|_2,$$

并以  $\alpha(\mathbf{p}, \mathbf{z})$  表示两条视线的夹角，则位置误差型 DOP 为

$$D(\mathbf{p}, \mathbf{z}) = \frac{\sqrt{r_1^2(\mathbf{z}) + r_2^2(\mathbf{p}, \mathbf{z})}}{|\sin \alpha(\mathbf{p}, \mathbf{z})|}.\tag{71}$$

若  $P_{j,t} = \{\mathbf{z} : \mathbf{a}_\ell^\top \mathbf{z} \leq b_\ell\}$ ，则定义区域深度和权重

$$\begin{aligned}d(\mathbf{z}) &= \min_\ell \frac{b_\ell - \mathbf{a}_\ell^\top \mathbf{z}}{\|\mathbf{a}_\ell\|_2}, \\ w(\mathbf{z}) &= \varepsilon_w + (1 - \varepsilon_w) \left( \frac{d(\mathbf{z})}{d^*} \right)^{p_w}, \\ d^* &= \max_{\mathbf{z} \in P_{j,t}} d(\mathbf{z})\end{aligned}\tag{72}$$

其中取  $\varepsilon_w = 0.2, p_w = 2$ 。为处理近共线点处的数值发散，实际计算正则化积分

$$J_\eta(\mathbf{p}) = \frac{\int_{P_{j,t}} w(\mathbf{z}) \frac{\sqrt{r_1^2(\mathbf{z}) + r_2^2(\mathbf{p}, \mathbf{z})}}{\max\{|\sin \alpha(\mathbf{p}, \mathbf{z})|, \eta\}} d\mathbf{z}}{\int_{P_{j,t}} w(\mathbf{z}) d\mathbf{z}}, \quad \eta = 0.02.\tag{73}$$

对于近共线区域权重占比过大的候选点直接予以排除。记候选池中的最大主效用为  $U^*$ ，仅在

$$U(\mathbf{p}) \geq (1 - \tau_{\text{eff}})U^*, \quad \tau_{\text{eff}} = 10^{-6}\tag{74}$$

候选中比较  $J_\eta$ ；只有 DOP 至少改善 2% 时才替换原主效用最优点，从而保持单位时间信息效率优先。

由图可见，单纯最小化加权 DOP 会增加移动与测量时间，单纯最大化单位时间信息又可能牺牲交会几何；所用近优决胜策略在主效用近优集内进一步比较加权 DOP，从而兼顾时间效率与定位稳定性。

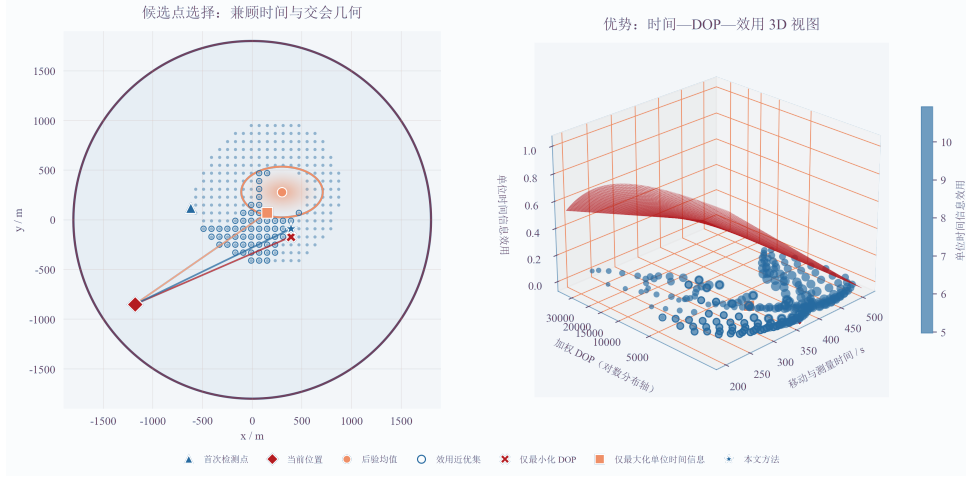


图 15 加权 DOP 近优决胜策略的优势对比图

### 7.5 试探清除与失败反馈

当  $r_{j,t}^* > 20$  米时，最小覆盖圆尚不能给出可靠清除保证，但规划后验可能已集中在某个半径为 20 米的圆内。对试探位置  $\mathbf{p}$ ，定义成功概率

$$p_s(\mathbf{p}) = \int_{P_{j,t} \cap \mathbb{B}(\mathbf{p}, 20)} \pi_{j,t}(\mathbf{z}) d\mathbf{z}. \quad (75)$$

设  $V_s$  和  $V_f$  分别为成功与失败后的预计剩余成本，则单次试探清除的期望成本为

$$Q_{\text{try}}(\mathbf{p}) = \frac{\|\mathbf{p} - \mathbf{q}_t\|_2}{5} + 3 + p_s(\mathbf{p})(2 + V_s) + [1 - p_s(\mathbf{p})]V_f. \quad (76)$$

设当前规划补测点为  $\mathbf{p}_m$ ，定位区域最小覆盖圆心为  $\mathbf{c}^*$ ，则参考成本取为

$$Q_m = \frac{\|\mathbf{p}_m - \mathbf{q}_t\|_2}{5} + 5 + \mathbf{1}_{\{f_t \neq j\}} + \frac{\|\mathbf{p}_m - \mathbf{c}^*\|_2}{5} + 5 + T_{\text{tail}}, \quad (77)$$

其中若存在下一任务节点  $\mathbf{q}_{\text{next}}$ ，则  $T_{\text{tail}} = \frac{\|\mathbf{c}^* - \mathbf{q}_{\text{next}}\|_2}{5}$ ，否则取零。

为避免连续试探的短视性，在即将执行的补测路段附近生成候选序列  $(\mathbf{p}_1, \dots, \mathbf{p}_H)$ ，其中  $H \leq 3$ 。定义第  $k$  次首次成功的互斥概率

$$a_k = \Pr(\mathbf{z}_j \in \mathbb{B}(\mathbf{p}_k, 20), \mathbf{z}_j \notin \mathbb{B}(\mathbf{p}_\ell, 20), \forall \ell < k \mid \mathcal{H}_t), \quad (78)$$

$$b_k = 1 - \sum_{\ell < k} a_\ell$$

其中  $b_k$  为能够执行到第  $k$  次试探的概率，且令  $\mathbf{p}_0 = \mathbf{q}_t$ 。序列的预计成本为

$$Q_{\text{seq}} = \sum_{k=1}^H b_k \left( \frac{\|\mathbf{p}_k - \mathbf{p}_{k-1}\|_2}{5} + 3 \right) + \sum_{k=1}^H a_k(2 + V_{s,k}) + b_{H+1}V_f. \quad (79)$$

互斥事件的定义避免重叠试探圆的成功概率被重复累计。对全部试探均失败的完整插入路径，限制其相对直接前往补测点的绕行量：

$$\sum_{k=1}^H \|\mathbf{p}_k - \mathbf{p}_{k-1}\|_2 + \|\mathbf{p}_H - \mathbf{p}_m\|_2 - \|\mathbf{p}_m - \mathbf{q}_t\|_2 \leq 5. \quad (80)$$

程序选择比式 (77) 更省时的试探序列，但仍只执行首个动作，并在获得反馈后重新计算。每个目标最多试探 5 次，首次成功概率不低于 0.2，试探时要求  $r_{j,t}^* \leq 80$  米，候选点不超过 12 个且预计节时非负。

若在  $\mathbf{p}$  处试探失败，则得到确定信息  $\|\mathbf{z}_j - \mathbf{p}\|_2 > 20$ ，后验更新为

$$\pi_{j,t+1}(\mathbf{z}) = \frac{\pi_{j,t}(\mathbf{z}) \mathbf{1}_{\{\|\mathbf{z}-\mathbf{p}\|_2 > 20\}}}{\int_{P_{j,t}} \pi_{j,t}(\mathbf{u}) \mathbf{1}_{\{\|\mathbf{u}-\mathbf{p}\|_2 > 20\}} d\mathbf{u}}. \quad (81)$$

排除圆盘后的位置集合可能非凸。为保证可靠性，本文继续保存凸外包  $P_{j,t}$ ，并单独记录失败圆，用于后验计算与后续试探筛选。失败后立即撤销旧计划、更新任务状态并重新规划；若式 (81) 的分母接近零，则按概率近似失效处理。

## 7.6 算法评价指标

模型执行时先初始化机器狗位置和频道状态，再动态维护任务集，一直持续执行下列步骤，直至满足条件为止：

1. 若下一任务为扫描点，则批量检测该点尚未检测的未知频道，并依据有无信号更新频道状态；
2. 若下一任务为目标，则先检查近距离提示和最小覆盖圆，满足可靠清除条件时前往最近可靠清除点，否则在强接收约束下优化单位时间信息增益，并用区域加权 DOP 比较主效用近优候选。
3. 比较补测方案与有限前瞻试探方案，只执行当前最优动作，获得反馈后更新集合成员区域、规划后验和任务路线。

记  $\mathcal{D}_t$  和  $\mathcal{A}_t$  分别为已清除频道集合和已认证不存在频道集合。由于干扰源数量上界为 16，判断全部任务完成的条件为

$$|\mathcal{D}_t| = 16 \quad \text{或} \quad |\mathcal{D}_t \cup \mathcal{A}_t| = 20. \quad (82)$$

采用题目规定的清除比例和平均定位清除时间评价策略：

$$\eta_{\text{clr}} = \frac{N_c}{N}, \quad \bar{T} = \frac{T}{N_c}.$$

## 7.7 蒙特卡洛仿真实验

为检验滚动规划策略在随机场景下的任务完成能力与计算效率，本文对 300 组随机案例分别执行完整的定位与清除策略，统计结果见表 3。

表 3 问题三蒙特卡洛仿真实验结果

| 随机案例数 | 全部清除成功率 | 干扰源总数  | 单源平均定位清除时间 | 每局平均定位清除时间  | 每局平均程序运行时间 | 每局平均检测次数   |
|-------|---------|--------|------------|-------------|------------|------------|
| 300 组 | 100.00% | 3889 个 | 261.31 秒/个 | 3328.41 秒/局 | 1.56 秒/局   | 121.42 次/局 |

300 组案例均完成全部干扰源清除，共处理 3889 个干扰源，说明所构建策略在随机场景下具有稳定的任务完成能力。单源平均定位清除时间为 261.31 秒，每局平均检测 121.42 次。

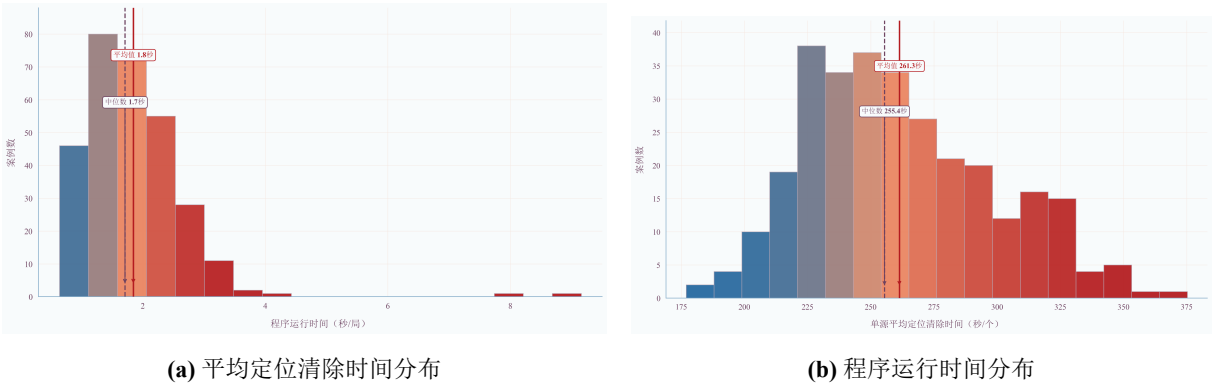


图 16 蒙特卡洛仿真结果时间分布图

两类分布表明，单源定位清除时间主要集中在均值附近并呈现一定右侧长尾；程序运行时间整体处于秒级，少量高耗时案例未改变其在线计算可行性。

7.8 正式测试结果

为检验所提策略在正式测试环境中的实际表现，本文按照统一口径记录三组测试的清除数量、单源平均定位清除时间与程序运行时间，结果见表 4。

表 4 问题三正式测试结果

| 测试案例编码 | 清除干扰源个数 | 平均定位清除时间   | 程序运行时间 |
|--------|---------|------------|--------|
| 测试 3   | 12 个    | 279.93 秒/个 | 2.73 秒 |
| 测试 1   | 14 个    | 262.52 秒/个 | 3.00 秒 |
| 测试 2   | 10 个    | 264.72 秒/个 | 1.74 秒 |

三组测试分别清除 14、10 和 12 个干扰源，单源平均定位清除时间为 262.52 秒、264.72 秒和 533.54 秒；对应的程序运行时间均不超过 4.50 秒。测试 3 的实际任务耗时相对较长，但计算耗时仍远低于定位清除过程用时，说明滚动规划能够满足正式测试中的在线计算要求。

## 八、问题四的模型建立与求解

问题四在全向干扰源中进一步加入发射朝向未知的定向源。对于定向源，机器狗进入接收半径并不必然获得信号，因而第三问的覆盖策略和排除规则不再成立。设频道  $j$  的源类型为  $\tau_j \in \{O, D\}$ ，分别表示全向源和定向源；定向源的固定发射朝向为  $\phi_j$ ，令

$$\mathbf{u}(\phi) = (\cos \phi, \sin \phi)^\top.$$

机器狗在位置  $\mathbf{q}$  可接收到该源，当且仅当

$$\|\mathbf{q} - \mathbf{z}_j\|_2 \leq \rho_j, \quad \tau_j = O \text{ 或 } \mathbf{u}(\phi_j)^\top(\mathbf{q} - \mathbf{z}_j) \geq 0. \quad (83)$$

### 8.1 全方向扫描点布局

为避免扫描点位于定向源的发射背面，本题要求扫描点集合  $\mathcal{S}$  满足

$$\forall \mathbf{z} \in \Omega, \forall \phi \in [0, 2\pi), \exists \mathbf{s} \in \mathcal{S}: \quad \|\mathbf{s} - \mathbf{z}\|_2 \leq 1000, \quad \mathbf{u}(\phi)^\top(\mathbf{s} - \mathbf{z}) \geq 0. \quad (84)$$

该条件对任意实际接收半径均成立，也同时覆盖全向源。定义位置  $\mathbf{z}$  附近的扫描点集合

$$\mathcal{S}_z(\rho) = \mathcal{S} \cap \mathbb{B}(\mathbf{z}, \rho).$$

据此，式 (84) 可等价地写为下列局部凸包条件：

$$\mathbf{z} \in \text{conv } \mathcal{S}_z(1000), \quad \forall \mathbf{z} \in \Omega. \quad (85)$$

为说明其等价性，若式 (85) 成立，则存在非负权重  $\lambda_i$ ，使

$$\mathbf{z} = \sum_i \lambda_i \mathbf{s}_i, \quad \sum_i \lambda_i = 1, \quad \mathbf{s}_i \in \mathcal{S}_z(1000).$$

对任意  $\phi$ ，有

$$\sum_i \lambda_i \mathbf{u}(\phi)^\top(\mathbf{s}_i - \mathbf{z}) = 0.$$

故这些附近扫描点不可能全部位于发射半平面的严格背面，至少存在一个点满足式 (84)。反之，若  $\mathbf{z} \notin \text{conv } \mathcal{S}_z(1000)$ ，由严格分离定理可找到方向  $\mathbf{u}(\phi)$ ，使全部附近扫描点均位于其严格背面，从而产生漏检。因此，两种条件等价。

局部凸包条件还说明圆外扫描点不可缺少。取边界目标  $\mathbf{z} = R_0 \boldsymbol{\nu}$ ，其中  $\|\boldsymbol{\nu}\|_2 = 1$ ，并令其沿  $\boldsymbol{\nu}$  向外发射。对任意圆域内扫描点  $\mathbf{s}$ ，均有

$$\boldsymbol{\nu}^\top(\mathbf{s} - \mathbf{z}) = \boldsymbol{\nu}^\top \mathbf{s} - R_0 \leq \|\mathbf{s}\|_2 - R_0 \leq 0.$$

进一步考察圆外扫描点对朝外边界状态的覆盖能力。设其极径为  $b > R_0$ ，与边界目标的极角差为  $\theta$ 。同时满足方向和距离条件要求

$$\cos \theta \geq \max \left\{ \frac{R_0}{b}, \frac{b^2 + R_0^2 - 1000^2}{2bR_0} \right\}. \quad (86)$$

当  $b = \sqrt{R_0^2 + 1000^2}$  时覆盖角区间最宽，其半宽为

$$\beta_{\max} = \arctan \frac{1000}{1800} \approx 29.0546^\circ.$$

因此，一个圆外点最多覆盖约  $58.1092^\circ$  的朝外边界状态，圆外点数至少为

$$N_{\text{out}} \geq \left\lceil \frac{360^\circ}{2\beta_{\max}} \right\rceil = 7.$$

除扫描点及扫描点连线构成的集合外，局部凸包至少需要三个附近扫描点。对覆盖重数积分，并利用圆外扫描点的接收圆与  $\Omega$  的交叠面积不超过半个接收圆面积，得到

$$\left(N - \frac{7}{2}\right) \pi (1000)^2 \geq 3\pi (1800)^2,$$

从而有

$$N \geq 14. \quad (87)$$

为降低连续选点问题的维数，采用中心加双环的参数化布局：

$$\left\{ \begin{array}{l} \mathcal{S}(n_1, n_2, r_1, r_2, \psi) = \{\mathbf{0}\} \cup S_1 \cup S_2, \quad N = 1 + n_1 + n_2, \\ S_1 = \left\{ r_1 \left( \cos \frac{2\pi k}{n_1}, \sin \frac{2\pi k}{n_1} \right) : k = 0, \dots, n_1 - 1 \right\}, \\ S_2 = \left\{ r_2 \left( \cos \left( \frac{2\pi l}{n_2} + \psi \right), \sin \left( \frac{2\pi l}{n_2} + \psi \right) \right) : l = 0, \dots, n_2 - 1 \right\}. \end{array} \right. \quad (88)$$

定义布局所需的最坏接收半径

$$\rho_*(\mathcal{S}) = \sup_{z \in \Omega} \inf \{ \rho \geq 0 : z \in \text{conv}(\mathcal{S} \cap \mathbb{B}(z, \rho)) \}. \quad (89)$$

当  $\rho_*(\mathcal{S}) \leq 1000$  时布局可行。本文依次进行整数点数与内外圈分配枚举、差分进化连续参数搜索、漏区检查和局部参数细化。对于 12 点外环，其正十二边形内切圆必须覆盖目标圆域，故

$$r_2 \cos 15^\circ \geq 1800, \quad r_2 \geq 1863.497125 \text{ m}.$$

在该必要下界附近细化后，得到

$$n_1 = 8, \quad n_2 = 12, \quad r_1 = 995 \text{ m}, \quad r_2 = 1865 \text{ m}, \quad \psi = 0. \quad (90)$$

即最终扫描点集合为

$$\left\{ \begin{array}{l} \mathcal{S}_4 = \{\mathbf{0}\} \cup S_1 \cup S_2, \\ S_1 = \left\{ 995 \left( \cos \frac{k\pi}{4}, \sin \frac{k\pi}{4} \right) : k = 0, \dots, 7 \right\}, \\ S_2 = \left\{ 1865 \left( \cos \frac{l\pi}{6}, \sin \frac{l\pi}{6} \right) : l = 0, \dots, 11 \right\}. \end{array} \right. \quad (91)$$

有限采样不能排除扫描点之间存在细小漏区，因此还需验证整个连续区域。取保守认证半径  $\rho_v = 999 \text{ m}$ ，将包含  $\Omega$  的正方形递归划分为方盒  $Q$ 。记其四角集合为  $V(Q)$ ，并定义能够完整覆盖该方盒的扫描点集合

$$\mathcal{S}_Q = \left\{ \mathbf{s} \in \mathcal{S}_4 : \max_{\mathbf{v} \in V(Q)} \|\mathbf{s} - \mathbf{v}\|_2 < \rho_v \right\}.$$

最终布局共处理 9797 个递归方盒节点，所有与目标圆域相交的部分均在 999 米半径下通过认证。因此，21 点布局满足题设 1000 米的全方向覆盖要求。

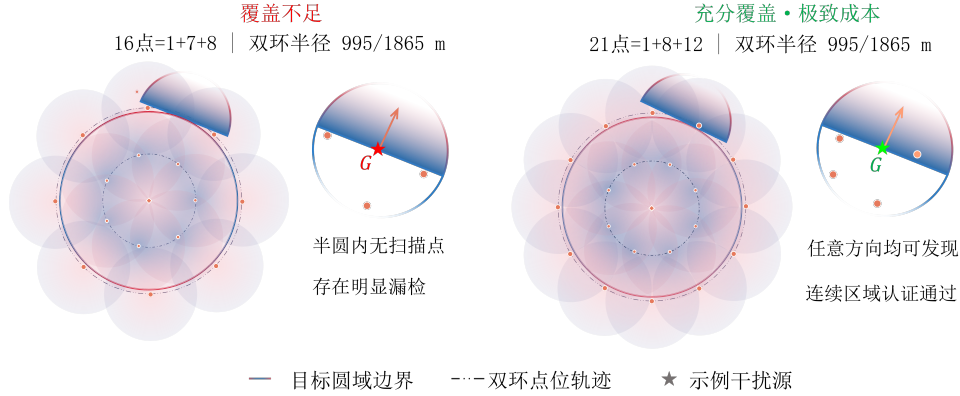


图 17 中心双圆环：全向与定向源的全覆盖设计图

该图直观展示了内外双环扫描点对中心、边界及不同发射方向的互补覆盖关系。

## 8.2 联合后验与定向补测决策

对已发现频道  $j$ ，将未知状态扩展为

$$\Theta_j = (\mathbf{z}_j, \tau_j, \phi_j, \rho_j).$$

规划先验取位置在目标圆域内均匀分布、 $\rho_j \sim U[1000, 1500]$ 、全向与定向类型概率各为 0.5，定向朝向在  $[0, 2\pi)$  上均匀分布。将类型与朝向合记为  $\eta$ ，定义方向可见函数

$$e_\eta(\mathbf{q}, \mathbf{z}) = \begin{cases} 1, & \eta = \text{O}, \\ \mathbf{1}_{\{\mathbf{u}(\phi)^\top(\mathbf{q}-\mathbf{z}) \geq 0\}}, & \eta = (\text{D}, \phi). \end{cases}$$

记阳性测站集合和阴性测站集合分别为  $\mathcal{Q}_{j,t}^+$  与  $\mathcal{Q}_{j,t}^-$ 。所有阳性测站必须位于候选状态的发射半平面内，并共同给出固定接收半径下界；只有候选状态会向某阴性测站发射时，该阴性才意味着超距。因而

$$\begin{aligned} \underline{\rho}_{j,t}(\mathbf{z}) &= \max \left\{ 1000, \max_{\mathbf{q} \in \mathcal{Q}_{j,t}^+} \|\mathbf{q} - \mathbf{z}\|_2 \right\}, \\ \bar{\rho}_{j,t}(\mathbf{z}, \eta) &= \min \left\{ 1500, \min_{\substack{\mathbf{q} \in \mathcal{Q}_{j,t}^- \\ e_\eta(\mathbf{q}, \mathbf{z})=1}} \|\mathbf{q} - \mathbf{z}\|_2 \right\}. \end{aligned} \tag{92}$$

若第二式的内层集合为空，则不增加上界约束。将同一个固定接收半径解析积分后，接收历史的相容权重为

$$L_{\text{RF}}(\mathbf{z}, \eta) = \mathbf{1}_{\{e_\eta(\mathbf{q}, \mathbf{z})=1, \forall \mathbf{q} \in \mathcal{Q}_{j,t}^+\}} \frac{[\bar{\rho}_{j,t}(\mathbf{z}, \eta) - \underline{\rho}_{j,t}(\mathbf{z})]_+}{500}. \quad (93)$$

测角误差尺度和角度似然沿用式 (67)。记其中的测角似然乘积为  $L_{\theta,j,t}(\mathbf{z})$ ，失败光学尝试位置集合为  $\mathcal{H}_{j,t}^{\text{fail}}$ ，则联合规划后验为

$$p_{j,t}(\mathbf{z}, \eta \mid \mathcal{H}_{j,t}) \propto p_0(\mathbf{z}, \eta) \mathbf{1}_{\{\mathbf{z} \in P_{j,t}\}} L_{\theta,j,t}(\mathbf{z}) L_{\text{RF}}(\mathbf{z}, \eta) \prod_{\mathbf{f} \in \mathcal{H}_{j,t}^{\text{fail}}} \mathbf{1}_{\{\|\mathbf{z}-\mathbf{f}\|_2 > 20\}}. \quad (94)$$

实际接收圆盘和发射半平面均为凸集，且包含全部阳性测站，因此

$$\mathbf{p} \in \text{conv } \mathcal{Q}_{j,t}^+ \implies \mathbf{p} \text{ 必能再次接收到频道 } j \text{ 的信号.}$$

对联合后验节点  $(\mathbf{z}, \eta)$ ，候选点  $\mathbf{p}$  成功接收的条件概率为

$$\pi_{\text{rec}}(\mathbf{p} \mid \mathbf{z}, \eta) = e_\eta(\mathbf{p}, \mathbf{z}) \frac{\left[ \bar{\rho}_{j,t}(\mathbf{z}, \eta) - \max\{\underline{\rho}_{j,t}(\mathbf{z}), \|\mathbf{p} - \mathbf{z}\|_2\} \right]_+}{\bar{\rho}_{j,t}(\mathbf{z}, \eta) - \underline{\rho}_{j,t}(\mathbf{z})}. \quad (95)$$

对式 (94) 积分得到预测接收概率  $P_{\text{rec}}(\mathbf{p})$ ；通过式 (8.2) 认证的候选直接取 1。考虑定向源可能无法接收后，主效用更新为

$$J(\mathbf{p}) = \frac{P_{\text{rec}}(\mathbf{p}) I(\mathbf{p})}{10 + \|\mathbf{p} - \mathbf{q}_t\|_2/5 + 0.5\|\mathbf{p} - \boldsymbol{\mu}_{j,t}\|_2/5}. \quad (96)$$

模型在位置均值和最小覆盖圆心周围生成多尺度方向点，并加入阳性测站之间及其通向位置均值的凸组合点，形成有限候选集。首先最大化式 (96)，再在主效用近优候选中沿用式 (73) 的区域深度加权 DOP；只有 DOP 改善达到 2% 时才替换主效用最优点，从而保持时间效率优先。

### 8.3 蒙特卡洛仿真实验

在配对测试的基础上，本文进一步通过 300 组随机案例考察含定向源场景下总体策略的稳定性与运行效率，统计结果见表 5。

表 5 问题四蒙特卡洛仿真实验结果

| 随机案例数 | 全部清除成功率 | 干扰源总数  | 单源平均定位清除时间 | 每局平均定位清除时间  | 每局平均程序运行时间 | 每局平均检测次数   |
|-------|---------|--------|------------|-------------|------------|------------|
| 300 组 | 100.00% | 3889 个 | 489.51 秒/个 | 6196.89 秒/局 | 2.24 秒/局   | 244.49 次/局 |

300 组实验共处理 3889 个干扰源，同样实现全部清除。与问题三相比，单源平均定位清除时间和每局平均检测次数均明显增加，说明未知发射朝向提高了有效观测与定位

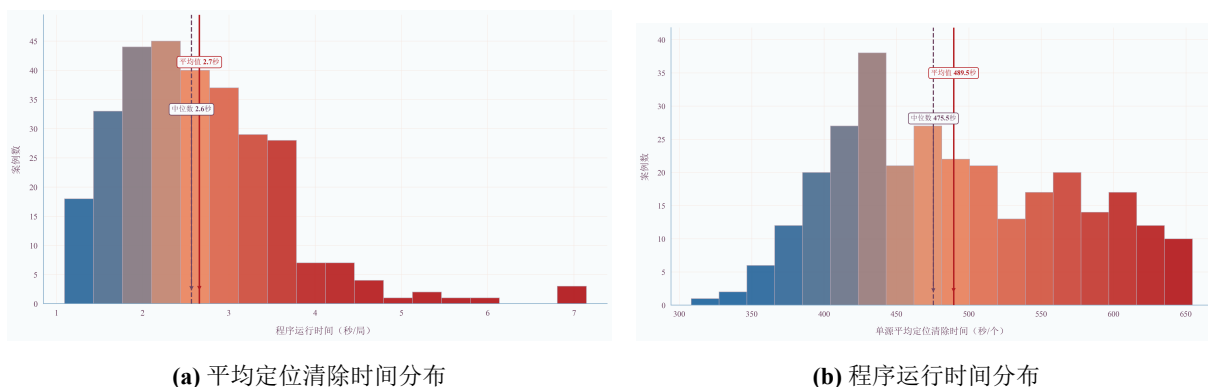


图 18 蒙特卡洛仿真结果时间分布图

清除的难度；但每局程序运行时间仍仅为 2.24 秒，相对于实际任务耗时较小，算法具有可接受的计算效率。

与问题三相比，定位清除时间分布整体向右移动且离散程度增大，反映了定向源方向未知带来的额外搜索成本；程序运行时间仍主要集中于秒级，说明联合状态更新未造成不可接受的计算负担。

## 8.4 正式测试结果

进一步在含定向干扰源的正式测试场景中运行问题四的联合更新策略，测试结果见表 6。

表 6 问题四正式测试结果

| 测试案例编码 | 清除干扰源个数 | 平均定位清除时间   | 程序运行时间 |
|--------|---------|------------|--------|
| 测试 1   | 12 个    | 533.54 秒/个 | 4.50 秒 |
| 测试 2   | 12 个    | 486.33 秒/个 | 3.91 秒 |
| 测试 3   | 10 个    | 613.65 秒/个 | 4.41 秒 |

三组测试分别清除 12、12 和 10 个干扰源，单源平均定位清除时间介于 486.33 秒至 613.65 秒，程序运行时间介于 3.91 秒至 4.50 秒。加入干扰源类型与发射朝向的联合更新后，算法仍能在数秒内完成规划计算，其计算开销相对于实际定位清除时间较小。

# 九、模型的评价

## 9.1 模型的优点

- 误差处理稳健：**针对题目仅给出示向度误差范围，采用集合成员估计将角度误差转化为半平面约束，避免人为假设误差分布带来的模型偏差；
- 兼顾可靠性和时间效率：**先依据最小有效接收半径建立全域覆盖扫描方案，保证全向干扰源能够被发现；在目标已被发现后，再结合最小覆盖圆和区域加权 DOP 等

动态决定补测或清除位置，从而减少不必要的移动和重复检测；

3. **算法具有较好的在线可执行性：**本文采用滚动决策，只执行当前最优动作，并在获得反馈后重新规划，避免一次性求解完整随机动态规划带来的巨大计算量。

## 9.2 模型的缺点

1. **几何近似具有一定保守性：**将带圆弧的定位区域外包为凸多边形，并在试探清除失败后继续维护凸外包。该方法能够保证真实目标不被错误排除，但会使部分补测和清除条件偏保守。
2. **扫描点布局受参数化形式限制：**本文在给定结构内优化点数、半径和相位。该方法显著降低了选点问题的维数，但所得方案只是在所设参数化布局中的较优解，不能保证是任意平面布点条件下的全局最优方案。

## 参考文献

- [1] GARULLI A, VICINO A. Set membership localization of mobile robots via angle measurements[J/OL]. IEEE Transactions on Robotics and Automation, 2001, 17(4):450-463. DOI: 10.1109/70.954757.
- [2] CALAFIORE G C. Geometry-aware set-membership multilateration: Directional bounds and anchor selection[EB/OL]. 2026. DOI: 10.48550/arXiv.2603.14263.
- [3] 罗双喜. 多站交叉定位相对 GDOP 及其测向站分布问题研究[J/OL]. 指挥控制与仿真, 2020, 42(2):7-11. DOI: 10.3969/j.issn.1673-3819.2020.02.002.
- [4] 姚胤博, 许睿, 徐悦, 等. 卫星导航干扰源无人机定位观测路径规划方法[J/OL]. 全球定位系统, 2024, 49(6):72-83. DOI: 10.12265/j.gnss.2024097.
- [5] VANDER HOOK J, TOKEKAR P, ISLER V. Bearing-only active target localization strategies for a system of two communicating mobile robots: Full technical report[R]. [S.l.]: Department of Computer Science, University of Minnesota, 2013.
- [6] OpenAI. GPT-5.6 Sol[EB/OL]. 2026. <https://developers.openai.com/api/docs/models/gpt-5.6-sol>.
- [7] DeepSeek. DeepSeek V4.1 Flash[EB/OL]. 2026. <https://www.deepseek.com/en/news/deepseek-v4-1-flash/>.

## 附录 A 文件列表

| 文件名            | 功能描述        |
|----------------|-------------|
| solve_q1.py    | 问题一求解主程序    |
| solve_q2.py    | 问题二求解主程序    |
| Q3final.py     | 问题三求解主程序    |
| simulate_t3.py | 问题三蒙特卡洛仿真代码 |
| Q4final.py     | 问题四求解主程序    |

## 附录 B 代码

solve\_q1.py

```
1 """Solve Question 1 of CUMCM 2026 B.
2
3 The input consists of detector coordinates and measured
4 bearings. Each bearing
5 is treated as a bounded error interval (default +/-1 degree),
6 which gives two
7 linear half-plane constraints. The script classifies their
8 intersection,
9 constructs the boundary polygon when it is bounded, and
10 computes:
11
12 * the region diameter (rotating calipers for a 2-D polygon),
13 * whether the diameter circle can cover the region, and
14 * the exact minimum enclosing circle of the boundary vertices.
15
16 No artificial bounding box is introduced. SciPy is used only
17 for feasibility
18 and coordinate-boundedness checks; all geometric output is
19 computed explicitly.
20
21 Accepted JSON input formats::
22
23     {"measurements": [{"x": 10, "y": 20, "svd_deg": 35.2},
24     ...],
25     "angle_error_deg": 1.0}
26     {"points": [[10, 20, 35.2], ...], "angle_error_deg": 1.0}
27
28 CSV input must contain x, y and one of svd_deg/theta_deg/theta
29 /angle_deg.
30 """
31
32 from __future__ import annotations
33
34 import argparse
35 import csv
36 import json
37 import math
38 import random
39 import sys
40 from dataclasses import dataclass
41 from pathlib import Path
42 from typing import Any, Iterable, Sequence
```

```

35
36 import numpy as np
37 from scipy.optimize import linprog
38
39
40 DEFAULT_ANGLE_ERROR_DEG = 1.0
41 _ANGLE_KEYS = ("svd_deg", "theta_deg", "theta", "angle_deg", "
    bearing_deg")
42
43
44 def _cross2(left: np.ndarray, right: np.ndarray) -> float:
45     """Return the scalar two-dimensional cross product."""
46
47     return float(left[0] * right[1] - left[1] * right[0])
48
49
50 @dataclass(frozen=True)
51 class HalfPlane:
52     """A half-plane represented by a directed boundary line
    and its normal."""
53
54     a: np.ndarray
55     b: float
56     p: np.ndarray
57     d: np.ndarray
58     angle: float
59     source_index: int
60
61
62 def _as_finite_float(value: Any, name: str) -> float:
63     try:
64         number = float(value)
65     except (TypeError, ValueError) as exc:
66         raise ValueError(f"{name} must be numeric") from exc
67     if not math.isfinite(number):
68         raise ValueError(f"{name} must be finite")
69     return number
70
71
72 def _extract_row(row: Any, index: int) -> tuple[float, float,
    float]:
73     if isinstance(row, dict):
74         try:
75             x = row["x"]
76             y = row["y"]
77         except KeyError as exc:
78             raise ValueError(f"measurement {index} must
    contain x and y") from exc
79     theta_key = next((key for key in _ANGLE_KEYS if key in
    row), None)
80     if theta_key is None:
81         raise ValueError(
82             f"measurement {index} must contain one of {'',
    '.join(_ANGLE_KEYS)}")
83     )
84     theta = row[theta_key]

```

```

85     elif isinstance(row, (list, tuple)) and len(row) >= 3:
86         x, y, theta = row[:3]
87     else:
88         raise ValueError(
89             f"measurement {index} must be an object or [x, y,
svd_deg] sequence"
90         )
91     return (
92         _as_finite_float(x, f"measurement {index}.x"),
93         _as_finite_float(y, f"measurement {index}.y"),
94         _as_finite_float(theta, f"measurement {index}.svd_deg"
95     ),
96 )
97
98 def parse_measurements(source: Any) -> tuple[list[tuple[float,
float, float]], float]:
99     """Parse a JSON-like object into measurements and angular
error in degrees."""
100
101     if isinstance(source, list):
102         rows = source
103         error_deg = DEFAULT_ANGLE_ERROR_DEG
104     elif isinstance(source, dict):
105         rows = source.get("measurements", source.get("points")
106     )
107     if rows is None:
108         raise ValueError("input must contain a '
measurements' or 'points' array")
109     error_deg = source.get("angle_error_deg",
DEFAULT_ANGLE_ERROR_DEG)
110     else:
111         raise ValueError("input must be a JSON object or an
array")
112
113     if not isinstance(rows, list) or not rows:
114         raise ValueError("measurements must be a non-empty
array")
115     error_deg = _as_finite_float(error_deg, "angle_error_deg")
116     if not (0.0 < error_deg < 90.0):
117         raise ValueError("angle_error_deg must be in (0, 90)")
118     return [_extract_row(row, i) for i, row in enumerate(rows)
119 ], error_deg
120
121 def load_input(path: str | Path) -> tuple[list[tuple[float,
float, float]], float]:
122     """Load JSON or CSV input from *path* (or '-' for stdin)."""
123
124     if str(path) == "-":
125         text = sys.stdin.read()
126         return parse_measurements(json.loads(text))
127     input_path = Path(path)
128     if input_path.suffix.lower() == ".csv":
129         with input_path.open("r", encoding="utf-8-sig",

```

```

129     newline="") as handle:
130         rows = list(csv.DictReader(handle))
131         return parse_measurements(rows)
132     with input_path.open("r", encoding="utf-8") as handle:
133         return parse_measurements(json.load(handle))
134
135 def build_constraints(
136     measurements: Sequence[tuple[float, float, float]],
137     angle_error_deg: float = 1.0
138 ) -> tuple[np.ndarray, np.ndarray]:
139     """Build 'A, b' for the constraints 'A @ z <= b'.

```

```

174     return _cross2(h.d, point - h.p) < -tol
175
176
177 def _make_halfplanes(a: np.ndarray, b: np.ndarray) -> list[
HalfPlane]:
178     halfplanes: list[HalfPlane] = []
179     for index, (normal, bound) in enumerate(zip(a, b)):
180         # The input normals have unit length, but
normalisation keeps this helper
181         # stable if build_constraints is reused with
externally supplied rows.
182         norm = float(np.linalg.norm(normal))
183         if norm == 0.0:
184             raise ValueError("a constraint normal cannot be
zero")
185         unit_a = np.asarray(normal, dtype=float) / norm
186         unit_b = float(bound) / norm
187         direction = np.array([-unit_a[1], unit_a[0]], dtype=
float)
188         angle = math.atan2(direction[1], direction[0]) % (2.0
* math.pi)
189         halfplanes.append(
190             HalfPlane(
191                 a=unit_a,
192                 b=unit_b,
193                 p=unit_b * unit_a,
194                 d=direction,
195                 angle=angle,
196                 source_index=index,
197             )
198         )
199     return halfplanes
200
201
202 def _deduplicate_directions(halfplanes: list[HalfPlane], tol:
float) -> list[HalfPlane]:
203     """Sort by direction and retain the strictest constraint
per direction."""
204
205     ordered = sorted(halfplanes, key=lambda h: h.angle)
206     result: list[HalfPlane] = []
207     for current in ordered:
208         if result:
209             previous = result[-1]
210             if abs(_cross2(previous.d, current.d)) <= 1e-13
and float(
211                 previous.d @ current.d
212             ) > 0:
213                 if current.b < previous.b - tol:
214                     result[-1] = current
215                 continue
216             result.append(current)
217     # The first and last bins meet at angle 0/2pi and need the
same treatment.
218     if len(result) > 1:
219         first, last = result[0], result[-1]

```

```

220         if abs(_cross2(first.d, last.d)) <= 1e-13 and float(
first.d @ last.d) > 0:
221             result[0] = first if first.b <= last.b else last
222             result.pop()
223         return result
224
225
226 def halfplane_intersection(
227     a: np.ndarray, b: np.ndarray, tol: float = 1e-8
228 ) -> list[np.ndarray]:
229     """Construct a full-dimensional bounded intersection using
a deque HPI.
230
231     A pairwise feasible-intersection fallback is retained for
lower-dimensional
232     bounded sets, where a standard deque HPI has fewer than
three boundary lines.
233     """
234
235     halfplanes = _deduplicate_directions(_make_halfplanes(a, b
), tol)
236     deque: list[HalfPlane] = []
237     for halfplane in halfplanes:
238         while len(deque) >= 2:
239             point = _intersection(deque[-2], deque[-1])
240             if point is None or not _outside(halfplane, point,
tol):
241                 break
242             deque.pop()
243         while len(deque) >= 2:
244             point = _intersection(deque[0], deque[1])
245             if point is None or not _outside(halfplane, point,
tol):
246                 break
247             deque.pop(0)
248             deque.append(halfplane)
249
250     while len(deque) >= 3:
251         point = _intersection(deque[-2], deque[-1])
252         if point is None or not _outside(deque[0], point, tol)
:
253             break
254         deque.pop()
255     while len(deque) >= 3:
256         point = _intersection(deque[0], deque[1])
257         if point is None or not _outside(deque[-1], point, tol
):
258             break
259         deque.pop(0)
260
261     if len(deque) >= 3 and all(
262         _intersection(deque[i], deque[(i + 1) % len(deque)])
is not None
263         for i in range(len(deque))
264     ):
265         vertices = [

```

```

266         _intersection(deque[i], deque[(i + 1) % len(deque)
267     ]
268     ]
269     return [point for point in vertices if point is not
None]
270     return []
271
272
273 def _pairwise_feasible_vertices(a: np.ndarray, b: np.ndarray,
tol: float) -> list[np.ndarray]:
274     candidates: list[np.ndarray] = []
275     for i in range(len(a)):
276         for j in range(i + 1, len(a)):
277             matrix = np.asarray((a[i], a[j]), dtype=float)
278             determinant = float(np.linalg.det(matrix))
279             if abs(determinant) <= 1e-13:
280                 continue
281             point = np.linalg.solve(matrix, np.asarray((b[i],
b[j]), dtype=float))
282             if np.all(a @ point <= b + tol):
283                 candidates.append(point)
284     return _deduplicate_points(candidates, tol)
285
286
287 def _deduplicate_points(points: Iterable[np.ndarray], tol:
float) -> list[np.ndarray]:
288     unique: list[np.ndarray] = []
289     for point in points:
290         if not any(float(np.linalg.norm(point - other)) <= tol
for other in unique):
291             unique.append(np.asarray(point, dtype=float))
292     return unique
293
294
295 def convex_hull(points: Sequence[np.ndarray], tol: float = 1e
-8) -> list[np.ndarray]:
296     """Return a counter-clockwise monotonic-chain hull."""
297
298     if len(points) == 0:
299         return []
300     # Cross-product tolerances must be translation invariant.
Using absolute
301     # coordinates here would incorrectly collapse a small
polygon near (1e12,
302     # 1e12) into a line even though its edge lengths are well
resolved.
303     anchor = np.asarray(points[0], dtype=float)
304     scale = max(1.0, max(float(np.max(np.abs(np.asarray(point)
- anchor))) for point in points))
305     cross_tol = tol * scale * scale
306     ordered = sorted(
307         _deduplicate_points((np.asarray(p, dtype=float) for p
in points), tol),
308         key=lambda p: (p[0], p[1]),
309     )

```

```

310     if len(ordered) == 1:
311         return ordered
312
313     def cross(o: np.ndarray, p: np.ndarray, q: np.ndarray) ->
float:
314         return _cross2(p - o, q - o)
315
316     lower: list[np.ndarray] = []
317     for point in ordered:
318         while len(lower) >= 2 and cross(lower[-2], lower[-1],
point) <= cross_tol:
319             lower.pop()
320             lower.append(point)
321     upper: list[np.ndarray] = []
322     for point in reversed(ordered):
323         while len(upper) >= 2 and cross(upper[-2], upper[-1],
point) <= cross_tol:
324             upper.pop()
325             upper.append(point)
326     return lower[:-1] + upper[:-1]
327
328
329 def _feasible_and_bounded(a: np.ndarray, b: np.ndarray) ->
tuple[str, np.ndarray | None]:
330     if len(a) == 0:
331         return "unbounded", None
332     feasibility = linprog(
333         np.zeros(2),
334         A_ub=a,
335         b_ub=b,
336         bounds=[(None, None), (None, None)],
337         method="highs",
338     )
339     if feasibility.status == 2:
340         return "empty", None
341     if feasibility.status != 0:
342         raise RuntimeError(f"linear feasibility check failed:
{feasibility.message}")
343     for objective in ([1.0, 0.0], [-1.0, 0.0], [0.0, 1.0],
[0.0, -1.0]):
344         bound_check = linprog(
345             np.asarray(objective),
346             A_ub=a,
347             b_ub=b,
348             bounds=[(None, None), (None, None)],
349             method="highs",
350         )
351         if bound_check.status == 3:
352             return "unbounded", np.asarray(feasibility.x,
dtype=float)
353         if bound_check.status not in (0, 4):
354             raise RuntimeError(f"boundedness check failed: {
bound_check.message}")
355         return "bounded", np.asarray(feasibility.x, dtype=float)
356
357

```

```

358 def _distance_sq(p: np.ndarray, q: np.ndarray) -> float:
359     difference = p - q
360     return float(difference @ difference)
361
362
363 def diameter_rotating_calipers(vertices: Sequence[np.ndarray],
364     tol: float = 1e-10) -> tuple[float, int, int]:
365     """Return diameter and endpoint indices of a CCW convex
366     polygon in O(m)."""
367
368     count = len(vertices)
369     if count < 2:
370         return 0.0, 0, 0
371     if count == 2:
372         return math.sqrt(_distance_sq(vertices[0], vertices
373     [1])), 0, 1
374     points = [np.asarray(point, dtype=float) for point in
375     vertices]
376     best_sq, best_pair = 0.0, (0, 0)
377
378     def update(i: int, j: int) -> None:
379         nonlocal best_sq, best_pair
380         value = _distance_sq(points[i], points[j])
381         if value > best_sq + tol:
382             best_sq, best_pair = value, (i, j)
383
384     j = 1
385     for i in range(count):
386         ni = (i + 1) % count
387         edge = points[ni] - points[i]
388         while True:
389             nj = (j + 1) % count
390             current_area = _cross2(edge, points[j] - points[i]
391         ])
392             next_area = _cross2(edge, points[nj] - points[i])
393             if next_area > current_area + tol:
394                 j = nj
395                 continue
396             break
397         update(i, j)
398         update(ni, j)
399         if abs(
400             _cross2(edge, points[(j + 1) % count] - points[i])
401             - _cross2(edge, points[j] - points[i])
402             ) <= tol:
403             update(i, (j + 1) % count)
404             update(ni, (j + 1) % count)
405     return math.sqrt(best_sq), best_pair[0], best_pair[1]
406
407
408 def _circle_from_three(p: np.ndarray, q: np.ndarray, r: np.
409     ndarray, tol: float) -> tuple[np.ndarray, float] | None:
410     """Return the circumcircle of three points, or 'None' if
411     near-collinear.
412
413     The equations are translated by 'p' before solving.

```

```

407     This avoids the
408     cancellation in 'q @ q - p @ p' when coordinates are
409     large but the
410     triangle itself is small.
411     """
412     q_relative = q - p
413     r_relative = r - p
414     determinant = 2.0 * _cross2(q_relative, r_relative)
415     local_scale = max(1.0, float(np.max(np.abs(np.vstack((
416     q_relative, r_relative))))))
417     if abs(determinant) <= tol * local_scale * local_scale:
418         return None
419     matrix = 2.0 * np.asarray((q_relative, r_relative), dtype=
420     float)
421     rhs = np.asarray((q_relative @ q_relative, r_relative @
422     r_relative), dtype=float)
423     center = p + np.linalg.solve(matrix, rhs)
424     return center, math.sqrt(_distance_sq(center, p))
425
426 def _circle_from_two(p: np.ndarray, q: np.ndarray) -> tuple[np
427     .ndarray, float]:
428     difference = q - p
429     # Translate before averaging; (p + q) / 2 loses the low
430     # bits when both
431     # coordinates share a large common offset.
432     center = p + difference / 2.0
433     return center, math.sqrt(float(difference @ difference)) /
434     2.0
435
436 def _contains_circle(
437     circle: tuple[np.ndarray, float, list[int]], point: np.
438     ndarray, tolerance: float
439 ) -> bool:
440     center, radius, _ = circle
441     return _distance_sq(point, center) <= (radius + tolerance)
442     ** 2
443
444 def _minimum_circle_for_support(
445     points: Sequence[np.ndarray], support: Sequence[int],
446     tolerance: float
447 ) -> tuple[np.ndarray, float, list[int]]:
448     """Find the smallest circle covering one, two, or three
449     support points."""
450
451     indices = list(dict.fromkeys(support))
452     if not indices:
453         return np.zeros(2, dtype=float), 0.0, []
454     if len(indices) == 1:
455         return np.asarray(points[indices[0]], dtype=float),
456         0.0, indices
457
458     candidates: list[tuple[np.ndarray, float, list[int]]] = []

```

```

450     for left in range(len(indices)):
451         for right in range(left + 1, len(indices)):
452             i, j = indices[left], indices[right]
453             center, radius = _circle_from_two(points[i],
points[j])
454             candidates.append((center, radius, [i, j]))
455             if len(indices) >= 3:
456                 i, j, k = indices[:3]
457                 circle = _circle_from_three(points[i], points[j],
points[k], tolerance)
458                 if circle is not None:
459                     center, radius = circle
460                     candidates.append((center, radius, [i, j, k]))
461
462             feasible = []
463             for center, radius, boundary in candidates:
464                 if all(_distance_sq(points[index], center) <= (radius
+ tolerance) ** 2 for index in indices):
465                     feasible.append((center, radius, boundary))
466             if not feasible:
467                 raise RuntimeError("failed to construct a circle for
support points")
468             return min(feasible, key=lambda item: item[1])
469
470
471 def _minimum_enclosing_circle_enumeration(
472     points: Sequence[np.ndarray], tolerance: float
473 ) -> tuple[np.ndarray, float]:
474     """Exact 2/3-point candidate enumeration used as a
numerical fallback."""
475
476     candidates: list[tuple[np.ndarray, float]] = []
477     count = len(points)
478     for i in range(count):
479         for j in range(i + 1, count):
480             candidates.append(_circle_from_two(points[i],
points[j]))
481     for i in range(count):
482         for j in range(i + 1, count):
483             for k in range(j + 1, count):
484                 circle = _circle_from_three(points[i], points[
j], points[k], tolerance)
485                 if circle is not None:
486                     candidates.append(circle)
487
488     for center, radius in sorted(candidates, key=lambda item:
item[1]):
489         if all(_distance_sq(point, center) <= (radius +
tolerance) ** 2 for point in points):
490             return center, radius
491     raise RuntimeError("failed to construct a covering circle"
)
492
493
494 def minimum_enclosing_circle(vertices: Sequence[np.ndarray],
tol: float = 1e-8) -> dict[str, Any]:

```

```

495     """Find the minimum enclosing circle with a randomized
496     incremental solver.
497     In the plane, the active support has at most three points.
498     The nested
499     boundary updates therefore implement Welzl's randomized
500     incremental method
501     in expected linear time. A candidate enumeration fallback
502     is retained for
503     rare floating-point inconsistencies.
504     """
505     points = [np.asarray(point, dtype=float) for point in
506     vertices]
507     if not points:
508         raise ValueError("minimum enclosing circle requires at
509         least one point")
510     if any(point.shape != (2,) or not np.all(np.isfinite(point
511     )) for point in points):
512         raise ValueError("minimum enclosing circle points must
513         be finite 2-D vectors")
514     anchor = points[0]
515     local_scale = max(1.0, max(float(np.max(np.abs(point -
516     anchor))) for point in points))
517     absolute_scale = max(1.0, max(float(np.max(np.abs(point)))
518     for point in points))
519     roundoff_tolerance = 32.0 * np.finfo(float).eps *
520     absolute_scale
521     tolerance = max(abs(float(tol)), 1e-12 * local_scale,
522     roundoff_tolerance)
523     if len(points) == 1:
524         return {"center": points[0], "radius": 0.0, "
525     support_indices": [0]}
526
527     order = list(range(len(points)))
528     random.Random(0xC0DEC0DE).shuffle(order)
529     circle = _minimum_circle_for_support(points, [], tolerance
530     )
531     for outer, index in enumerate(order):
532         if _contains_circle(circle, points[index], tolerance):
533             continue
534         circle = _minimum_circle_for_support(points, [index],
535     tolerance)
536         for middle in range(outer):
537             middle_index = order[middle]
538             if _contains_circle(circle, points[middle_index],
539     tolerance):
540                 continue
541             circle = _minimum_circle_for_support(points, [
542     index, middle_index], tolerance)
543             for inner in range(middle):
544                 inner_index = order[inner]
545                 if _contains_circle(circle, points[inner_index
546     ], tolerance):
547                     continue
548                 circle = _minimum_circle_for_support(

```

```

533         points, [index, middle_index, inner_index
534 ], tolerance
535     )
536     center, radius, _ = circle
537     distances = np.sqrt([_distance_sq(point, center) for point
538 in points])
539     maximum_distance = float(np.max(distances))
540     if maximum_distance > radius + 10.0 * tolerance:
541         center, radius = _minimum_enclosing_circle_enumeration
542 (points, tolerance)
543         distances = np.sqrt([_distance_sq(point, center) for
544 point in points])
545         radius = max(float(radius), float(np.max(distances)))
546         support_tolerance = max(10.0 * tolerance, 1e-10 * max(1.0,
547 radius))
548         support_indices = [
549             index for index, distance in enumerate(distances) if
550 abs(float(distance) - radius) <= support_tolerance
551 ]
552         return {
553             "center": np.asarray(center, dtype=float),
554             "radius": radius,
555             "support_indices": support_indices,
556         }
557
558 def _point_list(points: Sequence[np.ndarray]) -> list[list[
559 float]]:
560     return [[float(point[0]), float(point[1])] for point in
561 points]
562
563 def solve_question1(
564     measurements: Sequence[tuple[float, float, float]],
565     angle_error_deg: float = DEFAULT_ANGLE_ERROR_DEG,
566     tolerance: float | None = None,
567 ) -> dict[str, Any]:
568     """Solve Question 1 and return a JSON-serialisable result
569     dictionary."""
570
571     measurements = [
572         _extract_row(row, index) for index, row in enumerate(
573 measurements)
574     ]
575     if not measurements:
576         raise ValueError("measurements must be a non-empty
577 sequence")
578     a, b = build_constraints(measurements, angle_error_deg)
579     coordinate_array = np.asarray([row[:2] for row in
580 measurements], dtype=float)
581     anchor = coordinate_array[0]
582     extent = max(
583         1.0,
584         float(np.max(np.ptp(coordinate_array, axis=0))),
585         float(np.max(np.abs(coordinate_array - anchor))),

```

```

577 )
578 origin_scale = max(
579     1.0,
580     float(np.max(np.abs(coordinate_array))),
581     float(np.max(np.abs(b))) if len(b) else 1.0,
582 )
583 roundoff_scale = 32.0 * np.finfo(float).eps * origin_scale
584 tol = float(tolerance) if tolerance is not None else 1e-9
* extent + roundoff_scale
585 if not math.isfinite(tol) or tol < 0.0:
586     raise ValueError("tolerance must be a finite non-
negative number")
587 status, feasible_point = _feasible_and_bounded(a, b)
588 result: dict[str, Any] = {
589     "status": status,
590     "angle_error_deg": float(angle_error_deg),
591     "measurement_count": len(measurements),
592     "constraint_count": int(len(a)),
593     "vertices": [],
594     "diameter": None,
595     "diameter_endpoints": None,
596     "diameter_circle": None,
597     "minimum_cover_circle": None,
598 }
599 if status == "empty" or status == "unbounded":
600     if status == "unbounded" and feasible_point is not
None:
601         result["feasible_point"] = [float(feasible_point
[0]), float(feasible_point[1])]
602     return result
603
604 hpi_vertices = halfplane_intersection(a, b, tol=tol)
605 if hpi_vertices and all(np.all(a @ point <= b + tol) for
point in hpi_vertices):
606     candidates = hpi_vertices
607 else:
608     candidates = _pairwise_feasible_vertices(a, b, tol)
609 vertices = convex_hull(candidates, tol=tol)
610 result["vertices"] = _point_list(vertices)
611 if not vertices:
612     # A bounded non-empty polyhedron always has at least
one extreme point;
613     # reaching this branch indicates a numerical
inconsistency in the input.
614     raise RuntimeError("bounded feasible region has no
recoverable vertices")
615
616 dimension_status = {1: "point", 2: "segment", 3: "polygon"
}.get(len(vertices), "polygon")
617 result["status"] = dimension_status
618 diameter, first, second = diameter_rotating_calipers(
vertices, tol=tol * tol)
619 result["diameter"] = float(diameter)
620 result["diameter_endpoints"] = {
621     "p": result["vertices"][first],
622     "q": result["vertices"][second],

```

```

623     }
624     diameter_center = vertices[first] + (vertices[second] -
vertices[first]) / 2.0
625     diameter_radius = diameter / 2.0
626     max_distance = max(
627         (math.sqrt(_distance_sq(point, diameter_center)) for
point in vertices),
628         default=0.0,
629     )
630     result["diameter_circle"] = {
631         "center": [float(diameter_center[0]), float(
diameter_center[1])],
632         "radius": float(diameter_radius),
633         "max_vertex_distance": float(max_distance),
634         "covers": bool(max_distance <= diameter_radius + tol),
635     }
636     mec = minimum_enclosing_circle(vertices, tol=tol)
637     result["minimum_cover_circle"] = {
638         "center": [float(mec["center"][0]), float(mec["center"
][1])],
639         "radius": float(mec["radius"]),
640         "support_indices": mec["support_indices"],
641         "max_vertex_distance": float(
642             max(math.sqrt(_distance_sq(point, mec["center"]))
for point in vertices)
643         ),
644     }
645     return result
646
647
648 def _demo_input() -> dict[str, Any]:
649     return {
650         "angle_error_deg": 1.0,
651         "measurements": [
652             {"x": 1000.0, "y": 0.0, "svd_deg": 180.0},
653             {"x": -500.0, "y": 500.0 * math.sqrt(3.0), "
svd_deg": 300.0},
654             {"x": -500.0, "y": -500.0 * math.sqrt(3.0), "
svd_deg": 60.0},
655         ],
656     }
657
658
659 def main(argv: Sequence[str] | None = None) -> int:
660     parser = argparse.ArgumentParser(description=__doc__)
661     parser.add_argument("--input", help="JSON/CSV input path;
omit when using --demo")
662     parser.add_argument("--output", help="write result JSON to
this path")
663     parser.add_argument("--vertices-output", help="write
boundary vertices as CSV")
664     parser.add_argument("--angle-error-deg", type=float, help=
"override input angular error")
665     parser.add_argument(
666         "--tolerance",
667         type=float,

```

```

668         help="non-negative geometric tolerance in metres (
default: scale-aware)",
669     )
670     parser.add_argument("--demo", action="store_true", help="
run the six-vertex counterexample")
671     args = parser.parse_args(argv)
672     if args.demo:
673         measurements, error_deg = parse_measurements(
demo_input())
674     elif args.input:
675         measurements, error_deg = load_input(args.input)
676     else:
677         parser.error("provide --input or --demo")
678     if args.angle_error_deg is not None:
679         error_deg = args.angle_error_deg
680     result = solve_question1(measurements, error_deg,
tolerance=args.tolerance)
681     payload = json.dumps(result, ensure_ascii=False, indent=2)
682     if args.output:
683         Path(args.output).parent.mkdir(parents=True, exist_ok=
True)
684         Path(args.output).write_text(payload + "\n", encoding=
"utf-8")
685     else:
686         print(payload)
687     if args.vertices_output:
688         with Path(args.vertices_output).open("w", encoding="
utf-8", newline="") as handle:
689             writer = csv.writer(handle)
690             writer.writerow(("vertex_index", "x", "y"))
691             for index, point in enumerate(result["vertices"]):
692                 writer.writerow((index, point[0], point[1]))
693     return 0
694
695
696 if __name__ == "__main__":
697     raise SystemExit(main())

```

#### solve\_q2.py

```

1  """Continuous second-detector selection model for Question 2.
2
3  The first detector and its measured bearing define a
4  conservative continuous
5  target region. This module then solves the robust second-
6  detector problem
7  described in ''docs/问题二求解思路.md'':
8
9  * the target disk and the first-detector reception disk are
10     represented by
11     circumscribed regular polygons, so the target region remains
12     conservative;
13 * the weighted DOP is integrated over the polygon by
14     deterministic triangle
15     quadrature (the nodes are integration points, not target
16     representatives);
17 * the semi-infinite reception constraint is evaluated with a

```

```

12     geometric
13     separation oracle. For the inner disk part the maximum is
14     attained at
15     polygon/circle extreme points; for the outer part the
16     constraint is affine
17     in the target position and the circle support point is also
18     checked;
19 * a multi-start constrained optimization gives a numerical
20 reference optimum;
21 the requested candidate region is reported as a grid
22 approximation of the
23 continuous near-optimal level set, together with area and
24 connected-component
25 summaries.
26
27 Example::
28
29     python solve_q2.py --demo --output ../../outputs/t2/
30     t2_demo_result.json
31
32 JSON input accepts either::
33
34     {"s1": {"x": 0, "y": 0}, "svd_deg": 35}
35     {"detector": [0, 0], "bearing_deg": 35}
36
37 Optional model parameters can be placed in the same object or
38 supplied on the
39 command line. See '--help' for the complete list.
40 """
41
42 from __future__ import annotations
43
44 import argparse
45 import json
46 import math
47 from dataclasses import asdict, dataclass
48 from pathlib import Path
49 from typing import Any, Sequence
50
51 import numpy as np
52 from scipy.optimize import differential_evolution, linprog,
53     minimize
54
55 TAU_DEFAULTS = (0.01, 0.03, 0.05, 0.10, 0.20)
56 EPSILON_DEFAULTS = (0.1, 0.2, 0.3, 0.5)
57 P_DEFAULTS = (1.0, 2.0, 3.0)
58
59 def _finite_float(value: Any, name: str) -> float:
60     try:
61         number = float(value)
62     except (TypeError, ValueError) as exc:
63         raise ValueError(f"{name} must be numeric") from exc
64     if not math.isfinite(number):
65         raise ValueError(f"{name} must be finite")
66     return number

```

```

59
60
61 def _finite_int(value: Any, name: str) -> int:
62     number = _finite_float(value, name)
63     if number != round(number):
64         raise ValueError(f"{name} must be an integer")
65     return int(number)
66
67
68 def _as_point(value: Any, name: str) -> tuple[float, float]:
69     if isinstance(value, dict):
70         if "x" not in value or "y" not in value:
71             raise ValueError(f"{name} must contain x and y")
72         return _finite_float(value["x"], f"{name}.x"),
73             _finite_float(
74                 value["y"], f"{name}.y"
75             )
76     if isinstance(value, (list, tuple)) and len(value) >= 2:
77         return _finite_float(value[0], f"{name}[0]"),
78             _finite_float(
79                 value[1], f"{name}[1]"
80             )
81     raise ValueError(f"{name} must be an object with x/y or a
82     two-element array")
83
84 @dataclass
85 class Q2Config:
86     """Numerical and model parameters for Question 2."""
87
88     target_radius: float = 1800.0
89     min_receive_radius: float = 1000.0
90     max_receive_radius: float = 1500.0
91     angle_error_deg: float = 1.0
92     circle_sides: int = 360
93     epsilon_w: float = 0.2
94     p_w: float = 2.0
95     tau: float = 0.05
96     quadrature_order: int = 6
97     validation_quadrature_order: int = 8
98     candidate_grid: int = 101
99     seed: int = 20260911
100     use_global_search: bool = True
101     global_maxiter: int = 24
102     global_popsizes: int = 7
103     local_maxiter: int = 240
104     feasibility_tolerance_m: float = 1e-6
105     angle_sin_tolerance: float = 1e-10
106     sensitivity_epsilons: tuple[float, ...] = EPSILON_DEFAULTS
107     sensitivity_powers: tuple[float, ...] = P_DEFAULTS
108     sensitivity_taus: tuple[float, ...] = TAU_DEFAULTS
109
110     def validate(self) -> None:
111         for name in (
112             "target_radius",
113             "min_receive_radius",

```

```

112         "max_receive_radius",
113         "angle_error_deg",
114         "epsilon_w",
115         "p_w",
116         "tau",
117         "feasibility_tolerance_m",
118         "angle_sin_tolerance",
119     ):
120         value = _finite_float(getattr(self, name), name)
121         setattr(self, name, value)
122         self.circle_sides = _finite_int(self.circle_sides, "
circle_sides")
123         self.quadrature_order = _finite_int(self.
quadrature_order, "quadrature_order")
124         self.validation_quadrature_order = _finite_int(
125             self.validation_quadrature_order, "
validation_quadrature_order"
126         )
127         self.candidate_grid = _finite_int(self.candidate_grid,
"candidate_grid")
128         self.seed = _finite_int(self.seed, "seed")
129         self.global_maxiter = _finite_int(self.global_maxiter,
"global_maxiter")
130         self.global_popsiz = _finite_int(self.global_popsiz,
"global_popsiz")
131         self.local_maxiter = _finite_int(self.local_maxiter, "
local_maxiter")
132         if not isinstance(self.use_global_search, (bool, np.
bool_)):
133             raise ValueError("use_global_search must be
boolean")
134         self.use_global_search = bool(self.use_global_search)
135         if self.target_radius <= 0:
136             raise ValueError("target_radius must be positive")
137         if not (0 < self.min_receive_radius <= self.
max_receive_radius):
138             raise ValueError("receive-radius bounds must
satisfy 0 < min <= max")
139         if not (0 < self.angle_error_deg < 90):
140             raise ValueError("angle_error_deg must be in (0,
90)")
141         if not (0 < self.epsilon_w < 1):
142             raise ValueError("epsilon_w must be in (0, 1)")
143         if self.p_w < 1:
144             raise ValueError("p_w must be at least 1")
145         if self.tau < 0:
146             raise ValueError("tau must be non-negative")
147         if self.feasibility_tolerance_m < 0:
148             raise ValueError("feasibility_tolerance_m must be
non-negative")
149         if not (0 <= self.angle_sin_tolerance < 1):
150             raise ValueError("angle_sin_tolerance must be in
[0, 1)")
151         if self.circle_sides < 16:
152             raise ValueError("circle_sides must be at least 16
")

```

```

153         if self.quadrature_order < 2 or self.
validation_quadrature_order < 2:
154             raise ValueError("quadrature orders must be at
least 2")
155         if self.candidate_grid < 11:
156             raise ValueError("candidate_grid must be at least
11")
157         if self.global_maxiter < 1 or self.global_popsiz < 2
or self.local_maxiter < 1:
158             raise ValueError("optimizer iteration/population
parameters are too small")
159
160     def validate_sweep(name: str, values: Any, predicate:
Any, description: str) -> tuple[float, ...]:
161         if not isinstance(values, (list, tuple, np.ndarray
)) or len(values) == 0:
162             raise ValueError(f"{name} must be a non-empty
sequence")
163         result = tuple(_finite_float(value, f"{name}[{
index}]") for index, value in enumerate(values))
164         if not all(predicate(value) for value in result):
165             raise ValueError(f"{name} contains values
outside {description}")
166         return result
167
168     self.sensitivity_epsilons = validate_sweep(
169         "sensitivity_epsilons", self.sensitivity_epsilons,
lambda value: 0 < value < 1, "(0, 1)"
170     )
171     self.sensitivity_powers = validate_sweep(
172         "sensitivity_powers", self.sensitivity_powers,
lambda value: value >= 1, "[1, +inf)"
173     )
174     self.sensitivity_taus = validate_sweep(
175         "sensitivity_taus", self.sensitivity_taus, lambda
value: value >= 0, "[0, +inf)"
176     )
177
178
179 @dataclass
180 class PolygonRegion:
181     """A counter-clockwise convex polygon and its half-plane
description."""
182
183     vertices: np.ndarray
184     a: np.ndarray
185     b: np.ndarray
186     area: float
187     centroid: np.ndarray
188     d_star: float
189     incenter: np.ndarray
190
191
192 def _cross(left: np.ndarray, right: np.ndarray) -> float:
193     return float(left[0] * right[1] - left[1] * right[0])
194

```

```

195
196 def _circle_halfplanes(center: np.ndarray, radius: float,
197     sides: int) -> list[tuple[np.ndarray, float]]:
198     """Return tangent half-planes of an external regular
199     polygon."""
200     result: list[tuple[np.ndarray, float]] = []
201     for index in range(sides):
202         angle = 2.0 * math.pi * index / sides
203         normal = np.asarray((math.cos(angle), math.sin(angle))
204     , dtype=float)
205         result.append((normal, radius + float(normal @ center)
206     ))
207     return result
208
209 def _wedge_halfplanes(
210     s1: np.ndarray, bearing_deg: float, error_deg: float
211 ) -> list[tuple[np.ndarray, float]]:
212     lower = math.radians(bearing_deg - error_deg)
213     upper = math.radians(bearing_deg + error_deg)
214     lower_direction = np.asarray((math.cos(lower), math.sin(
215     lower)))
216     upper_direction = np.asarray((math.cos(upper), math.sin(
217     upper)))
218     lower_left_normal = np.asarray((-lower_direction[1],
219     lower_direction[0]))
220     upper_left_normal = np.asarray((-upper_direction[1],
221     upper_direction[0]))
222     # cross(lower_direction, G-S1) >= 0 and
223     # cross(upper_direction, G-S1) <= 0.
224     return [
225         (-lower_left_normal, float((-lower_left_normal) @ s1))
226     ,
227         (upper_left_normal, float(upper_left_normal @ s1)),
228     ]
229
230 def _external_circle_vertices(center: np.ndarray, radius:
231     float, sides: int) -> np.ndarray:
232     half_step = math.pi / sides
233     vertex_radius = radius / math.cos(half_step)
234     angles = 2.0 * math.pi * np.arange(sides, dtype=float) /
235     sides + half_step
236     return center + vertex_radius * np.column_stack((np.cos(
237     angles), np.sin(angles)))
238
239 def _clip_polygon(
240     polygon: np.ndarray,
241     normal: np.ndarray,
242     bound: float,
243     tolerance: float = 1e-10,
244 ) -> np.ndarray:
245     if len(polygon) == 0:
246         return polygon

```

```

239 output: list[np.ndarray] = []
240 previous = polygon[-1]
241 previous_value = float(bound - normal @ previous)
242 previous_inside = previous_value >= -tolerance
243 for current in polygon:
244     current_value = float(bound - normal @ current)
245     current_inside = current_value >= -tolerance
246     if current_inside != previous_inside:
247         denominator = previous_value - current_value
248         if abs(denominator) > np.finfo(float).eps:
249             fraction = previous_value / denominator
250             output.append(previous + fraction * (current -
previous))
251         if current_inside:
252             output.append(current.copy())
253             previous = current
254             previous_value = current_value
255             previous_inside = current_inside
256     if not output:
257         return np.empty((0, 2), dtype=float)
258 cleaned: list[np.ndarray] = []
259 for point in output:
260     if not cleaned or np.linalg.norm(point - cleaned[-1])
> 1e-8:
261         cleaned.append(point)
262     if len(cleaned) > 1 and np.linalg.norm(cleaned[0] -
cleaned[-1]) <= 1e-8:
263         cleaned.pop()
264     return np.asarray(cleaned, dtype=float)
265
266
267 def _polygon_area_centroid(vertices: np.ndarray) -> tuple[
float, np.ndarray]:
268     if len(vertices) < 3:
269         return 0.0, np.mean(vertices, axis=0) if len(vertices)
else np.zeros(2)
270     next_vertices = np.roll(vertices, -1, axis=0)
271     cross_values = vertices[:, 0] * next_vertices[:, 1] -
next_vertices[:, 0] * vertices[:, 1]
272     signed_twice_area = float(np.sum(cross_values))
273     area = abs(signed_twice_area) / 2.0
274     if abs(signed_twice_area) <= 1e-14:
275         return area, np.mean(vertices, axis=0)
276     centroid = np.sum((vertices + next_vertices) *
cross_values[:, None], axis=0) / (
277         3.0 * signed_twice_area
278     )
279     if signed_twice_area < 0:
280         vertices[:, :] = vertices[:, :-1]
281     return area, centroid
282
283
284 def _compute_incenter(a: np.ndarray, b: np.ndarray) -> tuple[
np.ndarray, float]:
285     norms = np.linalg.norm(a, axis=1)
286     if len(a) == 0:

```

```

287         raise ValueError("target region has no half-planes")
288     result = linprog(
289         np.asarray((0.0, 0.0, -1.0)),
290         A_ub=np.column_stack((a, norms)),
291         b_ub=b,
292         bounds=[(None, None), (None, None), (0.0, None)],
293         method="highs",
294     )
295     if result.status != 0 or result.x is None:
296         raise ValueError(f"failed to compute target-region
inradius: {result.message}")
297     return np.asarray(result.x[:2], dtype=float), float(result
.x[2])
298
299
300 def build_target_region(
301     s1: Sequence[float] | np.ndarray,
302     bearing_deg: float,
303     config: Q2Config | None = None,
304 ) -> PolygonRegion:
305     """Construct the conservative continuous target polygon ''
Omega_1''."""
306
307     config = config or Q2Config()
308     config.validate()
309     s1_array = np.asarray(s1, dtype=float)
310     if s1_array.shape != (2,) or not np.all(np.isfinite(
s1_array)):
311         raise ValueError("s1 must be a finite two-dimensional
point")
312     bearing_deg = _finite_float(bearing_deg, "bearing_deg") %
360.0
313
314     circle_target = _circle_halfplanes(
315         np.zeros(2), config.target_radius, config.circle_sides
316     )
317     circle_reception = _circle_halfplanes(
318         s1_array, config.max_receive_radius, config.
circle_sides
319     )
320     wedge = _wedge_halfplanes(s1_array, bearing_deg, config.
angle_error_deg)
321     halfplanes = circle_target + circle_reception + wedge
322     polygon = _external_circle_vertices(
323         np.zeros(2), config.target_radius, config.circle_sides
324     )
325     # Clipping an external target polygon by every half-plane
preserves the
326     # continuous half-plane model while avoiding pairwise line
intersections.
327     for normal, bound in circle_reception + wedge:
328         polygon = _clip_polygon(polygon, normal, bound)
329         if len(polygon) == 0:
330             raise ValueError("the first measurement yields an
empty target region")
331     area, centroid = _polygon_area_centroid(polygon)

```

```

332     if area <= 1e-8:
333         raise ValueError("the first measurement yields a zero-
area target region")
334     a = np.asarray([entry[0] for entry in halfplanes], dtype=
float)
335     b = np.asarray([entry[1] for entry in halfplanes], dtype=
float)
336     incenter, d_star = _compute_incenter(a, b)
337     if d_star <= 1e-8:
338         raise ValueError("target region has no positive
inradius")
339     return PolygonRegion(
340         vertices=polygon,
341         a=a,
342         b=b,
343         area=float(area),
344         centroid=np.asarray(centroid, dtype=float),
345         d_star=float(d_star),
346         incenter=incenter,
347     )
348
349
350 def _triangle_quadrature(order: int) -> tuple[np.ndarray, np.
ndarray]:
351     """Tensor Gauss-Legendre/Duffy quadrature on the unit
reference triangle."""
352
353     nodes_1d, weights_1d = np.polynomial.legendre.leggauss(
order)
354     nodes_1d = (nodes_1d + 1.0) / 2.0
355     weights_1d = weights_1d / 2.0
356     barycentric: list[tuple[float, float, float]] = []
357     weights: list[float] = []
358     for u, wu in zip(nodes_1d, weights_1d):
359         for v, wv in zip(nodes_1d, weights_1d):
360             # Reference coordinates are (u, (1-u)*v); the
Jacobian is 1-u.
361             barycentric.append(((1.0 - u) * (1.0 - v), u, (1.0
- u) * v))
362             weights.append(float(wu * wv * (1.0 - u)))
363     # The temporary tuple above is (lambda_0, lambda_1,
lambda_2) after
364     # interpreting the first coordinate as 1-u and the second
as u.
365     return np.asarray(barycentric, dtype=float), np.asarray(
weights, dtype=float)
366
367
368 def _integration_nodes(
369     vertices: np.ndarray, order: int
370 ) -> tuple[np.ndarray, np.ndarray, float]:
371     barycentric, reference_weights = _triangle_quadrature(
order)
372     center = np.mean(vertices, axis=0)
373     points: list[np.ndarray] = []
374     weights: list[float] = []

```

```

375     for index, first in enumerate(vertices):
376         second = vertices[(index + 1) % len(vertices)]
377         determinant = abs(_cross(second - center, first -
center))
378         if determinant <= 1e-14:
379             continue
380         triangle = (
381             barycentric[:, 0, None] * center
382             + barycentric[:, 1, None] * first
383             + barycentric[:, 2, None] * second
384         )
385         points.append(triangle)
386         weights.append(determinant * reference_weights)
387     if not points:
388         raise ValueError("failed to triangulate target polygon
")
389     points_array = np.vstack(points)
390     weights_array = np.concatenate(weights)
391     return points_array, weights_array, float(np.sum(
weights_array))
392
393
394 def _point_inside(region: PolygonRegion, point: np.ndarray,
tolerance: float = 1e-8) -> bool:
395     slack = region.b - region.a @ point
396     scale = max(1.0, float(np.max(np.abs(region.b))))
397     return bool(np.all(slack >= -tolerance * scale))
398
399
400 def _segment_circle_intersections(
401     first: np.ndarray, second: np.ndarray, center: np.ndarray,
radius: float
402 ) -> list[np.ndarray]:
403     direction = second - first
404     offset = first - center
405     aa = float(direction @ direction)
406     if aa <= 1e-20:
407         return []
408     bb = 2.0 * float(offset @ direction)
409     cc = float(offset @ offset) - radius * radius
410     discriminant = bb * bb - 4.0 * aa * cc
411     if discriminant < -1e-8 * max(1.0, bb * bb, abs(aa * cc)):
412         return []
413     discriminant = max(0.0, discriminant)
414     root = math.sqrt(discriminant)
415     result: list[np.ndarray] = []
416     for parameter in ((-bb - root) / (2.0 * aa), (-bb + root)
/ (2.0 * aa)):
417         if -1e-9 <= parameter <= 1.0 + 1e-9:
418             clipped = min(1.0, max(0.0, parameter))
419             point = first + clipped * direction
420             if not result or np.linalg.norm(point - result
[-1]) > 1e-7:
421                 result.append(point)
422     return result
423

```

```

424
425 class Question2Model:
426     """Numerical model with reusable geometry, quadrature and
        objective state."""
427
428     def __init__(
429         self,
430         s1: Sequence[float] | np.ndarray,
431         bearing_deg: float,
432         config: Q2Config,
433         quadrature_order: int | None = None,
434     ) -> None:
435         self.s1 = np.asarray(s1, dtype=float)
436         self.bearing_deg = float(bearing_deg) % 360.0
437         self.config = config
438         self.config.validate()
439         self.region = build_target_region(self.s1, self.
bearing_deg, config)
440         self._quadrature_cache: dict[int, dict[str, Any]] = {}
441         order = config.quadrature_order if quadrature_order is
None else _finite_int(
442             quadrature_order, "quadrature_order"
443         )
444         if order < 2:
445             raise ValueError("quadrature order must be at
least 2")
446         self.quadrature_order = order
447         default_data = self._quadrature_data(order)
448         self.nodes = default_data["nodes"]
449         self.quad_weights = default_data["quad_weights"]
450         self.quad_area = default_data["quad_area"]
451         self.depths = default_data["depths"]
452         self.depth_quantiles = default_data["depth_quantiles"]
453         self.r1_vectors = default_data["r1_vectors"]
454         self.r1_sq = default_data["r1_sq"]
455         self.r1 = default_data["r1"]
456         self.search_center = self.region.centroid.copy()
457         self.search_radius = max(
458             config.min_receive_radius,
459             float(np.linalg.norm(self.search_center - self.s1)
),
460         )
461         self.search_bounds = [
462             (
463                 float(self.search_center[0] - self.
search_radius),
464                 float(self.search_center[0] + self.
search_radius),
465             ),
466             (
467                 float(self.search_center[1] - self.
search_radius),
468                 float(self.search_center[1] + self.
search_radius),
469             ),
470         ]

```

```

471     # The reception oracle is evaluated thousands of times
472     when a
473     # marching-squares boundary or a Monte-Carlo
474     diagnostic is requested.
475     # Vertices and polygon/circle intersection points are
476     independent of
477     # the candidate detector position, so cache them once.
478     The only
479     # position-dependent support point is added by ''
480     receive_h'' below.
481     (
482         self._reception_static_inner,
483         self._reception_static_outer,
484     ) = self._build_reception_static_candidates()
485
486     def _build_reception_static_candidates(self) -> tuple[np.
487         ndarray, np.ndarray]:
488         """Return unique polygon/circle extreme points used by
489         ''H(P)''.

```

```

513         for point in unique
514         if float(np.linalg.norm(point - self.s1)) <=
radius + 1e-7
515     ]
516     outer = [
517         point
518         for point in unique
519         if float(np.linalg.norm(point - self.s1)) >=
radius - 1e-7
520     ]
521     # Circle intersections belong to both pieces. A full-
dimensional
522     # polygon has at least one candidate in each piece;
retain a centroid
523     # fallback only for defensive handling of degenerate
numerical input.
524     if not inner:
525         inner = [self.region.centroid.copy()]
526     if not outer:
527         outer = [self.region.centroid.copy()]
528     return np.asarray(inner, dtype=float), np.asarray(
outer, dtype=float)
529
530     def _quadrature_data(self, order: int) -> dict[str, Any]:
531         """Build and cache integration data for one quadrature
order."""
532
533         order = _finite_int(order, "quadrature_order")
534         if order < 2:
535             raise ValueError("quadrature order must be at
least 2")
536         cached = self._quadrature_cache.get(order)
537         if cached is not None:
538             return cached
539         nodes, quad_weights, quad_area = _integration_nodes(
self.region.vertices, order)
540         slacks = self.region.b[:, None] - self.region.a @
nodes.T
541         depths = np.maximum(
542             0.0,
543             np.min(slacks / np.linalg.norm(self.region.a, axis
=1)[:, None], axis=0),
544         )
545         depth_quantiles = np.minimum(1.0, depths / self.region
.d_star)
546         r1_vectors = nodes - self.s1[None, :]
547         r1_sq = np.einsum("ij,ij->i", r1_vectors, r1_vectors)
548         data = {
549             "nodes": nodes,
550             "quad_weights": quad_weights,
551             "quad_area": float(quad_area),
552             "depths": depths,
553             "depth_quantiles": depth_quantiles,
554             "r1_vectors": r1_vectors,
555             "r1_sq": r1_sq,
556             "r1": np.sqrt(r1_sq),

```

```

557     }
558     self._quadrature_cache[order] = data
559     return data
560
561     def _weights(
562         self,
563         epsilon_w: float | None = None,
564         p_w: float | None = None,
565         depth_quantiles: np.ndarray | None = None,
566     ) -> np.ndarray:
567         epsilon = self.config.epsilon_w if epsilon_w is None
568         else float(epsilon_w)
569         power = self.config.p_w if p_w is None else float(p_w)
570         if not (0.0 < epsilon < 1.0):
571             raise ValueError("epsilon_w must be in (0, 1)")
572         if power < 1.0:
573             raise ValueError("p_w must be at least 1")
574         quantiles = self.depth_quantiles if depth_quantiles is
575         None else depth_quantiles
576         return epsilon + (1.0 - epsilon) * np.power(quantiles,
577             power)
578
579     def _circle_boundary_candidates(self, point: np.ndarray)
580     -> list[np.ndarray]:
581         result: list[np.ndarray] = []
582         vertices = self.region.vertices
583         for index, first in enumerate(vertices):
584             second = vertices[(index + 1) % len(vertices)]
585             result.extend(
586                 _segment_circle_intersections(
587                     first, second, self.s1, self.config.
588                     min_receive_radius
589                 )
590             )
591         direction = point - self.s1
592         norm = float(np.linalg.norm(direction))
593         if norm > 1e-12:
594             # This is simultaneously the farthest point for
595             # the inner
596             # radius-1000 disk and the support point for the
597             # outer affine
598             # constraint.
599             support = self.s1 - self.config.min_receive_radius
600             * direction / norm
601             if _point_inside(self.region, support, 2e-8):
602                 result.append(support)
603         return result
604
605     def _reception_candidates(self, point: np.ndarray) ->
606     tuple[list[np.ndarray], list[np.ndarray]]:
607         inner: list[np.ndarray] = []
608         outer: list[np.ndarray] = []
609         radius = self.config.min_receive_radius
610         for vertex in self.region.vertices:
611             distance = float(np.linalg.norm(vertex - self.s1))
612             if distance <= radius + 1e-7:

```

```

604         inner.append(vertex)
605         if distance >= radius - 1e-7:
606             outer.append(vertex)
607     intersections = self._circle_boundary_candidates(point
)
608     for candidate in intersections:
609         inner.append(candidate)
610         outer.append(candidate)
611     # Remove duplicate points while preserving
deterministic order.
612     def unique(points: list[np.ndarray]) -> list[np.
ndarray]:
613         output: list[np.ndarray] = []
614         for candidate in points:
615             if not _point_inside(self.region, candidate, 5
e-7):
616                 continue
617             if not any(np.linalg.norm(candidate - prior)
<= 1e-7 for prior in output):
618                 output.append(candidate)
619         return output
620
621     return unique(inner), unique(outer)
622
623     def receive_h(self, point: Sequence[float] | np.ndarray)
-> float:
624         """Return the exact polygon/circle separation value H(
P), up to roundoff."""
625
626         point_array = np.asarray(point, dtype=float)
627         if point_array.shape != (2,) or not np.all(np.isfinite
(point_array)):
628             return math.inf
629         # Evaluate the cached extreme points in one vector
operation. Each
630         # point carries the correct piecewise radius 'max(r0
,||G-S1||)';
631         # this is algebraically identical to the previous
inner/outer list
632         # implementation and is substantially faster for dense
grids.
633         radius_sq = self.config.min_receive_radius**2
634         inner = self._reception_static_inner
635         outer = self._reception_static_outer
636         inner_values = np.einsum(
637             "ij,ij->i", inner - point_array[None, :], inner -
point_array[None, :]
638         ) - radius_sq
639         outer_radius_sq = np.einsum(
640             "ij,ij->i", outer - self.s1[None, :], outer - self
.s1[None, :]
641         )
642         outer_values = np.einsum(
643             "ij,ij->i", outer - point_array[None, :], outer -
point_array[None, :]
644         ) - outer_radius_sq

```

```

645         best = float(max(np.max(inner_values), np.max(
outer_values)))
646
647         # The support point of the radius-r0 circle is the
only extreme point
648         # that changes with P. It contributes when it lies in
the target
649         # polygon. Include it with the same piecewise radius
expression.
650         direction = point_array - self.s1
651         norm = float(np.linalg.norm(direction))
652         if norm > 1e-12:
653             support = self.s1 - self.config.min_receive_radius
* direction / norm
654             if _point_inside(self.region, support, 2e-8):
655                 support_value = float(np.sum((point_array -
support) ** 2) - radius_sq)
656                 best = max(best, support_value)
657                 if not math.isfinite(best):
658                     # A full-dimensional polygon always has at least
one extreme
659                     # point; retain a defensive fallback for malformed
input.
660                     target = self.region.centroid
661                     best = float(
662                         np.sum((point_array - target) ** 2)
663                         - max(radius_sq, np.sum((target - self.s1) **
2)))
664             )
665         return best
666
667     def receive_h_many(self, points: Sequence[Sequence[float]]
| np.ndarray) -> np.ndarray:
668         """Vectorised separation-oracle values for a batch of
detector points.
669
670         The returned array has one value per row of 'points
''. Invalid rows
671         receive '+inf'. Static extrema are evaluated by
matrix products;
672         the dynamic circle support point is handled in a small
vector loop.
673         This method is intended for grid diagnostics and does
not alter the
674         scalar oracle used by the optimiser.
675         """
676
677         array = np.asarray(points, dtype=float)
678         if array.ndim == 1:
679             if array.shape != (2,):
680                 raise ValueError("points must have shape (n,
2)")
681             array = array.reshape(1, 2)
682         if array.ndim != 2 or array.shape[1] != 2:
683             raise ValueError("points must have shape (n, 2)")
684         result = np.full(len(array), math.inf, dtype=float)

```

```

685         valid = np.all(np.isfinite(array), axis=1)
686         if not np.any(valid):
687             return result
688         query = array[valid]
689         radius_sq = self.config.min_receive_radius**2
690         inner = self._reception_static_inner
691         outer = self._reception_static_outer
692         #  $||P-G||^2 - r(G)^2 = ||P||^2 - 2 P \cdot G + ||G||^2 - r(G)^2$ .
693         query_sq = np.einsum("ij,ij->i", query, query)
694         inner_sq_minus_r = np.einsum("ij,ij->i", inner, inner)
        - radius_sq
695         outer_radius_sq = np.einsum(
696             "ij,ij->i", outer - self.s1[None, :], outer - self
        .s1[None, :])
697     )
698     outer_sq_minus_r = np.einsum("ij,ij->i", outer, outer)
        - outer_radius_sq
699     inner_values = query_sq[:, None] - 2.0 * (query @
inner.T) + inner_sq_minus_r[None, :]
700     outer_values = query_sq[:, None] - 2.0 * (query @
outer.T) + outer_sq_minus_r[None, :]
701     best = np.maximum(np.max(inner_values, axis=1), np.max
(outer_values, axis=1))
702
703     directions = query - self.s1[None, :]
704     norms = np.linalg.norm(directions, axis=1)
705     support_valid = norms > 1e-12
706     if np.any(support_valid):
707         support = np.zeros_like(query)
708         support[support_valid] = self.s1[None, :] - self.
config.min_receive_radius * directions[support_valid] /
norms[support_valid, None]
709         inside = np.zeros(len(query), dtype=bool)
710         for index, candidate in enumerate(support):
711             if support_valid[index]:
712                 inside[index] = _point_inside(self.region,
candidate, 2e-8)
713         if np.any(inside):
714             delta = query[inside] - support[inside]
715             support_values = np.einsum("ij,ij->i", delta,
delta) - radius_sq
716             best[inside] = np.maximum(best[inside],
support_values)
717         result[valid] = best
718         return result
719
720     def reception_h_tolerance(self) -> float:
721         """Return the numerical H tolerance used for
feasibility acceptance.
722
723         'H' is a squared-distance residual (m^2), while the
public
724         configuration keeps the more interpretable distance-
scale
725         parameter
        'feasibility_tolerance_m'. The search radius
converts that parameter

```

```

726         to the residual scale used by the optimizer and grid
classifier.
727         """
728
729         return float(self.config.feasibility_tolerance_m *
self.search_radius)
730
731     def reception_status(self, point: Sequence[float] | np.
ndarray) -> dict[str, Any]:
732         """Return strict and tolerance-aware reception
feasibility flags."""
733
734         h_value = float(self.receive_h(point))
735         finite = math.isfinite(h_value)
736         h_tolerance = self.reception_h_tolerance()
737         return {
738             "h": h_value,
739             "h_tolerance": h_tolerance,
740             "strict_feasible": bool(finite and h_value <= 0.0)
741         },
742         {
743             "feasible_with_tolerance": bool(finite and h_value
<= h_tolerance),
744         }
745
746     def _line_intersects_target(self, point: np.ndarray) ->
bool:
747         """Check the DOP singularity condition for the line S1
--P."""
748
749         direction = point - self.s1
750         norm = float(np.linalg.norm(direction))
751         if norm <= 1e-10:
752             return True
753         direction /= norm
754         lower, upper = -math.inf, math.inf
755         scale = max(1.0, float(np.max(np.abs(self.region.b))))
756         coefficient_tolerance = 1e-13
757         for normal, bound in zip(self.region.a, self.region.b)
758         :
759             coefficient = float(normal @ direction)
760             residual = float(bound - normal @ self.s1)
761             if abs(coefficient) <= coefficient_tolerance:
762                 if residual < -1e-9 * scale:
763                     return False
764                 continue
765             candidate = residual / coefficient
766             if coefficient > 0:
767                 upper = min(upper, candidate)
768             else:
769                 lower = max(lower, candidate)
770         if lower > upper + 1e-10:
771             return False
772         if lower < -1e-8 or upper > 1e-8 or upper - lower > 1e
-8:
773             return True
774         # The line only touches the polygon at S1. A

```

```

direction in the local
772     # tangent cone still creates a logarithmic DOP
singularity at the
773     # wedge apex, so reject it explicitly.
774     active = [
775         normal
776         for normal, bound in zip(self.region.a, self.
region.b)
777         if abs(float(bound - normal @ self.s1)) <= 1e-7 *
scale
778     ]
779     if not active:
780         return True
781     for signed_direction in (direction, -direction):
782         if all(float(normal @ signed_direction) <= 1e-9
for normal in active):
783             return True
784     return False

785
786     def geometry_is_finite(self, point: Sequence[float] | np.
ndarray) -> bool:
787         point_array = np.asarray(point, dtype=float)
788         if point_array.shape != (2,) or not np.all(np.isfinite
(point_array)):
789             return False
790         return not self._line_intersects_target(point_array)
791
792     def objective(
793         self,
794         point: Sequence[float] | np.ndarray,
795         epsilon_w: float | None = None,
796         p_w: float | None = None,
797         quadrature_order: int | None = None,
798     ) -> float:
799         """Evaluate the weighted position-error DOP J(P)."""
800
801         point_array = np.asarray(point, dtype=float)
802         if point_array.shape != (2,) or not np.all(np.isfinite
(point_array)):
803             return math.inf
804         if not self.geometry_is_finite(point_array):
805             return math.inf
806         order = self.quadrature_order if quadrature_order is
None else _finite_int(
807             quadrature_order, "quadrature_order"
808         )
809         data = self._quadrature_data(order)
810         nodes = data["nodes"]
811         quad_weights = data["quad_weights"]
812         r1_vectors = data["r1_vectors"]
813         r1_sq = data["r1_sq"]
814         r1 = data["r1"]
815         vectors_2 = nodes - point_array[None, :]
816         r2_sq = np.einsum("ij,ij->i", vectors_2, vectors_2)
817         r2 = np.sqrt(r2_sq)
818         determinants = np.abs(

```

```

819         r1_vectors[:, 0] * vectors_2[:, 1]
820         - r1_vectors[:, 1] * vectors_2[:, 0]
821     )
822     denominator = r1 * r2
823     with np.errstate(divide="ignore", invalid="ignore",
over="ignore"):
824         sine = np.divide(determinants, denominator, out=np
.zeros_like(denominator), where=denominator > 0)
825         if np.any(sine <= self.config.angle_sin_tolerance)
:
826             return math.inf
827         dop = r1 * r2 * np.sqrt(r1_sq + r2_sq) /
determinants
828         if not np.all(np.isfinite(dop)):
829             return math.inf
830         weights = self._weights(epsilon_w, p_w, data["
depth_quantiles"])
831         normalization = float(np.sum(quad_weights * weights))
832         if normalization <= 0:
833             return math.inf
834         value = float(np.sum(quad_weights * weights * dop) /
normalization)
835         return value if math.isfinite(value) else math.inf
836
837     def objective_many(
838         self,
839         points: Sequence[Sequence[float]] | np.ndarray,
840         epsilon_w: float | None = None,
841         p_w: float | None = None,
842         quadrature_order: int | None = None,
843         *,
844         chunk_size: int = 512,
845     ) -> np.ndarray:
846         """Evaluate 'J(P)' for many detector points.
847
848         This is the batch counterpart of :meth:'objective',
849         used by
850         convergence tables and marching-squares extraction.
851         It preserves the
852         scalar method's explicit '+inf' treatment of
853         collinear points and
854         singular quadrature nodes. 'chunk_size' bounds
855         temporary memory for
856         high-resolution grids.
857         """
858
859         array = np.asarray(points, dtype=float)
860         if array.ndim == 1:
861             if array.shape != (2,):
862                 raise ValueError("points must have shape (n,
2)")
863             array = array.reshape(1, 2)
864         if array.ndim != 2 or array.shape[1] != 2:
865             raise ValueError("points must have shape (n, 2)")
866         try:
867             chunk = max(1, _finite_int(chunk_size, "chunk_size

```

```

864     "))
865     except ValueError as exc:
866         raise ValueError("chunk_size must be a positive
integer") from exc
867     result = np.full(len(array), math.inf, dtype=float)
868     if len(array) == 0:
869         return result
870     order = self.quadrature_order if quadrature_order is
None else _finite_int(
871         quadrature_order, "quadrature_order"
872     )
873     data = self._quadrature_data(order)
874     nodes = data["nodes"]
875     quad_weights = data["quad_weights"]
876     r1_vectors = data["r1_vectors"]
877     r1_sq = data["r1_sq"]
878     r1 = data["r1"]
879     weights = self._weights(epsilon_w, p_w, data["
depth_quantiles"])
880     normalization = float(np.sum(quad_weights * weights))
881     if normalization <= 0 or not math.isfinite(
normalization):
882         return result
883     valid_finite = np.all(np.isfinite(array), axis=1)
884     if not np.any(valid_finite):
885         return result
886     # The line/polygon test is intentionally kept scalar:
it is a cheap
887     # robust half-plane interval calculation and avoids
changing the
888     # singularity semantics of the public objective.
889     valid_geometry = np.zeros(len(array), dtype=bool)
890     for index in np.flatnonzero(valid_finite):
891         valid_geometry[index] = self.geometry_is_finite(
array[index])
892     valid_indices = np.flatnonzero(valid_geometry)
893     if not len(valid_indices):
894         return result
895     for start in range(0, len(valid_indices), chunk):
896         indices = valid_indices[start : start + chunk]
897         query = array[indices]
898         vectors_2 = nodes[None, :, :] - query[:, None, :]
899         r2_sq = np.einsum("mij,mij->mi", vectors_2,
vectors_2)
900         r2 = np.sqrt(r2_sq)
901         determinants = np.abs(
902             r1_vectors[None, :, 0] * vectors_2[:, :, 1]
903             - r1_vectors[None, :, 1] * vectors_2[:, :, 0]
904         )
905         denominator = r1[None, :] * r2
906         with np.errstate(divide="ignore", invalid="ignore"
, over="ignore"):
907             sine = np.divide(
908                 determinants,
909                 denominator,
910                 out=np.zeros_like(denominator),

```

```

910         where=denominator > 0,
911     )
912     finite_rows = np.all(
913         np.isfinite(sine) & (sine > self.config.
angle_sin_tolerance),
914         axis=1,
915     )
916     dop = np.divide(
917         r1[None, :] * r2 * np.sqrt(r1_sq[None, :]
+ r2_sq),
918         determinants,
919         out=np.full_like(determinants, math.inf),
920         where=determinants > 0,
921     )
922     finite_rows &= np.all(np.isfinite(dop), axis=1)
923     if np.any(finite_rows):
924         numerators = np.sum(dop[finite_rows] * (
quad_weights * weights)[None, :], axis=1)
925         values = numerators / normalization
926         good = np.isfinite(values)
927         result[indices[finite_rows][good]] = values[
good]
928     return result
929
930     def constrained_objective(self, point: Sequence[float] |
np.ndarray) -> float:
931         """Finite objective used by global and local numerical
optimizers.
932
933         The strict public objective intentionally returns '+'
inf'' at a
934         collinearity singularity. Derivative-based optimizers
cannot finite-
935         difference an infinite value, so this wrapper
substitutes a large
936         finite barrier while retaining the exact objective for
all valid
937         points.
938         """
939
940         value = self.objective(point)
941         violation = max(0.0, self.receive_h(point)) / max(1.0,
self.search_radius**2)
942         if not math.isfinite(value):
943             value = 1.0e12
944         return float(value + 2.0e7 * violation * violation +
2.0e5 * violation)
945
946     def _candidate_starts(self, rng: np.random.Generator) ->
list[np.ndarray]:
947         starts: list[np.ndarray] = [self.region.centroid.copy
(), self.region.incenter.copy()]
948         radii = (0.25, 0.5, 0.75, 1.0)
949         for fraction in radii:
950             for angle in np.linspace(0.0, 2.0 * math.pi, 16,
endpoint=False):

```

```

951         starts.append(
952             self.s1
953             + fraction
954             * self.search_radius
955             * np.asarray((math.cos(angle), math.sin(
angle))))
956     )
957     for _ in range(18):
958         starts.append(
959             np.asarray(
960                 [
961                     rng.uniform(*self.search_bounds[0]),
962                     rng.uniform(*self.search_bounds[1]),
963                 ]
964             )
965         )
966     clipped: list[np.ndarray] = []
967     for start in starts:
968         point = np.asarray(start, dtype=float)
969         point = np.clip(
970             point,
971             np.asarray([self.search_bounds[0][0], self.
search_bounds[1][0]]),
972             np.asarray([self.search_bounds[0][1], self.
search_bounds[1][1]]),
973         )
974         if not any(np.linalg.norm(point - previous) <= 1e
-9 for previous in clipped):
975             clipped.append(point)
976     return clipped
977
978     def optimize(
979         self,
980         *,
981         seed: int | None = None,
982         max_local_starts: int | None = None,
983         starts: Sequence[Sequence[float]] | None = None,
984     ) -> dict[str, Any]:
985         """Run global penalized search followed by constrained
local polishing.
986
987         ''seed'' and ''max_local_starts'' are optional
experiment controls;
988         omitting them retains the original deterministic
configuration. A
989         caller may also provide explicit ''starts'' to audit
separate
990         optimisation branches without changing the production
defaults.
991         """
992
993         run_seed = self.config.seed if seed is None else
_finite_int(seed, "seed")
994         rng = np.random.default_rng(run_seed)
995         candidates: list[tuple[float, np.ndarray, str]] = []
996         global_result: Any = None

```

```

997         if self.config.use_global_search:
998             global_result = differential_evolution(
999                 self.constrained_objective,
1000                 self.search_bounds,
1001                 seed=run_seed,
1002                 maxiter=self.config.global_maxiter,
1003                 popsize=self.config.global_popsize,
1004                 polish=False,
1005                 updating="immediate",
1006                 workers=1,
1007                 tol=1e-5,
1008             )
1009             global_point = np.asarray(global_result.x, dtype=
float)
1010             global_value = self.objective(global_point)
1011             global_h = self.receive_h(global_point)
1012             global_feasible = math.isfinite(global_value) and
(
1013                 global_h <= self.reception_h_tolerance()
1014             )
1015             if global_feasible:
1016                 candidates.append((global_value, global_point,
"global"))
1017                 if starts is None:
1018                     starts_list = self._candidate_starts(rng)
1019                 else:
1020                     starts_list = []
1021                     lower = np.asarray([self.search_bounds[0][0], self
.search_bounds[1][0]])
1022                     upper = np.asarray([self.search_bounds[0][1], self
.search_bounds[1][1]])
1023                     for start in starts:
1024                         point = np.asarray(start, dtype=float)
1025                         if point.shape != (2,) or not np.all(np.
isfinite(point)):
1026                             continue
1027                         starts_list.append(np.clip(point, lower, upper
))
1028                     if not starts_list:
1029                         starts_list = self._candidate_starts(rng)
1030                 if max_local_starts is not None:
1031                     requested = _finite_int(max_local_starts, "
max_local_starts")
1032                     if requested < 1:
1033                         raise ValueError("max_local_starts must be
positive")
1034                     # Preserve the centroid, incenter, and a balanced
angular subset
1035                     # before filling the remaining slots. This keeps
both symmetric
1036                     # branches represented in low-cost convergence
audits.
1037                     if len(starts_list) > requested:
1038                         keep: list[np.ndarray] = []
1039                         preferred = starts_list[:2]
1040                         keep.extend(preferred[:requested])

```

```

1041         remaining = starts_list[2:]
1042         if requested > len(keep):
1043             stride = max(1, len(remaining) // max(1,
requested - len(keep)))
1044             for candidate in remaining[::stride]:
1045                 if len(keep) >= requested:
1046                     break
1047                 keep.append(candidate)
1048                 starts_list = keep[:requested]
1049             starts = starts_list
1050             if global_result is not None:
1051                 starts.insert(0, np.asarray(global_result.x, dtype
=float))
1052             scale = max(1.0, self.search_radius**2)
1053
1054             def feasibility_constraint(point: np.ndarray) -> float
:
1055                 return -self.receive_h(point) / scale
1056
1057             local_results: list[dict[str, Any]] = []
1058             for start in starts:
1059                 result = minimize(
1060                     self.constrained_objective,
1061                     np.asarray(start, dtype=float),
1062                     method="SLSQP",
1063                     bounds=self.search_bounds,
1064                     constraints=[{"type": "ineq", "fun":
feasibility_constraint}],
1065                     options={"maxiter": self.config.local_maxiter,
"ftol": 1e-7, "disp": False},
1066                 )
1067                 point = np.asarray(result.x, dtype=float)
1068                 value = self.objective(point)
1069                 h_value = self.receive_h(point)
1070                 feasible = math.isfinite(value) and h_value <=
self.reception_h_tolerance()
1071                 local_results.append(
1072                     {
1073                         "success": bool(result.success),
1074                         "message": str(result.message),
1075                         "point": point,
1076                         "objective": value,
1077                         "objective_with_penalty": float(self.
constrained_objective(point)),
1078                         "reception_h": h_value,
1079                         "reception_h_tolerance": self.
reception_h_tolerance(),
1080                         "strict_feasible": bool(math.isfinite(
h_value) and h_value <= 0.0),
1081                         "feasible_with_tolerance": bool(
1082                             math.isfinite(h_value)
1083                             and h_value <= self.
reception_h_tolerance()
1084                             ),
1085                         "feasible": feasible,
1086                         "iterations": int(getattr(result, "nit",

```

```

0) or 0),
    "function_evaluations": int(getattr(result
, "nfev", 0) or 0),
    }
    )
    if feasible:
        candidates.append((value, point, "local"))
    if not any(
        math.isfinite(value)
        and self.receive_h(point)
        <= self.reception_h_tolerance()
        for value, point, _ in candidates
    ):
        # A finite point can be hard for SLSQP to reach if
        # most starts lie
        # on a bearing ray. Probe directions outside that
        # ray from S1;
        # S1 itself is reception-feasible but
        # geometrically singular.
        fallback_angles = [
            self.bearing_deg + 90.0,
            self.bearing_deg - 90.0,
            self.bearing_deg + 45.0,
            self.bearing_deg - 45.0,
        ]
        fallback_fractions = (1e-4, 1e-3, 1e-2, 5e-2, 0.1,
                               0.25, 0.5, 0.75, 1.0)
        for angle_deg in fallback_angles:
            direction = np.asarray(
                (
                    math.cos(math.radians(angle_deg)),
                    math.sin(math.radians(angle_deg)),
                ),
                dtype=float,
            )
            for fraction in fallback_fractions:
                raw_point = self.s1 + fraction * self.
search_radius * direction
                point = np.clip(
                    raw_point,
                    np.asarray([self.search_bounds[0][0],
self.search_bounds[1][0]]),
                    np.asarray([self.search_bounds[0][1],
self.search_bounds[1][1]]),
                )
                value = self.objective(point)
                h_value = self.receive_h(point)
                if (
                    math.isfinite(value)
                    and h_value <= self.
reception_h_tolerance()
                ):
                    candidates.append((value, point, "
direction-fallback"))
                    break
            if any(

```

```

1132         math.isfinite(value)
1133         and self.receive_h(candidate)
1134         <= self.reception_h_tolerance()
1135         for value, candidate, _ in candidates
1136     ):
1137         break
1138     feasible_candidates = [
1139         item
1140         for item in candidates
1141         if math.isfinite(item[0]) and self.receive_h(item
1142 [1]) <= self.reception_h_tolerance()
1143     ]
1144     if not feasible_candidates:
1145         raise RuntimeError(
1146             "no finite reception-feasible second detector
1147 was found; "
1148             "increase circle_sides/search iterations or
1149 inspect the input bearing"
1150         )
1151     best_value, best_point, source = min(
1152 feasible_candidates, key=lambda item: item[0])
1153     best_point = np.asarray(best_point, dtype=float)
1154     selected_local_run: dict[str, Any] | None = None
1155     if source == "local":
1156         matching_runs = [
1157             run
1158             for run in local_results
1159             if math.isfinite(float(run["objective"]))
1160             and np.linalg.norm(np.asarray(run["point"],
1161 dtype=float) - best_point) <= 1e-7
1162             and abs(float(run["objective"]) - best_value)
1163 <= 1e-7
1164         ]
1165     if matching_runs:
1166         selected_local_run = matching_runs[0]
1167         bearing_from_s1 = math.degrees(math.atan2(best_point
1168 [1] - self.s1[1], best_point[0] - self.s1[0])) % 360.0
1169         bearing_difference = ((bearing_from_s1 - self.
1170 bearing_deg + 180.0) % 360.0) - 180.0
1171     return {
1172         "point": best_point,
1173         "j_star": float(best_value),
1174         "reception_h": float(self.receive_h(best_point)),
1175         "reception_h_tolerance": self.
1176 reception_h_tolerance(),
1177         "strict_feasible": bool(
1178             math.isfinite(self.receive_h(best_point))
1179             and self.receive_h(best_point) <= 0.0
1180         ),
1181         "feasible_with_tolerance": bool(
1182             math.isfinite(self.receive_h(best_point))
1183             and self.receive_h(best_point) <= self.
1184 reception_h_tolerance()
1185         ),
1186         "distance_from_s1": float(np.linalg.norm(
1187 best_point - self.s1)),

```

```

1177         "bearing_from_s1_deg": float(bearing_from_s1),
1178         "bearing_difference_deg": float(bearing_difference
),
1179         "source": source,
1180         "iterations": None
1181         if selected_local_run is None
1182         else int(selected_local_run["iterations"]),
1183         "function_evaluations": None
1184         if selected_local_run is None
1185         else int(selected_local_run["function_evaluations"
]),
1186         "local_run_count": int(len(local_results)),
1187         "global": None
1188         if global_result is None
1189         else {
1190             "objective_with_penalty": float(global_result.
fun),
1191             "objective": float(self.objective(
global_result.x)),
1192             "reception_h": float(self.receive_h(
global_result.x)),
1193             "reception_h_tolerance": self.
reception_h_tolerance(),
1194             "strict_feasible": bool(
1195                 math.isfinite(self.receive_h(global_result
.x))
1196                 and self.receive_h(global_result.x) <= 0.0
1197             ),
1198             "feasible_with_tolerance": bool(
1199                 math.isfinite(self.receive_h(global_result
.x))
1200                 and self.receive_h(global_result.x) <=
self.reception_h_tolerance()
1201             ),
1202             "feasible": bool(
1203                 math.isfinite(self.objective(global_result
.x))
1204                 and self.receive_h(global_result.x) <=
self.reception_h_tolerance()
1205             ),
1206             "point": [float(value) for value in
global_result.x],
1207             "iterations": int(global_result.nit),
1208             "function_evaluations": int(global_result.nfev
),
1209         },
1210         "local_runs": local_results,
1211     }
1212
1213     def candidate_region(
1214         self,
1215         optimum: dict[str, Any],
1216         tau: float | None = None,
1217         grid_size: int | None = None,
1218         include_values: bool = False,
1219     ) -> dict[str, Any]:

```

```

1220     """Approximate the continuous near-optimal level set
1221     on a regular grid."""
1222     tau_value = self.config.tau if tau is None else
1223     _finite_float(tau, "tau")
1224     if tau_value < 0:
1225         raise ValueError("tau must be non-negative")
1226     count = (
1227         self.config.candidate_grid
1228         if grid_size is None
1229         else _finite_int(grid_size, "grid_size")
1230     )
1231     if count < 11:
1232         raise ValueError("candidate grid must contain at
1233         least 11 points per axis")
1234     try:
1235         optimum_point = np.asarray(optimum["point"], dtype
1236         =float)
1237         optimum_j = _finite_float(optimum["j_star"], "
1238         optimum.j_star")
1239     except (KeyError, TypeError, ValueError) as exc:
1240         raise ValueError("optimum must contain a finite
1241         point and j_star") from exc
1242     if optimum_point.shape != (2,) or not np.all(np.
1243     isfinite(optimum_point)):
1244         raise ValueError("optimum.point must be a finite
1245         two-dimensional point")
1246     threshold = (1.0 + tau_value) * optimum_j
1247     x_edges = np.linspace(self.search_bounds[0][0], self.
1248     search_bounds[0][1], count + 1)
1249     y_edges = np.linspace(self.search_bounds[1][0], self.
1250     search_bounds[1][1], count + 1)
1251     x_values = (x_edges[:-1] + x_edges[1:]) / 2.0
1252     y_values = (y_edges[:-1] + y_edges[1:]) / 2.0
1253     mask = np.zeros((count, count), dtype=bool)
1254     values = np.full((count, count), np.nan, dtype=float)
1255     for iy, y_value in enumerate(y_values):
1256         for ix, x_value in enumerate(x_values):
1257             point = np.asarray((x_value, y_value), dtype=
1258             float)
1259             if self.receive_h(point) > self.
1260             reception_h_tolerance():
1261                 continue
1262             if not self.geometry_is_finite(point):
1263                 continue
1264             value = self.objective(point)
1265             if not math.isfinite(value):
1266                 continue
1267             values[iy, ix] = value
1268             mask[iy, ix] = value <= threshold
1269     step_x = float(x_edges[1] - x_edges[0])
1270     step_y = float(y_edges[1] - y_edges[0])
1271     area_per_cell = abs(step_x * step_y)
1272     components = _connected_components(mask)
1273     component_stats: list[dict[str, Any]] = []
1274     accepted_indices = np.argwhere(mask)

```

```

1264         for component in components:
1265             indices = np.asarray(component, dtype=int)
1266             yy = indices[:, 0]
1267             xx = indices[:, 1]
1268             points = np.column_stack((x_values[xx], y_values[
1269 yy]))
1269             distances = np.linalg.norm(points - self.s1[None,
1270 :], axis=1)
1270             component_stats.append(
1271                 {
1272                     "cell_count": int(len(indices)),
1273                     "area": float(len(indices) * area_per_cell
1274 ),
1274                     "bbox": {
1275                         "x_min": float(np.min(x_values[xx])),
1276                         "x_max": float(np.max(x_values[xx])),
1277                         "y_min": float(np.min(y_values[yy])),
1278                         "y_max": float(np.max(y_values[yy])),
1279                     },
1280                     "distance_from_s1_min": float(np.min(
1281 distances)),
1281                     "distance_from_s1_max": float(np.max(
1282 distances)),
1282                 }
1283             )
1284             if len(accepted_indices):
1285                 accepted_points = np.column_stack(
1286                     (x_values[accepted_indices[:, 1]], y_values[
1287 accepted_indices[:, 0]]))
1287                 distances = np.linalg.norm(accepted_points - self.
1288 s1[None, :], axis=1)
1289                 distance_min = float(np.min(distances))
1290                 distance_max = float(np.max(distances))
1291             else:
1292                 # An empty level-set estimate has no meaningful
1293 distance range.
1293                 # 'None' keeps the result JSON standards-
1294 compliant while
1294                 # preserving the distinction from a finite zero
1295 distance.
1295                 distance_min = distance_max = None
1296                 result: dict[str, Any] = {
1297                     "tau": tau_value,
1298                     "threshold": float(threshold),
1299                     "grid_shape": [count, count],
1300                     "x_min": float(x_edges[0]),
1301                     "x_max": float(x_edges[-1]),
1302                     "y_min": float(y_edges[0]),
1303                     "y_max": float(y_edges[-1]),
1304                     "x_step": step_x,
1305                     "y_step": step_y,
1306                     "area": float(len(accepted_indices) *
1307 area_per_cell),
1307                     "distance_from_s1_min": distance_min,
1308                     "distance_from_s1_max": distance_max,

```

```

1309         "component_count": len(component_stats),
1310         "components": component_stats,
1311         "sample_x": x_values.astype(float).tolist(),
1312         "sample_y": y_values.astype(float).tolist(),
1313         "optimum_point": [float(value) for value in
optimum_point],
1314         "optimum_j": float(optimum_j),
1315         "reception_h_tolerance": float(self.
reception_h_tolerance()),
1316         "feasibility_rule": (
1317             "A grid cell center is accepted only when  $H(P)$ 
<= "
1318             "reception_h_tolerance, the objective is
finite, and the "
1319             "S1--P line does not intersect the target
polygon."
1320         ),
1321         # Row iy corresponds to the cell centered at
sample_y[iy], and
1322         # each value is a 0/1 cell-center classification.
This is
1323         # intentionally a grid approximation of the
continuous level set;
1324         # the optimum is reported separately and is never
injected into
1325         # an infeasible or singular grid cell.
1326         "mask": mask.astype(np.uint8).tolist(),
1327     }
1328     if include_values:
1329         result["objective_grid"] = [
1330             None if not math.isfinite(float(value)) else
float(value) for value in row]
1331             for row in values
1332         ]
1333     return result
1334
1335     def quadrature_check(self, point: Sequence[float],
high_order: int | None = None) -> dict[str, Any]:
1336         order = (
1337             self.config.validation_quadrature_order
1338             if high_order is None
1339             else _finite_int(high_order, "
validation_quadrature_order")
1340         )
1341         if order < 2:
1342             raise ValueError("validation quadrature order must
be at least 2")
1343         high_model = Question2Model(
1344             self.s1,
1345             self.bearing_deg,
1346             self.config,
1347             quadrature_order=order,
1348         )
1349         low_value = self.objective(point)
1350         high_value = high_model.objective(point)
1351         return {

```

```

1352         "order_low": int(self.quadrature_order),
1353         "order_high": int(order),
1354         "j_low": float(low_value),
1355         "j_high": float(high_value),
1356         "absolute_difference": float(abs(low_value -
high_value)),
1357         "relative_difference": float(abs(low_value -
high_value) / max(1.0, abs(high_value))),
1358     }
1359
1360     def sensitivity(
1361         self,
1362         grid_size: int | None = None,
1363         *,
1364         use_global_search: bool | None = None,
1365         include_candidate_grids: bool = False,
1366     ) -> dict[str, Any]:
1367         """Run the documented epsilon/power/tau sensitivity
sweep.
1368
1369         Every weight pair is re-optimised. Candidate-region
statistics are
1370         reported for every 'tau' and remain explicitly
labelled as regular
1371         grid approximations. 'use_global_search' is an opt-
in override for
1372         the per-sweep optimisation configuration; when omitted
, the model's
1373         configured value is preserved.
1374         """
1375
1376         rows: list[dict[str, Any]] = []
1377         compact_grid = grid_size or min(self.config.
candidate_grid, 61)
1378         compact_grid = _finite_int(compact_grid, "grid_size")
1379         if compact_grid < 11:
1380             raise ValueError("sensitivity grid must contain at
least 11 points per axis")
1381         if use_global_search is None:
1382             sweep_use_global = bool(self.config.
use_global_search)
1383         elif isinstance(use_global_search, (bool, np.bool_)):
1384             sweep_use_global = bool(use_global_search)
1385         else:
1386             raise ValueError("use_global_search must be
boolean when supplied")
1387         reference_eps, reference_power = self.config.epsilon_w
, self.config.p_w
1388         for epsilon in self.config.sensitivity_epsilons:
1389             for power in self.config.sensitivity_powers:
1390                 local_config = Q2Config(**asdict(self.config))
1391                 local_config.epsilon_w = float(epsilon)
1392                 local_config.p_w = float(power)
1393                 local_config.use_global_search =
sweep_use_global
1394                 local_model = Question2Model(

```

```

1395         self.s1,
1396         self.bearing_deg,
1397         local_config,
1398         quadrature_order=self.config.
quadrature_order,
1399     )
1400     optimum = local_model.optimize()
1401     point = np.asarray(optimum["point"], dtype=
float)
1402     reference_j = self.objective(point,
reference_eps, reference_power)
1403     candidate_by_tau: list[dict[str, Any]] = []
1404     areas: dict[str, float] = {}
1405     # Candidate masks are computed with the same
continuous model
1406     # and only a smaller reporting grid to keep
the sweep tractable.
1407     for tau in self.config.sensitivity_taus:
1408         region = local_model.candidate_region(
optimum, tau=tau, grid_size=compact_grid)
1409         areas[f"{float(tau):g}"] = float(region["
area"])
1410         accepted_cell_count = int(np.sum(np.
asarray(region["mask"], dtype=np.uint8)))
1411         candidate_summary: dict[str, Any] = {
1412             "tau": float(region["tau"]),
1413             "threshold": float(region["threshold"
]),
1414             "area": float(region["area"]),
1415             "accepted_cell_count":
accepted_cell_count,
1416             "grid_shape": list(region["grid_shape"
]),
1417             "x_step": float(region["x_step"]),
1418             "y_step": float(region["y_step"]),
1419             # An empty grid estimate has no
distance interval;
1420             # preserve 'None' so JSON stays
standards-compliant
1421             # and downstream plots can distinguish
it from zero.
1422             "distance_from_s1_min": region["
distance_from_s1_min"],
1423             "distance_from_s1_max": region["
distance_from_s1_max"],
1424             "component_count": int(region["
component_count"]),
1425             "components": region["components"],
1426         }
1427         if include_candidate_grids:
1428             candidate_summary.update(
1429                 {
1430                     "x_min": float(region["x_min"
]),
1431                     "x_max": float(region["x_max"
]),

```

```

1432         "y_min": float(region["y_min"
1433     ]),
1434     "y_max": float(region["y_max"
1435 ]),
1436     "sample_x": region["sample_x"
1437 ],
1438     "sample_y": region["sample_y"
1439 ],
1440     "mask": region["mask"],
1441     }
1442     )
1443     candidate_by_tau.append(candidate_summary)
1444     local_runs = optimum.get("local_runs", [])
1445     local_success_count = sum(bool(run.get("
1446 success")) for run in local_runs)
1447     local_feasible_count = sum(bool(run.get("
1448 feasible")) for run in local_runs)
1449     global_summary = optimum.get("global")
1450     rows.append(
1451     {
1452         "epsilon_w": float(epsilon),
1453         "p_w": float(power),
1454         "point": [float(point[0]), float(point
1455 [1])],
1456         "j_star": float(optimum["j_star"]),
1457         "j_under_reference_weight": float(
1458 reference_j),
1459         "distance_from_s1": float(np.linalg.
1460 norm(point - self.s1)),
1461         "bearing_from_s1_deg": float(optimum["
1462 bearing_from_s1_deg"]),
1463         "bearing_difference_deg": float(
1464 optimum["bearing_difference_deg"]),
1465         "reception_h": float(optimum["
1466 reception_h"]),
1467         "reception_h_tolerance": float(optimum
1468 ["reception_h_tolerance"]),
1469         "strict_feasible": bool(optimum["
1470 strict_feasible"]),
1471         "feasible_with_tolerance": bool(
1472 optimum["feasible_with_tolerance"]),
1473         "optimization_source": str(optimum["
1474 source"]),
1475         "use_global_search": sweep_use_global,
1476         "selected_iterations": optimum["
1477 iterations"],
1478         "selected_function_evaluations":
1479 optimum["function_evaluations"],
1480         "local_run_count": int(optimum["
1481 local_run_count"]),
1482         "local_success_count": int(
1483 local_success_count),
1484         "local_feasible_count": int(
1485 local_feasible_count),
1486         "global": global_summary,
1487         "candidate_by_tau": candidate_by_tau,

```

```

1467         "candidate_area_by_tau": areas,
1468     }
1469 )
1470 return {
1471     "reference_weight": {
1472         "epsilon_w": float(reference_eps),
1473         "p_w": float(reference_power),
1474     },
1475     "epsilon_values": [float(value) for value in self.
config.sensitivity_epsilons],
1476     "p_values": [float(value) for value in self.config
.sensitivity_powers],
1477     "tau_values": [float(value) for value in self.
config.sensitivity_taus],
1478     "grid_size": int(compact_grid),
1479     "use_global_search": sweep_use_global,
1480     "grid_approximation": (
1481         "Each candidate area, distance range and
component count is "
1482         "computed from cell centres on the reported
regular grid; it "
1483         "is not a strict continuous-region measure."
1484     ),
1485     "rows": rows,
1486 }
1487
1488
1489 def _connected_components(mask: np.ndarray) -> list[list[tuple
[int, int]]]:
1490     height, width = mask.shape
1491     visited = np.zeros_like(mask, dtype=bool)
1492     components: list[list[tuple[int, int]]] = []
1493     for iy in range(height):
1494         for ix in range(width):
1495             if not mask[iy, ix] or visited[iy, ix]:
1496                 continue
1497             stack = [(iy, ix)]
1498             visited[iy, ix] = True
1499             component: list[tuple[int, int]] = []
1500             while stack:
1501                 cy, cx = stack.pop()
1502                 component.append((cy, cx))
1503                 for dy, dx in ((-1, 0), (1, 0), (0, -1), (0,
1504 1)):
1505                     ny, nx = cy + dy, cx + dx
1506                     if 0 <= ny < height and 0 <= nx < width
and mask[ny, nx] and not visited[ny, nx]:
1507                         visited[ny, nx] = True
1508                         stack.append((ny, nx))
1509             components.append(component)
1510     return components
1511
1512 def parse_input(source: Any) -> tuple[np.ndarray, float,
Q2Config]:
1513     """Parse JSON-like Question 2 input and return '(S1,

```

```

1514     bearing, config)'. """
1515     if not isinstance(source, dict):
1516         raise ValueError("Question 2 input must be a JSON
object")
1517     if "s1" in source:
1518         s1 = np.asarray(_as_point(source["s1"], "s1"), dtype=
float)
1519     elif "detector" in source:
1520         s1 = np.asarray(_as_point(source["detector"], "
detector"), dtype=float)
1521     elif "x1" in source and "y1" in source:
1522         s1 = np.asarray(
1523             (_finite_float(source["x1"], "x1"), _finite_float(
source["y1"], "y1")),
1524             dtype=float,
1525         )
1526     else:
1527         raise ValueError("input must contain s1/detector or x1
and y1")
1528     bearing_keys = ("svd_deg", "bearing_deg", "theta_deg", "
theta", "angle_deg")
1529     bearing_key = next((key for key in bearing_keys if key in
source), None)
1530     if bearing_key is None:
1531         raise ValueError("input must contain svd_deg or
bearing_deg")
1532     bearing = _finite_float(source[bearing_key], bearing_key)
% 360.0
1533     config = Q2Config()
1534     # Keep the JSON names identical to the dataclass fields.
In particular,
1535     # expose optimizer, numerical-protection, and sensitivity
parameters so a
1536     # paper run can be reproduced without editing the solver
source.
1537     config_fields = set(asdict(config))
1538     for source_key in config_fields:
1539         if source_key in source:
1540             setattr(config, source_key, source[source_key])
1541     config.validate()
1542     return s1, bearing, config
1543
1544
1545 def load_input(path: str | Path) -> tuple[np.ndarray, float,
Q2Config]:
1546     with Path(path).open("r", encoding="utf-8") as handle:
1547         return parse_input(json.load(handle))
1548
1549
1550 def solve_question2(
1551     s1: Sequence[float] | np.ndarray,
1552     bearing_deg: float,
1553     config: Q2Config | None = None,
1554     *,
1555     run_sensitivity: bool = False,

```

```

1556     include_grid_values: bool = False,
1557 ) -> dict[str, Any]:
1558     """Solve Question 2 and return a JSON-serialisable result
1559     dictionary."""
1560     config = config or Q2Config()
1561     config.validate()
1562     model = Question2Model(s1, bearing_deg, config)
1563     optimum = model.optimize()
1564     candidate = model.candidate_region(
1565         optimum,
1566         tau=config.tau,
1567         grid_size=config.candidate_grid,
1568         include_values=include_grid_values,
1569     )
1570     quadrature = model.quadrature_check(optimum["point"])
1571     result: dict[str, Any] = {
1572         "model": "question_2_continuous_robust_dop",
1573         "input": {
1574             "s1": [float(model.s1[0]), float(model.s1[1])],
1575             "svd_deg": float(model.bearing_deg),
1576         },
1577         "config": {
1578             key: (list(value) if isinstance(value, tuple) else
1579 value)
1580             for key, value in asdict(config).items()
1581         },
1582         "target_region": {
1583             "vertices": model.region.vertices.astype(float).
1584 tolist(),
1585             "halfplane_count": int(len(model.region.a)),
1586             "area": float(model.region.area),
1587             "centroid": model.region.centroid.astype(float).
1588 tolist(),
1589             "incenter": model.region.incenter.astype(float).
1590 tolist(),
1591             "inradius": float(model.region.d_star),
1592             "circle_approximation": "external circumscribed
1593 regular polygons",
1594         },
1595         "reception_constraint": {
1596             "description": (
1597                 "H(P)=sup_G [|P-G|^2-max("
1598                 f"{config.min_receive_radius:g}^2,||G-S1||^2)]
1599                 <= 0"
1600             ),
1601             "min_receive_radius": float(config.
1602 min_receive_radius),
1603             "search_center": model.search_center.astype(float)
1604 .tolist(),
1605             "search_radius": float(model.search_radius),
1606             "search_bounds": [[float(low), float(high)] for
1607 low, high in model.search_bounds],
1608             "h_at_optimum": float(optimum["reception_h"]),
1609             "h_tolerance": float(model.reception_h_tolerance())
1610 ),

```

```

1601         "strict_feasible_at_optimum": bool(optimum["
strict_feasible"]),
1602         "feasible_with_tolerance_at_optimum": bool(
1603             optimum["feasible_with_tolerance"]
1604         ),
1605         "tolerance_semantics": (
1606             "feasibility_tolerance_m is converted to an H
residual tolerance "
1607             "by multiplying by search_radius; H itself has
squared-distance units"
1608         ),
1609     },
1610     "quadrature": {
1611         "order": int(config.quadrature_order),
1612         "node_count": int(len(model.nodes)),
1613         "integrated_area": float(model.quad_area),
1614         "target_polygon_area": float(model.region.area),
1615         "area_absolute_difference": float(abs(model.
quad_area - model.region.area)),
1616         "check_at_optimum": quadrature,
1617     },
1618     "optimum": {
1619         key: (
1620             [float(value[0]), float(value[1])]
1621             if key == "point"
1622             else value
1623         )
1624         for key, value in optimum.items()
1625         if key != "local_runs"
1626     },
1627     "candidate_region": candidate,
1628     "assumptions": [
1629         "The target and first-reception disks are replaced
by external circumscribed polygons; this keeps the feasible
target region conservative.",
1630         (
1631             "The first bearing error is the closed
interval "
1632             f"[svd-{config.angle_error_deg:g}°, "
1633             f"svd+{config.angle_error_deg:g}°]."
1634         ),
1635         (
1636             f"The {config.min_receive_radius:g} m lower
reception radius is "
1637             "used in the robust minimum-radius constraint
exactly as "
1638             "specified in the model document."
1639         ),
1640         "A detector position whose S1--P line meets the
target polygon is assigned J=+infinity to exclude the
logarithmic collinearity singularity.",
1641         "The reported candidate area is a regular-grid
approximation of the continuous near-optimal level set; its
threshold is continuous  $J(P) \leq (1+\tau)J_*$ .",
1642     ],
1643 }

```

```

1644     if run_sensitivity:
1645         result["sensitivity"] = model.sensitivity()
1646     return result
1647
1648
1649 def _demo_input() -> dict[str, Any]:
1650     return {
1651         "s1": {"x": 0.0, "y": 0.0},
1652         "svd_deg": 32.0,
1653         "angle_error_deg": 1.0,
1654         "epsilon_w": 0.2,
1655         "p_w": 2.0,
1656         "tau": 0.05,
1657     }
1658
1659
1660 def _write_json(path: str | Path, value: Any) -> None:
1661     output = Path(path)
1662     output.parent.mkdir(parents=True, exist_ok=True)
1663     output.write_text(
1664         json.dumps(value, ensure_ascii=False, indent=2,
1665 allow_nan=False) + "\n",
1666         encoding="utf-8",
1667     )
1668
1669
1670 def main(argv: Sequence[str] | None = None) -> int:
1671     parser = argparse.ArgumentParser(description=__doc__)
1672     parser.add_argument("--input", help="Question 2 JSON input path")
1673     parser.add_argument("--output", help="write result JSON to this path")
1674     parser.add_argument("--demo", action="store_true", help="run a deterministic demonstration case")
1675     parser.add_argument("--s1-x", type=float, help="override first detector x")
1676     parser.add_argument("--s1-y", type=float, help="override first detector y")
1677     parser.add_argument("--svd-deg", type=float, help="override first measured bearing")
1678     parser.add_argument("--target-radius", type=float)
1679     parser.add_argument("--min-receive-radius", type=float)
1680     parser.add_argument("--max-receive-radius", type=float)
1681     parser.add_argument("--angle-error-deg", type=float, help="bearing error half-width")
1682     parser.add_argument("--epsilon-w", type=float, help="boundary base weight")
1683     parser.add_argument("--p-w", type=float, help="boundary-depth power")
1684     parser.add_argument("--tau", type=float, help="relative near-optimal tolerance")
1685     parser.add_argument("--circle-sides", type=int, help="sides used for external circle polygons")
1686     parser.add_argument("--quadrature-order", type=int, help="Gauss-Duffy order per triangle axis")
1687     parser.add_argument("--validation-quadrature-order", type=

```

```

int)
1687     parser.add_argument("--candidate-grid", type=int, help="
candidate-region grid points per axis")
1688     parser.add_argument("--seed", type=int)
1689     parser.add_argument("--global-maxiter", type=int)
1690     parser.add_argument("--global-popsiz", type=int)
1691     parser.add_argument("--local-maxiter", type=int)
1692     parser.add_argument("--feasibility-tolerance-m", type=
float)
1693     parser.add_argument("--angle-sin-tolerance", type=float)
1694     parser.add_argument(
1695         "--global-search",
1696         dest="use_global_search",
1697         action="store_true",
1698         default=None,
1699         help="enable differential-evolution global search",
1700     )
1701     parser.add_argument(
1702         "--no-global",
1703         dest="use_global_search",
1704         action="store_false",
1705         help="skip differential-evolution global search",
1706     )
1707     parser.add_argument("--sensitivity", action="store_true",
help="run epsilon/power/tau sensitivity sweep")
1708     parser.add_argument("--include-grid-values", action="
store_true", help="include objective values for every grid
cell")
1709     args = parser.parse_args(argv)
1710     if args.demo:
1711         s1, bearing, config = parse_input(_demo_input())
1712     elif args.input:
1713         s1, bearing, config = load_input(args.input)
1714     else:
1715         parser.error("provide --input or --demo")
1716     overrides = {
1717         "target_radius": args.target_radius,
1718         "min_receive_radius": args.min_receive_radius,
1719         "max_receive_radius": args.max_receive_radius,
1720         "angle_error_deg": args.angle_error_deg,
1721         "epsilon_w": args.epsilon_w,
1722         "p_w": args.p_w,
1723         "tau": args.tau,
1724         "circle_sides": args.circle_sides,
1725         "quadrature_order": args.quadrature_order,
1726         "validation_quadrature_order": args.
validation_quadrature_order,
1727         "candidate_grid": args.candidate_grid,
1728         "seed": args.seed,
1729         "global_maxiter": args.global_maxiter,
1730         "global_popsiz": args.global_popsiz,
1731         "local_maxiter": args.local_maxiter,
1732         "feasibility_tolerance_m": args.
feasibility_tolerance_m,
1733         "angle_sin_tolerance": args.angle_sin_tolerance,
1734     }

```

```

1735     for name, value in overrides.items():
1736         if value is not None:
1737             setattr(config, name, value)
1738     if args.s1_x is not None:
1739         s1[0] = args.s1_x
1740     if args.s1_y is not None:
1741         s1[1] = args.s1_y
1742     if args.svd_deg is not None:
1743         bearing = args.svd_deg
1744     if args.use_global_search is not None:
1745         config.use_global_search = args.use_global_search
1746     config.validate()
1747     result = solve_question2(
1748         s1,
1749         bearing,
1750         config,
1751         run_sensitivity=args.sensitivity,
1752         include_grid_values=args.include_grid_values,
1753     )
1754     payload = json.dumps(result, ensure_ascii=False, indent=2)
1755     if args.output:
1756         _write_json(args.output, result)
1757     else:
1758         print(payload)
1759     return 0
1760
1761 if __name__ == "__main__":
1762     raise SystemExit(main())
1763

```

### Q3final.py

```

1  #!/usr/bin/env python3
2  """问题三中心点加8个外围点滚动路径顺路补测版V2，外围半径939
   米，Python 3.9+，依赖 numpy scipy。
3  安装：python -m pip install numpy scipy
4  官方模拟器：python rolling939v2.py --robot-id 参赛队号
5  本地自测：python rolling939v2.py --self-test 10
6  默认布局为原点加半径939米的正八边形，并保留V1的同点批量频道检
   测。为已有目标选择下一检测点时，优先考察当前点至滚动路线下一
   必经点的线段和该必经点，在交会信息损失可接受时采用零绕行补测
   点。
7  只适用于全向源。统计假设为位置均匀先验、固定接收半径均匀先验、
   不同测点高斯测角近似。
8  各频道存在性为独立伯努利近似(默认0.65)，未建模10~16总数的联合
   概率约束。
9  这些概率只用于选址；完成条件和清除认证始终采用几何条件。
10 不是官方模拟器，不读取官方隐藏数据。--self-test 仅为本地合成案
   例。
11 """
12 import numpy as np
13 from scipy.optimize import minimize, linprog
14 import argparse
15 import csv
16 import hashlib
17 import json

```

```

18 import math
19 import random
20 import time
21 import uuid
22 from pathlib import Path
23 from urllib.request import Request, build_opener, ProxyHandler
24 from urllib.error import HTTPError, URLError
25
26 DELTA = math.radians(1.006) # ±1度加两位小数取整裕量
27 N_CIRCLE = 96
28
29 def dist(a, b):
30     return math.hypot(a[0]-b[0], a[1]-b[1])
31
32 def clip(poly, a, b):
33     """裁剪  $a \cdot x \leq b$ , 返回凸多边形; 边界使用微小外放容差。"""
34     if not poly:
35         return []
36     b += 1e-8
37     out = []
38     prev = poly[-1]
39     fp = a[0]*prev[0]+a[1]*prev[1]-b
40     for cur in poly:
41         fc = a[0]*cur[0]+a[1]*cur[1]-b
42         if (fp <= 0) != (fc <= 0):
43             t = fp/(fp-fc)
44             out.append((prev[0]+t*(cur[0]-prev[0]), prev[1]+t
45 *(cur[1]-prev[1])))
46             if fc <= 0:
47                 out.append(cur)
48             prev, fp = cur, fc
49     return out
50
51 def disk_clip(poly, center, radius):
52     # 圆的切线半平面: 外接近似, 不能用内接多边形排除真实目标。
53     for k in range(N_CIRCLE):
54         t = 2*math.pi*k/N_CIRCLE
55         a = (math.cos(t), math.sin(t))
56         poly = clip(poly, a, radius+a[0]*center[0]+a[1]*center
57 [1])
58     return poly
59
60 def initial_poly():
61     return disk_clip([(-1800,-1800),(1800,-1800),(1800,1800)
62 ,(-1800,1800)], (0,0),1800)
63
64 def update_poly(poly, q, angle):
65     t = math.radians(angle)
66     for a in [(math.sin(t-DELTA),-math.cos(t-DELTA)),
67 (-math.sin(t+DELTA),math.cos(t+DELTA))]:
68         poly = clip(poly,a,a[0]*q[0]+a[1]*q[1])
69     poly = disk_clip(poly,q,1500)
70     if not poly:
71         raise RuntimeError('定位约束为空: 检查角度、题目类型或
72 误差界; 停止, 避免误判完成。')
73     return poly

```

```

70
71 def contains(poly, p):
72     return all(((b[0]-a[0])*(p[1]-a[1])-(b[1]-a[1])*(p[0]-a[0])
73                 >= -1e-6
74                 for a,b in zip(poly,poly[1:]+poly[:1])))
75
76 def circum(a,b,c):
77     # 相对坐标计算，减少消减误差。
78     ux,uy=b[0]-a[0],b[1]-a[1]
79     vx,vy=c[0]-a[0],c[1]-a[1]
80     det=2*(ux*vy-uy*vx)
81     if abs(det)<1e-12:
82         return None
83     u2,v2=ux*ux+uy*uy,vx*vx+vy*vy
84     return (a[0]+(u2*vy-v2*uy)/det,a[1]+(ux*v2-vx*u2)/det)
85
86 def mec(poly):
87     """枚举2/3个支撑顶点，小型定位多边形适用；最终全顶点认证。"""
88     if len(poly)==1:
89         return poly[0],0.0
90     center=(sum(p[0] for p in poly)/len(poly),sum(p[1] for p
91 in poly)/len(poly))
92     best=max(dist(center,p) for p in poly)
93     def consider(c,r):
94         nonlocal center,best
95         if r<best and all(dist(c,p)<=r+1e-7 for p in poly):
96             center,best=c,max(dist(c,p) for p in poly)
97     for i,a in enumerate(poly):
98         for j in range(i):
99             b=poly[j]
100             c=((a[0]+b[0])/2,(a[1]+b[1])/2)
101             consider(c,dist(a,b)/2)
102             for k in range(j):
103                 c=circum(a,b,poly[k])
104                 if c is not None:
105                     consider(c,dist(c,a))
106     return center,max(dist(center,p) for p in poly)
107
108 def polygon_diameter(poly):
109     """问题一证书：凸多边形直径由顶点对取得；顶点少，直接准确枚举。"""
110     if len(poly)<2: return 0.
111     return max(dist(poly[i],poly[j]) for i in range(len(poly))
112 for j in range(i))
113
114 def problem1_geometry_certificate(poly,center=None,radius=None):
115     """记录 MEC 与 Jung 上界；只作证书，不参与任何调度决策。"""
116     if center is None or radius is None:center,radius=mec(poly)
117     diameter=polygon_diameter(poly);jung_upper=diameter/math.
sqrt(3)
118     tol=1e-7*max(1.,diameter)
119     ratio=radius/jung_upper if jung_upper>tol else (0. if

```

```

117     radius<=tol else math.inf)
118     radius_to_diameter=radius/diameter if diameter>tol else
(0. if radius<=tol else math.inf)
118     return {'vertex_count':len(poly),'diameter_m':diameter,'
mec_center':list(center),
119             'mec_radius_m':radius,'jung_upper_m':jung_upper,'
jung_ratio':ratio,
120             'radius_to_diameter':radius_to_diameter,
121             'jung_certified':radius<=jung_upper+tol,'
numeric_tolerance_m':tol}
122
123 # 可通过命令行调整。权重是选址偏好，不是目标位置的概率密度。
124 SCHEDULE_CONFIG = {'search_gain':0.2,'lookahead':1.0,'
cell_size':100.}
125
126 def mesh(poly, depth):
127     v=np.asarray(poly,dtype=float)
128     center=v.mean(axis=0)
129     tri=np.array([[center,a,b] for a,b in zip(v,np.roll(v,-1,
axis=0))])
130     for _ in range(depth):
131         a,b,c=tri[:,0],tri[:,1],tri[:,2]
132         ab,bc,ca=(a+b)/2,(b+c)/2,(c+a)/2
133         tri=np.concatenate([np.stack(x,axis=1) for x in
134                             [(a,ab,ca),(ab,b,bc),(ca,bc,c),(ab,bc,ca)]]
135                             a,b=tri[:,1]-tri[:,0],tri[:,2]-tri[:,0]
136                             area=np.abs(a[:,0]*b[:,1]-a[:,1]*b[:,0])/2
137                             return tri[area>1e-10],area[area>1e-10]
138
139 BAYES_CONFIG={'noise_deg':1/math.sqrt(3),'quad_depth':3,'
prior_exists':0.65}
140
141 # 问题二仅作 V2 主目标近乎同值时的决胜项；默认参数不改动任何
V2 配置。
142 Q12_LINK_CONFIG={'eps_w':0.2,'p_w':2.0,'tau':0.01,'
min_dop_improvement':0.02,
143                 'max_primary_loss_ratio':0.005,'
primary_tie_rel_tol':1e-6,
144                 'sin_floor':0.02,'collinear_sin_tol':1e-8,
145                 'max_collinear_weight_fraction':0.05,'
enable_dop_override':True}
146
147 def point_segment_distances(points,a,b):
148     """批量计算点到线段 ab 的欧氏距离，兼容退化边。"""
149     edge=b-a;length2=float(edge@edge)
150     if length2<=1e-24:return np.linalg.norm(points-a,axis=1)
151     t=np.clip((points-a)@edge/length2,0.,1.)
152     return np.linalg.norm(points-(a+t[:,None]*edge),axis=1)
153
154 def depth_weighted_dop(poly,station1,candidate,quad_depth=2):
155     """以三角形面积求积近似连续区域深度加权 DOP。"""
156     triangles,areas=mesh(poly,quad_depth);points=triangles.
mean(axis=1)
157     if not len(points) or not np.all(np.isfinite(points)):
return math.inf
158     vertices=np.asarray(poly,dtype=float)

```

```

159     depths=np.full(len(points),np.inf)
160     for a,b in zip(vertices,np.roll(vertices,-1,axis=0)):
161         depths=np.minimum(depths,point_segment_distances(
points,a,b))
162     d_star=float(np.max(depths))
163     if not math.isfinite(d_star) or d_star<=1e-12:return math.
inf
164     cfg=Q12_LINK_CONFIG
165     weights=areas*(cfg['eps_w']+(1-cfg['eps_w'])*(np.maximum(
depths,0.)/d_star)**cfg['p_w'])
166     u=points-np.asarray(station1,dtype=float);v=points-np.
asarray(candidate,dtype=float)
167     r1=np.linalg.norm(u,axis=1);r2=np.linalg.norm(v,axis=1)
168     denom=r1*r2
169     cross=np.abs(u[:,0]*v[:,1]-u[:,1]*v[:,0])
170     sin_alpha=np.divide(cross,denom,out=np.zeros_like(cross),
where=denom>1e-12)
171     bad=(denom<=1e-12)|(~np.isfinite(sin_alpha))|(sin_alpha<=
cfg['collinear_sin_tol'])
172     total=float(weights.sum())
173     if (not math.isfinite(total) or total<=1e-300 or
174         float(weights[bad].sum())/total>cfg['
max_collinear_weight_fraction']):return math.inf
175     dop=np.sqrt(r1*r1+r2*r2)/np.maximum(np.abs(sin_alpha),cfg[
'sin_floor'])
176     value=float(weights@dop/total)
177     return value if math.isfinite(value) else math.inf
178
179 def gaussian_moments(points,weights):
180     w=np.asarray(weights,dtype=float);total=float(w.sum())
181     if not math.isfinite(total) or total<=1e-300:raise
ValueError('后验积分权重退化')
182     w=w/total;mu=w@points;diff=points-mu
183     covariance=(diff*w[:,None]).T@diff
184     values,vectors=np.linalg.eigh((covariance+covariance.T)/2)
185     covariance=(vectors*np.maximum(values,1e-4))@vectors.T
186     return mu,covariance
187
188 def radius_interval(points,positives,negatives):
189     lower=np.full(len(points),1000.);upper=np.full(len(points)
,1500.)
190     for q in positives:lower=np.maximum(lower,np.linalg.norm(
points-q,axis=1))
191     for q in negatives:upper=np.minimum(upper,np.linalg.norm(
points-q,axis=1))
192     return lower,upper
193
194 def update_gaussian(target,q,angle):
195     """受几何支持集约束的假定密度滤波(ADF)。
196      $p_{\text{new}}(G) \propto N(G;\mu,\Sigma)*\text{测角似然}*\text{接收半径条件似然}*\text{I}(G\text{在可行域})$ 。
197     对非高斯后验积分求均值/协方差，再用二维高斯近似。
198     首次更新采用圆域内均匀先验； $R \sim U[1000,1500]$ 且固定，跨观测共
享。
199     """

```

```

200     triangles, areas=mesh(target['poly'], BAYES_CONFIG['
quad_depth'])
201     points=triangles.mean(axis=1)
202     positives=target['stations']; negatives=target.get('
negatives',[])
203     lower, upper=radius_interval(points, positives, negatives)
204     new_width=np.maximum(upper-lower, 0.)
205     delta=points-np.asarray(q); distance=np.linalg.norm(delta,
axis=1)
206     residual=(np.arctan2(delta[:,1], delta[:,0])-math.radians(
angle)+math.pi)%(2*math.pi)-math.pi
207     variance=math.radians(BAYES_CONFIG['noise_deg'])**2
208     logw=np.log(areas)-residual**2/(2*variance)
209     if 'mean' in target:
210         diff=points-target['mean']; inv=np.linalg.inv(target['
covariance'])
211         logw-=0.5*np.einsum('ij,jk,ik->i', diff, inv, diff)
212         old_lower, old_upper=radius_interval(points, positives
[:-1], negatives)
213         old_width=np.maximum(old_upper-old_lower, 0.)
214         likelihood=np.divide(new_width, old_width, out=np.
zeros_like(new_width), where=old_width>1e-10)
215     else:
216         # 首次有效示向同时结合此前全部无信号信息，避免逐次独立
重抽接收半径。
217         likelihood=new_width/500.
218         valid=(distance>5)&(likelihood>0)&(np.linalg.norm(points,
axis=1)<=1800+1e-6)
219         for station in positives: valid &= np.linalg.norm(points-np
.asarray(station), axis=1)>5
220         failure_support=np.ones(len(points), dtype=bool)
221         for failed in target.get('failed_clears', []):
failure_support &= np.linalg.norm(points-np.asarray(failed),
axis=1)>20
222         valid &= failure_support
223         note='gaussian_moment_projection'
224         if np.any(valid):
225             logw[valid]+=np.log(likelihood[valid]); logw[~valid]=-
np.inf
226             weights=np.exp(logw-np.max(logw))
227         else:
228             # 极窄/边界区域可能被有限求积漏采；回到几何区域的保守
矩近似，绝不判空。
229             weights=areas*failure_support
230             if weights.sum()<=0: weights=areas
231             note='geometric_moment_fallback'
232             mu, cov=gaussian_moments(points, weights)
233             target['mean']=mu; target['covariance']=cov
234             return {'mean':mu.tolist(), 'covariance':cov.tolist(), 'note
':note,
235                     'quadrature_points':len(points), 'noise_std_deg':
BAYES_CONFIG['noise_deg'],
236                     'gaussian_entropy_nats':float(math.log(2*math.pi*
math.e)+0.5*np.linalg.slogdet(cov)[1])}
237
238 def update_unknown_belief(belief, coverage_map, ch):

```

```

239     # 已存在目标的空间先验为圆内均匀；同一个固定R的全部
no_signal联合似然。
240     points=coverage_map.centers
241     _,upper=radius_interval(points,[],belief['negatives'])
242     weights=np.maximum(upper-1000.,0.)/500.
243     inside=np.linalg.norm(points,axis=1)<=1800
244     weights*=inside
245     evidence=float(weights.sum())/max(1,int(inside.sum()))
246     prior=BAYES_CONFIG['prior_exists']
247     belief['existence']=max(1e-9,prior*evidence/(1-prior+prior
*evidence))
248     note='negative_observation_moment_projection'
249     if weights.sum()<=1e-12:
250         weights=coverage_map.remaining[ch-1].astype(float)
251         note='grid_boundary_moment_fallback'
252     if weights.sum()>0:belief['mean'],belief['covariance']=
gaussian_moments(points,weights)
253     return {'mean':belief['mean'].tolist(),'covariance':belief
['covariance'].tolist(),
254             'existence_probability':belief['existence'],'note'
:note}
255
256 def binary_entropy(p):
257     if p<=0 or p>=1:return 0.
258     return -p*math.log(p)-(1-p)*math.log1p(-p)
259
260 def detection_information(belief,coverage_map,q):
261     # P(收到信号)在均匀空间/半径先验及所有负观测条件下的数值近似。
262     # H(检测结果)是关于完整潜在状态(存在性、位置、R)的二值观测
信息代理。
263     points=coverage_map.centers
264     _,upper=radius_interval(points,[],belief['negatives'])
265     old=np.maximum(upper-1000.,0.)
266     lower=np.maximum(1000.,np.linalg.norm(points-np.asarray(q)
,axis=1))
267     new=np.maximum(upper-lower,0.)
268     inside=np.linalg.norm(points,axis=1)<=1800
269     a=float(new[inside].sum())/max(1e-12,old[inside].sum())
270     probability=min(1.,max(0.,belief['existence']*a))
271     return binary_entropy(probability),probability
272
273 def bayes_select(poly,stations,current,target,deadline):
274     """局部高斯线性化下的预期熵下降/耗时；接收条件另作全域认
证。"""
275     begin=time.monotonic();mu=np.asarray(target['mean']);cov=
np.asarray(target['covariance'])
276     stations=np.asarray(stations);scale=1000.
277     variance=math.radians(BAYES_CONFIG['noise_deg'])*2
278     triangles,_=mesh(poly,2)
279     station_r2=np.sum((triangles[:, :, None, :] - stations[None,
None, :, :])**2,axis=3)
280     def receive(z):
281         d2=np.sum((triangles-z*scale)**2,axis=2)
282         upper=np.minimum(np.max(d2-1e6,axis=1),np.min(np.max(
d2[:, :, None] - station_r2,axis=1),axis=1))

```

```

283         return float(upper.max())/1e6
284     def separation(z):return float(np.linalg.norm(z*scale-mu
-20)/scale
285     def station_gap(z):return float(np.min(np.linalg.norm(z*
scale-stations,axis=1))-5)/scale
286     def information(z):
287         delta=mu-z*scale;r2=float(delta@delta)
288         if r2<1e-8:return 0.
289         h=np.array([-delta[1],delta[0]])/r2
290         return 0.5*math.log1p(max(0.,float(h@cov@h))/variance)
291     def duration(z):
292         p=z*scale
293         return 10+np.linalg.norm(p-current)/5+0.5*np.linalg.
norm(p-mu)/5
294     def objective(z):
295         if time.monotonic()>deadline-20:raise BudgetStop('信息
增益规划接近现实截止时间。')
296         return -information(z)/duration(z)
297     def feasible(z):return receive(z)<=-1e-10 and separation(z
)>=-1e-8 and station_gap(z)>=-1e-8
298     center,radius=mec(poly);center=np.asarray(center)
299     seeds=[]
300     for origin in (mu,center):
301         for radius0 in sorted(set([25.,50.,100.,200.,min(500.,
max(50.,radius*.5))])):
302             for angle in np.arange(12)*math.pi/6:
303                 z=(origin+radius0*np.array([math.cos(angle),
math.sin(angle)]))/scale
304                 if feasible(z):seeds.append((objective(z),z))
305         if not seeds:return None,{'reason':
no_feasible_information_point',
'reception_certificate':{'certified':False,'scope':
entire_convex_polygon',
'reason':
no_candidate_satisfied_hard_reception_constraint'}}
308         seeds.sort(key=lambda x:x[0]);pool=list(seeds);starts=[]
309         for value,z in seeds:
310             if not starts or all(np.linalg.norm(z-p)>0.1 for p in
starts):starts.append(z)
311             if len(starts)==4:break
312             anchor=triangles[0].mean(axis=0)
313             bound=max(1000.,float(np.max(np.linalg.norm(stations-
anchor,axis=1))))
314             box=[((anchor[k]-bound)/scale,(anchor[k]+bound)/scale) for
k in range(2)]
315             constraints=[{'type':'ineq','fun':lambda z:-receive(z)-1e
-9},
{'type':'ineq','fun':separation},{ 'type':'
ineq','fun':station_gap}]
317         for z in starts:
318             result=minimize(objective,z,method='SLSQP',bounds=box,
constraints=constraints,
options={'maxiter':60,'ftol':1e-8})
319             if feasible(result.x):pool.append((objective(result.x)
,result.x))
320         original_value,original_z=min(pool,key=lambda x:x[0])

```

```

322     original_score=max(0.,-float(original_value));
original_point=tuple(float(v) for v in original_z*scale)
323     original_dop=depth_weighted_dop(poly,target['stations'
][0],original_point)
324     chosen_value,chosen_z=original_value,original_z
325     chosen_dop=original_dop;override=False
326     cfg=Q12_LINK_CONFIG;tie_candidates=[]
327     for candidate_value,candidate_z in pool:
328         score=max(0.,-float(candidate_value))
329         primary_loss=(original_score-score)/max(original_score
,1e-12)
330         if primary_loss<=min(cfg['tau'],cfg['
max_primary_loss_ratio'],cfg['primary_tie_rel_tol'])+1e-15:
331             p=tuple(float(v) for v in candidate_z*scale)
332             dop=depth_weighted_dop(poly,target['stations'][0],
p)
333             tie_candidates.append((dop,primary_loss,
candidate_value,candidate_z))
334             finite=[x for x in tie_candidates if math.isfinite(x[0])]
335             if finite and math.isfinite(original_dop):
336                 best_dop=min(x[0] for x in finite)
337                 eligible=[x for x in finite if x[0]<=best_dop*(1+cfg['
tau'])]
338                 dop,primary_loss,candidate_value,candidate_z=min(
eligible,key=lambda x:(x[0],x[1]))
339                 improvement=(original_dop-dop)/max(original_dop,1e-12)
340                 if (cfg['enable_dop_override'] and improvement>=cfg['
min_dop_improvement'] and
341                     primary_loss<=cfg['max_primary_loss_ratio']):
342                     chosen_dop,chosen_value,chosen_z=dop,
candidate_value,candidate_z;override=True
343                     point=tuple(float(v) for v in chosen_z*scale)
344                     primary_loss=(original_score-max(0.,-float(chosen_value)))/max(original_score,1e-12)
345                     improvement=((original_dop-chosen_dop)/max(original_dop,1e
-12)
346                         if math.isfinite(original_dop) and math.
isfinite(chosen_dop) else 0.)
347                     receive_upper=receive(chosen_z)*1e6
348                     reception_certificate={'certified':receive_upper<=-1e-4,
349                         'scope':'entire_convex_polygon','method':
convex_quadratic_maximum_on_triangle_vertices',
350                         'receive_upper_m2':receive_upper,'constraint':
candidate_receives_for_every_G_in_polygon'}
351                     tiebreak={'original_position':original_point,'
chosen_position':point,
352                         'original_primary_objective':float(original_value),'
chosen_primary_objective':float(chosen_value),
353                         'original_dop':original_dop if math.isfinite(
original_dop) else None,
354                         'chosen_dop':chosen_dop if math.isfinite(chosen_dop)
else None,
355                         'relative_dop_improvement':improvement,'
primary_loss_ratio':primary_loss,
356                         'candidate_count':len(pool),'
near_primary_candidate_count':len(tie_candidates),

```

```

357         'override': override, 'reception_certificate':
reception_certificate}
358     return point, {'reason': 'selected', 'position': point, '
expected_information_nats': information(chosen_z),
359                 'information_per_second': -float(chosen_value
), 'estimated_action_chain_s': float(duration(chosen_z)),
360                 'receive_upper_m2': receive_upper, 'elapsed_s'
: time.monotonic() - begin,
361                 'reception_certificate':
reception_certificate, 'problem2_dop_tiebreak': tiebreak,
362                 'note': '
linearized_gaussian_information_surrogate_with_hard_reception_certificate'
}
363
364 ROUTE_MEASURE_CONFIG = {'min_information_ratio': 0.70, '
min_saving_s': 10.0,
365                        'segment_fractions': (0.25, 0.5, 0.75, 1.0),
366                        'min_stop_information_nats': 0.50, '
max_target_channels_per_stop': 6}
367
368 def mandatory_stop_measurement_score(target, q):
369     """评价必经扫描点是否适合作为已发现目标的顺路补测点。"""
370     q = np.asarray(q, dtype=float); poly = target.get('poly') or []
371     if not poly or 'mean' not in target or 'covariance' not in
target: return None
372     # 对凸定位区域，距固定点的最大值在顶点取得；不超过1000米即
保证能接收。
373     receive_bound = max(np.linalg.norm(np.asarray(v, dtype=float)
-q) for v in poly)
374     if receive_bound > 1000 - 1e-6: return None
375     stations = np.asarray(target.get('stations', []), dtype=float)
376     if len(stations) and float(np.min(np.linalg.norm(stations -
q, axis=1))) < 5 - 1e-7: return None
377     mu = np.asarray(target['mean'], dtype=float); delta = mu - q; r2 =
float(delta@delta)
378     if r2 < 20**2: return None
379     h = np.array([-delta[1], delta[0]])/r2
380     variance = math.radians(BAYES_CONFIG['noise_deg'])**2
381     info = 0.5 * math.log1p(max(0., float(h@np.asarray(target['
covariance'])@h))/variance)
382     if info < ROUTE_MEASURE_CONFIG['min_stop_information_nats']:
return None
383     return {'information_nats': info, 'receive_bound_m': float(
receive_bound)}
384
385 def route_aligned_measurement(poly, stations, current, target,
next_stop, free_point, free_detail):
386     """在当前点到下一必经点的线段上寻找可替代自由补测点。
387
388     仅当候选点满足保守接收条件、与既有测站/目标估计保持间距、
信息量达到
389     自由最优点的指定比例，并且预计“补测后去中心再回到下一节
点”的总链路
390     至少节省指定秒数时采用。这样不会仅为贴路线而牺牲过多交会质
量。

```

```

391     """
392     if next_stop is None or free_point is None: return
free_point, {**free_detail, 'route_aligned': False, '
route_reason': 'no_next_stop'}
393     mu=np.asarray(target['mean'], dtype=float); cov=np.asarray(
target['covariance'], dtype=float)
394     stations=np.asarray(stations, dtype=float); current=np.
asarray(current, dtype=float)
395     next_stop=np.asarray(next_stop, dtype=float); free_point=np.
asarray(free_point, dtype=float)
396     variance=math.radians(BAYES_CONFIG['noise_deg'])*2
397     triangles,_=mesh(poly,2)
398     station_r2=np.sum((triangles[:, :, None, :] - stations[None,
None, :, :])**2, axis=3)
399     def receive(p):
400         d2=np.sum((triangles-p)**2, axis=2)
401         upper=np.minimum(np.max(d2-1e6, axis=1), np.min(np.max(
d2[:, :, None] - station_r2, axis=1), axis=1))
402         return float(upper.max())
403     def information(p):
404         delta=mu-p; r2=float(delta@delta)
405         if r2<1e-8: return 0.
406         h=np.array([-delta[1], delta[0]])/r2
407         return 0.5*math.log1p(max(0., float(h@cov@h))/variance)
408     def feasible(p):
409         return (receive(p)<=-1e-4 and np.linalg.norm(p-mu)
>=20-1e-7 and
410             float(np.min(np.linalg.norm(p-stations, axis=1)
))>=5-1e-7)
411     def chain_seconds(p):
412         # 补测5秒; 频道切换对自由点和顺路点相同, 比较时可抵
消。
413         return (np.linalg.norm(p-current)+np.linalg.norm(p-mu)
+np.linalg.norm(mu-next_stop))/5+5
414         free_info=float(free_detail.get('expected_information_nats
', information(free_point)))
415         free_chain=chain_seconds(free_point)
416         candidates=[]
417         delta=next_stop-current
418         for fraction in ROUTE_MEASURE_CONFIG['segment_fractions']:
419             p=current+float(fraction)*delta
420             if feasible(p):
421                 info=information(p); chain=chain_seconds(p)
422                 candidates.append((chain, -info, float(fraction),
p, info))
423         if not candidates:
424             return tuple(free_point), {**free_detail, 'route_aligned
': False, 'route_reason': 'no_feasible_route_candidate'}
425         chain, _, fraction, p, info=min(candidates, key=lambda x: (x[0],
x[1]))
426         saving=free_chain-chain
427         ratio=info/max(free_info, 1e-12)
428         if ratio>=ROUTE_MEASURE_CONFIG['min_information_ratio']
and saving>=ROUTE_MEASURE_CONFIG['min_saving_s']:
429             point=tuple(float(v) for v in p)
430             return point, {'reason': 'route_aligned_selected', '

```

```

431     position':point,
432         'expected_information_nats':info,
433         'free_expected_information_nats':free_info,
434         'information_ratio':ratio,
435         'estimated_chain_s':chain,
436         'free_estimated_chain_s':free_chain,
437         'estimated_saving_s':saving,
438         'segment_fraction':fraction, 'next_stop':
439         tuple(float(v) for v in next_stop),
440         'route_aligned':True,
441         'note':'
feasible_point_on_current_to_next_planned_stop'}
442     return tuple(free_point),{**free_detail, 'route_aligned':
443     False,
444         'route_reason':
445         'insufficient_information_or_saving',
446         'best_route_information_ratio':
447         ratio,
448         'best_route_estimated_saving_s':
449         saving,
450         'next_stop':tuple(float(v) for v
451         in next_stop)}
452
453 PROBE_CONFIG={'max_attempts':5, 'min_probability':0.2, '
454 max_radius':80.}
455
456 JOINT_CONFIG={'max_detour_m':5., 'horizon':3, 'min_saving_s':0.,
457 'candidate_limit':12}
458
459 def probe_distribution(target):
460     """失败clear后保留条件分布的非高斯形状，不反复用高斯投影重
461     加权。
462     新测向到来时(测点数改变)重新以新高斯工作后验生成求积权重。
463     """
464     version=len(target.get('stations',[]))
465     cached=target.get('probe_base')
466     if cached is None or cached['version']!=version:
467         triangles,areas=mesh(target['poly'],4);points=
468         triangles.mean(axis=1)
469         diff=points-target['mean'];inv=np.linalg.inv(target['
470 covariance'])
471         logw=np.log(areas)-0.5*np.einsum('ij,jk,ik->i',diff,
472 inv,diff)
473         weights=np.exp(logw-np.max(logw));weights/=weights.sum
474         ()
475         cached={'version':version, 'points':points, 'weights':
476 weights};target['probe_base']=cached
477         points=cached['points'];weights=cached['weights'].copy()
478         for q in target.get('failed_clears',[]):weights[np.linalg.
479 norm(points-q,axis=1)<=20]=0.
480         if weights.sum()<=1e-300:return None
481         weights/=weights.sum()
482         return points,weights
483
484 def joint_probe_plan(target,current,current_channel,ch,anchor,
485 same_target,next_stop):

```

"""在实际即将执行的路段上插入1~3次clear，枚举成功/全部失败分支。

返回值为启发式预计节省，不是无条件时间上界；只执行第一步，反馈后滚动重算。

```
remaining_budget=PROBE_CONFIG['max_attempts']-target.get('
probe_attempts',0)
if remaining_budget<=0 or target['radius']>PROBE_CONFIG['
max_radius'] or target.get('planned') is None:return None
sample=probe_distribution(target)
if sample is None:return None
points,weights=sample;current=np.asarray(current,dtype=
float);anchor=np.asarray(anchor,dtype=float)
vector=anchor-current;length2=float(vector@vector);direct=
dist(current,anchor)
def project(q):
    if length2<=1e-12:return current.copy()
    return current+np.clip(float((np.asarray(q)-current)
@vector)/length2,0,1)*vector
seeds=[tuple(current),tuple(project(target['mean'])),tuple
(project(target['center']))]
if length2>1e-12:
    direction=vector/math.sqrt(length2);center=project(
target['mean'])
    for offset in (-40,-20,-10,10,20,40):seeds.append(
tuple(project(center+offset*direction)))
poly=np.asarray(target['poly']);low=poly.min(axis=0);high=
poly.max(axis=0)
n=np.maximum(1,np.ceil((high-low)/25).astype(int))
seeds.extend([tuple(target['mean']),tuple(target['center'
])])
for iy in range(n[1]):
    for ix in range(n[0]):
        q=low+(np.array([ix,iy])+.5)*(high-low)/n
        seeds.extend([tuple(q),tuple(project(q))])
candidates=[];seen=set()
for q in seeds:
    key=tuple(round(x,5) for x in q)
    if key in seen:continue
    seen.add(key)
    if any(dist(q,p)<1 for p in target.get('failed_clears'
,[])):continue
    extra=dist(current,q)+dist(q,anchor)-direct
    if extra>JOINT_CONFIG['max_detour_m']+1e-7:continue
    mask=np.linalg.norm(points-q,axis=1)<=20
    mass=float(weights[mask].sum())
    if mass<0.01:continue
    candidates.append({'q':tuple(q),'mask':mask,'mass':
mass,'extra':max(0.,extra)})
candidates.sort(key=lambda c:c['mass']/(3+dist(current,c['
q']))/5),reverse=True)
candidates=candidates[:JOINT_CONFIG['candidate_limit']]
if not candidates:return None
planned=target['planned'];center=target['center']
if same_target:
    # baseline: current->测向点->估计清除点->next_stop。
```

```

507         tail=5+(current_channel!=ch)+dist(anchor,center)/5+5
508         if next_stop is not None:tail+=dist(center,next_stop)
/5
509         success_tail=lambda q:dist(q,next_stop)/5 if next_stop
is not None else 0.
510     else:
511         # 顺路处理其他目标：保留既定路线，估算避免将来专门定位
该目标的成本。
512         tail=dist(anchor,planned)/5+5+(current_channel!=ch)+
dist(planned,center)/5+5
513         success_tail=lambda q:dist(q,anchor)/5
514         baseline=direct/5+tail
515         best=None;max_depth=min(JOINT_CONFIG['horizon'],
remaining_budget)
516         def visit(order,excluded,prob_remaining,travel_so_far,
action_expected,last,success_return):
517             nonlocal best
518             if len(order)>=max_depth:return
519             for i,candidate in enumerate(candidates):
520                 if i in order:continue
521                 q=candidate['q'];distance=dist(last,q)
522                 route_length=travel_so_far+distance
523                 detour=route_length+dist(q,anchor)-direct
524                 # 对“全部失败”的完整插入路径限制额外路程，而非分
别限制每一个点。
525                 if detour>JOINT_CONFIG['max_detour_m']+1e-7:
continue
526                 fresh=candidate['mask']&~excluded
527                 probability=.995*float(weights[fresh].sum()) # 预
留求积遗漏尾部
528                 if probability<=1e-6:continue
529                 conditional=probability/prob_remaining
530                 if not order and conditional<PROBE_CONFIG['
min_probability']:continue
531                 if order and conditional<0.05:continue
532                 after=max(.005,prob_remaining-probability)
533                 expected=action_expected+prob_remaining*(distance
/5+3)+2*probability
534                 returned=success_return+probability*success_tail(q
)
535                 total=expected+returned+after*(dist(q,anchor)/5+
tail)
536                 sequence=order+[i]
537                 saving=baseline-total
538                 if saving>=JOINT_CONFIG['min_saving_s'] and (best
is None or saving>best['expected_saving_s']):
539                     best={'position':candidates[sequence[0]]['q'],
540                           'sequence':[candidates[j]['q'] for j in
sequence],
541                                   'probability':.995*candidates[sequence
[0]]['mass'],
542                                   'sequence_success_probability':1-after,'
expected_time_s':total,
543                                   'reference_time_s':baseline,'
expected_saving_s':saving,
544                                   'worst_branch_extra_distance_m':max(0.,

```

```

detour), 'anchor': tuple(anchor),
545     'same_target': same_target, 'hypothesis': '
approximate_posterior_and_one_measurement_continuation'}
546     visit(sequence, excluded|candidate['mask'], after,
route_length, expected, q, returned)
547     visit([], np.zeros(len(points), dtype=bool), 1., 0., 0., tuple(
current), 0.)
548     return best
549
550 def condition_failed_clear(target, q):
551     target.setdefault('failed_clears', []).append(tuple(q))
552     sample = probe_distribution(target)
553     if sample is None: return {'note': '
quadrature_missed_remaining_region_keep_geometric_outer_bound
'}
554     target['mean'], target['covariance'] = gaussian_moments(*
sample)
555     target['planned'] = None
556     return {'note': '
posterior_conditioned_on_distance_greater_than_20',
557             'mean': target['mean'].tolist(), 'covariance': target
['covariance'].tolist()}
558
559 LAYOUT_PRESETS = {
560     'ring8_939': {'ring_points': 8, 'ring_radius': 939.0},
561 }
562 DEFAULT_LAYOUT = 'ring8_939'
563
564 def scan_points(ring_points, ring_radius):
565     """返回原点及均匀分布在指定半径圆周上的扫描点。"""
566     return [(0., 0.)] + [
567         (ring_radius*math.cos(2*math.pi*k/ring_points),
568          ring_radius*math.sin(2*math.pi*k/ring_points))
569         for k in range(ring_points)
570     ]
571
572 def coverage_certificate(ring_points=8, ring_radius=939.):
573     """解析认证原点+均匀环点对1800米圆域的1000米覆盖。
574
575     原点覆盖半径1000米的内盘。外环中任一点与最近环点夹角不超过
576      $\pi/n$ ; 距离平方关于目标极径为凸函数, 因此最坏值出现在1000或
577     1800米端点。该结论适用于任意  $n \geq 2$ 。
578     """
579     if isinstance(ring_points, bool) or not isinstance(
ring_points, int) or ring_points < 2:
580         raise ValueError('ring_points 必须是大于等于2的整数')
581     if not math.isfinite(ring_radius) or ring_radius <= 0:
582         raise ValueError('ring_radius 必须是有限正数')
583     half_sector = math.pi/ring_points
584     endpoint_bounds = {
585         str(r): math.sqrt(r*r+ring_radius*ring_radius-
586                             2*r*ring_radius*math.cos(half_sector
587
588         for r in (1000., 1800.)
589     }
590     outer = max(endpoint_bounds.values())

```

```

590     whole = max(1000., outer)
591     if outer > 1000.+1e-9:
592         raise ValueError(
593             f'布局不能保证覆盖: n={ring_points}, R={
ring_radius:g}米, '
594             f'外环最不利距离={outer:.6f}米 > 1000米')
595     return {'target_radius_m':1800.,
guaranteed_reception_radius_m':1000.,
596             'ring_points':ring_points, 'ring_radius_m':float(
ring_radius),
597             'half_sector_angle_deg':math.degrees(half_sector),
598             'endpoint_distance_bounds_m':endpoint_bounds,
599             'outer_annulus_distance_bound_m':outer, '
whole_domain_bound_m':whole,
600             'certified':True,
601             'method':
analytic_origin_disk_and_uniform_ring_angular_sectors'}
602
603 def resolve_layout(layout_name=DEFAULT_LAYOUT, ring_points=
None, ring_radius=None):
604     preset = LAYOUT_PRESETS[layout_name]
605     n = preset['ring_points'] if ring_points is None else
ring_points
606     radius = preset['ring_radius'] if ring_radius is None else
ring_radius
607     custom = ring_points is not None or ring_radius is not
None
608     name = layout_name if not custom else f'custom_n{n}_R{
radius:g}'
609     certificate = coverage_certificate(n, radius)
610     return {'layout_name':name, 'base_preset':layout_name, '
ring_points':n,
611             'ring_radius':float(radius), 'certificate':
certificate}
612
613 def rolling_open_route(current, nodes):
614     """访问全部当前任务点的开放欧氏路径; 服务时间在本轮固定,
不影响排序。
615     <=10个非当前位置节点时Held-Karp精确; 更大时多初值最近邻+2-
opt近似。
616     只决定本轮下一动作, 每次响应后重新建节点, 不承诺未知环境全
局最优。
617     """
618     if not nodes: return [], {'method':'empty', 'length_m':0., '
exact':True}
619     zero=[i for i, n in enumerate(nodes) if dist(current, n['
position'])<1e-6]
620     ids=[i for i in range(len(nodes)) if i not in zero]
621     n=len(ids)
622     if not n: return zero, {'method':'at_current_position', '
length_m':0., 'exact':True}
623     xy=np.asarray([current]+[nodes[i]['position'] for i in ids
], dtype=float)
624     d=np.linalg.norm(xy[:, None, :] - xy[None, :, :], axis=2)
625     if n<=10:
626         count=1<n; dp=np.full((count, n), np.inf); parent=np.full

```

```

((count,n),-1,dtype=int)
627     for j in range(n):dp[1<j,j]=d[0,j+1]
628     for mask in range(1,count):
629         for last in range(n):
630             if not(mask>>last&1):continue
631             prev=mask^(1<last)
632             if not prev:continue
633             candidates=[j for j in range(n) if prev>>j&1]
634             values=dp[prev,candidates]+d[np.asarray(
candidates)+1,last+1]
635             k=int(np.argmin(values));dp[mask,last]=values[
k];parent[mask,last]=candidates[k]
636             mask=count-1;last=int(np.argmin(dp[mask]));length=
float(dp[mask,last]);order=[]
637             while mask:
638                 order.append(last);old=int(parent[mask,last]);mask
^=1<last;last=old
639                 order.reverse();method='held_karp_exact';exact=True
640             else:
641                 best=None
642                 for first in np.argsort(d[0,1:][:min(n,6)]):
643                     order=[int(first)];remaining=set(range(n))-{int(
first)}
644                     while remaining:
645                         nxt=min(remaining,key=lambda j:d[order[-1]+1,j
+1]);order.append(nxt);remaining.remove(nxt)
646                     for _ in range(40):
647                         gain=0.;pair=None
648                         for i in range(n-1):
649                             a=0 if i==0 else order[i-1]+1;b=order[i]+1
650                             for j in range(i+1,n):
651                                 c=order[j]+1
652                                 delta=d[a,b]-d[a,c]
653                                 if j+1<n:
654                                     end=order[j+1]+1;delta+=d[c,end]-d
[b,end]
655                                     if delta>gain+1e-9:gain=delta;pair=(i,
j)
656                                     if pair is None:break
657                                     i,j=pair;order[i:j+1]=reversed(order[i:j+1])
658                                     length=float(d[0,order[0]+1]+sum(d[a+1,b+1] for a,
b in zip(order,order[1:])))
659                                     if best is None or length<best[0]:best=(length,
list(order))
660                                     length,order=best;method='multi_start_2opt_approximate
';exact=False
661                 return zero+[ids[i] for i in order],{'method':method,'
length_m':length,'exact':exact,'task_nodes':len(nodes)}
662
663 class CoverageMap:
664     """每频道独立的保守剩余区域地图。
665     与目标圆相交的方格全部保留；no_signal只删除四角均在1000米
内的格。
666     方格全部删除=>该频道无目标。格心仅用于选址，不作为真实目标
离散位置。
667     """

```

```

668     def __init__(self, cell_size=100.):
669         self.n=math.ceil(3600/cell_size)
670         self.step=3600/self.n
671         axis=-1800+np.arange(self.n)*self.step
672         x,y=np.meshgrid(axis,axis)
673         lo=np.column_stack([x.ravel(),y.ravel()]); hi=lo+self.
step
674         nearest=np.maximum(np.maximum(lo,-hi),0.)
675         self.valid=np.sum(nearest**2,axis=1)<=1800**2+1e-6
676         self.ids=np.flatnonzero(self.valid)
677         self.centers=(lo[self.valid]+hi[self.valid])/2
678         self.corners=np.stack([lo[self.valid],lo[self.valid]+[
self.step,0],
679                                 hi[self.valid],lo[self.valid
]+[0,self.step]],axis=1)
680         self.remaining=np.ones((20,len(self.ids)),dtype=bool)
681         self.observations=np.zeros(20,dtype=int)
682         def covered(self,q):
683             return np.max(np.sum((self.corners-np.asarray(q))*2,
axis=2),axis=1)<=(1000-1e-5)**2
684         def update_negative(self,ch,q):
685             before=int(self.remaining[ch-1].sum())
686             self.remaining[ch-1,self.covered(q)]=False
687             self.observations[ch-1]+=1
688             return before-int(self.remaining[ch-1].sum())
689 def plan_cpp(coverage_map):
690     """有限候选布局上的覆盖选点+开放路径动态规划。
691     圆域由保守方格外包，只有整格被某扫描点1000米圆盘覆盖才认
证。
692     不是在移动途中连续感知；也不声称连续平面上的全局最优。
693     """
694     families=[('raster_3x3',[(0.,0.)]+[(float(x),float(y)) for
y in (-1200,0,1200)
695                                     for x in (-1200,0,1200)
if (x,y)!= (0,0)])])
696     for radius in (1250,1300,1350,1400,1450,1500):
697         for offset in (0.,math.pi/6):
698             families.append((f'hex_{radius}_{round(math.
degrees(offset))}',[(0.,0.)]+
699                         [(radius*math.cos(offset+k*math.pi/3),radius*
math.sin(offset+k*math.pi/3)) for k in range(6)]))
700     records=[];best=None
701     for name,points in families:
702         covers=[coverage_map.covered(p) for p in points]
703         if not np.logical_or.reduce(covers).all():
704             records.append({'layout':name,'certified':False});
continue
705     n=len(points)-1; count=1<n
706     # dp[mask,j]:从原点出发访问mask，最后停在点j+1的最短开
放路径。
707     dp=np.full((count,n),np.inf);parent=np.full((count,n)
,-1,dtype=int)
708     union=[None]*count;union[0]=covers[0].copy()
709     for j in range(n):dp[1<j,j]=dist(points[0],points[j
+1])
710     best_family=None

```

```

711         for mask in range(1, count):
712             bit = mask & -mask; j = bit.bit_length() - 1
713             union[mask] = union[mask ^ bit] | covers[j + 1]
714             for last in range(n):
715                 if not (mask >> last & 1) or not math.isfinite(dp[
mask, last]): continue
716                 for nxt in range(n):
717                     if mask >> nxt & 1: continue
718                     newmask = mask | (1 << nxt)
719                     value = dp[mask, last] + dist(points[last + 1],
points[nxt + 1])
720                     if value < dp[newmask, nxt]:
721                         dp[newmask, nxt] = value; parent[newmask,
nxt] = last
722                 if not union[mask].all(): continue
723                 last = int(np.argmax(dp[mask])); length = float(dp[mask
, last])
724                 # 每个检测点都可从当前频道开始: 20次测量耗时20*5
秒,
725                 # 仅其余19个频道需要切换, 故每点均为119秒。
726                 # 这是统一比较布局的代理耗时; 实际会按频道状态减少
扫描。
727                 scan_time = 119 * (1 + bin(mask).count('1'))
728                 score = length / 5 + scan_time
729                 if best_family is None or score < best_family[0]:
best_family = (score, mask, last, length, scan_time)
730                 if best_family is None: raise RuntimeError('覆盖布局动
态规划失败')
731                 score, mask, last, length, scan_time = best_family
732                 order = []; work = mask
733                 while work:
734                     order.append(last + 1); previous = int(parent[work, last
]); work ^= 1 << last; last = previous
735                     route = [points[0]] + [points[i] for i in reversed(order)]
736                     record = {'layout': name, 'certified': True, 'waypoint_count
': len(route), 'route_length_m': length,
737                             'full_scan_surrogate_s': score, '
scan_time_surrogate_s': scan_time}
738                     records.append(record)
739                     if best is None or score < best['full_scan_surrogate_s'
]:
740                         best = {**record, 'waypoints': route, 'coverage_cells':
len(coverage_map.ids)}
741                     if best is None: raise RuntimeError('没有通过全域覆盖认证的
CPP布局')
742                     best['evaluated_layouts'] = records
743                     return best
744
745 class HttpBackend:
746     def __init__(self, url):
747         self.url = url.rstrip('/')
748         self.opener = build_opener(ProxyHandler({})) # 本机接口
不经过系统代理
749     def post(self, path, payload, timeout):
750         data = json.dumps(payload, ensure_ascii=False, allow_nan=

```

```

False).encode('utf-8')
751     req=Request(self.url+path,data=data,headers={'Content-
Type':'application/json'},method='POST')
752     with self.opener.open(req,timeout=timeout) as response
:
753         if response.status!=200:
754             raise RuntimeError('非200 HTTP响应')
755         return json.loads(response.read().decode('utf-8'))
756
757 class LocalBackend:
758     """独立生成10~16个全向源；同频道同位置误差固定，无真值传给
策略。"""
759     def __init__(self,seed,edge=False):
760         rng=random.Random(seed)
761         self.seed=seed
762         self.targets={}
763         for ch in rng.sample(range(1,21),rng.randint(10,16)):
764             r=1800 if edge else 1800*math.sqrt(rng.random())
765             t=rng.random()*2*math.pi
766             self.targets[ch]=((r*math.cos(t),r*math.sin(t))
,1000 if edge else rng.uniform(1000,1500))
767         self.total=len(self.targets)
768         self.done=set(); self.q=(0,0); self.ch=1; self.vt=0;
self.cache={}
769     def post(self,path,payload,timeout):
770         key=payload['request_id']
771         if key in self.cache:
772             return self.cache[key]
773         result={'accepted':True,'real_timestamp_ms':int(time.
time()*1000)}
774         if path=='/enter':
775             result.update(remaining_real_duration_s=1200,
max_virtual_duration_s=360000,max_real_duration_s=1200)
776         elif path in ('/measure','/clear'):
777             q=(payload['position']['x'],payload['position']['y
']); ch=payload['channel']
778             self.vt+=dist(self.q,q)/5; self.q=q
779             target=self.targets.get(ch)
780             d=dist(q,target[0]) if target and ch not in self.
done else math.inf
781             if path=='/measure':
782                 self.vt+=5+(ch!=self.ch); self.ch=ch
783                 kind='no_signal' if not target or d>target[1]
else ('near' if d<=5 else 'direction')
784                 result['measure_result']=kind
785                 if kind=='direction':
786                     token=f'{self.seed}:{ch}:{q[0]:.9f}:{q
[1]:.9f}'.encode()
787                     error=2*int.from_bytes(hashlib.sha256(
token).digest()[:8],'big')/(2**64-1)-1
788                     angle=math.degrees(math.atan2(target
[0][1]-q[1],target[0][0]-q[0]))
789                     result['svd_deg']=round((angle+error)
%360,2)%360
790                 else:
791                     success=d<=20; self.vt+=5 if success else 3

```

```

792         result['clear_result']='success' if success
else 'no_target_in_range'
793         if success: self.done.add(ch)
794     else:
795         result['exit_reason']='user_exit'
796         result['virtual_time_s']=self.vt
797         self.cache[key]=result
798         return result
799
800 class BudgetStop(RuntimeError):
801     pass
802
803 class Robot:
804     def __init__(self, backend, robot_id, out=None, quiet=False,
805                  layout=None):
806         self.backend=backend; self.robot_id=robot_id; self.out
807         =out; self.quiet=quiet
808         self.layout=layout or resolve_layout()
809         self.prefix=uuid.uuid4().hex[:12]; self.seq=0; self.q
810         =(0.,0.); self.ch=1
811         self.vt=0.; self.vmax=360000.; self.deadline=math.inf;
812         self.started=time.monotonic()
813         self.done=set(); self.absent=set(); self.searched={c:
814         set() for c in range(1,21)}
815         self.distance=0.; self.measures=0; self.switches=0;
816         self.clears=0; self.fallbacks=0; self.information_records=[]
817         self.problem1_geometry_records=[]
818         self.log=None
819         if out:
820             out.mkdir(parents=True, exist_ok=True)
821             self.log=(out/'actions.jsonl').open('w', encoding='
822             utf-8')
823         def write_log(self, record):
824             if self.log:
825                 self.log.write(json.dumps(record, ensure_ascii=
826                 False, allow_nan=False)+'\n'); self.log.flush()
827         def call(self, path, q=None, ch=None):
828             if path in ('/measure', '/clear'):
829                 cost=dist(self.q, q)/5+6
830                 if time.monotonic()>self.deadline-12 or self.vt+
831                 cost>self.vmax-1:
832                     raise BudgetStop('接近运行时限，任务未完成，保
833                     存当前结果并退出。')
834                 self.seq+=1
835                 payload={'arena_id':'default', 'robot_id':self.robot_id
836                 , 'request_id':f'{self.prefix}-{self.seq}'}
837                 if q is not None:
838                     payload.update(position={'x':float(q[0]), 'y':float
839                     (q[1])}, channel=ch)
840                 self.write_log({'event':'request', 'path':path, 'payload
841                 ':payload})
842                 for attempt in range(3):
843                     if time.monotonic()>=self.deadline:
844                         raise RuntimeError('本局已到现实截止时间；不再
845                         发送请求。')
846                 try:

```

```

833         result=self.backend.post(path,payload,max(0.1,
min(4.,self.deadline-time.monotonic()))))
834         break
835     except HTTPError as exc:
836         self.write_log({'event':'http_error','code':
exc.code,'request_id':payload['request_id']})
837         raise RuntimeError(f'HTTP {exc.code}, 请检查模
拟器状态及请求参数。') from exc
838     except (URLError,TimeoutError,ConnectionError,
OSError,json.JSONDecodeError) as exc:
839         self.write_log({'event':'network_error','
attempt':attempt+1,'detail':str(exc),'request_id':payload['
request_id']})
840         if attempt==2:
841             raise RuntimeError('连接失败或响应丢失, 动
作结果可能未知; 已停止, 不发送后续动作。请检查模拟器界面和日
志。') from exc
842         time.sleep(0.3) # 同一个payload/request_id重
试, 绝不并发
843         self.write_log({'event':'response','path':path,'
request_id':payload['request_id'],'response':result})
844         if result.get('accepted') is not True:
845             raise RuntimeError('请求被拒绝 accepted=false; 确
认参赛队号、接口就绪状态。')
846         self.vt=float(result['virtual_time_s'])
847         if path=='/enter':
848             self.deadline=time.monotonic()+float(result['
remaining_real_duration_s'])
849             self.vmax=float(result.get('max_virtual_duration_s
',360000))
850             if q is not None:
851                 self.distance+=dist(self.q,q); self.q=tuple(q)
852             if path=='/measure':
853                 self.measures+=1; self.switches+=ch!=self.ch; self
.ch=ch
854                 if result.get('measure_result') not in ('near','
direction','no_signal'):
855                     raise RuntimeError('未知检测响应')
856                 if path=='/clear':
857                     self.clears+=1
858                     if result.get('clear_result') not in ('success','
no_target_in_range'):
859                         raise RuntimeError('未知清除响应')
860                     if result['clear_result']=='success': self.done.
add(ch)
861                     if not self.quiet and q is not None:
862                         kind=result.get('measure_result',result.get('
clear_result'))
863                         print(f'{self.seq:4d} {path:8s} ch={ch:2d} ({q
[0]:8.1f},{q[1]:8.1f}) {kind:20s} T={self.vt:.1f}s cleared={
len(self.done)}',flush=True)
864                     return result
865                 def clear(self,ch,q):
866                     return self.call('/clear',q,ch)['clear_result']=='
success'

```

```

867     def clear_point(self, poly, center):
868         """在20米共同覆盖圆盘交集中选离当前位置近的清除点。"""
869         v=np.asarray(poly); current=np.asarray(self.q); scale
=1000.
870         def safe(z): return (19.8**2-np.sum((z*scale-v)**2,
axis=1))/400.
871         result=minimize(lambda z:float(np.sum((z-current/scale
)**2)),np.asarray(center)/scale,
872                         method='SLSQP',constraints=[{'type':'
ineq','fun':safe}],
873                         options={'maxiter':40,'ftol':1e-9})
874         q=tuple(result.x*scale)
875         return q if max(dist(q,p) for p in poly)<=19.80001
else center
876
877     def make_fallback(self, target):
878         """对当前保守凸区域作25米网格覆盖；失败后依次尝试剩余
格。"""
879         poly=target['poly']; cells=[]
880         xmin=min(p[0] for p in poly); xmax=max(p[0] for p in
poly)
881         ymin=min(p[1] for p in poly); ymax=max(p[1] for p in
poly)
882         for iy in range(max(1,math.ceil((ymax-ymin)/25))):
883             y0=ymin+25*iy; y1=min(ymax,y0+25); row=[]
884             for ix in range(max(1,math.ceil((xmax-xmin)/25))):
885                 x0=xmin+25*ix; x1=min(xmax,x0+25); part=poly
886                 for a,b in [((1,0),x1),((-1,0),-x0),((0,1),y1)
,((0,-1),-y0)]:
887                     part=clip(part,a,b)
888                     if part and not any(max(dist(v,failed) for v
in part)<=20 for failed in target.get('failed_clears',[])):
889                         row.append(((x0+x1)/2,(y0+y1)/2))
890                         cells.extend(row if iy%2==0 else row[::-1])
891                     if cells and dist(self.q,cells[-1])<dist(self.q,cells
[0]): cells.reverse()
892                 target['fallback']=cells; self.fallbacks+=1
893
894     def schedule_run(self):
895         self.targets={}
896         self.beliefs={ch:{'mean':np.zeros(2),'covariance':np.
eye(2)*1800**2/4,
897                         'existence':BAYES_CONFIG['
prior_exists'],'negatives':[]}} for ch in range(1,21)}
898         self.coverage_map=CoverageMap(SCHEDULE_CONFIG['
cell_size']) # 仅概率求积辅助，不作为完成证书
899         scans=scan_points(self.layout['ring_points'],self.
layout['ring_radius'])
900         certificate=self.layout['certificate']
901         self.cpp_plan={'layout_name':self.layout['layout_name'
],
902                       'ring_points':self.layout['ring_points'
],
903                       'ring_radius':self.layout['ring_radius'
],
904                       'waypoint_count':len(scans),'waypoints'

```

```

:scans,
905         'certificate':certificate}
906     if self.out:(self.out/'scan_plan.json').write_text(
json.dumps(self.cpp_plan,indent=2),encoding='utf-8')
907     self.write_log({'event':'coverage_certificate',**self.
cpp_plan})
908     self.scheduler_counts={'search_measures':0,'
localization_measures':0,'certified_clears':0,
909         'fallback_clears':0,'decisions':0,'probe_attempts'
:0,'probe_successes':0,
910         'joint_sequences_planned':0,'on_route_probes':0,'
exact_routes':0,'approximate_routes':0,
911         'batch_scan_visits':0,'batch_scan_channels':0,
912         'route_aligned_plans':0,'free_measurement_plans'
:0,
913         'dop_tie_break_decisions':0,'
dop_tie_break_overrides':0}
914     cache_key=None;cache_value=None
915     while len(self.done)<16 and len(self.done|self.absent)
<20:
916         if time.monotonic()>self.deadline-20:raise
BudgetStop('滚动规划接近现实截止时间。')
917         unknown=[ch for ch in range(1,21) if ch not in
self.targets and ch not in self.absent]
918         nodes=[]
919         for i,q in enumerate(scans):
920             channels=[ch for ch in unknown if i not in
self.searched[ch]]
921             if channels:nodes.append({'type':'scan','id':f
'scan-{i}','index':i,'position':q,'channels':channels})
922             for ch,target in self.targets.items():
923                 if ch not in self.done:
924                     nodes.append({'type':'source','id':f'
source-{ch}','channel':ch,
925                         'position':tuple(target.get(
'mean',target['center']))})
926             if not nodes:raise RuntimeError('无任务节点，但未
能证明全部清除。')
927             key=(tuple(self.q),tuple((n['id'],tuple(n['
position']))) for n in nodes))
928             if key!=cache_key:
929                 cache_value=rolling_open_route(self.q,nodes);
cache_key=key
930             order,route_info=cache_value
931             self.scheduler_counts['exact_routes' if route_info
['exact'] else 'approximate_routes']+=1
932             node=nodes[order[0]]
933             next_stop=nodes[order[1]]['position'] if len(order
)>1 else None
934             self.scheduler_counts['decisions']+=1
935             self.write_log({'event':'rolling_route','nodes':
nodes,'order':[nodes[i]['id'] for i in order]**route_info})
936             if node['type']=='scan':
937                 q=node['position']
938                 # 到达扫描点后批量检测本节点全部尚未搜索频道。
当前频道优先，

```

```

939         # 其余频道按环形顺序检测，避免同一扫描点被反复
加入滚动路线。
940         pending=list(node['channels'])
941         # V2: 把当前扫描点同时当作已发现目标的顺路补测
点。只有当整个
942         # 定位多边形都位于1000米接收圆内，且预计信息量
达标时才加入。
943         route_targets=[]
944         for target_ch,target in self.targets.items():
945             if target_ch in self.done:continue
946             score=mandatory_stop_measurement_score(
target,q)
947             if score is not None:route_targets.append
((score['information_nats'],target_ch,score))
948             route_targets.sort(reverse=True)
949             route_targets=route_targets[:
ROUTE_MEASURE_CONFIG['max_target_channels_per_stop']]
950             pending.extend(ch for _,ch,_ in route_targets
if ch not in pending)
951             if self.ch in pending:
952                 start=pending.index(self.ch)
953                 pending=pending[start:]+pending[:start]
954                 self.scheduler_counts['batch_scan_visits']+=1
955                 self.scheduler_counts['batch_scan_channels']+=
len(pending)
956                 self.write_log({'event':'batch_scan_start','
scan_id':node['id'],'index':node['index'],
957                               'position':q,'channels':
pending,'start_channel':self.ch,
958                               'route_aligned_targets':[{
channel':ch,**score} for _,ch,score in route_targets]})
959                 measured=[]
960                 for ch in pending:
961                     # 前面的测量可能已将频道清除或认证为不存
在；这种情况下跳过。
962                     if ch in self.done or ch in self.absent or
node['index'] in self.searched[ch]:
963                         continue
964                     was_target=ch in self.targets
965                     res=self.call('/measure',q,ch)
966                     if was_target:
967                         self.scheduler_counts['
route_aligned_plans']+=1
968                         self.scheduler_counts['
localization_measures']+=1
969                     else:
970                         self.scheduler_counts['search_measures
']+=1
971                         measured.append(ch)
972                         self.searched[ch].add(node['index'])
973                         if res['measure_result']=='no_signal':
974                             self.coverage_map.update_negative(ch,q
)
975                             self.beliefs[ch]['negatives'].append(q
)
976                             belief=update_unknown_belief(self.

```

```

beliefs[ch],self.coverage_map,ch)
977         self.write_log({'event': '
negative_update', 'channel':ch,**belief})
978         # 仅尚未发现的频道可由全覆盖扫描认证为
不存在；已发现目标的
979         # no_signal 只作为固定接收半径的负观
测，不得误判为无源。
980         if ch in self.targets:
981             self.targets[ch].setdefault('
negatives',[ ]).append(q)
982             self.targets[ch]['planned']=None
983             elif len(self.searched[ch])==len(scans
):
984                 self.absent.add(ch);self.beliefs[
ch]['existence']=0.
985             else:
986                 self.accept_bearing(ch,q,res)
987                 self.write_log({'event': 'batch_scan_end', '
scan_id':node['id'], 'index':node['index'],
988                                 'position':q, '
measured_channels':measured,
989                                 'new_current_channel':self.ch,
'done':sorted(self.done),
990                                 'absent':sorted(self.absent)})
991         # 批次结束后再统一重建源节点与剩余扫描节点并规
划下一段路径。
992         cache_key=None
993         continue
994         ch=node['channel'];target=self.targets[ch]
995         if target.get('near') is not None or target['
radius']<=19.8:
996             q=target.get('near')
997             if q is None:q=self.clear_point(target['poly'
],target['center'])
998             self.write_log({'event': 'local_decision', '
channel':ch, 'choice': 'certified_clear'})
999             if not self.clear(ch,q):raise RuntimeError('可
靠清除与响应矛盾')
1000             self.scheduler_counts['certified_clears']+=1;
continue
1001             if target.get('fallback') is not None:
1002                 if not target['fallback']:raise RuntimeError('
网格兜底耗尽而目标未清除')
1003                 q=target['fallback'].pop(0);self.clear(ch,q)
1004                 self.scheduler_counts['fallback_clears']+=1;
continue
1005                 if target.get('planned') is None:
1006                     if len(target['stations'])>=8:self.
make_fallback(target);continue
1007                     free_q,free_detail=bayes_select(target['poly'
],target['stations'],self.q,target,self.deadline)
1008                     dop_detail=free_detail.get('
problem2_dop_tiebreak')
1009                     if dop_detail is not None:
1010                         self.scheduler_counts['

```

```

1011         dop_tie_break_decisions']+=1
1012         self.scheduler_counts['
dop_tie_break_overrides']+=int(dop_detail['override'])
1013         self.write_log({'event': '
problem2_dop_tiebreak', 'channel':ch,**dop_detail})
1014         if free_q is None:
1015             detail=free_detail;q=None
1016         else:
1017             q,detail=route_aligned_measurement(
                target['poly'],target['stations'],self
1018             .q,target,next_stop,free_q,free_detail)
1019             key_name='route_aligned_plans' if detail.
get('route_aligned') else 'free_measurement_plans'
1020             self.scheduler_counts[key_name]+=1
1021             self.information_records.append(detail)
1022             self.write_log({'event': 'measurement_plan', '
channel':ch,**detail})
1023             if q is None:self.make_fallback(target);
continue
1024             target['planned']=q
1025             q=target['planned']
1026             measure_cost=dist(self.q,q)/5+5+(self.ch!=ch)+dist
(q,target['center'])/5+5
1027             if next_stop is not None:measure_cost+=dist(target
['center'],next_stop)/5
1028             probe=joint_probe_plan(target,self.q,self.ch,ch,q,
True,next_stop)
1029             self.write_log({'event': 'local_decision', 'channel'
:ch,
1030                 'expected_measure_then_clear_s':measure_cost,
1031                 'expected_probe_sequence_s':probe['
expected_time_s'] if probe else None,
1032                 'probe_plan':probe,'choice': 'probe' if probe
else 'measure',
1033                 'note': '补测一次后清除为预测代理；成功仍需收到
clear成功响应。'})
1034             if probe:
1035                 target['probe_attempts']=target.get('
probe_attempts',0)+1
1036                 self.scheduler_counts['probe_attempts']+=1
1037                 self.scheduler_counts['joint_sequences_planned
']+=int(len(probe['sequence'])>=2)
1038                 self.scheduler_counts['on_route_probes']+=int(
probe['worst_branch_extra_distance_m']<1e-5)
1039                 success=self.clear(ch,probe['position'])
1040                 if success:self.scheduler_counts['
probe_successes']+=1
1041                 else:
1042                     info=condition_failed_clear(target,probe['
position'])
1043                     self.beliefs[ch].update(mean=target['mean'
],covariance=target['covariance'])
1044                     self.write_log({'event': '
failed_clear_update', 'channel':ch,**info})
1045                     self.write_log({'event': 'probe_result', '
channel':ch, 'success':success, 'plan':probe})

```

```

1045         else:
1046             res=self.call('/measure',q,ch);self.
scheduler_counts['localization_measures']+=1
1047             if res['measure_result']=='no_signal':raise
RuntimeError('接收认证点返回无信号, 请确认问题3全向案例。')
1048             self.accept_bearing(ch,q,res)
1049             if len(self.done)<10:raise RuntimeError('清除数低于题
设下界10, 未确认完成。')
1050
1051     def accept_bearing(self,ch,q,result):
1052         target=self.targets.setdefault(ch,{ 'poly':initial_poly
(), 'stations':[], 'planned':None, 'fallback':None,
1053         'negatives':list(self.
beliefs[ch]['negatives'])})
1054         self.beliefs[ch]['existence']=1.;target['planned']=
None
1055         if result['measure_result']=='near':
1056             target.update(near=q,center=q,radius=5.,mean=np.
asarray(q),covariance=np.eye(2)*25/4)
1057             self.beliefs[ch].update(mean=target['mean'],
covariance=target['covariance'])
1058             if not self.clear(ch,q):raise RuntimeError('near后
清除失败')
1059             self.scheduler_counts['certified_clears']+=1
1060         else:
1061             target['angle']=result['svd_deg'];target['poly']=
update_poly(target['poly'],q,target['angle'])
1062             target['stations'].append(q);target['center'],
target['radius']=mec(target['poly'])
1063             geometry=problem1_geometry_certificate(target['
poly'],target['center'],target['radius'])
1064             self.problem1_geometry_records.append(geometry)
1065             posterior=update_gaussian(target,q,target['angle'
])
1066             self.beliefs[ch].update(mean=target['mean'],
covariance=target['covariance'])
1067             self.write_log({'event':'gaussian_update','channel
':ch,**posterior,
1068                             'problem1_geometry':geometry})
1069
1070     def run(self):
1071         status='incomplete'; error=None; entered=False
1072         try:
1073             self.call('/enter'); entered=True
1074             self.schedule_run()
1075             status='complete'
1076             self.call('/exit')
1077         except BudgetStop as exc:
1078             status='budget_stop'; error=str(exc)
1079             if entered and time.monotonic()<self.deadline:
1080                 try: self.call('/exit')
1081                     except Exception as exit_exc: error+='; 退出失
败: '+str(exit_exc)
1082             except (Exception,KeyboardInterrupt) as exc:
1083                 status='error'; error=str(exc) or '用户中断'

```

```

1084         # 通信结果可能未知，绝不在错误后继续发送新动作或探
1085         测/exit。
1086         geometry_records=[g for g in self.
1087         problem1_geometry_records
1088             if math.isfinite(g['jung_ratio'])
1089             and math.isfinite(g['radius_to_diameter'])]
1090         geometry_summary={ 'sample_count':len(self.
1091         problem1_geometry_records),
1092             'finite_sample_count':len(geometry_records),
1093             'max_jung_ratio':max((g['jung_ratio'] for g in
1094             geometry_records),default=None),
1095             'mean_radius_to_diameter':(sum(g['
1096             radius_to_diameter'] for g in geometry_records)/len(
1097             geometry_records)
1098                 if geometry_records
1099             else None),
1100             'jung_violation_count':sum(not g['jung_certified']
1101             for g in self.problem1_geometry_records)}
1102         summary={ 'status':status, 'error':error, 'cleared_count'
1103         :len(self.done),
1104             'cleared_channels':sorted(self.done), '
1105             absent_channels':sorted(self.absent),
1106             'virtual_time_s':self.vt, '
1107             average_clear_time_s':self.vt/len(self.done) if self.done
1108             else None,
1109             'program_runtime_s':time.monotonic()-self.
1110             started, 'distance_m':self.distance,
1111             'measure_count':self.measures, 'switch_count':
1112             self.switches, 'clear_attempts':self.clears,
1113             'failed_clear_attempts':self.clears-len(self.
1114             done), 'fallback_count':self.fallbacks,
1115             'information_selections':len(self.
1116             information_records),
1117             'strategy':
1118             parameterized_ring_rolling_open_route',
1119             'layout_name':self.layout['layout_name'],
1120             'ring_points':self.layout['ring_points'], '
1121             ring_radius':self.layout['ring_radius'],
1122             'joint_config':dict(JOINT_CONFIG), '
1123             probe_config':dict(PROBE_CONFIG),
1124             'bayes_config':dict(BAYES_CONFIG), '
1125             q12_link_config':dict(Q12_LINK_CONFIG),
1126             'problem1_geometry':geometry_summary,
1127             'scheduler':getattr(self, 'scheduler_counts'
1128             ,{}), 'scheduler_config':dict(SCHEDULE_CONFIG),
1129             'clear_ratio':None, 'note':'真实总数仅演练结束
1130             后可见; clear_ratio需用实际总数计算。'}
1131         expected=self.distance/5+self.switches+5*self.measures
1132         +3*self.clears+2*len(self.done)
1133         summary['accounting_delta_s']=self.vt-expected
1134         if abs(summary['accounting_delta_s'])<1e-9:summary['
1135         accounting_delta_s']=0.
1136         if self.out:
1137             if hasattr(self, 'beliefs'):
1138                 posteriors={str(ch):{'mean':b['mean'].tolist()
1139                 , 'covariance':b['covariance'].tolist()},

```

```

1114         'existence_probability':b
1115         ['existence'],
1116         'status':'cleared' if ch
1117         in self.done else ('absent' if ch in self.absent else ('
1118         detected' if ch in self.targets else 'unknown'))}
1119         for ch,b in self.beliefs.items()}
1120         (self.out/'gaussian_posteriors.json').
1121         write_text(json.dumps(posteriors,ensure_ascii=False,indent
1122         =2),encoding='utf-8')
1123         if hasattr(self,'coverage_map'):
1124             fm=self.coverage_map
1125             snapshot={'cell_size_m':fm.step,'centers':fm.
1126             centers.tolist(),
1127             'remaining_cell_indices':{str(ch):np
1128             .flatnonzero(fm.remaining[ch-1]).tolist()
1129             for ch in
1130             range(1,21) if ch not in self.targets},
1131             'absent_channels':sorted(self.absent
1132             ), 'cleared_channels':sorted(self.done),
1133             'note':'方格外包连续位置；未发现频道
1134             仅凭整格无信号认证删除。'}
1135             (self.out/'coverage_map.json').write_text(json
1136             .dumps(snapshot,ensure_ascii=False),encoding='utf-8')
1137             (self.out/'summary.json').write_text(json.dumps(
1138             summary,ensure_ascii=False,indent=2),encoding='utf-8')
1139             with (self.out/'summary.csv').open('w',encoding='
1140             utf-8-sig',newline='') as f:
1141                 writer=csv.DictWriter(f,fieldnames=list(
1142                 summary)); writer.writeheader(); writer.writerow(summary)
1143                 if self.log: self.log.close()
1144                 return summary
1145
1146 def write_csv(path, rows):
1147     if not rows:
1148         return
1149     with path.open('w',encoding='utf-8-sig',newline='') as f:
1150         writer=csv.DictWriter(f,fieldnames=list(rows[0]))
1151         writer.writeheader();writer.writerows(rows)
1152
1153 def scenario_record(backend):
1154     return {'seed':backend.seed,'total':backend.total,
1155             'sources':[{'channel':ch,'position':list(value[0])
1156             ,
1157             'reception_radius_m':value[1]}
1158             for ch,value in sorted(backend.targets.
1159             items())]}
1160
1161 def run_local_case(seed, case_number, layout, case_dir):
1162     backend=LocalBackend(seed,edge=False)
1163     truth=scenario_record(backend)
1164     robot=Robot(backend,'LOCAL-TEST',case_dir,quiet=True,
1165     layout=layout)
1166     result=robot.run()
1167     result.update(true_total=backend.total,
1168     clear_ratio=len(backend.done)/backend.total,
1169     seed=seed,case=case_number)

```

```

1153     # Robot.run先写基础summary；补充仅本地评估器可知的真值指
1154     标。
1155     (case_dir/'summary.json').write_text(
1156         json.dumps(result,ensure_ascii=False,indent=2),
1157         encoding='utf-8')
1158     write_csv(case_dir/'summary.csv',[result])
1159     (case_dir/'scenario.json').write_text(
1160         json.dumps(truth,ensure_ascii=False,indent=2),encoding
1161         ='utf-8')
1162     success=(result['status']=='complete' and len(backend.done
1163 )==backend.total and
1164         abs(result['accounting_delta_s'])<1e-5 and
1165         all(len(robot.searched[ch])==layout['ring_points'
1166 ]+1
1167         for ch in result['absent_channels']))
1168     row={'layout_name':layout['layout_name'],'ring_points':
1169 layout['ring_points'],
1170         'ring_radius':layout['ring_radius'],'case':
1171 case_number,'seed':seed,
1172         'true_total':backend.total,'total':backend.total,
1173         'cleared_count':len(backend.done),'cleared':len(
1174 backend.done),
1175         'success':success,'status':result['status'],'
1176 clear_ratio':result['clear_ratio'],
1177         'virtual_time_s':result['virtual_time_s'],
1178         'average_clear_time_s':result['average_clear_time_s'
1179 ],
1180         'distance_m':result['distance_m'],'measure_count':
1181 result['measure_count'],
1182         'failed_clear_attempts':result['failed_clear_attempts
1183 '],
1184         'program_runtime_s':result['program_runtime_s'],
1185         'probe_attempts':result['scheduler'].get('
1186 probe_attempts',0),
1187         'probe_successes':result['scheduler'].get('
1188 probe_successes',0),
1189         'exact_routes':result['scheduler'].get('exact_routes'
1190 ,0),
1191         'approximate_routes':result['scheduler'].get('
1192 approximate_routes',0),
1193         'error':result['error']}
1194     return row,truth
1195
1196 def summarize_layout(layout, rows):
1197     def mean(key):return sum(float(r[key]) for r in rows)/len(
1198 rows)
1199     total_times=[float(r['virtual_time_s']) for r in rows]
1200     avg_time=sum(total_times)/len(total_times)
1201     variance=sum((x-avg_time)**2 for x in total_times)/len(
1202 total_times)
1203     total_sources=sum(int(r['true_total']) for r in rows)
1204     return {'layout_name':layout['layout_name'],'ring_points':
1205 layout['ring_points'],
1206         'ring_radius':layout['ring_radius'],'case_count':
1207 len(rows),
1208         'success_count':sum(bool(r['success']) for r in

```

```

rows),
1189         'mean_clear_ratio':mean('clear_ratio'),
1190         'mean_virtual_time_s':avg_time,
1191         'std_virtual_time_s':math.sqrt(variance),
1192         'pooled_time_per_source_s':sum(total_times)/
total_sources,
1193         'mean_distance_m':mean('distance_m'),
1194         'mean_measure_count':mean('measure_count'),
1195         'mean_program_runtime_s':mean('program_runtime_s')
,
1196         'failed_clear_attempts':sum(int(r['
failed_clear_attempts']) for r in rows)}
1197
1198 def self_test(count,seed_start=2026091200,output_dir=None,
layout=None):
1199     """运行单布局本地案例；目录为 root/layout_name/case_NN。"""
1200     root=Path(output_dir or 'local_tests');root.mkdir(parents=
True,exist_ok=True)
1201     layout=layout or resolve_layout();seeds=list(range(
seed_start,seed_start+count))
1202     rows=[]
1203     for i,seed in enumerate(seeds,1):
1204         case=root/layout['layout_name']/f'case_{i:02d}'
1205         row,_=run_local_case(seed,i,layout,case)
1206         rows.append(row);print(json.dumps(row,ensure_ascii=
False),flush=True)
1207     summary=summarize_layout(layout,rows)
1208     report={'note':'本地随机合成环境，非官方演练；仿真真值只供
评估器使用',
1209            'layout':layout,'seeds':seeds,'cases':rows,'
summary':summary,
1210            'mean_total_time_s':summary['mean_virtual_time_s'
],
1211            'pooled_time_per_source_s':summary['
pooled_time_per_source_s']}
1212     (root/'test_results.json').write_text(
1213         json.dumps(report,ensure_ascii=False,indent=2),
encoding='utf-8')
1214     print(json.dumps(summary,ensure_ascii=False,indent=2))
1215     return report
1216
1217 def compare_layouts(count,seed_start=2026091200,output_dir=
None):
1218     """以共同随机数对全部预设布局作配对仿真并保存完整过程。"""
1219     root=Path(output_dir or 'layout_comparison');root.mkdir(
parents=True,exist_ok=True)
1220     seeds=list(range(seed_start,seed_start+count));all_rows
=[];summaries=[]
1221     scenario_fingerprints={}
1222     resolved=[resolve_layout(name) for name in LAYOUT_PRESETS]
1223     for layout in resolved:
1224         rows=[]
1225         for i,seed in enumerate(seeds,1):
1226             case=root/layout['layout_name']/f'case_{i:02d}'
1227             row,truth=run_local_case(seed,i,layout,case)

```

```

1228         fingerprint=hashlib.sha256(json.dumps(truth,
1229 sort_keys=True).encode()).hexdigest()
1229         old=scenario_fingerprints.setdefault(seed,
1230 fingerprint)
1230         if old!=fingerprint:
1231             raise RuntimeError(f'seed {seed} 在不同布局生
成了不同真值')
1232         rows.append(row);all_rows.append(row)
1233         print(json.dumps(row,ensure_ascii=False),flush=
True)
1234         summaries.append(summarize_layout(layout,rows))
1235         write_csv(root/'comparison_cases.csv',all_rows)
1236         write_csv(root/'comparison_summary.csv',summaries)
1237         results={'note':'所有布局共享相同seed与LocalBackend生成规
则，可进行配对比较',
1238 'seeds':seeds,'cases':all_rows,'summaries':
summaries}
1239         (root/'comparison_results.json').write_text(
1240             json.dumps(results,ensure_ascii=False,indent=2),
encoding='utf-8')
1241         manifest={'experiment':
q3_multi_layout_common_random_numbers',
1242 'created_at':time.strftime('%Y-%m-%dT%H:%M:%S%z'
),
1243 'description':'问题三多布局本地合成仿真；各布局
使用完全相同的seed和目标真值。',
1244 'case_count':count,'seed_start':seed_start,'
seeds':seeds,
1245 'presets':LAYOUT_PRESETS,
1246 'coverage_certificates':{x['layout_name']:x['
certificate'] for x in resolved},
1247 'output_structure':'<layout_name>/case_NN/保存每
局完整过程文件'}
1248         (root/'experiment_manifest.json').write_text(
1249             json.dumps(manifest,ensure_ascii=False,indent=2),
encoding='utf-8')
1250         print(json.dumps({'summaries':summaries},ensure_ascii=
False,indent=2))
1251         return results
1252
1253 def main():
1254     parser=argparse.ArgumentParser(description=__doc__,
formatter_class=argparse.RawDescriptionHelpFormatter)
1255     parser.add_argument('--search-gain',type=float,default
=0.2,help='扫描批次收益权重，默认0.2，非存在概率')
1256     parser.add_argument('--cell-size',type=float,default=100.,
help='保守方格边长，默认100米，50到200米')
1257     parser.add_argument('--lookahead',type=float,default=1.0,
help='插入任务的后续绕行权重，默认1.0')
1258     parser.add_argument('--quad-depth',type=int,default=3,help
='后验积分三角形细分层数，1到4')
1259     parser.add_argument('--noise-deg',type=float,default=1/
math.sqrt(3),help='近似测角标准差，默认0.577度；不改变硬误差
界')
1260     parser.add_argument('--prior-exists',type=float,default

```

```

1261     =0.65,help='每频道初始存在概率近似，默认0.65')
1262     parser.add_argument('--clear-detour',type=float,default
1263     =5.,help='失败分支相对既定路线最多增加的米数；0为严格顺路')
1264     parser.add_argument('--clear-horizon',type=int,default=3,
1265     help='联合规划深度，1到3')
1266     parser.add_argument('--min-clear-saving',type=float,
1267     default=0.,help='预计至少节省的秒数')
1268     parser.add_argument('--max-probes',type=int,default=5,help
1269     ='每目标试探clear次数上限；0为禁用')
1270     parser.add_argument('--min-probe-prob',type=float,default
1271     =0.2,help='试探成功概率估计下限')
1272     parser.add_argument('--probe-radius',type=float,default
1273     =80.,help='允许试探的定位覆盖圆半径上限，米')
1274     parser.add_argument('--layout',choices=list(LAYOUT_PRESETS
1275     ),default=DEFAULT_LAYOUT,
1276     help='预设布局，默认 ring8_939')
1277     parser.add_argument('--ring-points',type=int,
1278     help='自定义外围点数，覆盖所选预设的 n
1279     ')
1280     parser.add_argument('--ring-radius',type=float,
1281     help='自定义布局半径（米），覆盖所选预
1282     设的 R')
1283     parser.add_argument('--compare-layouts',action='store_true
1284     ',
1285     help='对全部预设布局使用相同seed批量比
1286     较（需同时指定--self-test N）')
1287     parser.add_argument('--robot-id',help='与模拟器登录参赛队
1288     号完全一致')
1289     parser.add_argument('--base-url',default='http
1290     ://127.0.0.1:2026')
1291     parser.add_argument('--output',help='日志目录；默认按时间
1292     创建')
1293     parser.add_argument('--seed-start',type=int,default
1294     =2026091200,help='本地测试起始随机种子')
1295     parser.add_argument('--test-output',default='local_tests',
1296     help='本地测试结果目录')
1297     parser.add_argument('--self-test',type=int,metavar='N',
1298     help='运行N个本地随机案例，不连接官方模拟器')
1299     args=parser.parse_args()
1300     if not (0<args.noise_deg<=5 and 0<args.prior_exists<1 and
1301     1<=args.quad_depth<=4):
1302         parser.error('0<noise-deg<=5, 0<prior-exists<1, 1<=
1303     quad-depth<=4')
1304     if not (0<args.search_gain<=5 and 0<=args.lookahead<=1 and
1305     50<=args.cell_size<=200):
1306         parser.error('0<search-gain<=5, 0<=lookahead<=1, 50<=
1307     cell-size<=200')
1308     SCHEDULE_CONFIG.update(search_gain=args.search_gain,
1309     lookahead=args.lookahead,cell_size=args.cell_size)
1310     if not (0<=args.max_probes<=20 and 0<args.min_probe_prob<1
1311     and 20<=args.probe_radius<=200):
1312         parser.error('0<=max-probes<=20, 0<min-probe-prob<1,
1313     20<=probe-radius<=200')
1314     if not (0<=args.clear_detour<=50 and 1<=args.clear_horizon

```

```

1290     <=3 and args.min_clear_saving>=0):
1291         parser.error('0<=clear-detour<=50, 1<=clear-horizon<=3
, min-clear-saving>=0')
1292     JOINT_CONFIG.update(max_detour_m=args.clear_detour,horizon
=args.clear_horizon,min_saving_s=args.min_clear_saving)
1293     PROBE_CONFIG.update(max_attempts=args.max_probes,
min_probability=args.min_probe_prob,max_radius=args.
probe_radius)
1294     BAYES_CONFIG.update(noise_deg=args.noise_deg,prior_exists=
args.prior_exists,quad_depth=args.quad_depth)
1295     if args.compare_layouts and args.self_test is None:
1296         parser.error('--compare-layouts 必须与 --self-test N
同时使用')
1297     if args.compare_layouts and (args.ring_points is not None
or args.ring_radius is not None):
1298         parser.error('--compare-layouts 比较固定预设, 不接受
--ring-points/--ring-radius')
1299     try:
1300         layout=resolve_layout(args.layout,args.ring_points,
args.ring_radius)
1301     except (ValueError,KeyError) as exc:
1302         parser.error(str(exc))
1303     if args.self_test is not None:
1304         if args.self_test<1: parser.error('N必须大于0')
1305         if args.compare_layouts:
1306             report=compare_layouts(args.self_test,args.
seed_start,args.test_output)
1307             if any(x['success_count']<args.self_test for x in
report['summaries']):
1308                 raise SystemExit(1)
1309             else:
1310                 report=self_test(args.self_test,args.seed_start,
args.test_output)
1311                 if report['summary']['success_count']<args.
self_test:raise SystemExit(1)
1312             return
1313     if not args.robot_id: parser.error('请提供 --robot-id 参赛
队号')
1314     out=Path(args.output) if args.output else Path('runs')/(
layout['layout_name']+'_'+time.strftime('%Y%m%d_%H%M%S
')+ '_' +uuid.uuid4().hex[:6])
1315     print(f"问题3多布局滚动路径版: 布局={layout['layout_name
']}, 外围点数={layout['ring_points']}, "
1316         f"半径={layout['ring_radius']:g}米。请确保官方模拟器
接口已就绪。日志目录: {out.resolve()}")
1317     result=Robot(HttpBackend(args.base_url),args.robot_id,out,
layout=layout).run()
1318     print("==== 最终结果 =====")
1319     print(f"实际清除总数: {result['cleared_count']}")
1320     print(f"实际消耗总时间: {result['virtual_time_s']:.2f} s")
1321     if result['average_clear_time_s'] is None:
1322         print("实际平均消耗时间: N/A")
1323     else:
1324         print(f"实际平均消耗时间: {result['
average_clear_time_s']:.2f} s")

```

```

1325
1326     if result['status'] != 'complete':
1327         print('本次未确认全部完成; 请保留日志并检查模拟器界
1328             面。')
1329         raise SystemExit(1)
1329 if __name__ == '__main__':
1330     main()

```

simulate\_t3.py

```

1  #!/usr/bin/env python3
2  """Run reproducible synthetic simulations for Question 3.
3
4  The official simulator hides the source truth, so this module
5  uses the
6  existing 'LocalBackend' only for offline strategy validation
7  . It records
8  case-level success/time distributions and source-level
9  localization/clear
10 errors. No simulator result or official case code is inferred
11 from these
12 synthetic runs.
13 """
14 from __future__ import annotations
15
16 import argparse
17 import csv
18 import json
19 import math
20 import statistics
21 import sys
22 import time
23 from pathlib import Path
24 from typing import Any, Iterable
25
26 import numpy as np
27
28 CODE_ROOT = Path(__file__).resolve().parent
29 PROJECT_ROOT = CODE_ROOT.parents[1]
30 if str(CODE_ROOT) not in sys.path:
31     sys.path.insert(0, str(CODE_ROOT))
32 import t3 # noqa: E402
33
34 DEFAULT_OUTPUT = PROJECT_ROOT / "outputs" / "t3" / "simulation
35 "
36
37 def _finite(value: Any) -> float | None:
38     try:
39         value = float(value)
40     except (TypeError, ValueError):
41         return None
42     return value if math.isfinite(value) else None
43
44 def _json_safe(value: Any) -> Any:

```

```

43     """Convert non-finite numeric values to JSON 'null'
    recursively.
44
45     CSV files intentionally retain 'nan' for unavailable
    metrics, while the
46     machine-readable JSON contract must remain strict ('
    allow_nan=False').
47     This matters for unsuccessful runs where no successful
    clear distance is
48     available.
49     """
50     if isinstance(value, float):
51         return value if math.isfinite(value) else None
52     if isinstance(value, dict):
53         return {key: _json_safe(item) for key, item in value.
    items()}
54     if isinstance(value, (list, tuple)):
55         return [_json_safe(item) for item in value]
56     return value
57
58
59 def _quantiles(values: Iterable[float]) -> dict[str, float |
    None]:
60     values = [float(v) for v in values if _finite(v) is not
    None]
61     if not values:
62         return {"mean": None, "std": None, "p05": None, "
    median": None, "p95": None}
63     q05, q50, q95 = np.quantile(np.asarray(values, dtype=float
    ), [0.05, 0.50, 0.95])
64     return {
65         "mean": float(statistics.fmean(values)),
66         "std": float(statistics.pstdev(values)) if len(values)
    > 1 else 0.0,
67         "p05": float(q05),
68         "median": float(q50),
69         "p95": float(q95),
70     }
71
72
73 def _write_csv(path: Path, rows: list[dict[str, Any]]) -> None
    :
74     if not rows:
75         return
76     path.parent.mkdir(parents=True, exist_ok=True)
77     with path.open("w", encoding="utf-8-sig", newline="") as
    handle:
78         writer = csv.DictWriter(handle, fieldnames=list(rows
    [0]))
79         writer.writeheader()
80         writer.writerows(rows)
81
82
83 def _clear_records(case_dir: Path) -> dict[int, list[dict[str,
    Any]]]:
84     """Read clear request/response pairs from the existing

```

```

JSONL audit log."""
85     records: dict[int, list[dict[str, Any]]] = {}
86     path = case_dir / "actions.jsonl"
87     if not path.exists():
88         return records
89     pending: dict[str, dict[str, Any]] = {}
90     for line in path.read_text(encoding="utf-8").splitlines():
91         try:
92             event = json.loads(line)
93         except json.JSONDecodeError:
94             continue
95         if event.get("event") == "request" and event.get("path
96 ") == "/clear":
97             payload = event.get("payload", {})
98             request_id = str(payload.get("request_id", ""))
99             pending[request_id] = payload
100         elif event.get("event") == "response" and event.get("
101 path") == "/clear":
102             request_id = str(event.get("request_id", ""))
103             payload = pending.get(request_id, {})
104             response = event.get("response", {})
105             try:
106                 ch = int(payload["channel"])
107                 position = payload["position"]
108                 point = (float(position["x"]), float(position[
109 "y"])))
110             except (KeyError, TypeError, ValueError):
111                 continue
112             records.setdefault(ch, []).append(
113                 {"point": point, "result": response.get("
114 clear_result")})
115         )
116     return records
117
118 def _source_rows(
119     backend: Any,
120     robot: Any,
121     case_dir: Path,
122     case_number: int,
123     seed: int,
124 ) -> list[dict[str, Any]]:
125     clears = _clear_records(case_dir)
126     rows: list[dict[str, Any]] = []
127     for channel, (truth, reception_radius) in sorted(backend.
128 targets.items()):
129         target = robot.targets.get(channel, {})
130         center = target.get("center")
131         mean = target.get("mean")
132         center_error = t3.dist(truth, center) if center is not
133 None else math.nan
134         mean_error = t3.dist(truth, mean) if mean is not None
135 else math.nan
136         attempts = clears.get(channel, [])
137         successful = [item for item in attempts if item.get("
138 result") == "success"]

```

```

132         clear_distance = (
133             t3.dist(truth, successful[-1]["point"]) if
successful else math.nan
134         )
135         rows.append(
136             {
137                 "case": case_number,
138                 "seed": seed,
139                 "channel": channel,
140                 "reception_radius_m": float(reception_radius),
141                 "center_error_m": center_error,
142                 "mean_error_m": mean_error,
143                 "clear_distance_m": clear_distance,
144                 "clear_attempts": len(attempts),
145                 "clear_success": bool(successful),
146                 "station_count": len(target.get("stations",
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178

```

```

179     "failed_clear_attempts": int(result.get("
180 failed_clear_attempts", 0)),
181     "program_runtime_s": float(result.get("
182 program_runtime_s", math.nan)),
183     "mean_clear_distance_m": float(statistics.fmean(errors
184 )) if errors else math.nan,
185     "p95_clear_distance_m": float(np.quantile(errors,
186 0.95)) if errors else math.nan,
187     "mean_center_error_m": float(statistics.fmean(
188 center_errors)) if center_errors else math.nan,
189     "error_hit_rate_20m": float(sum(v <= 20.0 for v in
190 errors) / len(errors)) if errors else math.nan,
191     "error_hit_rate_50m": float(sum(v <= 50.0 for v in
192 errors) / len(errors)) if errors else math.nan,
193     "error_hit_rate_100m": float(sum(v <= 100.0 for v in
194 errors) / len(errors)) if errors else math.nan,
195     "probe_attempts": int(result.get("scheduler", {}).get(
196 "probe_attempts", 0)),
197     "probe_successes": int(result.get("scheduler", {}).get(
198 "probe_successes", 0)),
199     "error": result.get("error"),
200 }
201
202 def _aggregate(case_rows: list[dict[str, Any]], source_rows:
203 list[dict[str, Any]]) -> dict[str, Any]:
204     success = [bool(row["success"]) for row in case_rows]
205     times = [row["virtual_time_s"] for row in case_rows]
206     per_source = [row["time_per_cleared_source_s"] for row in
207 case_rows]
208     clear_ratios = [row["clear_ratio"] for row in case_rows]
209     clear_errors = [row["clear_distance_m"] for row in
210 source_rows if _finite(row["clear_distance_m"]) is not None]
211     center_errors = [row["center_error_m"] for row in
212 source_rows if _finite(row["center_error_m"]) is not None]
213     return {
214         "case_count": len(case_rows),
215         "source_count": len(source_rows),
216         "success_count": int(sum(success)),
217         "success_rate": float(sum(success) / len(success)) if
218 success else 0.0,
219         "clear_ratio": _quantiles(clear_ratios),
220         "virtual_time_s": _quantiles(times),
221         "time_per_cleared_source_s": _quantiles(per_source),
222         "clear_distance_m": _quantiles(clear_errors),
223         "center_error_m": _quantiles(center_errors),
224         "hit_rate_clear_distance": {
225             f"le_{limit:g}m": float(sum(v <= limit for v in
226 clear_errors) / len(clear_errors))
227             if clear_errors else None
228             for limit in (5, 20, 50, 100)
229         },
230         "failed_clear_attempts": int(sum(int(row["
231 failed_clear_attempts"]) for row in case_rows)),
232         "mean_measure_count": float(statistics.fmean(row["

```

```

measure_count"] for row in case_rows)) if case_rows else
None,
217     "mean_program_runtime_s": float(statistics.fmean(row["
program_runtime_s"] for row in case_rows)) if case_rows else
None,
218     }
219
220
221 def run_simulation(
222     cases: int,
223     seed_start: int,
224     output_dir: Path,
225     layout_name: str = t3.DEFAULT_LAYOUT,
226     edge_fraction: float = 0.0,
227 ) -> dict[str, Any]:
228     if cases < 1:
229         raise ValueError("cases must be positive")
230     if not 0.0 <= edge_fraction <= 1.0:
231         raise ValueError("edge_fraction must be in [0, 1]")
232     layout = t3.resolve_layout(layout_name)
233     output_dir.mkdir(parents=True, exist_ok=True)
234     case_rows: list[dict[str, Any]] = []
235     source_rows: list[dict[str, Any]] = []
236     seeds = list(range(int(seed_start), int(seed_start) + int(
cases)))
237     edge_cases = int(math.ceil(cases * edge_fraction)) if
edge_fraction > 0 else 0
238     for case_number, seed in enumerate(seeds, start=1):
239         case_dir = output_dir / "cases" / f"case_{case_number
:03d}"
240         # Deterministically assign the requested fraction to
the first cases;
241         # unlike modulo rounding this still exercises an edge
case for small
242         # smoke runs and never silently rounds a nonzero
fraction to zero.
243         backend = t3.LocalBackend(seed, edge=(case_number <=
edge_cases))
244         truth = t3.scenario_record(backend)
245         robot = t3.Robot(backend, "LOCAL-SIM", case_dir, quiet
=True, layout=layout)
246         result = robot.run()
247         truth_rows = _source_rows(backend, robot, case_dir,
case_number, seed)
248         case_rows.append(_case_row(result, truth, case_number,
seed, truth_rows))
249         source_rows.extend(truth_rows)
250         (case_dir / "scenario.json").write_text(json.dumps(
truth, ensure_ascii=False, indent=2), encoding="utf-8")
251         print(json.dumps(case_rows[-1], ensure_ascii=False),
flush=True)
252         aggregate = _aggregate(case_rows, source_rows)
253         _write_csv(output_dir / "case_metrics.csv", case_rows)
254         _write_csv(output_dir / "source_metrics.csv", source_rows)
255         payload = {
256             "schema_version": 1,

```

```

257     "experiment": "q3_localbackend_monte_carlo",
258     "official_simulator_test": False,
259     "generated_at": time.strftime("%Y-%m-%dT%H:%M:%S%z"),
260     "layout": layout,
261     "seed_start": int(seed_start),
262     "seeds": seeds,
263     "edge_fraction": float(edge_fraction),
264     "cases": case_rows,
265     "aggregate": aggregate,
266     "source_metrics_csv": "source_metrics.csv",
267     "case_metrics_csv": "case_metrics.csv",
268     "limitations": [
269         "LocalBackend is a synthetic offline evaluator and
270         not the official simulator.",
271         "The source truth is used only after each local
272         run to score errors.",
273         "Program runtime is local Python wall time;
274         virtual_time_s is the task cost metric.",
275     ],
276     }
277     (output_dir / "simulation_results.json").write_text(
278         json.dumps(_json_safe(payload), ensure_ascii=False,
279         indent=2, allow_nan=False), encoding="utf-8"
280     )
281     print(json.dumps(aggregate, ensure_ascii=False, indent=2),
282           flush=True)
283     return payload
284
285 def build_parser() -> argparse.ArgumentParser:
286     parser = argparse.ArgumentParser(description=__doc__)
287     parser.add_argument("--cases", type=int, default=30)
288     parser.add_argument("--seed-start", type=int, default
289     =2026091300)
290     parser.add_argument("--output-dir", type=Path, default=
291     DEFAULT_OUTPUT)
292     parser.add_argument("--layout", choices=list(t3.
293     LAYOUT_PRESETS), default=t3.DEFAULT_LAYOUT)
294     parser.add_argument("--edge-fraction", type=float, default
295     =0.0)
296     return parser
297
298 if __name__ == "__main__":
299     args = build_parser().parse_args()
300     run_simulation(args.cases, args.seed_start, args.
301     output_dir, args.layout, args.edge_fraction)

```

#### Q4final.py

```

1  #!/usr/bin/env python3
2  """ 问题四B版：位置—类型—朝向—固定接收半径联合估计。
3  Python 3.9+, 依赖 numpy scipy。
4  运行：python rolling21_q4.py --robot-id 参赛队号
5  本地：python rolling21_q4.py --self-test 10 --test-output
6  q4_tests
7  压力：python rolling21_q4.py --self-test 3 --source-mode

```

```

    outward --edge
7  固定布局：中心+995米8点内圈+1865米12点外圈，运行前验证全方向
   覆盖。
8  无信号更新联合候选状态及其条件半径区间；不直接删除硬位置圆盘，
   探索次数受限。
9  成功测站的凸组合保证定向/全向接收；其他候选明确为探索性补测。
10 方向概率和高斯信息量均为调度代理，不构成几何证书。
11 光学清除独立于发射方向；保留25米网格兜底。时限内未完成会如实报
   告。
12 本地环境为合成验证，不是官方模拟器，不读取官方隐藏数据。
13 ""
14 import numpy as np
15 from scipy.optimize import minimize, linprog
16 from scipy.spatial import ConvexHull, QhullError
17 import argparse
18 import csv
19 import hashlib
20 import json
21 import math
22 import random
23 import time
24 import uuid
25 from pathlib import Path
26 from urllib.request import Request, build_opener, ProxyHandler
27 from urllib.error import HTTPError, URLError
28
29 DELTA = math.radians(1.006) # ±1度加两位小数取整裕量
30 N_CIRCLE = 96
31
32 def dist(a, b):
33     return math.hypot(a[0]-b[0], a[1]-b[1])
34
35 def clip(poly, a, b):
36     """裁剪  $a \cdot x \leq b$ ，返回凸多边形；边界使用微小外放容差。"""
37     if not poly:
38         return []
39     b += 1e-8
40     out = []
41     prev = poly[-1]
42     fp = a[0]*prev[0]+a[1]*prev[1]-b
43     for cur in poly:
44         fc = a[0]*cur[0]+a[1]*cur[1]-b
45         if (fp <= 0) != (fc <= 0):
46             t = fp/(fp-fc)
47             out.append((prev[0]+t*(cur[0]-prev[0]), prev[1]+t
48                 *(cur[1]-prev[1])))
49             if fc <= 0:
50                 out.append(cur)
51             prev, fp = cur, fc
52     return out
53
54 def disk_clip(poly, center, radius):
55     # 圆的切线半平面：外接近似，不能用内接多边形排除真实目标。
56     for k in range(N_CIRCLE):
57         t = 2*math.pi*k/N_CIRCLE

```

```

57     a = (math.cos(t), math.sin(t))
58     poly = clip(poly, a, radius+a[0]*center[0]+a[1]*center
59     [1])
60     return poly
61 def initial_poly():
62     return disk_clip([(-1800,-1800),(1800,-1800),(1800,1800)
63     ,(-1800,1800)], (0,0),1800)
64 def update_poly(poly, q, angle):
65     t = math.radians(angle)
66     for a in [(math.sin(t-DELTA),-math.cos(t-DELTA)),
67     (-math.sin(t+DELTA),math.cos(t+DELTA))]:
68         poly = clip(poly,a,a[0]*q[0]+a[1]*q[1])
69     poly = disk_clip(poly,q,1500)
70     if not poly:
71         raise RuntimeError('定位约束为空：检查角度、题目类型或
72         误差界；停止，避免误判完成。')
73     return poly
74 def contains(poly, p):
75     return all((b[0]-a[0])*(p[1]-a[1])-(b[1]-a[1])*(p[0]-a[0])
76     >= -1e-6
77     for a,b in zip(poly,poly[1:]+poly[:1]))
78 def circum(a,b,c):
79     # 相对坐标计算，减少消减误差。
80     ux,uy=b[0]-a[0],b[1]-a[1]
81     vx,vy=c[0]-a[0],c[1]-a[1]
82     det=2*(ux*vy-uy*vx)
83     if abs(det)<1e-12:
84         return None
85     u2,v2=ux*ux+uy*uy,vx*vx+vy*vy
86     return (a[0]+(u2*vy-v2*uy)/det,a[1]+(ux*v2-vx*u2)/det)
87 def mec(poly):
88     """枚举2/3个支撑顶点，小型定位多边形适用；最终全顶点认证。
89     """
90     if len(poly)==1:
91         return poly[0],0.0
92     center=(sum(p[0] for p in poly)/len(poly),sum(p[1] for p
93     in poly)/len(poly))
94     best=max(dist(center,p) for p in poly)
95     def consider(c,r):
96         nonlocal center,best
97         if r<best and all(dist(c,p)<=r+1e-7 for p in poly):
98             center,best=c,max(dist(c,p) for p in poly)
99     for i,a in enumerate(poly):
100         for j in range(i):
101             b=poly[j]
102             c=((a[0]+b[0])/2,(a[1]+b[1])/2)
103             consider(c,dist(a,b)/2)
104             for k in range(j):
105                 c=circum(a,b,poly[k])
106                 if c is not None:
107                     consider(c,dist(c,a))

```

```

107     return center,max(dist(center,p) for p in poly)
108
109 def polygon_diameter(poly):
110     """问题一证书：凸多边形直径由顶点对取得；顶点少，直接准确
    枚举。"""
111     if len(poly)<2:return 0.
112     return max(dist(poly[i],poly[j]) for i in range(len(poly))
    for j in range(i))
113
114 def problem1_geometry_certificate(poly,center=None,radius=None
    ):
115     """记录 MEC 与 Jung 上界；只作证书，不参与任何调度决策。"""
116     if center is None or radius is None:center,radius=mec(poly
    )
117     diameter=polygon_diameter(poly);jung_upper=diameter/math.
    sqrt(3)
118     tol=1e-7*max(1.,diameter)
119     ratio=radius/jung_upper if jung_upper>tol else (0. if
    radius<=tol else math.inf)
120     radius_to_diameter=radius/diameter if diameter>tol else
    (0. if radius<=tol else math.inf)
121     return {'vertex_count':len(poly),'diameter_m':diameter,'
    mec_center':list(center),
122             'mec_radius_m':radius,'jung_upper_m':jung_upper,'
    jung_ratio':ratio,
123             'radius_to_diameter':radius_to_diameter,
124             'jung_certified':radius<=jung_upper+tol,'
    numeric_tolerance_m':tol}
125
126 # 可通过命令行调整。权重是选址偏好，不是目标位置的概率密度。
127 SCHEDULE_CONFIG = {'search_gain':0.2,'lookahead':1.0,'
    cell_size':100.}
128
129 def mesh(poly, depth):
130     v=np.asarray(poly,dtype=float)
131     center=v.mean(axis=0)
132     tri=np.array([[center,a,b] for a,b in zip(v,np.roll(v,-1,
    axis=0))])
133     for _ in range(depth):
134         a,b,c=tri[:,0],tri[:,1],tri[:,2]
135         ab,bc,ca=(a+b)/2,(b+c)/2,(c+a)/2
136         tri=np.concatenate([np.stack(x,axis=1) for x in
137                             [(a,ab,ca),(ab,b,bc),(ca,bc,c),(ab,bc,ca)]]
138                             )
139         a,b=tri[:,1]-tri[:,0],tri[:,2]-tri[:,0]
140         area=np.abs(a[:,0]*b[:,1]-a[:,1]*b[:,0])/2
141         return tri[area>1e-10],area[area>1e-10]
142
143 BAYES_CONFIG={'noise_deg':1/math.sqrt(3),'quad_depth':3,'
    prior_exists':0.65}
144
145 # 问题二仅作 V2 主目标近乎同值时的决胜项；默认参数不改动任何
    V2 配置。
146 Q12_LINK_CONFIG={'eps_w':0.2,'p_w':2.0,'tau':0.01,'
    min_dop_improvement':0.02,
    'max_primary_loss_ratio':0.005,'

```

```

147     'primary_tie_rel_tol':1e-6,
148         'sin_floor':0.02, 'collinear_sin_tol':1e-8,
149         'max_collinear_weight_fraction':0.05, '
150     enable_dop_override':True}
151
152 def point_segment_distances(points,a,b):
153     """批量计算点到线段 ab 的欧氏距离，兼容退化边。"""
154     edge=b-a;length2=float(edge@edge)
155     if length2<=1e-24:return np.linalg.norm(points-a,axis=1)
156     t=np.clip((points-a)@edge/length2,0.,1.)
157     return np.linalg.norm(points-(a+t[:,None]*edge),axis=1)
158
159 def depth_weighted_dop(poly,station1,candidate,quad_depth=2):
160     """以三角形面积求积近似连续区域深度加权 DOP。"""
161     triangles,areas=mesh(poly,quad_depth);points=triangles.
162     mean(axis=1)
163     if not len(points) or not np.all(np.isfinite(points)):
164     return math.inf
165     vertices=np.asarray(poly,dtype=float)
166     depths=np.full(len(points),np.inf)
167     for a,b in zip(vertices,np.roll(vertices,-1,axis=0)):
168     depths=np.minimum(depths,point_segment_distances(
169     points,a,b))
170     d_star=float(np.max(depths))
171     if not math.isfinite(d_star) or d_star<=1e-12:return math.
172     inf
173     cfg=Q12_LINK_CONFIG
174     weights=areas*(cfg['eps_w']+(1-cfg['eps_w'])*(np.maximum(
175     depths,0.)/d_star)**cfg['p_w'])
176     u=points-np.asarray(station1,dtype=float);v=points-np.
177    .asarray(candidate,dtype=float)
178     r1=np.linalg.norm(u,axis=1);r2=np.linalg.norm(v,axis=1)
179     denom=r1*r2
180     cross=np.abs(u[:,0]*v[:,1]-u[:,1]*v[:,0])
181     sin_alpha=np.divide(cross,denom,out=np.zeros_like(cross),
182     where=denom>1e-12)
183     bad=(denom<=1e-12)|(~np.isfinite(sin_alpha))|(sin_alpha<=
184     cfg['collinear_sin_tol'])
185     total=float(weights.sum())
186     if (not math.isfinite(total) or total<=1e-300 or
187     float(weights[bad].sum())/total>cfg['
188     max_collinear_weight_fraction']):return math.inf
189     dop=np.sqrt(r1*r1+r2*r2)/np.maximum(np.abs(sin_alpha),cfg[
190     'sin_floor'])
191     value=float(weights@dop/total)
192     return value if math.isfinite(value) else math.inf
193
194 def gaussian_moments(points,weights):
195     w=np.asarray(weights,dtype=float);total=float(w.sum())
196     if not math.isfinite(total) or total<=1e-300:raise
197     ValueError('后验积分权重退化')
198     w=w/total;mu=w@points;diff=points-mu
199     covariance=(diff*w[:,None]).T@diff
200     values,vectors=np.linalg.eigh((covariance+covariance.T)/2)
201     covariance=(vectors*np.maximum(values,1e-4))@vectors.T
202     return mu,covariance

```

```

190
191 def radius_interval(points, positives, negatives):
192     lower=np.full(len(points),1000.);upper=np.full(len(points)
193     ,1500.)
194     for q in positives:lower=np.maximum(lower,np.linalg.norm(
195     points-q,axis=1))
196     for q in negatives:upper=np.minimum(upper,np.linalg.norm(
197     points-q,axis=1))
198     return lower,upper
199
200 def update_gaussian(target,q,angle):
201     """受几何支持集约束的假定密度滤波(ADF)。
202      $p_{\text{new}}(G) \propto N(G;\mu,\Sigma)$ *测角似然*接收半径条件似然*I(G在可行域)。
203     对非高斯后验积分求均值/协方差，再用二维高斯近似。
204     首次更新采用圆域内均匀先验： $R \sim U[1000,1500]$ 且固定，跨观测共享。
205     第四问仅保留阳性接收的半径下界；阴性与朝向似然不进入位置矩更新。
206     """
207     triangles,areas=mesh(target['poly'],BAYES_CONFIG['
208     quad_depth'])
209     points=triangles.mean(axis=1)
210     positives=target['stations'];negatives=[] # 定向负观测不能
211     转成接收半径上界
212     lower,upper=radius_interval(points,positives,negatives)
213     new_width=np.maximum(upper-lower,0.)
214     delta=points-np.asarray(q);distance=np.linalg.norm(delta,
215     axis=1)
216     residual=(np.arctan2(delta[:,1],delta[:,0])-math.radians(
217     angle)+math.pi)%(2*math.pi)-math.pi
218     variance=math.radians(BAYES_CONFIG['noise_deg'])**2
219     logw=np.log(areas)-residual**2/(2*variance)
220     if 'mean' in target:
221         diff=points-target['mean'];inv=np.linalg.inv(target['
222         covariance'])
223         logw-=0.5*np.einsum('ij,jk,ik->i',diff,inv,diff)
224         old_lower,old_upper=radius_interval(points,positives
225        [:-1],negatives)
226         old_width=np.maximum(old_upper-old_lower,0.)
227         likelihood=np.divide(new_width,old_width,out=np.
228         zeros_like(new_width),where=old_width>1e-10)
229     else:
230         # 仅结合有效示向的接收半径下界，阴性不作为位置似然。
231         likelihood=new_width/500.
232         valid=(distance>5)&(likelihood>0)&(np.linalg.norm(points,
233         axis=1)<=1800+1e-6)
234         for station in positives:valid &= np.linalg.norm(points-np
235         .asarray(station),axis=1)>5
236         failure_support=np.ones(len(points),dtype=bool)
237         for failed in target.get('failed_clears',[]):
238             failure_support &= np.linalg.norm(points-np.asarray(failed),
239             axis=1)>20
240         valid &= failure_support
241         note='gaussian_moment_projection'

```

```

228     if np.any(valid):
229         logw[valid]+=np.log(likelihood[valid]);logw[~valid]=-
np.inf
230         weights=np.exp(logw-np.max(logw))
231     else:
232         # 极窄/边界区域可能被有限求积漏采；回到几何区域的保守
矩近似，绝不判空。
233         weights=areas*failure_support
234         if weights.sum()<=0:weights=areas
235         note='geometric_moment_fallback'
236         mu,cov=gaussian_moments(points,weights)
237         target['mean']=mu;target['covariance']=cov
238         return {'mean':mu.tolist(),'covariance':cov.tolist(),'note
':note,
239                 'quadrature_points':len(points),'noise_std_deg':
BAYES_CONFIG['noise_deg'],
240                 'gaussian_entropy_nats':float(math.log(2*math.pi*
math.e)+0.5*np.linalg.slogdet(cov)[1])}
241
242 def update_unknown_belief(belief,coverage_map,ch):
243     # 不把未收信号解释为目标在1000米圆盘外；存在性仅由完整21点
扫描认证。
244     return {'mean':belief['mean'].tolist(),'covariance':belief
['covariance'].tolist(),
245             'existence_probability':belief['existence'],
246             'note':'
directional_negative_recorded_without_position_exclusion'}
247
248 def binary_entropy(p):
249     if p<=0 or p>=1:return 0.
250     return -p*math.log(p)-(1-p)*math.log1p(-p)
251
252 def reception_hull_certified(stations,p):
253     """成功测站凸包内保持固定半径圆盘及发射半平面内，二者均
凸。"""
254     a=np.unique(np.asarray(stations,dtype=float).reshape(-1,2)
,axis=0)
255     if not len(a):return False
256     p=np.asarray(p,dtype=float)
257     if len(a)>=3:
258         try:
259             eq=ConvexHull(a).equations
260             return bool(np.max(eq[:,2])@p+eq[:,2])<=1e-8)
261         except QhullError:pass
262     i,j=max(((i,j) for i in range(len(a)) for j in range(len(a)
))),key=lambda ij:np.linalg.norm(a[ij[0]]-a[ij[1]])
263     v=a[j]-a[i];den=float(v@v)
264     if den<1e-16:return bool(np.linalg.norm(p-a[i])<1e-8)
265     t=float((p-a[i])@v/den)
266     return bool(-1e-12<=t<=1+1e-12 and np.linalg.norm(p-a[i]-t
*v)<1e-8)
267
268 def direction_proxy(target,p):
269     # 在位置均值处对固定朝向做角度求积；仅是启发式成功概率。
270     mu=np.asarray(target['mean']);angles=np.arange(360)*math.
pi/180

```

```

271 normals=np.c_[np.cos(angles),np.sin(angles)]
272 positive=np.asarray(target['stations'])-mu
273 allowed=np.all(normals@positive.T>=-1e-8,axis=1)
274 omni_possible=True
275 for q in target.get('negatives',[]):
276     v=np.asarray(q)-mu
277     if np.linalg.norm(v)<1000:
278         allowed &= normals@v<0
279         omni_possible=False
280 if not allowed.any():return .5
281 frac=float(np.mean(normals[allowed]@(np.asarray(p)-mu)>=0)
)
282 # 0.5全向先验为选址假设，不据此判定真实源类型。
283 return max(.05,min(.99,(.5+.5*frac) if omni_possible else
frac))
284
285 def bayes_select(poly,stations,current,target,deadline):
286     """先保证原有距离强约束；辐射接收可认证则认证，否则明确探
索并限次。"""
287     begin=time.monotonic();mu=np.asarray(target['mean']);cov=
target['covariance']
288     st=np.asarray(stations);tri,_=mesh(poly,2)
289     station_r2=np.sum((tri[:, :, None, :] - st[None, None, :, :])**2,
axis=3)
290     variance=math.radians(BAYES_CONFIG['noise_deg'])**2
291     candidates=[]
292     for origin in (mu,np.asarray(target['center'])):
293         for radius in (25.,50.,100.,200.,400.):
294             for theta in np.arange(16)*math.pi/8:
295                 candidates.append(origin+radius*np.array([math
.cos(theta),math.sin(theta)]))
296     for q in st:
297         for fraction in (.25,.5,.75,.9):candidates.append((1-
fraction)*q+fraction*mu)
298     for i in range(len(st)):
299         for j in range(i):
300             for t in (.25,.5,.75):candidates.append((1-t)*st[i
]+t*st[j])
301     rows=[];seen=set()
302     for p in candidates:
303         if time.monotonic()>deadline-20:raise BudgetStop('第四
问选址接近截止时间')
304         key=tuple(np.round(p,6))
305         if key in seen:continue
306         seen.add(key)
307         if np.min(np.linalg.norm(st-p,axis=1))<5 or np.linalg.
norm(p-mu)<20:continue
308         if any(dist(p,q)<5 for q in target.get('negatives',[]
)):continue
309         d2=np.sum((tri-p)**2,axis=2)
310         upper=float(np.max(np.minimum(np.max(d2-1e6,axis=1),np
.min(np.max(d2[:, :, None] - station_r2, axis=1), axis=1))))
311         certified=reception_hull_certified(st,p)
312         if not certified and upper> -1e-4:continue
313         delta=mu-p;r2=float(delta@delta);h=np.array([-delta
[1],delta[0]])/r2

```

```

314         info=.5*math.log1p(max(0.,float(h@cov@h))/variance)
315         probability=1. if certified else direction_proxy(
target,p)
316         duration=10+dist(current,p)/5+.5*dist(p,mu)/5
317         score=probability*info/duration
318         rows.append(dict(p=tuple(float(x) for x in p),score=
score,info=info,probability=probability,
319                             duration=duration,certified=certified
,range_upper=upper))
320         if not rows:return None,{ 'reason':'no_feasible_new_point',
'reception_certificate':{'certified':False}}
321         original=max(rows,key=lambda x:x['score']);chosen=original
;cfg=Q12_LINK_CONFIG
322         tol=min(cfg['tau'],cfg['max_primary_loss_ratio'],cfg['
primary_tie_rel_tol'])
323         tied=[r for r in rows if (original['score']-r['score'])/
max(original['score'],1e-12)<=tol]
324         olddop=depth_weighted_dop(poly,stations[0],original['p'])
325         bestdop=olddop
326         for row in tied:
327             dop=depth_weighted_dop(poly,stations[0],row['p'])
328             if cfg['enable_dop_override'] and math.isfinite(olddop
) and dop<bestdop and dop<=olddop*(1-cfg['
min_dop_improvement']):
329                 chosen=row;bestdop=dop
330                 cert={'certified':chosen['certified'],
331                       'method':'convex_combination_of_positive_stations'
if chosen['certified'] else 'exploratory_direction_unknown',
332                       'range_certified':True}
333                 return chosen['p'],{ 'reason':'selected','position':chosen[
'p'],
334                                     'expected_information_nats':chosen['probability']*
chosen['info'],
335                                     'conditional_bearing_information_nats':chosen['info'],
336                                     'detection_probability_proxy':chosen['probability'],
337                                     'information_per_second':chosen['score'],
estimated_action_chain_s':chosen['duration'],
338                                     'reception_certificate':cert,'elapsed_s':time.
monotonic()-begin,
339                                     'problem2_dop_tiebreak':{'override':chosen is not
original,'candidate_count':len(rows),
340                                     'original_dop':olddop if math.isfinite(olddop)
else None,
341                                     'chosen_dop':bestdop if math.isfinite(bestdop)
else None},
342                                     'note':'position_mean_orientation_proxy;
uncertified_direction_candidates_may_return_no_signal'}
343
344 ROUTE_MEASURE_CONFIG={'min_information_ratio':0.70,'
min_saving_s':10.0,
345                       'segment_fractions':(0.25,0.5,0.75,1.0),
346                       'min_stop_information_nats':0.50,'
max_target_channels_per_stop':6}
347
348 def mandatory_stop_measurement_score(target,q):
349     """评价必经扫描点是否适合作为已发现目标的顺路补测点。"""

```

```

350     q=np.asarray(q,dtype=float);poly=target.get('poly') or []
351     if not poly or 'mean' not in target or 'covariance' not in
target:return None
352     # 对凸定位区域,距固定点的最大值在顶点取得;距离约束之外还
需成功测站凸包的辐射方向认证。
353     receive_bound=max(np.linalg.norm(np.asarray(v,dtype=float)
-q) for v in poly)
354     if receive_bound>1000-1e-6 or not reception_hull_certified
(target['stations'],q):return None
355     stations=np.asarray(target.get('stations',[]),dtype=float)
356     if len(stations) and float(np.min(np.linalg.norm(stations-
q,axis=1)))<5-1e-7:return None
357     mu=np.asarray(target['mean'],dtype=float);delta=mu-q;r2=
float(delta@delta)
358     if r2<20**2:return None
359     h=np.array([-delta[1],delta[0]])/r2
360     variance=math.radians(BAYES_CONFIG['noise_deg'])**2
361     info=0.5*math.log1p(max(0.,float(h@np.asarray(target['
covariance'])@h))/variance)
362     if info<ROUTE_MEASURE_CONFIG['min_stop_information_nats']:
return None
363     return {'information_nats':info,'receive_bound_m':float(
receive_bound)}
364
365 def route_aligned_measurement(poly,stations,current,target,
next_stop,free_point,free_detail):
366     """在当前点到下一必经点的线段上寻找可替代自由补测点。
367
368     仅当候选点满足保守接收条件、与既有测站/目标估计保持间距、
信息量达到
369     自由最优点的指定比例,并且预计“补测后去中心再回到下一节
点”的总链路
370     至少节省指定秒数时采用。这样不会仅为贴路线而牺牲过多交会质
量。
371     """
372     if next_stop is None or free_point is None:return
free_point,{**free_detail,'route_aligned':False,'
route_reason':'no_next_stop'}
373     mu=np.asarray(target['mean'],dtype=float);cov=np.asarray(
target['covariance'],dtype=float)
374     stations=np.asarray(stations,dtype=float);current=np.
asarray(current,dtype=float)
375     next_stop=np.asarray(next_stop,dtype=float);free_point=np.
asarray(free_point,dtype=float)
376     variance=math.radians(BAYES_CONFIG['noise_deg'])**2
377     triangles,_=mesh(poly,2)
378     station_r2=np.sum((triangles[:, :, None, :]-stations[None,
None, :, :])**2,axis=3)
379     def receive(p):
380         d2=np.sum((triangles-p)**2,axis=2)
381         upper=np.minimum(np.max(d2-1e6,axis=1),np.min(np.max(
d2[:, :, None]-station_r2,axis=1),axis=1))
382         return float(upper.max())
383     def information(p):
384         delta=mu-p;r2=float(delta@delta)

```

```

385         if r2<1e-8:return 0.
386         h=np.array([-delta[1],delta[0]])/r2
387         return 0.5*math.log1p(max(0.,float(h@cov@h))/variance)
388     def feasible(p):
389         return (reception_hull_certified(stations,p) and
receive(p)<=-1e-4 and np.linalg.norm(p-mu)>=20-1e-7 and
390             float(np.min(np.linalg.norm(p-stations,axis=1)
))>=5-1e-7)
391     def chain_seconds(p):
392         # 补测5秒; 频道切换对自由点和顺路点相同, 比较时可抵
消。
393         return (np.linalg.norm(p-current)+np.linalg.norm(p-mu)
+np.linalg.norm(mu-next_stop))/5+5
394         free_info=float(free_detail.get('expected_information_nats
',information(free_point)))
395         free_chain=chain_seconds(free_point)
396         candidates=[]
397         delta=next_stop-current
398         for fraction in ROUTE_MEASURE_CONFIG['segment_fractions']:
399             p=current+float(fraction)*delta
400             if feasible(p):
401                 info=information(p);chain=chain_seconds(p)
402                 candidates.append((chain, -info, float(fraction),
p, info))
403             if not candidates:
404                 return tuple(free_point),{**free_detail,'route_aligned
':False,'route_reason':'no_feasible_route_candidate'}
405             chain,_,fraction,p,info=min(candidates,key=lambda x:(x[0],
x[1]))
406             saving=free_chain-chain
407             ratio=info/max(free_info,1e-12)
408             if ratio>=ROUTE_MEASURE_CONFIG['min_information_ratio']
and saving>=ROUTE_MEASURE_CONFIG['min_saving_s']:
409                 point=tuple(float(v) for v in p)
410                 return point,{'reason':'route_aligned_selected','
position':point,
411                             'expected_information_nats':info,'
free_expected_information_nats':free_info,
412                             'information_ratio':ratio,'
estimated_chain_s':chain,
413                             'free_estimated_chain_s':free_chain,'
estimated_saving_s':saving,
414                             'segment_fraction':fraction,'next_stop':
tuple(float(v) for v in next_stop),
415                             'route_aligned':True,
416                             'reception_certificate':{'certified':
True,'method':'convex_combination_of_positive_stations'},
417                             'note':'
feasible_point_on_current_to_next_planned_stop'}
418                 return tuple(free_point),{**free_detail,'route_aligned':
False,
419                             'route_reason':
insufficient_information_or_saving',
420                             'best_route_information_ratio':
ratio,
421                             'best_route_estimated_saving_s':

```

```

422         saving,
423         'next_stop':tuple(float(v) for v
424         in next_stop))}
425
426 PROBE_CONFIG={'max_attempts':5,'min_probability':0.2,'
427 max_radius':80.}
428
429 JOINT_CONFIG={'max_detour_m':5.,'horizon':3,'min_saving_s':0.,
430 'candidate_limit':12}
431
432 def probe_distribution(target):
433     """失败clear后保留条件分布的非高斯形状，不反复用高斯投影重
434     加权。
435     新测向到来时(测点数改变)重新以新高斯工作后验生成求积权重。
436     """
437     version=len(target.get('stations',[]))
438     cached=target.get('probe_base')
439     if cached is None or cached['version']!=version:
440         triangles,areas=mesh(target['poly'],4);points=
441         triangles.mean(axis=1)
442         diff=points-target['mean'];inv=np.linalg.inv(target['
443         covariance'])
444         logw=np.log(areas)-0.5*np.einsum('ij,jk,ik->i',diff,
445         inv,diff)
446         weights=np.exp(logw-np.max(logw));weights/=weights.sum
447         ()
448         cached={'version':version,'points':points,'weights':
449         weights};target['probe_base']=cached
450         points=cached['points'];weights=cached['weights'].copy()
451         for q in target.get('failed_clears',[]):weights[np.linalg.
452         norm(points-q,axis=1)<=20]=0.
453         if weights.sum()<=1e-300:return None
454         weights/=weights.sum()
455         return points,weights
456
457 def joint_probe_plan(target,current,current_channel,ch,anchor,
458 same_target,next_stop):
459     """在实际即将执行的路段上插入1~3次clear，枚举成功/全部失败
460     分支。
461     返回值为启发式预计节省，不是无条件时间上界；只执行第一步，
462     反馈后滚动重算。
463     """
464     remaining_budget=PROBE_CONFIG['max_attempts']-target.get('
465     probe_attempts',0)
466     if remaining_budget<=0 or target['radius']>PROBE_CONFIG['
467     max_radius'] or target.get('planned') is None:return None
468     sample=probe_distribution(target)
469     if sample is None:return None
470     points,weights=sample;current=np.asarray(current,dtype=
471     float);anchor=np.asarray(anchor,dtype=float)
472     vector=anchor-current;length2=float(vector@vector);direct=
473     dist(current,anchor)
474     def project(q):
475         if length2<=1e-12:return current.copy()
476         return current+np.clip(float((np.asarray(q)-current)
477         @vector)/length2,0,1)*vector

```

```

459     seeds=[tuple(current),tuple(project(target['mean'])),tuple
460     (project(target['center']))]
461     if length2>1e-12:
462         direction=vector/math.sqrt(length2);center=project(
463         target['mean'])
464         for offset in (-40,-20,-10,10,20,40):seeds.append(
465         tuple(project(center+offset*direction)))
466         poly=np.asarray(target['poly']);low=poly.min(axis=0);high=
467         poly.max(axis=0)
468         n=np.maximum(1,np.ceil((high-low)/25).astype(int))
469         seeds.extend([tuple(target['mean']),tuple(target['center'
470         ])])
471         for iy in range(n[1]):
472             for ix in range(n[0]):
473                 q=low+(np.array([ix,iy])+.5)*(high-low)/n
474                 seeds.extend([tuple(q),tuple(project(q))])
475         candidates=[];seen=set()
476         for q in seeds:
477             key=tuple(round(x,5) for x in q)
478             if key in seen:continue
479             seen.add(key)
480             if any(dist(q,p)<1 for p in target.get('failed_clears'
481             ,[])):continue
482             extra=dist(current,q)+dist(q,anchor)-direct
483             if extra>JOINT_CONFIG['max_detour_m']+1e-7:continue
484             mask=np.linalg.norm(points-q,axis=1)<=20
485             mass=float(weights[mask].sum())
486             if mass<0.01:continue
487             candidates.append({'q':tuple(q),'mask':mask,'mass':
488             mass,'extra':max(0.,extra)})
489             candidates.sort(key=lambda c:c['mass']/(3+dist(current,c['
490             q'])/5),reverse=True)
491             candidates=candidates[:JOINT_CONFIG['candidate_limit']]
492             if not candidates:return None
493             planned=target['planned'];center=target['center']
494             if same_target:
495                 # baseline: current->测向点->估计清除点->next_stop。
496                 tail=5+(current_channel!=ch)+dist(anchor,center)/5+5
497                 if next_stop is not None:tail+=dist(center,next_stop)
498                 /5
499                 success_tail=lambda q:dist(q,next_stop)/5 if next_stop
500                 is not None else 0.
501             else:
502                 # 顺路处理其他目标：保留既定路线，估算避免将来专门定位
503                 该目标的成本。
504                 tail=dist(anchor,planned)/5+5+(current_channel!=ch)+
505                 dist(planned,center)/5+5
506                 success_tail=lambda q:dist(q,anchor)/5
507                 baseline=direct/5+tail
508                 best=None;max_depth=min(JOINT_CONFIG['horizon'],
509                 remaining_budget)
510                 def visit(order,excluded,prob_remaining,travel_so_far,
511                 action_expected,last,success_return):
512                     nonlocal best
513                     if len(order)>=max_depth:return
514                     for i,candidate in enumerate(candidates):

```

```

501         if i in order:continue
502         q=candidate['q'];distance=dist(last,q)
503         route_length=travel_so_far+distance
504         detour=route_length+dist(q,anchor)-direct
505         # 对“全部失败”的完整插入路径限制额外路程，而非分
别限制每一个点。
506         if detour>JOINT_CONFIG['max_detour_m']+1e-7:
continue
507         fresh=candidate['mask']&~excluded
508         probability=.995*float(weights[fresh].sum()) # 预
留求积遗漏尾部
509         if probability<=1e-6:continue
510         conditional=probability/prob_remaining
511         if not order and conditional<PROBE_CONFIG['
min_probability']:continue
512         if order and conditional<0.05:continue
513         after=max(.005,prob_remaining-probability)
514         expected=action_expected+prob_remaining*(distance
/5+3)+2*probability
515         returned=success_return+probability*success_tail(q
)
516         total=expected+returned+after*(dist(q,anchor)/5+
tail)
517         sequence=order+[i]
518         saving=baseline-total
519         if saving>=JOINT_CONFIG['min_saving_s'] and (best
is None or saving>best['expected_saving_s']):
520             best={'position':candidates[sequence[0]]['q'],
521                 'sequence':[candidates[j]['q'] for j in
sequence],
522                 'probability':.995*candidates[sequence
[0]]['mass'],
523                 'sequence_success_probability':1-after,'
expected_time_s':total,
524                 'reference_time_s':baseline,'
expected_saving_s':saving,
525                 'worst_branch_extra_distance_m':max(0.,
detour),'anchor':tuple(anchor),
526                 'same_target':same_target,'hypothesis':
approximate_posterior_and_one_measurement_continuation'}
527             visit(sequence,excluded|candidate['mask'],after,
route_length,expected,q,returned)
528             visit([],np.zeros(len(points),dtype=bool),1.,0.,0.,tuple(
current),0.)
529             return best
530
531 def condition_failed_clear(target,q):
532     target.setdefault('failed_clears',[]).append(tuple(q))
533     sample=probe_distribution(target)
534     if sample is None:return {'note':
quadrature_missed_remaining_region_keep_geometric_outer_bound
'}
535     target['mean'],target['covariance']=gaussian_moments(*
sample)
536     target['planned']=None
537     return {'note':

```

```

538     posterior_conditioned_on_distance_greater_than_20',
        'mean':target['mean'].tolist(), 'covariance':target
        ['covariance'].tolist())
539
540 def certify(p,radius=1.,minside=1e-5):
541     stack=[(0.,0.,1.8)]; count=0; maxdepth=0
542     while stack:
543         x,y,h=stack.pop();count+=1
544         if np.hypot(max(abs(x)-h,0),max(abs(y)-h,0))>1.8:continue
545         corn=np.array([[x-h,y-h],[x-h,y+h],[x+h,y-h],[x+h,y+h]])
546         ok=np.max(np.sum((p[:,None,:]-corn[None,:,:])**2,axis=2),
                    axis=1)<(radius-1e-10)**2
547         q=p[ok]
548         if len(q)>=3:
549             try:
550                 eq=ConvexHull(q).equations
551                 if np.max(corn@eq[:,2].T+eq[:,2])<-1e-10:continue
552             except:pass
553         if h*2<minside:return dict(pass_all=False,box=[x,y,h],boxes=
            count)
554         hh=h/2
555         stack.extend((x+dx*hh,y+dy*hh,hh) for dx in [-1,1] for dy in
            [-1,1])
556     return dict(pass_all=True,boxes=count)
557
558 LAYOUT_PRESETS={'q4_21':{'ring_points':20,'ring_radius'
        :1865.}}
559 DEFAULT_LAYOUT='q4_21'
560
561 def scan_points(ring_points=20,ring_radius=1865.):
562     if ring_points!=20 or ring_radius!=1865.:raise ValueError(
        '第四问仅接受已验证的21点布局')
563     return [(0.,0.)]+[(r*math.cos(2*math.pi*k/n),r*math.sin(2*
        math.pi*k/n))
564                 for n,r in [(8,995.),(12,1865.)] for k
        in range(n)]
565
566 def resolve_layout(layout_name=DEFAULT_LAYOUT,ring_points=None
        ,ring_radius=None):
567     if layout_name!=DEFAULT_LAYOUT or ring_points is not None
        or ring_radius is not None:
568         raise ValueError('第四问固定21点，不接受自定义单环参数
        ')
569     check=certify(np.asarray(scan_points())/1000.,radius=.999)
570     if not check['pass_all']:raise ValueError('21点连续覆盖验
        证未通过')
571     return {'layout_name':DEFAULT_LAYOUT,'ring_points':20,'
        ring_radius':1865.,
572             'inner_points':8,'inner_radius':995.,'outer_points
        ':12,'outer_radius':1865.,
573             'certificate':{'certified':True,'method':
        'adaptive_box_local_convex_hull_float_certificate',
574                             'target_radius_m':1800.,
        'verified_reception_radius_m':999.,
575                             'all_emission_directions':True,'
        boxes':check['boxes'],

```

```

576         'note': 'double precision with
conservative tolerance; not interval arithmetic'}}
577
578 def rolling_open_route(current, nodes):
579     """访问全部当前任务点的开放欧氏路径；服务时间在本轮固定，
    不影响排序。
580     <=10个非当前位置节点时Held-Karp精确；更大时多初值最近邻+2-
    opt近似。
581     只决定本轮下一动作，每次响应后重新建节点，不承诺未知环境全
    局最优。
582     """
583     if not nodes: return [], {'method': 'empty', 'length_m': 0., '
exact': True}
584     zero = [i for i, n in enumerate(nodes) if dist(current, n['
position']) < 1e-6]
585     ids = [i for i in range(len(nodes)) if i not in zero]
586     n = len(ids)
587     if not n: return zero, {'method': 'at_current_position', '
length_m': 0., 'exact': True}
588     xy = np.asarray([current] + [nodes[i]['position'] for i in ids
], dtype=float)
589     d = np.linalg.norm(xy[:, None, :] - xy[None, :, :], axis=2)
590     if n <= 10:
591         count = 1 << n; dp = np.full((count, n), np.inf); parent = np.full
((count, n), -1, dtype=int)
592         for j in range(n): dp[1 << j, j] = d[0, j+1]
593         for mask in range(1, count):
594             for last in range(n):
595                 if not (mask >> last & 1): continue
596                 prev = mask ^ (1 << last)
597                 if not prev: continue
598                 candidates = [j for j in range(n) if prev >> j & 1]
599                 values = dp[prev, candidates] + d[np.asarray(
candidates) + 1, last + 1]
600                 k = int(np.argmin(values)); dp[mask, last] = values[
k]; parent[mask, last] = candidates[k]
601                 mask = count - 1; last = int(np.argmin(dp[mask])); length =
float(dp[mask, last]); order = []
602                 while mask:
603                     order.append(last); old = int(parent[mask, last]); mask
^= 1 << last; last = old
604                 order.reverse(); method = 'held_karp_exact'; exact = True
605             else:
606                 best = None
607                 for first in np.argsort(d[0, 1:])[ : min(n, 6)]:
608                     order = [int(first)]; remaining = set(range(n)) - {int(
first)}
609                     while remaining:
610                         nxt = min(remaining, key=lambda j: d[order[-1] + 1, j
+ 1]); order.append(nxt); remaining.remove(nxt)
611                     for _ in range(40):
612                         gain = 0.; pair = None
613                         for i in range(n-1):
614                             a = 0 if i == 0 else order[i-1] + 1; b = order[i] + 1
615                             for j in range(i+1, n):
616                                 c = order[j] + 1

```

```

617         delta=d[a,b]-d[a,c]
618         if j+1<n:
619             end=order[j+1]+1;delta+=d[c,end]-d
        [b,end]
620         if delta>gain+1e-9:gain=delta;pair=(i,
        j)
621         if pair is None:break
622         i,j=pair;order[i:j+1]=reversed(order[i:j+1])
623         length=float(d[0,order[0]+1]+sum(d[a+1,b+1] for a,
        b in zip(order,order[1:])))
624         if best is None or length<best[0]:best=(length,
        list(order))
625         length,order=best;method='multi_start_2opt_approximate
        ';exact=False
626         return zero+[ids[i] for i in order],{'method':method,'
        length_m':length,'exact':exact,'task_nodes':len(nodes)}
627
628 class CoverageMap:
629     """每频道独立的保守剩余区域地图。
630     与目标圆相交的方格全部保留；定向no_signal不删除任何格。
631     格心仅为位置矩辅助，不作为真实目标离散位置或缺席证书。
632     """
633     def __init__(self, cell_size=100.):
634         self.n=math.ceil(3600/cell_size)
635         self.step=3600/self.n
636         axis=-1800+np.arange(self.n)*self.step
637         x,y=np.meshgrid(axis,axis)
638         lo=np.column_stack([x.ravel(),y.ravel()]); hi=lo+self.
        step
639         nearest=np.maximum(np.maximum(lo,-hi),0.)
640         self.valid=np.sum(nearest**2,axis=1)<=1800**2+1e-6
641         self.ids=np.flatnonzero(self.valid)
642         self.centers=(lo[self.valid]+hi[self.valid])/2
643         self.corners=np.stack([lo[self.valid],lo[self.valid]+[
        self.step,0],
644                                hi[self.valid],lo[self.valid
        ]+[0,self.step]],axis=1)
645         self.remaining=np.ones((20,len(self.ids)),dtype=bool)
646         self.observations=np.zeros(20,dtype=int)
647         def covered(self,q):
648             return np.max(np.sum((self.corners-np.asarray(q))*2,
        axis=2),axis=1)<=(1000-1e-5)**2
649         def update_negative(self,ch,q):
650             # 不删除任何位置格；定向角未知，单次阴性不是位置排除证
        据。
651             self.observations[ch-1]+=1
652             return 0
653
654 class HttpBackend:
655     def __init__(self,url):
656         self.url=url.rstrip('/')
657         self.opener=build_opener(ProxyHandler({})) # 本机接口
        不经过系统代理
658         def post(self,path,payload,timeout):
659             data=json.dumps(payload,ensure_ascii=False,allow_nan=
        False).encode('utf-8')

```

```

660         req=Request(self.url+path,data=data,headers={'Content-
Type':'application/json'},method='POST')
661         with self.opener.open(req,timeout=timeout) as response
:
662             if response.status!=200:
663                 raise RuntimeError('非200 HTTP响应')
664             return json.loads(response.read().decode('utf-8'))
665
666 LOCAL_CONFIG={'source_mode':'mixed','edge':False}
667
668 class LocalBackend:
669     """独立生成10~16个全向/定向混合源；同频道同位置误差固定，
    无真值传给策略。"""
670     def __init__(self,seed,edge=False):
671         edge=edge or LOCAL_CONFIG['edge']
672         rng=random.Random(seed)
673         self.seed=seed
674         self.targets={}
675         for ch in rng.sample(range(1,21),rng.randint(10,16)):
676             r=1800 if edge else 1800*math.sqrt(rng.random())
677             t=rng.random()*2*math.pi
678             self.targets[ch]=((r*math.cos(t),r*math.sin(t))
,1000 if edge else rng.uniform(1000,1500))
679         orientation_rng=random.Random(seed+910921)
680         mode=LOCAL_CONFIG['source_mode']
681         keys=sorted(self.targets); self.directions={}
682         for i,ch in enumerate(keys):
683             directional=(mode in ('directional','outward') or
(mode=='mixed' and i%2==0))
684             p=self.targets[ch][0]
685             self.directions[ch]=(math.atan2(p[1],p[0]) if mode
=='outward' else orientation_rng.uniform(0,2*math.pi)) if
directional else None
686             self.total=len(self.targets)
687             self.done=set(); self.q=(0,0); self.ch=1; self.vt=0;
self.cache={}
688         def post(self,path,payload,timeout):
689             key=payload['request_id']
690             if key in self.cache:
691                 return self.cache[key]
692             result={'accepted':True,'real_timestamp_ms':int(time.
time()*1000)}
693             if path=='/enter':
694                 result.update(remaining_real_duration_s=1200,
max_virtual_duration_s=360000,max_real_duration_s=1200)
695             elif path in ('/measure','/clear'):
696                 q=(payload['position']['x'],payload['position']['y
']); ch=payload['channel']
697                 self.vt+=dist(self.q,q)/5; self.q=q
698                 target=self.targets.get(ch)
699                 d=dist(q,target[0]) if target and ch not in self.
done else math.inf
700                 if path=='/measure':
701                     self.vt+=5+(ch!=self.ch); self.ch=ch
702                     theta=self.directions.get(ch)
703                     visible=(theta is None or (q[0]-target[0][0])*

```

```

704     math.cos(theta)+(q[1]-target[0][1])*math.sin(theta)>=-1e-9)
       kind='no_signal' if not target or d>target[1]
or not visible else ('near' if d<=5 else 'direction')
705         result['measure_result']=kind
706         if kind=='direction':
707             token=f'{self.seed}:{ch}:{q[0]:.9f}:{q
[1]:.9f}'.encode()
708             error=2*int.from_bytes(hashlib.sha256(
token).digest()[:8], 'big')/(2**64-1)-1
709             angle=math.degrees(math.atan2(target
[0][1]-q[1],target[0][0]-q[0]))
710             result['svd_deg']=round((angle+error)
%360,2)%360
711         else:
712             success=d<=20; self.vt+=5 if success else 3
713             result['clear_result']='success' if success
else 'no_target_in_range'
714             if success: self.done.add(ch)
715         else:
716             result['exit_reason']='user_exit'
717             result['virtual_time_s']=self.vt
718             self.cache[key]=result
719             return result
720
721 class BudgetStop(RuntimeError):
722     pass
723
724 class Robot:
725     def __init__(self,backend,robot_id,out=None,quiet=False,
layout=None):
726         self.backend=backend; self.robot_id=robot_id; self.out
=out; self.quiet=quiet
727         self.layout=layout or resolve_layout()
728         self.prefix=uuid.uuid4().hex[:12]; self.seq=0; self.q
=(0.,0.); self.ch=1
729         self.vt=0.; self.vmax=360000.; self.deadline=math.inf;
self.started=time.monotonic()
730         self.done=set(); self.absent=set(); self.searched={c:
set() for c in range(1,21)}
731         self.distance=0.; self.measures=0; self.switches=0;
self.clears=0; self.fallbacks=0; self.information_records=[]
732         self.problem1_geometry_records=[]
733         self.log=None
734         if out:
735             out.mkdir(parents=True,exist_ok=True)
736             self.log=(out/'actions.jsonl').open('w',encoding='
utf-8')
737         def write_log(self,record):
738             if self.log:
739                 self.log.write(json.dumps(record,ensure_ascii=
False,allow_nan=False)+'\n'); self.log.flush()
740         def call(self,path,q=None,ch=None):
741             if path in ('/measure','/clear'):
742                 cost=dist(self.q,q)/5+6
743                 if time.monotonic()>self.deadline-12 or self.vt+
cost>self.vmax-1:

```

```

744         raise BudgetStop('接近运行时限，任务未完成，保
存当前结果并退出。')
745         self.seq+=1
746         payload={'arena_id':'default','robot_id':self.robot_id
,'request_id':f'{self.prefix}-{self.seq}'}
747         if q is not None:
748             payload.update(position={'x':float(q[0]),'y':float
(q[1])},channel=ch)
749             self.write_log({'event':'request','path':path,'payload
':payload})
750             for attempt in range(3):
751                 if time.monotonic()>=self.deadline:
752                     raise RuntimeError('本局已到现实截止时间；不再
发送请求。')
753                 try:
754                     result=self.backend.post(path,payload,max(0.1,
min(4.,self.deadline-time.monotonic()))))
755                     break
756                 except HTTPError as exc:
757                     self.write_log({'event':'http_error','code':
exc.code,'request_id':payload['request_id']})
758                     raise RuntimeError(f'HTTP {exc.code}，请检查模
拟器状态及请求参数。') from exc
759                 except (URLError,TimeoutError,ConnectionError,
OSError,json.JSONDecodeError) as exc:
760                     self.write_log({'event':'network_error','
attempt':attempt+1,'detail':str(exc),'request_id':payload['
request_id']})
761                     if attempt==2:
762                         raise RuntimeError('连接失败或响应丢失，动
作结果可能未知；已停止，不发送后续动作。请检查模拟器界面和日
志。') from exc
763                     time.sleep(0.3) # 同一个payload/request_id重
试，绝不并发
764                     self.write_log({'event':'response','path':path,'
request_id':payload['request_id'],'response':result})
765                     if result.get('accepted') is not True:
766                         raise RuntimeError('请求被拒绝 accepted=false；确
认参赛队号、接口就绪状态。')
767                     self.vt=float(result['virtual_time_s'])
768                     if path=='/enter':
769                         self.deadline=time.monotonic()+float(result['
remaining_real_duration_s'])
770                         self.vmax=float(result.get('max_virtual_duration_s
',360000))
771                     if q is not None:
772                         self.distance+=dist(self.q,q); self.q=tuple(q)
773                     if path=='/measure':
774                         self.measures+=1; self.switches+=ch!=self.ch; self
.ch=ch
775                     if result.get('measure_result') not in ('near','
direction','no_signal'):
776                         raise RuntimeError('未知检测响应')
777                     if path=='/clear':
778                         self.clears+=1

```

```

779         if result.get('clear_result') not in ('success', '
no_target_in_range'):
780             raise RuntimeError('未知清除响应')
781         if result['clear_result']=='success': self.done.
add(ch)
782         if not self.quiet and q is not None:
783             kind=result.get('measure_result',result.get('
clear_result'))
784             print(f'{self.seq:4d} {path:8s} ch={ch:2d} ({q
[0]:8.1f},{q[1]:8.1f}) {kind:20s} T={self.vt:.1f}s cleared={
len(self.done)}',flush=True)
785             return result
786         def clear(self,ch,q):
787             return self.call('/clear',q,ch)['clear_result']=='
success'
788         def clear_point(self, poly, center):
789             """在20米共同覆盖圆盘交集中选离当前位置近的清除点。"""
790             v=np.asarray(poly); current=np.asarray(self.q); scale
=1000.
791             def safe(z): return (19.8**2-np.sum((z*scale-v)**2,
axis=1))/400.
792             result=minimize(lambda z:float(np.sum((z-current/scale
)**2)),np.asarray(center)/scale,
793                             method='SLSQP',constraints=[{'type':'
ineq','fun':safe}],
794                             options={'maxiter':40,'ftol':1e-9})
795             q=tuple(result.x*scale)
796             return q if max(dist(q,p) for p in poly)<=19.80001
else center
797
798         def make_fallback(self, target):
799             """对当前保守凸区域作25米网格覆盖；失败后依次尝试剩余
格。"""
800             poly=target['poly']; cells=[]
801             xmin=min(p[0] for p in poly); xmax=max(p[0] for p in
poly)
802             ymin=min(p[1] for p in poly); ymax=max(p[1] for p in
poly)
803             for iy in range(max(1,math.ceil((ymax-ymin)/25))):
804                 y0=ymin+25*iy; y1=min(ymax,y0+25); row=[]
805                 for ix in range(max(1,math.ceil((xmax-xmin)/25))):
806                     x0=xmin+25*ix; x1=min(xmax,x0+25); part=poly
807                     for a,b in [((1,0),x1),((-1,0),-x0),((0,1),y1)
,((0,-1),-y0)]:
808                         part=clip(part,a,b)
809                         if part and not any(max(dist(v,failed) for v
in part)<=20 for failed in target.get('failed_clears',[])):
810                             row.append(((x0+x1)/2,(y0+y1)/2))
811                             cells.extend(row if iy%2==0 else row[::-1])
812                     if cells and dist(self.q,cells[-1])<dist(self.q,cells
[0]): cells.reverse()
813                     target['fallback']=cells; self.fallbacks+=1
814
815         def schedule_run(self):
816             self.targets={}
817             self.beliefs={ch:{'mean':np.zeros(2),'covariance':np.

```

```

818 eye(2)*1800**2/4,
      'existence':BAYES_CONFIG['
prior_exists'],'negatives':[]}] for ch in range(1,21))}
819 self.coverage_map=CoverageMap(SCHEDULE_CONFIG['
cell_size']) # 仅概率求积辅助, 不作为完成证书
820 scans=scan_points(self.layout['ring_points'],self.
layout['ring_radius'])
821 certificate=self.layout['certificate']
822 self.cpp_plan={'layout_name':self.layout['layout_name'
],
      'ring_points':self.layout['ring_points'
],
      'ring_radius':self.layout['ring_radius'
],
      'waypoint_count':len(scans),'waypoints'
:scans,
      'certificate':certificate}
826 if self.out:(self.out/'scan_plan.json').write_text(
827 json.dumps(self.cpp_plan,indent=2),encoding='utf-8')
828 self.write_log({'event':'coverage_certificate',**self.
cpp_plan})
829 self.scheduler_counts={'search_measures':0,'
localization_measures':0,'certified_clears':0,
830 'fallback_clears':0,'decisions':0,'probe_attempts'
:0,'probe_successes':0,
831 'joint_sequences_planned':0,'on_route_probes':0,'
exact_routes':0,'approximate_routes':0,
832 'batch_scan_visits':0,'batch_scan_channels':0,
833 'route_aligned_plans':0,'free_measurement_plans'
:0,
834 'dop_tie_break_decisions':0,'
dop_tie_break_overrides':0}
835 cache_key=None;cache_value=None
836 while len(self.done)<16 and len(self.done|self.absent)
<20:
837 if time.monotonic()>self.deadline-20:raise
BudgetStop('滚动规划接近现实截止时间。')
838 unknown=[ch for ch in range(1,21) if ch not in
self.targets and ch not in self.absent]
839 nodes=[]
840 for i,q in enumerate(scans):
841 channels=[ch for ch in unknown if i not in
self.searched[ch]]
842 if channels:nodes.append({'type':'scan','id':f
'scan-{i}','index':i,'position':q,'channels':channels})
843 for ch,target in self.targets.items():
844 if ch not in self.done:
845 nodes.append({'type':'source','id':f'
source-{ch}','channel':ch,
846 'position':tuple(target.get(
'mean',target['center']))})
847 if not nodes:raise RuntimeError('无任务节点, 但未
能证明全部清除。')
848 key=(tuple(self.q),tuple((n['id'],tuple(n['
position']))) for n in nodes))
849 if key!=cache_key:

```

```

850         cache_value=rolling_open_route(self.q,nodes);
cache_key=key
851         order,route_info=cache_value
852         self.scheduler_counts['exact_routes' if route_info
['exact'] else 'approximate_routes']+=1
853         node=nodes[order[0]]
854         next_stop=nodes[order[1]]['position'] if len(order
)>1 else None
855         self.scheduler_counts['decisions']+=1
856         self.write_log({'event':'rolling_route','nodes':
nodes,'order':[nodes[i]['id'] for i in order]**route_info})
857         if node['type']=='scan':
858             q=node['position']
859             # 到达扫描点后批量检测本节点全部尚未搜索频道。
当前频道优先,
860             # 其余频道按环形顺序检测,避免同一扫描点被反复
加入滚动路线。
861             pending=list(node['channels'])
862             # V2: 把当前扫描点同时当作已发现目标的顺路补测
点。只有当整个
863             # 定位多边形都位于1000米接收圆内,且预计信息量
达标时才加入。
864             route_targets=[]
865             for target_ch,target in self.targets.items():
866                 if target_ch in self.done:continue
867                 score=mandatory_stop_measurement_score(
target,q)
868                 if score is not None:route_targets.append
((score['information_nats'],target_ch,score))
869                 route_targets.sort(reverse=True)
870                 route_targets=route_targets[:
ROUTE_MEASURE_CONFIG['max_target_channels_per_stop']]
871                 pending.extend(ch for _,ch,_ in route_targets
if ch not in pending)
872                 if self.ch in pending:
873                     start=pending.index(self.ch)
874                     pending=pending[start:]+pending[:start]
875                     self.scheduler_counts['batch_scan_visits']+=1
876                     self.scheduler_counts['batch_scan_channels']+=
len(pending)
877                     self.write_log({'event':'batch_scan_start','
scan_id':node['id'],'index':node['index'],
878                                     'position':q,'channels':
pending,'start_channel':self.ch,
879                                     'route_aligned_targets':[{
'channel':ch,**score} for _,ch,score in route_targets]})
880                     measured=[]
881                     for ch in pending:
882                         # 前面的测量可能已将频道清除或认证为不存
在;这种情况下跳过。
883                         if ch in self.done or ch in self.absent or
node['index'] in self.searched[ch]:
884                             continue
885                         was_target=ch in self.targets
886                         res=self.call('/measure',q,ch)

```

```

887         if was_target:
888             self.scheduler_counts['
route_aligned_plans']+=1
889             self.scheduler_counts['
localization_measures']+=1
890         else:
891             self.scheduler_counts['search_measures
']+=1
892             measured.append(ch)
893             self.searched[ch].add(node['index'])
894             if res['measure_result']=='no_signal':
895                 self.coverage_map.update_negative(ch,q
)
896                 self.beliefs[ch]['negatives'].append(q
)
897                 belief=update_unknown_belief(self.
beliefs[ch],self.coverage_map,ch)
898                 self.write_log({'event': '
negative_update', 'channel':ch,**belief})
899                 # 仅尚未发现的频道可由全覆盖扫描认证为
不存在；已发现目标的
900                 # no_signal 只记录位置并限制重复探索，
不得排除目标位置或误判为无源。
901                 if ch in self.targets:
902                     self.targets[ch].setdefault('
negatives',[ ]).append(q)
903                     self.targets[ch]['planned']=None
904                     self.targets[ch]['
negative_attempts']=self.targets[ch].get('negative_attempts'
,0)+1
905                     joint_info=update_joint(self.
targets[ch])
906                     self.beliefs[ch].update(mean=self.
targets[ch]['mean'],covariance=self.targets[ch]['covariance'
])
907                     self.write_log({'event': '
joint_negative_update', 'channel':ch,**joint_info})
908                     elif len(self.searched[ch])==len(scans
):
909                         self.absent.add(ch);self.beliefs[
ch]['existence']=0.
910                     else:
911                         self.accept_bearing(ch,q,res)
912                         self.write_log({'event': 'batch_scan_end', '
scan_id':node['id'], 'index':node['index'],
'position':q, '
measured_channels':measured,
913                                     'new_current_channel':self.ch,
914                                     'done':sorted(self.done),
915                                     'absent':sorted(self.absent)})
916                         # 批次结束后再统一重建源节点与剩余扫描节点并规
划下一段路径。
917                         cache_key=None
918                         continue
919                         ch=node['channel'];target=self.targets[ch]
920                         if target.get('near') is not None or target['

```

```

radius']<=19.8:
921         q=target.get('near')
922         if q is None:q=self.clear_point(target['poly'
],target['center'])
923         self.write_log({'event':'local_decision','
channel':ch,'choice':'certified_clear'})
924         if not self.clear(ch,q):raise RuntimeError('可
靠清除与响应矛盾')
925         self.scheduler_counts['certified_clears']+=1;
continue
926         if target.get('fallback') is not None:
927             if not target['fallback']:raise RuntimeError('
网格兜底耗尽而目标未清除')
928             q=target['fallback'].pop(0);self.clear(ch,q)
929             self.scheduler_counts['fallback_clears']+=1;
continue
930         if target.get('planned') is None:
931             if len(target['stations'])+target.get('
negative_attempts',0)>=8:self.make_fallback(target);continue
932             free_q,free_detail=bayes_select(target['poly'
],target['stations'],self.q,target,self.deadline)
933             dop_detail=free_detail.get('
problem2_dop_tiebreak')
934             if dop_detail is not None:
935                 self.scheduler_counts['
dop_tie_break_decisions']+=1
936                 self.scheduler_counts['
dop_tie_break_overrides']+=int(dop_detail['override'])
937                 self.write_log({'event':'
problem2_dop_tiebreak','channel':ch,**dop_detail})
938                 if free_q is None:
939                     detail=free_detail;q=None
940                 else:
941                     q,detail=route_aligned_measurement(
942                         target['poly'],target['stations'],self
.q,target,next_stop,free_q,free_detail)
943                     key_name='route_aligned_plans' if detail.
get('route_aligned') else 'free_measurement_plans'
944                     self.scheduler_counts[key_name]+=1
945                     self.information_records.append(detail)
946                     self.write_log({'event':'measurement_plan','
channel':ch,**detail})
947                     if q is None:self.make_fallback(target);
continue
948                     target['planned']=q
949                     q=target['planned']
950                     measure_cost=dist(self.q,q)/5+5+(self.ch!=ch)+dist
(q,target['center'])/5+5
951                     if next_stop is not None:measure_cost+=dist(target
['center'],next_stop)/5
952                     probe=joint_probe_plan(target,self.q,self.ch,ch,q,
True,next_stop)
953                     self.write_log({'event':'local_decision','channel'
:ch,
954                                     'expected_measure_then_clear_s':measure_cost,
955                                     'expected_probe_sequence_s':probe['

```

```

expected_time_s'] if probe else None,
956     'probe_plan':probe,'choice':'probe' if probe
else 'measure',
957     'note':'补测一次后清除为预测代理；成功仍需收到
clear成功响应。'})
958     if probe:
959         target['probe_attempts']=target.get('
probe_attempts',0)+1
960         self.scheduler_counts['probe_attempts']+=1
961         self.scheduler_counts['joint_sequences_planned
']+=int(len(probe['sequence'])>=2)
962         self.scheduler_counts['on_route_probes']+=int(
probe['worst_branch_extra_distance_m']<1e-5)
963         success=self.clear(ch,probe['position'])
964         if success:self.scheduler_counts['
probe_successes']+=1
965         else:
966             info=condition_failed_clear(target,probe['
position'])
967             self.beliefs[ch].update(mean=target['mean'
],covariance=target['covariance'])
968             self.write_log({'event':'
failed_clear_update','channel':ch,**info})
969             self.write_log({'event':'probe_result','
channel':ch,'success':success,'plan':probe})
970         else:
971             res=self.call('/measure',q,ch);self.
scheduler_counts['localization_measures']+=1
972             if res['measure_result']=='no_signal':
973                 target.setdefault('negatives',[]).append(
tuple(q))
974                 target['negative_attempts']=target.get('
negative_attempts',0)+1
975                 target['planned']=None
976                 joint_info=update_joint(target)
977                 self.beliefs[ch].update(mean=target['mean'
],covariance=target['covariance'])
978                 self.write_log({'event':'
joint_negative_update','channel':ch,**joint_info})
979                 self.write_log({'event':'
directional_local_no_signal','channel':ch,'position':q,
980                                'note':'
keep_geometric_region_and_replan'})
981             else:self.accept_bearing(ch,q,res)
982             if len(self.done)<10:raise RuntimeError('清除数低于题
设下界10，未确认完成。')
983
984     def accept_bearing(self,ch,q,result):
985         target=self.targets.setdefault(ch,{'poly':initial_poly
(), 'stations':[], 'planned':None, 'fallback':None,
986                                'negatives':list(self.
beliefs[ch]['negatives'])})
987         self.beliefs[ch]['existence']=1.;target['planned']=
None
988         if result['measure_result']=='near':
989             target.update(near=q,center=q,radius=5.,mean=np.

```

```

990         asarray(q), covariance=np.eye(2)*25/4)
991         self.beliefs[ch].update(mean=target['mean'],
          covariance=target['covariance'])
          if not self.clear(ch,q):raise RuntimeError('near后
          清除失败')
992         self.scheduler_counts['certified_clears']+=1
993         else:
994             target['angle']=result['svd_deg'];target['poly']=
          update_poly(target['poly'],q,target['angle'])
995             target['stations'].append(q);target['center'],
          target['radius']=mec(target['poly'])
996             geometry=problem1_geometry_certificate(target['
          poly'],target['center'],target['radius'])
997             self.problem1_geometry_records.append(geometry)
998             posterior=update_gaussian(target,q,target['angle'
          ])
999             self.beliefs[ch].update(mean=target['mean'],
          covariance=target['covariance'])
1000             self.write_log({'event':'gaussian_update','channel
          ':ch,**posterior,
          'problem1_geometry':geometry})
1001
1002     def run(self):
1003         status='incomplete'; error=None; entered=False
1004         try:
1005             self.call('/enter'); entered=True
1006             self.schedule_run()
1007             status='complete'
1008             self.call('/exit')
1009         except BudgetStop as exc:
1010             status='budget_stop'; error=str(exc)
1011             if entered and time.monotonic()<self.deadline:
1012                 try: self.call('/exit')
1013                 except Exception as exit_exc: error+='; 退出失
1014 败: '+str(exit_exc)
1015             except (Exception,KeyboardInterrupt) as exc:
1016                 status='error'; error=str(exc) or '用户中断'
1017                 # 通信结果可能未知, 绝不在错误后继续发送新动作或探
          测/exit.
1018             geometry_records=[g for g in self.
          problem1_geometry_records
          if math.isfinite(g['jung_ratio'])
          and math.isfinite(g['radius_to_diameter'])]
1019             geometry_summary={'sample_count':len(self.
          problem1_geometry_records),
          'finite_sample_count':len(geometry_records),
          'max_jung_ratio':max((g['jung_ratio'] for g in
          geometry_records),default=None),
          'mean_radius_to_diameter':(sum(g['
          radius_to_diameter'] for g in geometry_records)/len(
          geometry_records)
          if geometry_records
          else None),
          'jung_violation_count':sum(not g['jung_certified']
          for g in self.problem1_geometry_records)}
1020             summary={'status':status,'error':error,'cleared_count'

```

```

: len(self.done),
1027         'cleared_channels': sorted(self.done), '
absent_channels': sorted(self.absent),
1028         'virtual_time_s': self.vt, '
average_clear_time_s': self.vt/len(self.done) if self.done
else None,
1029         'program_runtime_s': time.monotonic()-self.
started, 'distance_m': self.distance,
1030         'measure_count': self.measures, 'switch_count':
self.switches, 'clear_attempts': self.clears,
1031         'failed_clear_attempts': self.clears-len(self.
done), 'fallback_count': self.fallbacks,
1032         'information_selections': len(self.
information_records),
1033         'strategy': '
q4_21_joint_position_orientation_v1',
1034         'coverage_waypoints': 21, 'inner_ring': {'points
': 8, 'radius_m': 995},
1035         'outer_ring': {'points': 12, 'radius_m': 1865},
1036         'directional_negative_count': sum(t.get('
negative_attempts', 0) for t in getattr(self, 'targets', {})).
values()),
1037         'layout_name': self.layout['layout_name'],
1038         'ring_points': self.layout['ring_points'], '
ring_radius': self.layout['ring_radius'],
1039         'joint_config': dict(JOINT_CONFIG), '
probe_config': dict(PROBE_CONFIG),
1040         'bayes_config': dict(BAYES_CONFIG), '
q12_link_config': dict(Q12_LINK_CONFIG),
1041         'problem1_geometry': geometry_summary,
1042         'scheduler': getattr(self, 'scheduler_counts'
, {}), 'scheduler_config': dict(SCHEDULE_CONFIG),
1043         'clear_ratio': None, 'note': '真实总数仅演练结束
后可见; clear_ratio需用实际总数计算。'}
1044         expected=self.distance/5+self.switches+5*self.measures
+3*self.clears+2*len(self.done)
1045         summary['accounting_delta_s']=self.vt-expected
1046         if abs(summary['accounting_delta_s'])<1e-9: summary['
accounting_delta_s']=0.
1047         if self.out:
1048             if hasattr(self, 'beliefs'):
1049                 posteriors={str(ch): {'mean': b['mean'].tolist()
, 'covariance': b['covariance'].tolist(),
1050                                     'existence_probability': b
['existence'],
1051                                     'status': 'cleared' if ch
in self.done else ('absent' if ch in self.absent else ('
detected' if ch in self.targets else 'unknown'))}
1052                             for ch, b in self.beliefs.items()}
1053                 (self.out/'gaussian_posteriors.json').
write_text(json.dumps(posteriors, ensure_ascii=False, indent
=2), encoding='utf-8')
1054                 if hasattr(self, 'coverage_map'):
1055                     fm=self.coverage_map
1056                     snapshot={'cell_size_m': fm.step, 'centers': fm.
centers.tolist(),

```

```

1057         'remaining_cell_indices':{str(ch):np
.flatnonzero(fm.remaining[ch-1]).tolist()
1058                                     for ch in
range(1,21) if ch not in self.targets},
1059         'absent_channels':sorted(self.absent
), 'cleared_channels':sorted(self.done),
1060         'note':'方格仅为位置矩辅助；定向阴性
不删除方格，缺席由完整21点扫描认证。'}
1061         (self.out/'coverage_map.json').write_text(json
.dumps(snapshot,ensure_ascii=False),encoding='utf-8')
1062         (self.out/'summary.json').write_text(json.dumps(
summary,ensure_ascii=False,indent=2),encoding='utf-8')
1063         with (self.out/'summary.csv').open('w',encoding='
utf-8-sig',newline='') as f:
1064             writer=csv.DictWriter(f,fieldnames=list(
summary)); writer.writeheader(); writer.writerow(summary)
1065             if self.log: self.log.close()
1066             return summary
1067
1068 def write_csv(path, rows):
1069     if not rows:
1070         return
1071     with path.open('w',encoding='utf-8-sig',newline='') as f:
1072         writer=csv.DictWriter(f,fieldnames=list(rows[0]))
1073         writer.writeheader();writer.writerows(rows)
1074
1075 def scenario_record(backend):
1076     return {'seed':backend.seed,'total':backend.total,
1077            'sources':[{'channel':ch,'position':list(value[0])
,
1078                        'reception_radius_m':value[1],'
direction_rad':backend.directions[ch]}
1079                for ch,value in sorted(backend.targets.
items())]}
1080
1081 def run_local_case(seed, case_number, layout, case_dir):
1082     backend=LocalBackend(seed,edge=False)
1083     truth=scenario_record(backend)
1084     robot=Robot(backend,'LOCAL-TEST',case_dir,quiet=True,
layout=layout)
1085     result=robot.run()
1086     result.update(true_total=backend.total,
1087                  clear_ratio=len(backend.done)/backend.total,
1088                  seed=seed,case=case_number)
1089     # Robot.run先写基础summary；补充仅本地评估器可知的真值指
标。
1090     (case_dir/'summary.json').write_text(
1091         json.dumps(result,ensure_ascii=False,indent=2),
encoding='utf-8')
1092     write_csv(case_dir/'summary.csv',[result])
1093     (case_dir/'scenario.json').write_text(
1094         json.dumps(truth,ensure_ascii=False,indent=2),encoding
='utf-8')
1095     success=(result['status']=='complete' and len(backend.done
)==backend.total and
1096             abs(result['accounting_delta_s'])<1e-5 and

```

```

11097         all(len(robot.searched[ch])==layout['ring_points'
11098 ]+1
11099         for ch in result['absent_channels']))
11099     row={'layout_name':layout['layout_name'],'ring_points':
11100 layout['ring_points'],
11101         'ring_radius':layout['ring_radius'],'case':
11102 case_number,'seed':seed,
11101         'true_total':backend.total,'total':backend.total,
11102         'cleared_count':len(backend.done),'cleared':len(
11103 backend.done),
11103         'success':success,'status':result['status'],'
11104 clear_ratio':result['clear_ratio'],
11104         'virtual_time_s':result['virtual_time_s'],
11105         'average_clear_time_s':result['average_clear_time_s'
11106 ],
11106         'distance_m':result['distance_m'],'measure_count':
11107 result['measure_count'],
11107         'failed_clear_attempts':result['failed_clear_attempts
11108 '],
11108         'program_runtime_s':result['program_runtime_s'],
11109         'probe_attempts':result['scheduler'].get('
11110 probe_attempts',0),
11110         'probe_successes':result['scheduler'].get('
11111 probe_successes',0),
11111         'exact_routes':result['scheduler'].get('exact_routes'
11112 ,0),
11112         'approximate_routes':result['scheduler'].get('
11113 approximate_routes',0),
11113         'error':result['error']}
11114     return row,truth
11115
11116 def summarize_layout(layout, rows):
11117     def mean(key):return sum(float(r[key]) for r in rows)/len(
11118 rows)
11118     total_times=[float(r['virtual_time_s']) for r in rows]
11119     avg_time=sum(total_times)/len(total_times)
11120     variance=sum((x-avg_time)**2 for x in total_times)/len(
11121 total_times)
11121     total_sources=sum(int(r['true_total']) for r in rows)
11122     return {'layout_name':layout['layout_name'],'ring_points':
11123 layout['ring_points'],
11123         'ring_radius':layout['ring_radius'],'case_count':
11124 len(rows),
11124         'success_count':sum(bool(r['success']) for r in
11125 rows),
11125         'mean_clear_ratio':mean('clear_ratio'),
11126         'mean_virtual_time_s':avg_time,
11127         'std_virtual_time_s':math.sqrt(variance),
11128         'pooled_time_per_source_s':sum(total_times)/
11129 total_sources,
11129         'mean_distance_m':mean('distance_m'),
11130         'mean_measure_count':mean('measure_count'),
11131         'mean_program_runtime_s':mean('program_runtime_s')
11132     ,
11132         'failed_clear_attempts':sum(int(r['
11133 failed_clear_attempts']) for r in rows)}

```

```

1133
1134 def self_test(count, seed_start=2026091200, output_dir=None,
1135              layout=None):
1136     """运行单布局本地案例；目录为 root/layout_name/case_NN。"""
1137     root=Path(output_dir or 'local_tests');root.mkdir(parents=
1138               True,exist_ok=True)
1139     layout=layout or resolve_layout();seeds=list(range(
1140               seed_start,seed_start+count))
1141     rows=[]
1142     for i,seed in enumerate(seeds,1):
1143         case=root/layout['layout_name']/f'case_{i:02d}'
1144         row,_=run_local_case(seed,i,layout,case)
1145         rows.append(row);print(json.dumps(row,ensure_ascii=
1146               False),flush=True)
1147     summary=summarize_layout(layout,rows)
1148     report={'note':'本地随机合成环境，非官方演练；仿真真值只供
1149             评估器使用',
1150            'layout':layout,'seeds':seeds,'cases':rows,'
1151            summary':summary,
1152            'mean_total_time_s':summary['mean_virtual_time_s'
1153            ],
1154            'pooled_time_per_source_s':summary['
1155            pooled_time_per_source_s']}
1156     (root/'test_results.json').write_text(
1157         json.dumps(report,ensure_ascii=False,indent=2),
1158         encoding='utf-8')
1159     print(json.dumps(summary,ensure_ascii=False,indent=2))
1160     return report
1161
1162 def compare_layouts(count, seed_start=2026091200, output_dir=
1163                    None):
1164     """以共同随机数对全部预设布局作配对仿真并保存完整过程。"""
1165     root=Path(output_dir or 'layout_comparison');root.mkdir(
1166         parents=True,exist_ok=True)
1167     seeds=list(range(seed_start,seed_start+count));all_rows
1168     =[];summaries=[]
1169     scenario_fingerprints={}
1170     resolved=[resolve_layout(name) for name in LAYOUT_PRESETS]
1171     for layout in resolved:
1172         rows=[]
1173         for i,seed in enumerate(seeds,1):
1174             case=root/layout['layout_name']/f'case_{i:02d}'
1175             row,truth=run_local_case(seed,i,layout,case)
1176             fingerprint=hashlib.sha256(json.dumps(truth,
1177             sort_keys=True).encode()).hexdigest()
1178             old=scenario_fingerprints.setdefault(seed,
1179             fingerprint)
1180             if old!=fingerprint:
1181                 raise RuntimeError(f'seed {seed} 在不同布局生
1182                 成了不同真值')
1183             rows.append(row);all_rows.append(row)
1184             print(json.dumps(row,ensure_ascii=False),flush=
1185                   True)
1186         summaries.append(summarize_layout(layout,rows))
1187     write_csv(root/'comparison_cases.csv',all_rows)

```

```

1172     write_csv(root/'comparison_summary.csv', summaries)
1173     results={'note': '所有布局共享相同seed与LocalBackend生成规则，可进行配对比较',
1174             'seeds': seeds, 'cases': all_rows, 'summaries':
summaries}
1175     (root/'comparison_results.json').write_text(
1176         json.dumps(results, ensure_ascii=False, indent=2),
encoding='utf-8')
1177     manifest={'experiment': '
q4_fixed_layout_common_random_numbers',
1178             'created_at': time.strftime('%Y-%m-%dT%H:%M:%S%z'
),
1179             'description': '问题四混合源本地合成仿真；各布局
使用完全相同的seed和目标真值。',
1180             'case_count': count, 'seed_start': seed_start, '
seeds': seeds,
1181             'presets': LAYOUT_PRESETS,
1182             'coverage_certificates': {x['layout_name']: x['
certificate'] for x in resolved},
1183             'output_structure': '<layout_name>/case_NN/保存每
局完整过程文件'}
1184     (root/'experiment_manifest.json').write_text(
1185         json.dumps(manifest, ensure_ascii=False, indent=2),
encoding='utf-8')
1186     print(json.dumps({'summaries': summaries}, ensure_ascii=
False, indent=2))
1187     return results
1188
1189 JOINT_DIRECTION_CONFIG={'direction_bins': 96, 'position_depth'
: 3, 'predictive_states': 1536, 'prior_omni': .5}
1190
1191 def update_joint(target):
1192     """在当前硬几何多边形内重算全历史联合后验，固定半径区间解
析积分。
1193     离散朝向/三角求积只影响调度，不删除硬几何支持集，不认证缺席。
1194     """
1195     cfg=JOINT_DIRECTION_CONFIG
1196     triangles, areas=mesh(target['poly'], cfg['position_depth'])
;points=triangles.mean(axis=1)
1197     n=len(points); angles=(np.arange(cfg['direction_bins']+.5)
*2*math.pi/cfg['direction_bins'])
1198     normals=np.c_[np.cos(angles), np.sin(angles)]; k=len(angles)
1199     lower=np.full(n, 1000.); upper=np.full((n, k+1), 1500.)
1200     valid=np.linalg.norm(points, axis=1)<=1800+1e-7
1201     orientation_valid=np.ones((n, k+1), dtype=bool)
1202     log_position=np.log(areas)
1203     variance=math.radians(BAYES_CONFIG['noise_deg'])*2
1204     for q, angle in target.get('bearing_history', []):
1205         v=np.asarray(q)-points; distance=np.linalg.norm(v, axis
=1)
1206         lower=np.maximum(lower, distance); valid &= distance>5
1207         predicted=np.arctan2(-v[:, 1], -v[:, 0])
1208         residual=(predicted-math.radians(angle)+math.pi)%(2*
math.pi)-math.pi

```

```

1209         log_position-=residual**2/(2*variance)
1210         orientation_valid[:,1:] &= v@normals.T>=-1e-8
1211         for q in target.get('negatives',[]):
1212             v=np.asarray(q)-points;distance=np.linalg.norm(v,axis
=1)
1213             emits=np.c_[np.ones(n,bool),v@normals.T>=-1e-8]
1214             upper=np.minimum(upper,np.where(emits,distance[:,None
],1500.))
1215             for q in target.get('failed_clears',[]):valid &= np.linalg
.norm(points-q,axis=1)>20
1216             width=np.maximum(upper-lower[:,None],0.)
1217             prior=np.r_[cfg['prior_omni'],np.full(k,(1-cfg['prior_omni
']/k))]
1218             position=np.exp(log_position-log_position.max())*valid
1219             weights=position[:,None]*width*orientation_valid*prior[
None,:]/500.
1220             total=float(weights.sum())
1221             if not math.isfinite(total) or total<=1e-280:
1222                 target.pop('joint_posterior',None)
1223                 return {'note':'
joint_quadrature_degenerate_keep_geometric_and_existing_moments
'}
1224             weights/=total;marginal=weights.sum(axis=1)
1225             target['mean'],target['covariance']=gaussian_moments(
points,marginal)
1226             # 确定性等质量压缩仅用于多候选点的接收概率预测。
1227             flat=weights.ravel();size=cfg['predictive_states']
1228             indexes=np.searchsorted(np.cumsum(flat),(np.arange(size)
+.5)/size)
1229             ip,ia=np.unravel_index(np.minimum(indexes,len(flat)-1),
weights.shape)
1230             target['joint_posterior']=dict(points=points,weights=
marginal,
1231             predictive_points=points[ip],predictive_angles=ia,
1232             predictive_lower=lower[ip],predictive_upper=upper[ip,
ia],normals=normals,
1233             omni_probability=float(weights[:,0].sum()))
1234             target.pop('probe_base',None)
1235             return {'note':'
joint_position_orientation_fixed_radius_full_history_quadrature
',
1236                     'mean':target['mean'].tolist(),'covariance':target
['covariance'].tolist(),
1237                     'omni_probability':float(weights[:,0].sum()),'
position_nodes':n,'direction_bins':k}
1238
1239 # 替换基版函数；硬几何更新仍由原 accept_bearing 执行。
1240 _base_position_update=update_gaussian
1241
1242 def update_gaussian(target,q,angle):
1243     # 先保留基版作为求积退化时的工作矩回退，再以完整历史联合后
    验替换。
1244     result=_base_position_update(target,q,angle)
1245     target.setdefault('bearing_history',[]).append((tuple(q),
float(angle)))
1246     result.update(update_joint(target))

```

```

1247     return result
1248
1249 def direction_proxy(target,p):
1250     posterior=target.get('joint_posterior')
1251     if posterior is None: return .5
1252     v=np.asarray(p)-posterior['predictive_points'];distance=np
1253     .linalg.norm(v,axis=1)
1254     ia=posterior['predictive_angles'];normal=posterior['
1255     normals'][np.maximum(ia-1,0)]
1256     emits=(ia==0)|(np.einsum('ij,ij->i',v,normal)>=-1e-8)
1257     lower=posterior['predictive_lower'];upper=posterior['
1258     predictive_upper']
1259     probability=np.maximum(upper-np.maximum(lower,distance),0)
1260     /np.maximum(upper-lower,1e-12)
1261     return float(np.clip(np.mean(probability*emits),.01,.999))
1262
1263 _base_probe_distribution=probe_distribution
1264
1265 def probe_distribution(target):
1266     posterior=target.get('joint_posterior')
1267     if posterior is None: return _base_probe_distribution(
1268     target)
1269     points=posterior['points'];weights=posterior['weights'].
1270     copy()
1271     for q in target.get('failed_clears',[]): weights[np.linalg.
1272     norm(points-q,axis=1)<=20]=0
1273     if weights.sum()<=1e-280: return None
1274     return points,weights/weights.sum()
1275
1276 _base_failed_clear=condition_failed_clear
1277
1278 def condition_failed_clear(target,q):
1279     result=_base_failed_clear(target,q)
1280     result.update(update_joint(target))
1281     return result
1282
1283 def main():
1284     parser=argparse.ArgumentParser(description=__doc__,
1285     formatter_class=argparse.RawDescriptionHelpFormatter)
1286     parser.add_argument('--source-mode',choices=['mixed','omni
1287     ','directional','outward'],default='mixed',help='仅本地：混
1288     合/全向/定向/朝外定向')
1289     parser.add_argument('--edge',action='store_true',help='仅
1290     本地：目标位于1800米边界，接收半径1000米')
1291     parser.add_argument('--search-gain',type=float,default
1292     =0.2,help='扫描批次收益权重，默认0.2，非存在概率')
1293     parser.add_argument('--cell-size',type=float,default=100.,
1294     help='保守方格边长，默认100米，50到200米')
1295     parser.add_argument('--lookahead',type=float,default=1.0,
1296     help='插入任务的后续绕行权重，默认1.0')
1297     parser.add_argument('--quad-depth',type=int,default=3,help
1298     ='后验积分三角形细分层数，1到4')
1299     parser.add_argument('--noise-deg',type=float,default=1/
1300     math.sqrt(3),help='近似测角标准差，默认0.577度；不改变硬误差

```

```

1286     parser.add_argument('--prior-exists',type=float,default
1287     =0.65,help='每频道初始存在概率近似，默认0.65')
1288     parser.add_argument('--clear-detour',type=float,default
1289     =5.,help='失败分支相对既定路线最多增加的米数；0为严格顺路')
1290     parser.add_argument('--clear-horizon',type=int,default=3,
1291     help='联合规划深度，1到3')
1292     parser.add_argument('--min-clear-saving',type=float,
1293     default=0.,help='预计至少节省的秒数')
1294     parser.add_argument('--max-probes',type=int,default=5,help
1295     ='每目标试探clear次数上限；0为禁用')
1296     parser.add_argument('--min-probe-prob',type=float,default
1297     =0.2,help='试探成功概率估计下限')
1298     parser.add_argument('--probe-radius',type=float,default
1299     =80.,help='允许试探的定位覆盖圆半径上限，米')
1300     parser.add_argument('--layout',choices=list(LAYOUT_PRESETS
1301     ),default=DEFAULT_LAYOUT,
1302     help='预设布局，默认 q4_21')
1303     parser.add_argument('--ring-points',type=int,
1304     help='自定义外围点数，覆盖所选预设的 n
1305     ')
1306     parser.add_argument('--ring-radius',type=float,
1307     help='自定义布局半径（米），覆盖所选预
1308     设的 R')
1309     parser.add_argument('--compare-layouts',action='store_true
1310     ',
1311     help='对全部预设布局使用相同seed批量比
1312     较（需同时指定--self-test N）')
1313     parser.add_argument('--robot-id',help='与模拟器登录参赛队
1314     号完全一致')
1315     parser.add_argument('--base-url',default='http
1316     ://127.0.0.1:2026')
1317     parser.add_argument('--output',help='日志目录；默认按时间
1318     创建')
1319     parser.add_argument('--seed-start',type=int,default
1320     =2026091200,help='本地测试起始随机种子')
1321     parser.add_argument('--test-output',default='local_tests',
1322     help='本地测试结果目录')
1323     parser.add_argument('--self-test',type=int,metavar='N',
1324     help='运行N个本地随机案例，不连接官方模拟器')
1325     args=parser.parse_args()
1326     LOCAL_CONFIG.update(source_mode=args.source_mode,edge=args
1327     .edge)
1328     if not (0<args.noise_deg<=5 and 0<args.prior_exists<1 and
1329     1<=args.quad_depth<=4):
1330         parser.error('0<noise-deg<=5, 0<prior-exists<1, 1<=
1331     quad-depth<=4')
1332     if not (0<args.search_gain<=5 and 0<=args.lookahead<=1 and
1333     50<=args.cell_size<=200):
1334         parser.error('0<search-gain<=5, 0<=lookahead<=1, 50<=
1335     cell-size<=200')
1336     SCHEDULE_CONFIG.update(search_gain=args.search_gain,
1337     lookahead=args.lookahead,cell_size=args.cell_size)
1338     if not (0<=args.max_probes<=20 and 0<args.min_probe_prob<1

```

```

1315     and 20<=args.probe_radius<=200):
1316         parser.error('0<=max-probes<=20, 0<min-probe-prob<1,
20<=probe-radius<=200')
1317     if not (0<=args.clear_detour<=50 and 1<=args.clear_horizon
<=3 and args.min_clear_saving>=0):
1318         parser.error('0<=clear-detour<=50, 1<=clear-horizon<=3
, min-clear-saving>=0')
1319     JOINT_CONFIG.update(max_detour_m=args.clear_detour,horizon
=args.clear_horizon,min_saving_s=args.min_clear_saving)
1320     PROBE_CONFIG.update(max_attempts=args.max_probes,
min_probability=args.min_probe_prob,max_radius=args.
probe_radius)
1321     BAYES_CONFIG.update(noise_deg=args.noise_deg,prior_exists=
args.prior_exists,quad_depth=args.quad_depth)
1322     if args.compare_layouts and args.self_test is None:
1323         parser.error('--compare-layouts 必须与 --self-test N
同时使用')
1324     if args.compare_layouts and (args.ring_points is not None
or args.ring_radius is not None):
1325         parser.error('--compare-layouts 比较固定预设, 不接受
--ring-points/--ring-radius')
1326     try:
1327         layout=resolve_layout(args.layout,args.ring_points,
args.ring_radius)
1328     except (ValueError,KeyError) as exc:
1329         parser.error(str(exc))
1330     if args.self_test is not None:
1331         if args.self_test<1: parser.error('N必须大于0')
1332         if args.compare_layouts:
1333             report=compare_layouts(args.self_test,args.
seed_start,args.test_output)
1334             if any(x['success_count']<args.self_test for x in
report['summaries']):
1335                 raise SystemExit(1)
1336             else:
1337                 report=self_test(args.self_test,args.seed_start,
args.test_output,layout)
1338                 if report['summary']['success_count']<args.
self_test:raise SystemExit(1)
1339             return
1340     if not args.robot_id: parser.error('请提供 --robot-id 参赛
队号')
1341     out=Path(args.output) if args.output else Path('runs')/(
layout['layout_name']+'_'+time.strftime('%Y%m%d_%H%M%S
')+ '_' +uuid.uuid4().hex[:6])
1342     print(f"问题4定向混合滚动路径版: 布局={layout['layout_name
']}, 外围点数={layout['ring_points']}, "
1343         f"半径={layout['ring_radius']:g}米。请确保官方模拟器
接口已就绪。日志目录: {out.resolve()}")
1344     result=Robot(HttpBackend(args.base_url),args.robot_id,out,
layout=layout).run()
1345     print("==== 最终结果 =====")
1346     print(f"实际清除总数: {result['cleared_count']}")
1347     print(f"实际消耗总时间: {result['virtual_time_s']:.2f} s")
1348     if result['average_clear_time_s'] is None:

```

```
1349         print("实际平均消耗时间: N/A")
1350     else:
1351         print(f"实际平均消耗时间: {result['average_clear_time_s']:.2f} s")
1352
1353     if result['status'] != 'complete':
1354         print('本次未确认全部完成; 请保留日志并检查模拟器界面。')
1355         raise SystemExit(1)
1356 if __name__ == '__main__':
1357     main()
```

q2\_NIPT\_time.py